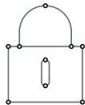
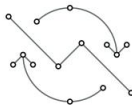
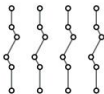
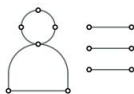
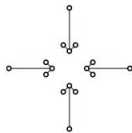
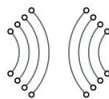
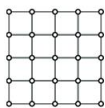


01

INTRODUÇÃO



O ritmo em que os sistemas de computadores mudam foi, é e continua sendo avassalador. De 1945, quando a era moderna dos computadores começou, até cerca de 1985, os computadores eram grandes e caros. Além disso, sem uma maneira de conectá-los, esses computadores operavam independentemente uns dos outros.

A partir de meados da década de 1980, no entanto, dois avanços na tecnologia começaram a mudar essa situação. O primeiro foi o desenvolvimento de microprocessadores poderosos. Inicialmente, eram máquinas de 8 bits, mas logo CPUs de 16, 32 e 64 bits se tornaram comuns. Com CPUs multicore poderosas, agora estamos novamente enfrentando o desafio de adaptar e desenvolver programas para explorar o paralelismo. Em qualquer caso, a geração atual de máquinas tem o poder de computação dos mainframes implantados há 30 ou 40 anos, mas por 1/1000 do preço ou menos.

O segundo desenvolvimento foi a invenção de redes de computadores de alta velocidade. **Redes locais** ou **LANs** permitem que milhares de máquinas dentro de um edifício sejam conectadas de tal forma que pequenas quantidades de informação podem ser transferidas em alguns microssegundos ou mais. Quantidades maiores de dados podem ser movidas entre máquinas a taxas de bilhões de bits por segundo (**bps**). **Redes de longa distância** ou **WANs** permitem que centenas de milhões de máquinas em todo o planeta sejam conectadas a velocidades que variam de dezenas de milhares a centenas de milhões de bps e mais.

Paralelamente ao desenvolvimento de máquinas cada vez mais poderosas e em rede, também pudemos testemunhar a miniaturização de sistemas de computadores, com talvez o smartphone como o resultado mais impressionante. Cheios de sensores, muita memória e uma CPU multicore poderosa, esses dispositivos são nada menos que computadores de ponta. Claro, eles também têm recursos de rede. Na mesma linha, os chamados **nanocomputadores** tornaram-se prontamente disponíveis. Esses pequenos computadores de placa única, geralmente do tamanho de um cartão de crédito, podem facilmente oferecer desempenho próximo ao de um desktop. Exemplos bem conhecidos incluem os sistemas Raspberry Pi e Arduino.

E a história continua. À medida que a digitalização da nossa sociedade continua, nos tornamos cada vez mais conscientes de quantos computadores estão realmente sendo usados, regularmente incorporados em outros sistemas, como carros, aviões, edifícios, pontes, rede elétrica e assim por diante. Essa conscientização é, infelizmente, aumentada quando tais sistemas de repente se tornam hackeáveis. Por exemplo, em 2021, um oleoduto de combustível nos Estados Unidos foi efetivamente fechado por um ataque de ransomware. Neste caso, o sistema de computador consistia em uma mistura de sensores, atuadores, controladores, computadores incorporados, servidores, etc., todos reunidos em um único sistema. O que muitos de nós não percebemos é que infraestruturas vitais, como oleodutos de combustível, são monitoradas e controladas por **sistemas de computadores em rede**. Na mesma linha, pode ser hora de começar a perceber que um carro moderno é, na verdade, um computador em rede móvel e autônomo. Neste caso, em vez de o computador móvel ser carregado por uma pessoa, precisamos lidar com o computador móvel transportando pessoas.

O tamanho de um sistema de computador em rede pode variar de um punhado de dispositivos a milhões de computadores. A rede de interconexão pode ser com fio, sem fio ou uma combinação de ambos. Além disso, esses sistemas são frequentemente altamente dinâmicos, no sentido de que os computadores podem entrar e sair, com a topologia e o desempenho da rede subjacente mudando quase continuamente.

É difícil pensar em sistemas de computador que não estejam em rede. E, de fato, a maioria dos sistemas de computador em rede pode ser acessada de qualquer lugar do mundo porque eles estão conectados à Internet. Estudar para entender esses sistemas pode facilmente se tornar extremamente complexo. Neste capítulo, começamos esclarecendo o que precisa ser compreendido para construir o quadro geral sem se perder.

1.1 De sistemas em rede para sistemas distribuídos

Antes de nos aprofundarmos nos vários aspectos dos sistemas distribuídos, vamos primeiro considerar o que a distribuição, ou descentralização, realmente implica.

1.1.1 Sistemas distribuídos versus descentralizados

Ao considerar várias fontes, há algumas opiniões sobre sistemas distribuídos versus descentralizados. Frequentemente, a distinção é ilustrada por três organizações diferentes de sistemas de computadores em rede, como mostrado na Figura 1.1, onde cada nó representa um sistema de computador e uma aresta, um link de comunicação entre dois nós.

Até que ponto tais distinções são úteis ainda está para ser visto, especialmente quando as discussões se abrem sobre os prós e contras de cada organização. Por exemplo, é frequentemente afirmado que organizações centralizadas não escalam bem. Da mesma forma, organizações distribuídas são consideradas mais robustas contra falhas. Como veremos, nenhuma dessas afirmações é geralmente verdadeira.

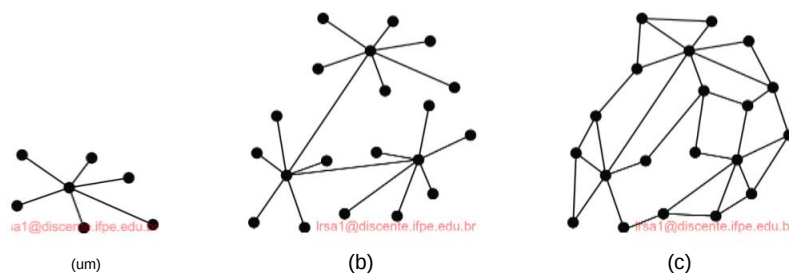


Figura 1.1: A organização de um sistema (a) centralizado, (b) descentralizado e (c) distribuído, de acordo com várias fontes populares. Adotamos uma abordagem diferente, pois figuras como essas não são realmente tão significativas.

Nós adotamos uma abordagem diferente. Se pensarmos em um sistema de computadores em rede como uma coleção de computadores conectados em uma rede, podemos nos perguntar como esses computadores se conectaram uns aos outros em primeiro lugar. Existem basicamente duas visões que se pode adotar.

A primeira **visão, integrativa**, é que havia uma necessidade de conectar sistemas de computadores existentes (em rede) uns aos outros. Normalmente, isso acontece quando serviços em execução em um sistema precisam ser disponibilizados para usuários e aplicativos que não foram pensados antes. Isso pode acontecer, por exemplo, ao integrar serviços financeiros com serviços de gerenciamento de projetos, como geralmente é o caso dentro de uma única organização. No domínio da pesquisa científica, vimos esforços para conectar uma miríade de recursos frequentemente caros (computadores de propósito especial, supercomputadores, sistemas de banco de dados muito grandes, etc.) no que veio a ser conhecido como um computador de grade.

A segunda **visão expansiva** é que um sistema existente requer uma extensão por meio de computadores adicionais. Essa visão é a mais frequentemente relacionada ao campo de sistemas distribuídos. Ela envolve expandir um sistema com computadores para manter recursos próximos de onde esses recursos são necessários. Uma expansão também pode ser motivada pela necessidade de melhorar a confiabilidade: se um computador falhar, então há outros que podem assumir. Um tipo importante de expansão é quando um serviço precisa ser disponibilizado para usuários e aplicativos remotos, por exemplo, oferecendo uma interface da Web ou um aplicativo para smartphone. Este último exemplo também mostra que a distinção entre uma visão integrativa e uma visão expansiva não é clara.

Em ambos os casos, vemos que o sistema em rede executa serviços, onde cada serviço é implementado como uma coleção de processos e recursos espalhados por vários computadores. As duas visões levam a uma distinção natural entre dois tipos de sistemas de computadores em rede:

- Um **sistema descentralizado** é um sistema de computador em rede no qual os processos e recursos são **necessariamente** distribuídos por vários computadores.
- Um **sistema distribuído** é um sistema de computador em rede no qual os processos e recursos são **suficientemente** distribuídos por vários computadores.

Antes de discutir por que essa distinção é importante, vejamos alguns exemplos de cada tipo de sistema.

Os sistemas descentralizados estão principalmente relacionados à visão integrativa de sistemas de computadores em rede. Eles surgem porque queremos conectar sistemas, mas podem ser impedidos por limites administrativos. Por exemplo, muitas aplicações no domínio da inteligência artificial exigem grandes quantidades de dados para construir modelos preditivos confiáveis. Normalmente, os dados são levados aos computadores de alto desempenho que literalmente treinam os modelos antes que eles possam ser usados. Mas quando os dados precisam permanecer dentro do perímetro de uma organização (e pode haver muitas razões pelas quais isso é necessário), precisamos levar o treinamento aos dados. O resultado é conhecido como **aprendizado federado**,

e é implementado por um sistema descentralizado, onde a necessidade de distribuição de processos e recursos é ditada por políticas administrativas.

Outro exemplo de um sistema descentralizado é o de **livro-razão distribuído**, também conhecido como **blockchain**. Neste caso, precisamos lidar com a situação em que as partes participantes não confiam umas nas outras o suficiente para criar esquemas simples de colaboração. Em vez disso, o que elas fazem é essencialmente tornar as transações entre si totalmente públicas (e verificáveis) por um livro-razão somente extensível que mantém registros dessas transações. O livro-razão em si é totalmente distribuído entre os participantes, e os participantes são aqueles que validam as transações (de outros) antes de admiti-las no livro-razão. O resultado é um sistema descentralizado no qual os processos e recursos são, de fato, necessariamente distribuídos entre vários computadores, neste caso devido à falta de confiança.

Como último exemplo de um sistema descentralizado, considere sistemas que são naturalmente dispersos geograficamente. Isso ocorre tipicamente com sistemas nos quais um local real precisa ser monitorado, por exemplo, no caso de uma usina de energia, um edifício, um ambiente natural específico e assim por diante. O sistema, controlando os monitores e onde as decisões são tomadas, pode facilmente ser colocado em outro lugar que não o local que está sendo monitorado. Um exemplo óbvio é o monitoramento e controle de satélites, mas também situações mais mundanas como monitoramento e controle de tráfego, trens, etc. Nesses exemplos, a necessidade de espalhar processos e recursos vem de um argumento espacial.

Como mencionamos, os sistemas distribuídos estão principalmente relacionados à visão expansiva de sistemas de computadores em rede. Um exemplo bem conhecido é o uso de serviços de e-mail, como o Google Mail. O que geralmente acontece é que um usuário faz login no sistema por meio de uma interface da Web para ler e enviar e-mails. Mais frequentemente, no entanto, é que os usuários configuram seu computador pessoal (como um laptop) para usar um cliente de e-mail específico. Para isso, eles precisam configurar algumas configurações, como o servidor de entrada e saída. No caso do Google Mail, são `imap.gmail.com` e `smtp.gmail.com`, respectivamente. Logicamente, parece que esses dois servidores lidarão com todos os seus e-mails. No entanto, com uma estimativa de quase 2 bilhões de usuários em 2022, é improvável que apenas dois computadores possam lidar com todos os seus e-mails (que foi estimado em mais de 300 bilhões por ano, ou seja, cerca de 10.000 e-mails por segundo). Nos bastidores, é claro, todo o serviço Google Mail foi implementado e espalhado por muitos computadores, formando em conjunto um sistema distribuído. Esse sistema foi configurado para garantir que muitos usuários possam processar seus e-mails (ou seja, garante escalabilidade), mas também que o risco de perda de e-mails devido a falhas seja mínimo (ou seja, o sistema garante tolerância a falhas). Para o usuário, no entanto, a imagem de apenas dois servidores é mantida (ou seja, a distribuição em si é altamente transparente para o usuário). O sistema distribuído que implementa um serviço de e-mail, como o Google Mail, normalmente se expande (ou encolhe) conforme ditado pelos requisitos de confiabilidade, por sua vez, dependendo do número de seus

Um tipo totalmente diferente de sistema distribuído é formado pela coleção das chamadas **Redes de Distribuição de Conteúdo**, ou **CDNs** para abreviar. Um exemplo bem conhecido é a Akamai com, em 2022, mais de 400.000 servidores em todo o mundo. Discutiremos o princípio de funcionamento das CDNs mais adiante no Capítulo 3. O que se resume é que o conteúdo de um site real é copiado e espalhado por vários servidores da CDN. Ao visitar um site, o usuário é redirecionado de forma transparente para um servidor próximo que contém todo ou parte do conteúdo desse site. A escolha de qual servidor direcionar um usuário pode depender de muitos fatores, mas certamente quando se trata de conteúdo de streaming, é selecionado um servidor que garanta um bom desempenho em termos de latência e largura de banda. O CDN garante dinamicamente que o servidor selecionado terá o conteúdo necessário prontamente disponível, além de atualizar esse conteúdo quando necessário ou removê-lo do servidor quando não houver usuários ou houver poucos usuários para atendê-lo. Enquanto isso, o usuário não sabe nada sobre o que está acontecendo nos bastidores (o que, novamente, é uma forma de transparência de distribuição). Também vemos neste exemplo que o conteúdo não é copiado para todos os servidores, mas apenas para onde faz sentido, ou seja, suicientemente, e por razões de desempenho. CDNs também copiam conteúdo para vários servidores para fornecer altos níveis de confiabilidade.

Como um sistema distribuído ãnal, muito menor, considere uma configuração baseada em um **dispositivo de armazenamento conectado à rede**, também chamado de **NAS**. Para uso doméstico, um NAS típico consiste em 2–4 slots para discos rígidos internos. O NAS opera como um servidor de arquivos: ele é acessível por meio de uma rede (geralmente sem fio) para qualquer dispositivo autorizado e, como tal, pode oferecer serviços como armazenamento compartilhado, backups automatizados, streaming de mídia e assim por diante. O NAS em si pode ser melhor visto como um único computador otimizado para armazenar arquivos e oferecer a capacidade de compartilhar facilmente esses arquivos. O último é importante e, junto com vários usuários, temos essencialmente uma configuração de um sistema distribuído. Os usuários trabalharão com um conjunto de arquivos que são localmente (ou seja, de seus laptops) facilmente acessíveis (na verdade, aparentemente integrados ao sistema de arquivos local), enquanto também diretamente acessíveis por e para outros usuários. Novamente, onde e como os arquivos compartilhados são armazenados é oculto (ou seja, a distribuição é transparente). Assumindo que compartilhar arquivos é o objetivo, então vemos que de fato um NAS pode fornecer distribuição suiciente de processos e recursos.

Nota 1.1 (Mais informações: Soluções centralizadas são ruins?)

Parece haver um equívoco persistente de que soluções centralizadas não podem ser escaláveis. Além disso, elas são quase sempre associadas à introdução de um único ponto de falha. Ambas as razões são frequentemente vistas como suficientes para descartar soluções centralizadas como sendo uma boa escolha ao projetar sistemas distribuídos.

O que muitas pessoas esquecem é que uma diferença deve ser feita entre designs lógicos e físicos. Uma solução logicamente centralizada pode ser implementada de uma maneira distribuída altamente escalável. Um excelente exemplo é o **Domain Name System (DNS)**, que discutimos extensivamente no Capítulo 6. Logicamente, o DNS é

organizado como uma árvore enorme, onde cada caminho da raiz para um nó folha representa um nome totalmente qualificado, como `www.distributed-systems.net`. Seria um erro pensar que o nó raiz é implementado por apenas um único servidor. Na verdade, o nó raiz é implementado por 13 servidores raiz diferentes, cada servidor, por sua vez, implementado como um grande computador de cluster. A organização física do DNS também mostra que a raiz não é um único ponto de falha. Sendo altamente replicado, seriam necessários esforços sérios para derrubar essa raiz e, até agora, todas as tentativas de fazê-lo falharam.

Soluções centralizadas não são ruins só porque parecem ser centralizadas.

Na verdade, como veremos muitas vezes ao longo deste livro, (logicamente, e até fisicamente) soluções centralizadas são frequentemente muito melhores do que equivalentes distribuídas pela simples razão de que há um único ponto de falha. Isso as torna muito mais fáceis de gerenciar, por exemplo, e certamente em comparação onde pode haver múltiplos pontos de falha. Além disso, esse único ponto de falha pode ser reforçado contra muitos tipos de falhas, bem como muitos tipos de ataques de segurança. Quando se trata de ser um gargalo de desempenho, também veremos que muitas coisas podem ser feitas para garantir que mesmo isso não possa ser mantido contra a centralização.

Nesse sentido, não esqueçamos que soluções centralizadas têm se mostrado extremamente escaláveis e robustas. Elas são chamadas de soluções baseadas em nuvem. Novamente, suas implementações podem fazer uso de soluções distribuídas muito sofisticadas, mas mesmo assim, veremos que mesmo essas soluções podem às vezes precisar depender de um pequeno conjunto de máquinas físicas, mesmo que seja apenas para garantir o desempenho.

1.1.2 Por que fazer a distinção é relevante

Por que fazemos essa distinção entre sistemas descentralizados e distribuídos? É importante perceber que soluções centralizadas são geralmente muito mais simples, e também mais simples ao longo de diferentes critérios. Descentralização, isto é, o ato de espalhar a implementação de um serviço em vários computadores porque acreditamos que é necessário, é uma decisão que precisa ser considerada cuidadosamente. De fato, soluções distribuídas e descentralizadas são inerentemente difíceis:

- Existem muitas dependências, muitas vezes inesperadas, que dificultam a compreensão do comportamento desses sistemas.
- Sistemas distribuídos e descentralizados sofrem quase continuamente com **falhas parciais**: algum processo ou recurso, em algum lugar de um dos computadores participantes, não está operando de acordo com as expectativas. Descobrir essa falha pode levar algum tempo, mas essas falhas são preferencialmente mascaradas (ou seja, passam despercebidas para usuários e aplicativos), incluindo a recuperação de falhas.

- Muito relacionado a falhas parciais é o fato de que em muitos sistemas de computadores em rede, nós participantes, processos, recursos e assim por diante vêm e vão. Isso torna esses sistemas altamente dinâmicos, exigindo por sua vez formas de gerenciamento e manutenção automatizados, aumentando por sua vez a complexidade.
- O fato de sistemas distribuídos e descentralizados serem conectados em rede, usados por muitos usuários e aplicativos e frequentemente cruzarem vários limites administrativos os torna particularmente vulneráveis a ataques de segurança. Portanto, entender esses sistemas e seu comportamento requer que entendamos como eles podem ser, e são protegidos. Infelizmente, entender a segurança não é tão fácil.

Nossa distinção é entre suíciência e necessidade de espalhar processos e recursos por vários computadores. Ao longo deste livro, assumimos o ponto de vista de que a descentralização nunca pode ser um objetivo em si mesma, e que ela deve se concentrar na suíciência para espalhar processos e recursos por computadores. Em princípio, quanto menos espalhar, melhor. No entanto, ao mesmo tempo, precisamos perceber que espalhar é às vezes realmente necessário, como ilustrado pelos exemplos de sistemas descentralizados. Deste ponto de suíciência, o livro é realmente sobre sistemas distribuídos e, quando apropriado, falaremos de sistemas descentralizados.

Na mesma linha, considerando que sistemas distribuídos e descentralizados são inerentemente complexos, é igualmente importante considerar soluções que sejam tão simples quanto possível. Portanto, dificilmente discutiremos otimizações para soluções, acreditando firmemente que o impacto de sua contribuição negativa para o aumento da complexidade supera a importância de sua contribuição positiva para um aumento de qualquer tipo de desempenho.

1.1.3 Estudando sistemas distribuídos

Considerando que sistemas distribuídos são inerentemente difíceis, é importante adotar uma abordagem sistemática para estudá-los. Uma das nossas principais preocupações é que há muitas dependências explícitas e implícitas em sistemas distribuídos. Por exemplo, não existe um módulo de comunicação separado ou um módulo de segurança separado. Nossa abordagem é dar uma olhada em sistemas distribuídos de um número limitado, mas de perspectivas diferentes. Cada perspectiva é considerada em um capítulo separado.

- Há muitas maneiras pelas quais os sistemas distribuídos são organizados. Começamos nossa discussão adotando a **perspectiva arquitetônica**: o que são organizações comuns, quais são estilos comuns? A perspectiva arquitetônica ajudará a obter uma primeira noção de como vários componentes de sistemas existentes interagem e dependem uns dos outros.

- Sistemas distribuídos são todos sobre processos. A **perspectiva de processo** é toda sobre entender as diferentes formas de processos que ocorrem em sistemas distribuídos, sejam eles threads, sua virtualização de processos de hardware, clientes, servidores e assim por diante. Os processos formam a espinha dorsal do software de sistemas distribuídos, e sua compreensão é essencial para entender sistemas distribuídos.
- Obviamente, com vários computadores em jogo, a comunicação entre processos é essencial. A **perspectiva da comunicação** diz respeito às facilidades que os sistemas distribuídos fornecem para trocar dados entre processos. Ela essencialmente envolve imitar chamadas de procedimentos em vários computadores, passagem de mensagens de alto nível com uma riqueza de opções semânticas e vários tipos de comunicação entre conjuntos de processos.
- Para que os sistemas distribuídos funcionem, o que acontece nos bastidores, sobre os quais os aplicativos são executados, é que os processos coordenam as coisas. Eles coordenam conjuntamente, por exemplo, para compensar a falta de relógio global, para realizar acesso exclusivo mútuo a recursos compartilhados, e assim por diante. A **perspectiva de coordenação** descreve uma série de tarefas fundamentais de coordenação que precisam ser realizadas como parte da maioria dos sistemas distribuídos.
- Para acessar processos e recursos, precisamos de nomenclatura. Em particular, precisamos de esquemas de nomenclatura que, quando usados, levarão ao processo, recursos ou qualquer outro tipo de entidade que está sendo nomeada. Por mais simples que isso possa parecer, a nomenclatura não só acaba sendo crucial em sistemas distribuídos, como também há muitas maneiras pelas quais a nomenclatura é suportada. A **perspectiva de nomenclatura** foca inteiramente em resolver um nome para o ponto de acesso da entidade nomeada.
- Um aspecto crítico dos sistemas distribuídos é que eles têm um bom desempenho em termos de eficiência e em termos de confiabilidade. O instrumento-chave para ambos os aspectos é a replicação de recursos. O único problema com a replicação é que atualizações podem acontecer, o que implica que todas as cópias de um recurso também precisam ser atualizadas. É aqui que manter a aparência de um sistema não distribuído se torna desafiador. A **perspectiva de consistência e replicação** concentra-se essencialmente nas compensações entre consistência, replicação e desempenho.
- Já mencionamos que sistemas distribuídos estão sujeitos a falhas parciais. A **perspectiva da tolerância a falhas** mergulha nos meios para mascarar falhas e sua recuperação. Ela provou ser uma das perspectivas mais difíceis para entender sistemas distribuídos, principalmente porque há muitas compensações a serem feitas, e também porque mascarar completamente falhas e sua recuperação é comprovadamente impossível.

- Como também mencionado, não existe um sistema distribuído não seguro. A **perspectiva de segurança** foca em como garantir acesso autorizado a recursos. Para esse fim, precisamos discutir a confiança em sistemas distribuídos, juntamente com a autenticação, ou seja, verificar uma identidade reivindicada.

A perspectiva de segurança vem por último, mas mais adiante neste capítulo discutiremos alguns instrumentos básicos necessários para entender o papel da segurança nas perspectivas anteriores.

1.2 Objetivos do projeto

Só porque é possível construir sistemas distribuídos não significa necessariamente que seja uma boa ideia. Nesta seção, discutimos quatro objetivos importantes que devem ser atingidos para fazer com que a construção de um sistema distribuído valha o esforço. Um sistema distribuído deve tornar os recursos facilmente acessíveis; deve ocultar o fato de que os recursos são distribuídos por uma rede; deve ser aberto; e deve ser escalável.

1.2.1 Compartilhamento de recursos

Um objetivo importante de um sistema distribuído é facilitar para usuários (e aplicativos) acessar e compartilhar recursos remotos. Recursos podem ser praticamente qualquer coisa, mas exemplos típicos incluem periféricos, instalações de armazenamento, dados, arquivos, serviços e redes, para citar apenas alguns. Há muitas razões para querer compartilhar recursos. Uma razão óbvia é a economia. Por exemplo, é mais barato ter uma única instalação de armazenamento confiável de ponta compartilhada do que ter que comprar e manter armazenamento para cada usuário separadamente.

Conectar usuários e recursos também facilita a colaboração e a troca de informações, como é ilustrado pelo sucesso da Internet com seus protocolos simples para troca de arquivos, e-mails, documentos, áudio e vídeo.

A conectividade da Internet permitiu que grupos de pessoas geograficamente dispersos trabalhassem juntos por meio de todos os tipos de **groupware**, ou seja, software para edição colaborativa, teleconferência e assim por diante, como é ilustrado por empresas multinacionais de desenvolvimento de software que terceirizaram grande parte de sua produção de código para a Ásia, mas também pela miríade de ferramentas de colaboração que se tornaram (mais facilmente) disponíveis devido à pandemia da COVID-19.

O compartilhamento de recursos em sistemas distribuídos também é ilustrado pelo sucesso de redes peer-to-peer de compartilhamento de arquivos como o **BitTorrent**. Esses sistemas distribuídos simplificam para os usuários o compartilhamento de arquivos pela Internet. Redes peer-to-peer são frequentemente associadas à distribuição de arquivos de mídia, como áudio e vídeo. Em outros casos, a tecnologia é usada para distribuir grandes quantidades de dados, como no caso de atualizações de software, serviços de backup e sincronização de dados em vários servidores.

A integração perfeita de instalações de compartilhamento de recursos em um ambiente de rede também é comum agora. Um grupo de usuários pode simplesmente colocar arquivos em um

pasta compartilhada especial que é mantida por um terceiro em algum lugar na Internet. Usando um software especial, a pasta compartilhada é quase indistinguível de outras pastas no computador de um usuário. Na verdade, esses serviços substituem o uso de um diretório compartilhado em um sistema de arquivos distribuído local, disponibilizando dados para usuários independentemente da organização à qual pertencem e independentemente de onde estejam. O serviço é oferecido para diferentes sistemas operacionais. Onde exatamente os dados são armazenados é completamente oculto do usuário final.

1.2.2 Transparência na distribuição

Um objetivo importante de um sistema distribuído é esconder o fato de que seus processos e recursos são fisicamente distribuídos por vários computadores, possivelmente separados por grandes distâncias. Em outras palavras, ele tenta tornar a distribuição de processos e recursos **transparente**, ou seja, invisível, para usuários finais e aplicativos. Como discutiremos mais extensivamente no Capítulo 2, a obtenção da transparência da distribuição é realizada por meio do que é conhecido como **middleware**, esboçado na Figura 1.2 (veja Gazis e Katsiri [2022] para uma primeira introdução).

Em essência, o que os aplicativos conseguem ver é a mesma interface em todos os lugares, enquanto por trás dessa interface, onde e como os processos e recursos estão e como eles são acessados é mantido transparente. Existem diferentes tipos de transparência, que discutiremos a seguir.

Tipos de transparência de distribuição

O conceito de transparência pode ser aplicado a vários aspectos de um sistema distribuído, dos quais os mais importantes estão listados na Figura 1.3. Usamos o termo objeto para significar um processo ou um recurso.

A **transparência de acesso** lida com a ocultação de diferenças na representação de dados e na maneira como os objetos podem ser acessados. Em um nível básico, queremos ocultar diferenças em arquiteturas de máquinas, mas o mais importante é que alcancemos

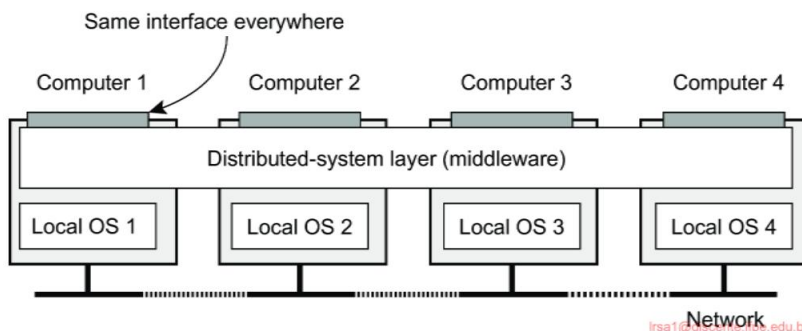


Figura 1.2: Realizando transparência de distribuição por meio de uma camada de middleware.

Descrição da transparência	
Acesso	Ocultar diferenças na representação de dados e como um objeto é acessado
Localização	Ocultar onde um objeto está localizado
Realocação	Ocultar que um objeto pode ser movido para outro local enquanto em uso
Migração	Ocultar que um objeto pode se mover para outro local
Replicação	Ocultar que um objeto é replicado
Concorrência	Ocultar que um objeto pode ser compartilhado por vários independentes Usuários
Falha	Ocultar a falha e recuperação de um objeto

Figura 1.3: Diferentes formas de transparência em um sistema distribuído (ver ISO [1995]). Um objeto pode ser um recurso ou um processo.

acordo sobre como os dados devem ser representados por diferentes máquinas e sistemas operacionais. Por exemplo, um sistema distribuído pode ter sistemas de computador que executam sistemas operacionais diferentes, cada um com suas próprias convenções de nomenclatura de arquivos. Diferenças nas convenções de nomenclatura, diferenças nas operações de arquivos ou diferenças em como a comunicação de baixo nível com outros processos deve ocorrer local, são exemplos de problemas de acesso que devem ser preferencialmente ocultados usuários e aplicativos.

Um grupo importante de tipos de transparência diz respeito à localização de um processo ou recurso. **A transparência de localização** refere-se ao fato de que os usuários não pode dizer onde um objeto está fisicamente localizado no sistema. Nomeação desempenha um papel importante na obtenção da transparência de localização. Em particular, a transparência de localização pode frequentemente ser alcançada atribuindo apenas nomes lógicos para recursos, ou seja, nomes em que a localização de um recurso não é secretamente codificado. Um exemplo de tal nome é o **localizador uniforme de recursos (URL)** <https://www.distributed-systems.net/>, que não dá nenhuma pista sobre a localização real do servidor Web onde este livro é oferecido. O URL também não dá nenhuma pista se os arquivos naquele site sempre estiveram em seu local atual ou foram recentemente movidos para lá. Por exemplo, todo o site pode ter sido movidos de um centro de dados para outro, mas os usuários não devem notar. O último é um exemplo de **transparência na realocação**, que está se tornando cada vez mais importante no contexto da **computação em nuvem**: o fenômeno pelo qual os serviços são fornecidos por enormes coleções de servidores remotos. Retornamos a computação em nuvem nos capítulos subsequentes e, em particular, no Capítulo 2.

Onde a transparência de realocação se refere a ser movido pelo distribuído sistema, **a transparência da migração** é oferecida por um sistema distribuído quando suporta a mobilidade de processos e recursos iniciados por usuários, sem afetar a comunicação e as operações em andamento. Um exemplo típico é a comunicação entre telemóveis: independentemente de duas pessoas

estão realmente se movendo, os celulares permitirão que eles continuem suas conversas. Outros exemplos que vêm à mente incluem rastreamento e rastreamento on-line de mercadorias enquanto elas estão sendo transportadas de um lugar para outro, e teleconferência (parcialmente) usando dispositivos equipados com Internet móvel.

Como veremos, a replicação desempenha um papel importante em sistemas distribuídos. Por exemplo, os recursos podem ser replicados para aumentar a disponibilidade ou melhorar o desempenho, colocando uma cópia perto do local onde são acessados.

A transparência de replicação lida com a ocultação do fato de que várias cópias de um recurso existem, ou que vários processos estão operando em alguma forma de modo lockstep para que um possa assumir quando outro falhar. Para ocultar a replicação dos usuários, é necessário que todas as réplicas tenham o mesmo nome. Consequentemente, um sistema que suporta transparência de replicação deve geralmente suportar transparência de localização também, porque de outra forma seria impossível se referir a réplicas em diferentes localizações.

Já mencionamos que um objetivo importante dos sistemas distribuídos é permitir o compartilhamento de recursos. Em muitos casos, o compartilhamento de recursos é feito cooperativamente, como no caso de canais de comunicação. No entanto, também há muitos exemplos de compartilhamento competitivo de recursos. Por exemplo, dois usuários independentes podem ter armazenado seus arquivos no mesmo servidor de arquivos ou podem estar acessando as mesmas tabelas em um banco de dados compartilhado. Em tais casos, é importante que cada usuário não perceba que o outro está fazendo uso do mesmo recurso. Esse fenômeno é chamado de **transparência de simultaneidade**.

Uma questão importante é que o acesso simultâneo a um recurso compartilhado deixa esse recurso em um estado consistente. A consistência pode ser alcançada por meio de mecanismos de bloqueio, pelos quais os usuários recebem, por sua vez, acesso exclusivo ao recurso desejado. Um mecanismo mais refinado é fazer uso de transações, mas estas podem ser difíceis de implementar em um sistema distribuído, notavelmente quando a escalabilidade é um problema.

Por último, mas certamente não menos importante, é importante que um sistema distribuído forneça **transparência de falhas**. Isso significa que um usuário ou aplicativo não percebe que alguma parte do sistema não funciona corretamente, e que o sistema subsequentemente (e automaticamente) se recupera dessa falha. Mascaram falhas é um dos problemas mais difíceis em sistemas distribuídos e é até mesmo impossível quando certas suposições aparentemente realistas são feitas, como discutiremos no Capítulo 8. A principal dificuldade em mascarar e se recuperar transparentemente de falhas está na incapacidade de distinguir entre um processo morto e um que responde dolorosamente lento. Por exemplo, ao contatar um servidor Web ocupado, um navegador eventualmente atingirá o tempo limite e informará que a página Web não está disponível. Nesse ponto, o usuário não consegue dizer se o servidor está realmente inativo ou se a rede está muito congestionada.

Grau de transparência da distribuição

Embora a transparência da distribuição seja geralmente considerada preferível para qualquer sistema distribuído, há situações em que tentar ocultar cegamente todos os aspectos da distribuição dos usuários não é uma boa ideia. Um exemplo simples é solicitar que seu jornal eletrônico apareça em sua caixa de correio antes das 7h, horário local, como de costume, enquanto você está atualmente do outro lado do mundo, vivendo em um fuso horário diferente. Seu jornal matutino não será o jornal matutino ao qual você está acostumado.

Da mesma forma, não se pode esperar que um sistema distribuído de área ampla que conecta um processo em São Francisco a um processo em Amsterdã esconda o fato de que a Mãe Natureza não permitirá que ele envie uma mensagem de um processo para o outro em menos de aproximadamente 35 milissegundos. A prática mostra que, na verdade, leva várias centenas de milissegundos usando uma rede de computadores. A transmissão de sinal não é limitada apenas pela velocidade da luz, mas também por capacidades de processamento limitadas e atrasos nos switches intermediários.

Há também um trade-off entre um alto grau de transparência e o desempenho de um sistema. Por exemplo, muitos aplicativos de Internet tentam repetidamente contatar um servidor antes de finalmente desistir. Consequentemente, tentar mascarar uma falha transitória de servidor antes de tentar outra pode tornar o sistema lento como um todo. Nesse caso, pode ter sido melhor desistir antes, ou pelo menos deixar o usuário cancelar as tentativas de fazer contato.

Outro exemplo é onde precisamos garantir que várias réplicas, localizadas em diferentes continentes, devam ser consistentes o tempo todo. Em outras palavras, se uma cópia for alterada, essa alteração deve ser propagada para todas as cópias antes de permitir qualquer outra operação. Uma única operação de atualização pode agora levar até segundos para ser concluída, algo que não pode ser escondido dos usuários.

Finalmente, há situações em que não é nada óbvio que esconder a distribuição seja uma boa ideia. Como os sistemas distribuídos estão se expandindo para dispositivos que as pessoas carregam por aí e onde a própria noção de localização e consciência de contexto está se tornando cada vez mais importante, pode ser melhor expor a distribuição em vez de tentar escondê-la. Um exemplo óbvio é fazer uso de serviços baseados em localização, que muitas vezes podem ser encontrados em telefones celulares, como encontrar uma loja mais próxima ou amigos próximos.

Existem outros argumentos contra a transparência da distribuição. Reconhecendo que a transparência total da distribuição é simplesmente impossível, devemos nos perguntar se é mesmo sensato fingir que podemos alcançá-la. Pode ser muito melhor tornar a distribuição explícita para que o usuário e o desenvolvedor do aplicativo nunca sejam enganados a acreditar que existe algo como transparência. O resultado será que os usuários entenderão muito melhor o comportamento (às vezes inesperado) de um sistema distribuído e, portanto, estarão muito mais bem preparados para lidar com esse comportamento.

A conclusão é que almejar a transparência da distribuição pode ser um bom objetivo ao projetar e implementar sistemas distribuídos, mas que

deve ser considerado junto com outras questões como desempenho e compreensibilidade. O preço para atingir transparência total pode ser surpreendentemente alto.

Nota 1.2 (Discussão: Contra a transparência da distribuição)

Vários pesquisadores argumentaram que ocultar a distribuição levará apenas a complicar ainda mais o desenvolvimento de sistemas distribuídos, exatamente pela razão de que a transparência total da distribuição nunca pode ser alcançada. Uma técnica popular para atingir a transparência de acesso é estender chamadas de procedimento para servidores remotos. No entanto, [Waldo et al. \[1997\]](#) já apontaram que tentar ocultar a distribuição por tais chamadas de procedimento remotas pode levar a uma semântica mal compreendida, pela simples razão de que uma chamada de procedimento muda quando executada por um link de comunicação defeituoso.

Como alternativa, vários pesquisadores e praticantes agora estão argumentando por menos transparência, por exemplo, usando mais explicitamente a comunicação no estilo de mensagem, ou postando mais explicitamente solicitações para, e obtendo resultados de máquinas remotas, como é feito na Web ao buscar páginas. Tais soluções serão discutidas em detalhes no próximo capítulo.

Um ponto de vista um tanto radical foi adotado por [Wams \[2012\]](#) ao afirmar que falhas parciais impedem a confiança na execução bem-sucedida de um serviço remoto. Se tal confiabilidade não puder ser garantida, é melhor sempre executar apenas execuções locais, levando ao princípio **de copiar antes de usar**. De acordo com esse princípio, os dados podem ser acessados somente após terem sido transferidos para a máquina do processo que deseja esses dados. Além disso, a modificação de um item de dados não deve ser feita. Em vez disso, ele só pode ser atualizado para uma nova versão. Não é difícil imaginar que muitos outros problemas surgirão. No entanto, [Wams](#) mostra que muitos aplicativos existentes podem ser retroajustados para essa abordagem alternativa sem sacrificar a funcionalidade.

1.2.3 Abertura

Outro objetivo importante dos sistemas distribuídos é a abertura. Um **sistema distribuído aberto** é essencialmente um sistema que oferece componentes que podem ser facilmente usados por, ou integrados a outros sistemas. Ao mesmo tempo, um sistema distribuído aberto em si frequentemente consistirá de componentes que se originam de outro lugar.

Interoperabilidade, componibilidade e extensibilidade

Ser aberto significa que os componentes devem aderir a regras padrão que descrevem a sintaxe e a semântica do que esses componentes têm a oferecer (ou seja, qual serviço eles fornecem). Uma abordagem geral é definir serviços por meio de **interfaces** usando uma **Interface Definition Language (IDL)**. As definições de interface escritas em uma IDL quase sempre capturam apenas a sintaxe dos serviços. Em outras palavras, elas especificam precisamente os nomes das funções que estão disponíveis.

juntamente com os tipos de parâmetros, valores de retorno, possíveis exceções que podem ser levantadas, e assim por diante. A parte difícil é especificar precisamente o que esses serviços fazem, ou seja, a semântica das interfaces. Na prática, tais especificações são dadas de forma informal pela linguagem natural.

Se for especificada corretamente, uma definição de interface permite que um processo arbitrário que precisa de uma certa interface, converse com outro processo que fornece essa interface. Ela também permite que duas partes independentes construam implementações inteiramente diferentes dessas interfaces, levando a dois componentes separados que operam exatamente da mesma maneira.

As especificações adequadas são completas e neutras. Completo significa que tudo o que é necessário para fazer uma implementação foi de fato especificado. No entanto, muitas definições de interface não são de todo completas, de modo que é necessário que um desenvolvedor adicione detalhes especificados de implementação.

Tão importante quanto isso é o fato de que as especificações não prescrevem como uma implementação deve ser; elas devem ser neutras.

Conforme apontado em Blair e Stefani [1998], completude e neutralidade são importantes para interoperabilidade e portabilidade. **Interoperabilidade** caracteriza a extensão pela qual duas implementações de sistemas ou componentes de diferentes fabricantes podem coexistir e trabalhar juntas meramente confiando nos serviços uma da outra conforme especificado por um padrão comum.

Portabilidade caracteriza até que ponto um aplicativo desenvolvido para um sistema distribuído A pode ser executado, sem modificação, em um sistema distribuído B diferente que implementa as mesmas interfaces que A.

Outro objetivo importante para um sistema distribuído aberto é que ele deve ser fácil de configurar o sistema a partir de diferentes componentes (possivelmente de diferentes desenvolvedores). Além disso, deve ser fácil adicionar novos componentes ou substituir os existentes sem afetar os componentes que permanecem no lugar.

Em outras palavras, um sistema distribuído aberto também deve ser **extensível**. Por exemplo, em um sistema extensível, deve ser relativamente fácil adicionar partes que rodem em um sistema operacional diferente, ou mesmo substituir um sistema de arquivos inteiro. Exemplos relativamente simples de extensibilidade são plug-ins para navegadores da Web, mas também aqueles para sites, como os usados no WordPress.

Nota 1.3 (Discussão: Sistemas abertos na prática)

Claro, o que acabamos de descrever é uma situação ideal. A prática mostra que muitos sistemas distribuídos não são tão abertos quanto gostaríamos, e que ainda é preciso muito esforço para juntar vários pedaços e partes para fazer um sistema distribuído. Uma maneira de sair da falta de abertura é simplesmente revelar todos os detalhes sangrentos de um componente e fornecer aos desenvolvedores o código-fonte real. Essa abordagem está se tornando cada vez mais popular, levando aos chamados projetos de código aberto, onde grandes grupos de pessoas contribuem para melhorar e depurar sistemas. É verdade que isso é o mais aberto que um sistema pode ser, mas se é a melhor maneira é questionável.

Separando a política do mecanismo

Para atingir a flexibilidade em sistemas distribuídos abertos, é crucial que o sistema seja organizado como uma coleção de componentes relativamente pequenos e facilmente substituíveis ou adaptáveis. Isso implica que devemos fornecer definições não apenas das interfaces de nível mais alto, ou seja, aquelas vistas por usuários e aplicativos, mas também definições para interfaces para partes internas do sistema e descrever como essas partes interagem. Essa abordagem é relativamente nova. Muitos sistemas mais antigos e até mesmo contemporâneos são construídos usando uma abordagem monolítica na qual os componentes são apenas separados logicamente, mas implementados como um programa enorme. Essa abordagem dificulta a substituição ou adaptação de um componente sem afetar todo o sistema. Os sistemas monolíticos, portanto, tendem a ser fechados em vez de abertos.

A necessidade de alterar um sistema distribuído geralmente é causada por um componente que não fornece a política ideal para um usuário ou aplicativo específico. Como exemplo, considere o cache em navegadores da Web. Há muitos parâmetros diferentes que precisam ser considerados:

Armazenamento: Onde os dados serão armazenados em cache? Normalmente, haverá um cache na memória ao lado do armazenamento em disco. No último caso, a posição exata no sistema de arquivos local precisa ser considerada.

Isenção: Quando o cache fica cheio, quais dados devem ser removidos para que páginas recém-obtidas podem ser armazenadas?

Compartilhamento: Cada navegador usa um cache privado ou o cache deve ser compartilhado entre navegadores de diferentes usuários?

Atualização: Quando um navegador verifica se os dados em cache ainda estão atualizados? Os caches são mais eficazes quando um navegador pode retornar páginas sem ter que contatar o site original. No entanto, isso traz o risco de retornar dados obsoletos. Observe também que as taxas de atualização são altamente dependentes de quais dados são realmente armazenados em cache: enquanto os horários dos trens dificilmente mudam, esse não é o caso das páginas da Web que mostram as condições atuais do tráfego rodoviário ou, pior ainda, os preços das ações.

O que precisamos é de uma separação entre **política e mecanismo**. No caso do cache da Web, por exemplo, um navegador deve idealmente fornecer facilidades para armazenar apenas documentos (ou seja, um mecanismo) e, ao mesmo tempo, permitir que os usuários decidam quais documentos são armazenados e por quanto tempo (ou seja, uma política). Na prática, isso pode ser implementado oferecendo um rico conjunto de parâmetros que o usuário pode definir (dinamicamente). Ao levar isso um passo adiante, um navegador pode até oferecer facilidades para conectar políticas que um usuário implementou como um componente separado.

Nota 1.4 (Discussão: Uma separação rigorosa é realmente o que precisamos?)

Em teoria, separar rigorosamente as políticas dos mecanismos parece ser o caminho a seguir. No entanto, há uma compensação importante a considerar: quanto mais rigorosa for a separação, mais precisamos garantir que oferecemos o conjunto apropriado de mecanismos. Na prática, isso significa que um rico conjunto de recursos é oferecido, o que, por sua vez, leva a muitos parâmetros de configuração. Como exemplo, o popular navegador Firefox vem com algumas centenas de parâmetros de configuração. Imagine como o espaço de configuração explode ao considerar grandes sistemas distribuídos que consistem em muitos componentes. Em outras palavras, a separação estrita de políticas e mecanismos pode levar a problemas de configuração altamente complexos.

Uma opção para aliviar esses problemas é fornecer padrões razoáveis, e é isso que geralmente acontece na prática. Uma abordagem alternativa é aquela em que o sistema observa seu próprio uso e altera dinamicamente as configurações de parâmetros. Isso leva ao que é conhecido como **sistemas autoconfiguráveis**. No entanto, o fato por si só de que muitos mecanismos precisam ser oferecidos para dar suporte a uma ampla gama de políticas frequentemente torna a codificação de sistemas distribuídos muito complicada. Codificar políticas em um sistema distribuído pode reduzir a complexidade consideravelmente, mas ao preço de menos flexibilidade.

1.2.4 Confiabilidade

Como o próprio nome sugere, **a confiabilidade** se refere ao grau em que se pode confiar que um sistema de computador opere conforme o esperado. Em contraste com sistemas de computador único, a confiabilidade em sistemas distribuídos pode ser bastante complexa devido a **falhas parciais**: em algum lugar há um componente falhando enquanto o sistema como um todo ainda parece estar correspondendo às expectativas (até um certo ponto ou momento). Embora sistemas de computador único também possam sofrer de falhas que não aparecem imediatamente, ter uma coleção potencialmente grande de sistemas de computador em rede complica as coisas consideravelmente. Na verdade, deve-se presumir que, a qualquer momento, sempre há falhas parciais ocorrendo. Um objetivo importante dos sistemas distribuídos é mascarar essas falhas, bem como mascarar a recuperação dessas falhas. Esse mascaramento é a essência de ser capaz de tolerar falhas, consequentemente chamado de **tolerância a falhas**.

Conceitos básicos

Confiabilidade é um termo que abrange vários requisitos úteis para sistemas distribuídos, incluindo os seguintes [Kopetz e Verissimo, 1993]:

- Disponibilidade
- Confiabilidade
-
- Segurança • Manutenibilidade

Disponibilidade é definida como a propriedade de que um sistema está pronto para ser usado imediatamente. Em geral, refere-se à probabilidade de que o sistema esteja operando corretamente em qualquer momento e esteja disponível para executar suas funções em nome de seus usuários. Em outras palavras, um sistema altamente disponível é aquele que provavelmente estará funcionando em um determinado instante no tempo.

Confiabilidade se refere à propriedade de que um sistema pode funcionar continuamente sem falhas. Em contraste com a disponibilidade, a confiabilidade é definida em termos de um intervalo de tempo em vez de um instante no tempo. Um sistema altamente confiável é aquele que provavelmente continuará a funcionar sem interrupção durante um período de tempo relativamente longo. Esta é uma diferença sutil, mas importante, quando comparada à disponibilidade. Se um sistema cair em média por um milissegundo aparentemente aleatório a cada hora, ele tem uma disponibilidade de mais de 99,9999 por cento, mas ainda não é confiável. Da mesma forma, um sistema que nunca falha, mas é desligado por duas semanas específicas todo mês de agosto, tem alta confiabilidade, mas apenas 96 por cento de disponibilidade. Os dois não são a mesma coisa.

Segurança se refere à situação em que, quando um sistema falha temporariamente em operar corretamente, nenhum evento catastrófico acontece. Por exemplo, muitos sistemas de controle de processo, como aqueles usados para controlar usinas nucleares ou enviar pessoas ao espaço, são necessários para fornecer um alto grau de segurança. Se tais sistemas de controle falharem temporariamente por apenas um breve momento, os efeitos podem ser desastrosos. Muitos exemplos do passado (e provavelmente muitos mais ainda por vir) mostram o quão difícil é construir sistemas seguros.

Por fim, a **manutenibilidade** se refere à facilidade com que um sistema com falha pode ser reparado. Um sistema altamente sustentável também pode mostrar um alto grau de disponibilidade, especialmente se as falhas puderem ser detectadas e reparadas automaticamente. No entanto, como veremos, a recuperação automática de falhas é mais fácil de dizer do que fazer.

Tradicionalmente, a tolerância a falhas tem sido relacionada às três métricas a seguir:

- **Tempo médio até a falha (MTTF):** O tempo médio até que um componente falha.
- **Tempo médio de reparo (MTTR):** O tempo médio necessário para reparar um componente.
- **Tempo médio entre falhas (MTBF):** simplesmente $MTTF + MTTR$.

Note que essas métricas só fazem sentido se tivermos uma noção precisa do que uma falha realmente é. Como veremos no Capítulo 8, identificar a ocorrência de uma falha pode não ser tão óbvio.

Falhas, erros, falhas

Diz-se que um sistema **falha** quando não consegue cumprir suas promessas. Em particular, se um sistema distribuído é projetado para fornecer a seus usuários vários serviços, o sistema falhou quando um ou mais desses serviços não podem ser (completamente) fornecidos. Um **erro** é uma parte do estado de um sistema que pode levar a uma falha. Por

por exemplo, ao transmitir pacotes por uma rede, é de se esperar que alguns pacotes tenham sido danificados quando chegam ao receptor. Danificado neste contexto significa que o receptor pode detectar incorretamente um valor de bit (por exemplo, lendo 1 em vez de 0), ou pode até mesmo ser incapaz de detectar que algo chegou.

A causa de um erro é chamada de **falha**. Claramente, descobrir o que causou um erro é importante. Por exemplo, um meio de transmissão errado ou ruim pode facilmente causar danos aos pacotes. Nesse caso, é relativamente fácil remover a falha. No entanto, erros de transmissão também podem ser causados por más condições climáticas, como em redes sem fio. Alterar o clima para reduzir ou evitar erros é um pouco mais complicado.

Como outro exemplo, um programa travado é claramente uma falha, que pode ter acontecido porque o programa entrou em um branch de código contendo um bug de programação (ou seja, um erro de programação). A causa desse bug é tipicamente um programador. Em outras palavras, o programador é a causa do erro (bug de programação), por sua vez levando a uma falha (um programa travado).

Construir sistemas confiáveis está intimamente relacionado ao controle de falhas. Conforme explicado por Avizienis et al. [2004], uma distinção pode ser feita entre prevenir, tolerar, remover e prever falhas. Para nossos propósitos, a questão mais importante é **a tolerância a falhas**, o que significa que um sistema pode fornecer seus serviços mesmo na presença de falhas. Por exemplo, aplicando códigos de correção de erros para transmissão de pacotes, é possível tolerar, até certo ponto, linhas de transmissão relativamente ruins e reduzir a probabilidade de que um erro (um pacote danificado) possa levar a uma falha.

As falhas são geralmente classificadas como transitórias, intermitentes ou permanentes. **Falhas transitórias** ocorrem uma vez e depois desaparecem. Se a operação for repetida, a falha desaparece. Um pássaro voando através do feixe de um transmissor de micro-ondas pode causar bits perdidos em alguma rede (sem mencionar um pássaro assado). Se a transmissão expirar e for tentada novamente, provavelmente funcionará na segunda vez.

Uma **falha intermitente** ocorre, depois desaparece por conta própria, depois reaparece, e assim por diante. Um contato frouxo em um conector frequentemente causará uma falha intermitente. Falhas intermitentes causam muito aborrecimento porque são difíceis de diagnosticar. Normalmente, quando o médico de falhas aparece, o sistema funciona bem.

Uma **falha permanente** é aquela que continua existindo até que o componente defeituoso seja substituído. Chips queimados, bugs de software e travamentos de cabeça de disco são exemplos de falhas permanentes.

Sistemas confiáveis também são necessários para fornecer segurança, especialmente em termos de confidencialidade e integridade. **Confidencialidade** é a propriedade de que as informações são divulgadas apenas para partes autorizadas, enquanto **a integridade** se relaciona a garantir que alterações em vários ativos possam ser feitas apenas de forma autorizada. De fato, podemos falar de um sistema confiável quando confidencialidade e integridade não estão em vigor? Retornamos à segurança em seguida.

1.2.5 Segurança

Um sistema distribuído que não é seguro não é confiável. Conforme mencionado, atenção especial é necessária para garantir a confidencialidade e a integridade, ambas diretamente acopladas à divulgação autorizada e ao acesso de informações e recursos. Em qualquer sistema de computador, **a autorização** é feita verificando se uma entidade identificada tem direitos de acesso adequados. Por sua vez, isso significa que o sistema deve saber que está realmente lidando com a entidade adequada. Por esse motivo, **a autenticação** é essencial: verificar a existência de uma identidade reivindicada. Igualmente importante é a noção de **confiança**. Se um sistema pode autenticar positivamente uma pessoa, quanto vale essa autenticação se a pessoa não é confiável? Somente por esse motivo, a autorização adequada é importante, pois pode ser usada para limitar qualquer dano que uma pessoa, que em retrospecto não poderia ser confiável, possa causar. Por exemplo, em sistemas financeiros, a autorização pode limitar a quantia de dinheiro que uma pessoa tem permissão para transferir entre várias contas. Discutiremos confiança, autenticação e autorização detalhadamente no Capítulo 9.

Elementos-chave necessários para entender a segurança

Uma técnica essencial para tornar os sistemas distribuídos seguros é a criptografia. Este não é o lugar neste livro para discutir extensivamente a criptografia (que também adiamos para o Capítulo 9), mas para entender como a segurança se encaixa em várias perspectivas nos capítulos seguintes, apresentamos informalmente alguns de seus elementos básicos.

Mantendo as coisas simples, a segurança em sistemas distribuídos é toda sobre **criptografar e descriptografar** dados usando **chaves de segurança**. A maneira mais fácil de considerar **uma** chave de segurança K é vê-la como uma função operando em alguns dados. Usamos a notação $K(\text{dados})$ para expressar o fato de que a chave K opera em dados.

Existem duas maneiras de criptografar e descriptografar dados. Em um criptosistema **simétrico**, a criptografia e a descriptografia ocorrem com uma única chave. Denotando por $EK(\text{data})$ a criptografia de dados usando a chave EK , e da mesma forma $DK(\text{data})$ para descriptografia com a chave DK , então em um criptosistema simétrico, a mesma chave é usada para criptografia e descriptografia, ou seja,

$$\text{se dados} = DK(EK(\text{dados})) \text{ então } DK = EK.$$

Note que em um criptosistema simétrico, a chave precisará ser mantida em segredo por todas as partes autorizadas a criptografar ou descriptografar dados. Em um criptosistema **assimétrico**, as chaves usadas para criptografia e descriptografia são diferentes. Em particular, há uma **chave pública** PK que pode ser usada por qualquer um, e uma **chave secreta** SK que, como o nome sugere, deve ser mantida em segredo. Criptosistemas assimétricos também são chamados de **sistemas de chave pública**.

Criptografia e descryptografia em sistemas de chave pública podem ser usadas de duas maneiras fundamentalmente diferentes. Primeiro, se Alice quiser criptografar dados que podem ser descryptografados somente por Bob, ela deve usar a chave pública de Bob, PK_B , levando aos dados criptografados $PK_B(data)$. Somente o detentor da chave secreta associada pode descryptografar essas informações, ou seja, Bob, que aplicará a operação $SK_B(PK_B(data))$, que retorna dados.

Um segundo caso de uso amplamente aplicado é o de realizar **assinaturas digitais**. Suponha que Alice disponibilize alguns dados para os quais é importante que qualquer parte, mas vamos supor que seja Bob, precise saber com certeza que eles vêm de Alice. Nesse caso, Alice pode criptografar os dados com sua chave secreta SK_A , levando a $SK_A(data)$. Se for possível garantir que a chave pública associada PK_A realmente pertence a Alice, então descryptografar com sucesso $SK_A(data)$ para data é prova de que Alice sabe sobre os dados: ela é a única que detém a chave secreta SK_A . Claro, precisamos fazer a suposição de que Alice é de fato a única que detém SK_A . Retornaremos a algumas dessas suposições no Capítulo 9.

Como se vê, provar que uma entidade viu ou sabe sobre alguns dados retorna frequentemente em sistemas distribuídos seguros. A colocação prática de assinaturas digitais é geralmente mais eficiente por uma **função hash**. Uma função hash H tem a propriedade de que, ao operar em alguns dados, ou seja, $H(dados)$, ela retorna uma string de comprimento fixo, independentemente do comprimento dos dados. Qualquer alteração de dados para $dados'$ levará a um valor hash diferente $H(dados')$. Além disso, dado um valor hash h , é computacionalmente impossível na prática descobrir os dados originais. O que tudo isso significa é que, para colocar uma assinatura digital, Alice calcula $sig = SK_A(H(dados))$ como sua assinatura e informa Bob sobre dados, H e sig . Bob, por sua vez, pode então verificar essa assinatura calculando $PK_A(sig)$ e verificando se ela corresponde ao valor $H(dados)$.

Usando criptografia em sistemas distribuídos

A aplicação da criptografia em sistemas distribuídos ocorre de muitas formas. Além de seu uso geral para criptografia e assinaturas digitais, a criptografia forma a base para realizar um **canal seguro** entre duas partes comunicantes. Tais canais basicamente permitem que duas partes saibam com certeza que estão de fato se comunicando com as entidades com as quais esperavam se comunicar. Em outras palavras, um canal de comunicação que suporta autenticação mútua. Um exemplo prático de canal seguro é usar https ao acessar sites. Agora, muitos navegadores exigem que os sites suportem esse protocolo e, no mínimo, avisarão o usuário quando esse não for o caso. Em geral, usar criptografia é necessário para realizar autenticação (e autorização) em sistemas distribuídos.

A criptografia também é usada para criar estruturas de dados distribuídas seguras. Um exemplo bem conhecido é o de um **blockchain**, que é, literalmente, uma cadeia de blocos. A ideia básica é simples: fazer hash dos dados em um bloco B_i , e colocar esse valor de hash como parte dos dados em seu bloco subsequente B_{i+1} . Qualquer alteração em

Bi (por exemplo, como resultado de um ataque), exigirá que o invasor também altere o valor de hash armazenado em Bi+1. No entanto, como o sucessor de Bi+1 contém o hash computado sobre os dados em Bi+1 e, portanto, incluindo o valor de hash original de Bi, o invasor também terá que alterar o novo valor de hash de Bi conforme armazenado em Bi+1. No entanto, alterar esse valor também significa alterar o valor de hash de Bi+1 e, portanto, o valor armazenado em Bi+2, por sua vez, exigindo que um novo valor de hash seja computado para Bi+2 e assim por diante. Em outras palavras, ao vincular blocos com segurança em uma cadeia, qualquer alteração bem-sucedida em um bloco exige que todos os blocos sucessivos também sejam modificados. Essas modificações devem passar despercebidas, o que é virtualmente impossível.

A criptografia também é usada para outro mecanismo importante em sistemas distribuídos: delegar direitos de acesso. A ideia básica é que Alice pode querer delegar alguns direitos a Bob, que, por sua vez, pode querer passar alguns desses direitos para Chuck. Usando meios apropriados (que discutimos no Capítulo 9, um serviço pode verificar com segurança se Chuck foi realmente autorizado a executar certas operações, sem a necessidade de que o serviço verifique com Alice se a delegação está em vigor. Observe que a delegação é algo a que estamos acostumados: muitos de nós delegam direitos de acesso que temos como usuário para aplicativos específicos, como um cliente de e-mail.

Uma aplicação distribuída futura da criptografia é a chamada computação **multipartidária**: os meios para duas ou três partes calcularem um valor para o qual os dados dessas partes são necessários, mas sem ter que realmente compartilhar esses dados. Um exemplo frequentemente usado é calcular o número de votos sem ter que saber quem votou em quem.

Veremos muitos outros exemplos de segurança em sistemas distribuídos nos capítulos seguintes. Com as breves explicações da base criptográfica, deve ser suficiente ver como a segurança é aplicada. Usaremos consistentemente as notações conforme mostrado na Figura 1.4. Alternativamente, os exemplos de segurança podem ser pulados até que tenhamos estudado o Capítulo 9.

Descrição da	notação
KA,B	Chave secreta compartilhada por A e B
PKA	Chave pública de A
SKA	Chave privada (secreta) de A
EK(dados)	Criptografia de dados usando a chave EK (ou chave K)
DK(dados)	Descritografia de dados (criptografados) usando a chave DK (ou chave K)
H(dados)	O hash de dados calculado usando a função H

Figura 1.4: Notações para criptosistemas usados neste livro.

1.2.6 Escalabilidade

Para muitos de nós, a conectividade mundial pela Internet é tão comum quanto poder enviar um pacote para qualquer pessoa em qualquer lugar do mundo. Além disso, onde até recentemente estávamos acostumados a ter computadores de mesa relativamente poderosos para aplicativos de escritório e armazenamento, agora estamos testemunhando que tais aplicativos e serviços estão sendo colocados no que foi cunhado como "a nuvem", o que, por sua vez, leva a um aumento de dispositivos em rede muito menores, como tablets ou até mesmo laptops somente em nuvem, como o Chromebook do Google. Com isso em mente, a escalabilidade se tornou um dos objetivos de design mais importantes para desenvolvedores de sistemas distribuídos.

Dimensões de escalabilidade

A escalabilidade de um sistema pode ser medida em pelo menos três dimensões diferentes (ver [Neuman, 1994]):

Escalabilidade de tamanho: Um sistema pode ser escalável em relação ao seu tamanho, o que significa que podemos facilmente adicionar mais usuários e recursos ao sistema sem nenhuma perda perceptível de desempenho.

Escalabilidade geográfica: Um sistema geograficamente escalável é aquele em que os usuários e recursos podem estar distantes, mas o fato de que atrasos na comunicação podem ser significativos dificilmente é notado.

Escalabilidade administrativa: Um sistema administrativamente escalável é aquele que ainda pode ser facilmente gerenciado, mesmo que abranja muitas organizações administrativas independentes.

Vamos analisar mais de perto cada uma dessas três dimensões de escalabilidade.

Escalabilidade de tamanho Quando um sistema precisa ser escalonado, tipos muito diferentes de problemas precisam ser resolvidos. Vamos primeiro considerar o escalonamento em relação ao tamanho. Se mais usuários ou recursos precisam ser suportados, frequentemente somos confrontados com as limitações de serviços centralizados, embora frequentemente por razões muito diferentes. Por exemplo, muitos serviços são centralizados no sentido de que são implementados por um único **servidor** em execução em uma máquina específica no sistema distribuído. Em um cenário mais moderno, podemos ter um grupo de servidores colaborativos co-localizados em um cluster de máquinas fortemente acopladas fisicamente colocadas no mesmo local. O problema com esse esquema é óbvio: o servidor, ou grupo de servidores, pode simplesmente se tornar um gargalo quando precisa processar um número crescente de solicitações. Para ilustrar como isso pode acontecer, vamos supor que um serviço seja implementado em uma única máquina. Nesse caso, existem essencialmente três causas raiz para se tornar um gargalo:

- A capacidade computacional, limitada pelas CPUs
- A capacidade de armazenamento, incluindo a taxa de transferência de E/S

- A rede entre o usuário e o serviço centralizado

Vamos primeiro considerar a capacidade computacional. Imagine um serviço para calcular rotas ótimas levando em conta informações de tráfego em tempo real. Não é difícil imaginar que este pode ser principalmente um serviço limitado à computação, exigindo vários (dezenas de) segundos para concluir uma solicitação. Se houver apenas uma única máquina disponível, então até mesmo um sistema moderno de ponta acabará tendo problemas se o número de solicitações aumentar além de um certo ponto.

Da mesma forma, mas por razões diferentes, encontraremos problemas ao ter um serviço que é principalmente limitado por E/S. Um exemplo típico é um mecanismo de busca centralizado mal projetado. O problema com consultas de busca baseadas em conteúdo é que essencialmente precisamos corresponder uma consulta a um conjunto de dados inteiro. Mesmo com técnicas avançadas de indexação, ainda podemos enfrentar o problema de ter que processar uma enorme quantidade de dados excedendo a capacidade da memória principal da máquina que executa o serviço. Como consequência, grande parte do tempo de processamento será determinado pelos acessos relativamente lentos ao disco e pela transferência de dados entre o disco e a memória principal. Simplesmente adicionar mais discos ou discos de velocidade mais alta não será uma solução sustentável, pois o número de solicitações continua a aumentar.

Por fim, a rede entre o usuário e o serviço também pode ser a causa da baixa escalabilidade. Imagine um serviço de vídeo sob demanda que precisa transmitir vídeo de alta qualidade para vários usuários. Um fluxo de vídeo pode facilmente exigir uma largura de banda de 8 a 10 Mbps, o que significa que se um serviço configurar conexões ponto a ponto com seus clientes, ele pode em breve atingir os limites da capacidade de rede de suas próprias linhas de transmissão de saída.

Existem várias soluções para escalabilidade de tamanho de ataque, que discutiremos abaixo, após analisar a escalabilidade geográfica e administrativa.

Nota 1.5 (Avançado: Analisando a escalabilidade do tamanho)

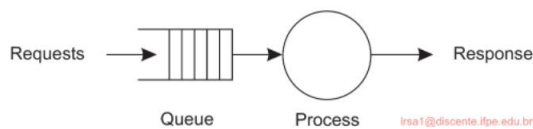


Figura 1.5: Um modelo simples de um serviço como um sistema de filas.

Problemas de escalabilidade de tamanho para serviços centralizados podem ser formalmente analisados usando a teoria de filas e fazendo algumas suposições simplificadoras. Em um nível conceitual, um serviço centralizado pode ser modelado como o sistema de filas simples mostrado na Figura 1.5: as solicitações são enviadas ao serviço, onde são enfileiradas até novo aviso. Assim que o processo pode lidar com uma próxima solicitação, ele a busca na fila, faz seu trabalho e produz uma resposta. Seguimos amplamente Menasce e Almeida [2002] ao explicar o desempenho de um serviço centralizado.

Frequentemente, podemos assumir que a fila tem uma capacidade infinita, o que significa que não há restrição no número de solicitações que podem ser aceitas para processamento posterior. Estritamente falando, isso significa que a taxa de chegada de solicitações não é influenciada pelo que está atualmente na fila ou sendo processado. Assumindo que a taxa de chegada de solicitações é γ solicitações por segundo, e que a capacidade de processamento do serviço é μ solicitações por segundo, pode-se calcular que a fração de tempo p_k em que há k solicitações no sistema é igual a:

$$p_k = \gamma \cdot \frac{1}{\mu} \cdot \left(\frac{\gamma}{\mu} \right)^{k-1} \cdot p_0$$

Se definirmos a **utilização** U de um serviço como a fração de tempo em que ele está ocupado, então claramente,

$$U = \sum_{k=0}^{\infty} k \cdot p_k = \frac{\gamma}{\mu} \cdot \sum_{k=0}^{\infty} k \cdot p_k = (1 - U) \cdot \sum_{k=0}^{\infty} k \cdot p_k = (1 - U) \cdot U$$

Podemos então calcular o número médio N de solicitações no sistema como

$$N = \sum_{k=0}^{\infty} k \cdot p_k = \frac{\gamma}{\mu} \cdot \sum_{k=0}^{\infty} k \cdot p_k = (1 - U) \cdot \sum_{k=0}^{\infty} k \cdot p_k = \frac{(1 - U) \cdot U}{(1 - U)^2} = \frac{U}{1 - U}$$

O que realmente nos interessa é o tempo de resposta R : quanto tempo leva para o serviço processar uma solicitação, incluindo o tempo gasto na fila.

Para isso, precisamos da vazão média X . Considerando que o serviço está "ocupado" quando pelo menos uma requisição está sendo processada, e que isso acontece então com uma vazão de μ requisições por segundo, e durante uma fração U do tempo total, temos:

$$X = U \cdot \mu + (1 - U) \cdot 0 = \mu \cdot U$$

servidor no trabalho servidor ocioso

Usando a fórmula de Little [Trivedi, 2002], podemos então derivar o tempo de resposta como

$$R = \frac{N}{X} = \frac{S}{1 - U} \quad R = \frac{1}{\mu(1 - U)}$$

onde $S = \frac{1}{\mu}$, o tempo de serviço real. Observe que se U for pequeno, a taxa de tempo de resposta para serviço é próxima de 1, o que significa que uma solicitação é processada virtualmente instantaneamente e na velocidade máxima possível. No entanto, assim que a utilização se aproxima de 1, vemos que a taxa de tempo de resposta para servidor aumenta rapidamente para valores muito altos, o que significa efetivamente que o sistema está chegando perto de uma parada brusca. É aqui que vemos problemas de escalabilidade surgirem. A partir deste modelo simples, podemos ver que a única solução é reduzir o tempo de serviço S . Deixamos como um exercício para o leitor explorar como S pode ser diminuído.

Escalabilidade geográfica A escalabilidade geográfica tem seus próprios problemas.

Uma das principais razões pelas quais ainda é difícil escalar os sistemas distribuídos existentes

que foram projetados para redes locais é que muitos deles são baseados em **comunicação síncrona**. Nessa forma de comunicação, uma parte solicitando um serviço, geralmente chamada de **cliente**, bloqueia até que uma resposta seja enviada de volta do **servidor** que implementa o serviço. Mais especificamente, frequentemente vemos um padrão de comunicação consistindo em muitas interações cliente-servidor, como pode ser o caso com transações de banco de dados. Essa abordagem geralmente funciona bem em LANs, onde a comunicação entre duas máquinas é frequentemente, na pior das hipóteses, algumas centenas de microssegundos. No entanto, em um sistema de área ampla, precisamos considerar que a comunicação entre processos pode ser de centenas de milissegundos, três ordens de magnitude mais lenta. Construir aplicativos usando comunicação síncrona em sistemas de área ampla requer muito cuidado (e não apenas um pouco de paciência), notavelmente com um rico padrão de interação entre cliente e servidor.

Outro problema que dificulta a escalabilidade geográfica é que a comunicação em redes de longa distância é inerentemente muito menos confiável do que em redes de área local. Além disso, geralmente também precisamos lidar com largura de banda limitada. O efeito é que soluções desenvolvidas para redes locais nem sempre podem ser facilmente portadas para um sistema de área ampla. Um exemplo típico é o streaming de vídeo.

Em uma rede doméstica, mesmo tendo apenas links sem fio, garantir um fluxo estável e rápido de quadros de vídeo de alta qualidade de um servidor de mídia para um monitor é bem simples. Simplesmente colocar o mesmo servidor bem longe e usar uma conexão TCP padrão para o monitor certamente falhará: limitações de largura de banda surgirão instantaneamente, mas também manter o mesmo nível de confiabilidade pode facilmente causar dores de cabeça.

Outro problema que surge quando os componentes ficam muito distantes é o fato de que os sistemas de área ampla geralmente têm apenas recursos muito limitados para comunicação multiponto. Em contraste, as redes de área local geralmente suportam mecanismos de transmissão eficientes. Esses mecanismos provaram ser extremamente úteis para descobrir componentes e serviços, o que é essencial de uma perspectiva de gerenciamento. Em sistemas de área ampla, precisamos desenvolver serviços separados, como serviços de nomenclatura e diretório, para os quais as consultas podem ser enviadas. Esses serviços de suporte, por sua vez, precisam ser escaláveis também e, muitas vezes, não existem soluções óbvias, como encontraremos em capítulos posteriores.

Escalabilidade administrativa Finalmente, uma questão difícil, e frequentemente aberta, é como escalar um sistema distribuído em vários domínios administrativos independentes. Um grande problema que precisa ser resolvido é o de políticas conflitantes sobre uso de recursos (e pagamento), gerenciamento e segurança. Para ilustrar, há muitos anos os cientistas buscam soluções para compartilhar seus equipamentos (geralmente caros) no que é conhecido como **grade computacional**.

Nessas grades, um sistema global descentralizado é construído como uma federação de sistemas distribuídos locais, permitindo que um programa executado em um computador em uma organização A acesse diretamente recursos na organização B.

Muitos componentes de um sistema distribuído que residem em um único domínio podem frequentemente ser confiáveis por usuários que operam dentro desse mesmo domínio. Em tais casos, a administração do sistema pode ter testado e certificado aplicativos, e pode ter tomado medidas especiais para garantir que tais componentes não possam ser adulterados. Em essência, os usuários confiam em seus administradores de sistema. No entanto, essa confiança não se expande naturalmente além dos limites do domínio.

Se um sistema distribuído se expande para outro domínio, dois tipos de medidas de segurança precisam ser tomadas. Primeiro, o sistema distribuído tem que se proteger contra ataques maliciosos do novo domínio. Por exemplo, usuários do novo domínio podem ter apenas acesso de leitura ao sistema de arquivos em seu domínio original. Da mesma forma, recursos como setters de imagem caros ou computadores de alto desempenho podem não ser disponibilizados para usuários não autorizados. Segundo, o novo domínio tem que se proteger contra ataques maliciosos do sistema distribuído. Um exemplo típico é o de baixar programas, como no caso do aprendizado federado. Basicamente, o novo domínio não sabe o que esperar desse código estrangeiro. O problema, como veremos no Capítulo 9, é como impor essas limitações.

Como um contraexemplo de sistemas distribuídos abrangendo múltiplos domínios administrativos que aparentemente não sofrem de problemas de escalabilidade administrativa, considere as modernas redes peer-to-peer de compartilhamento de arquivos. Nesses casos, os usuários finais simplesmente instalam um programa que implementa funções de busca e download distribuídas e em minutos podem começar a baixar arquivos. Outros exemplos incluem aplicativos peer-to-peer para telefonia pela Internet, como os sistemas Skype mais antigos [Baset e Schulzrinne, 2006], e (novamente mais antigos) aplicativos de streaming de áudio assistidos por pares, como o Spotify [Kreitz e Niemelä, 2010]. Um aplicativo mais moderno (que ainda precisa se provar em termos de escalabilidade) são os blockchains. O que esses sistemas descentralizados têm em comum é que os usuários finais, e não as entidades administrativas, colaboram para manter o sistema funcionando. Na melhor das hipóteses, organizações administrativas subjacentes, como **os provedores de serviços de Internet (ISPs)**, podem policiar o tráfego de rede causado por esses sistemas ponto a ponto.

Técnicas de dimensionamento

Tendo discutido alguns problemas de escalabilidade, chegamos à questão de como esses problemas podem ser resolvidos em geral. Na maioria dos casos, problemas de escalabilidade em sistemas distribuídos aparecem como problemas de desempenho causados pela capacidade limitada de servidores e rede. Simplesmente melhorar sua capacidade (por exemplo, aumentando a memória, atualizando CPUs ou substituindo módulos de rede) é frequentemente uma solução, chamada de **escalonamento**. Quando se trata de **escalonamento horizontal**, ou seja, expandir o sistema distribuído essencialmente implantando mais máquinas, existem basicamente apenas três técnicas que podemos aplicar: ocultar latências de comunicação, distribuição de trabalho e replicação (veja também Neuman [1994]).

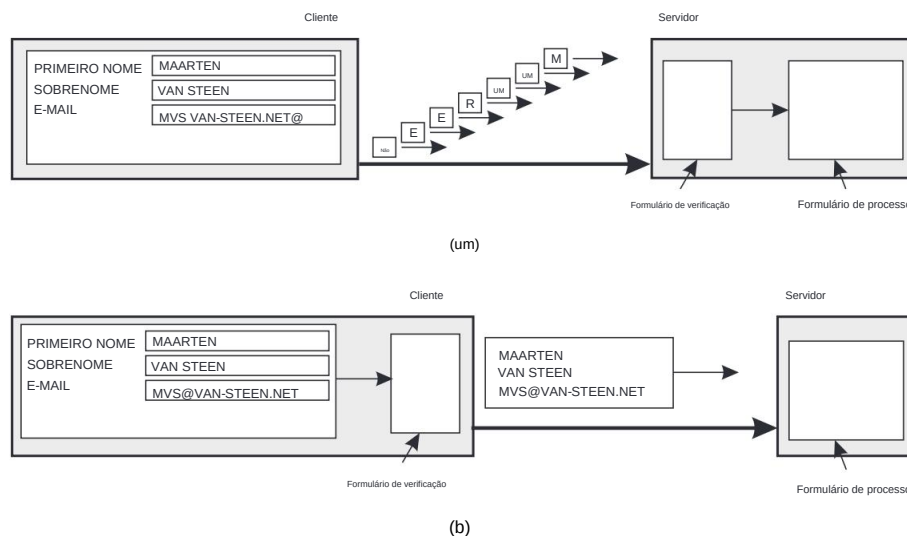


Figura 1.6: A diferença entre deixar (a) um servidor ou (b) um cliente verificar os formulários enquanto eles estão sendo preenchidos.

Ocultando latências de comunicação Ocultar latências de comunicação é aplicável no caso de escalabilidade geográfica. A ideia básica é simples: tente evitar esperar por respostas a solicitações de serviço remoto o máximo possível. Por exemplo, quando um serviço é solicitado em uma máquina remota, uma alternativa para esperar por uma resposta do servidor é fazer outro trabalho útil do lado do solicitante. Essencialmente, isso significa construir o aplicativo solicitante de tal forma que ele use apenas **comunicação assíncrona**. Quando uma resposta chega, o aplicativo é interrompido e um manipulador especial é chamado para concluir a solicitação emitida anteriormente. A comunicação assíncrona pode frequentemente ser usada em sistemas de processamento em lote e aplicativos paralelos, nos quais tarefas independentes podem ser agendadas para execução enquanto outra tarefa aguarda a conclusão da comunicação. Alternativamente, um novo thread de controle pode ser iniciado para executar a solicitação. Embora ele bloqueie a espera pela resposta, outros threads no processo podem continuar.

No entanto, há muitos aplicativos que não podem fazer uso efetivo da comunicação assíncrona. Por exemplo, em aplicativos interativos, quando um usuário envia uma solicitação, ele geralmente não terá nada melhor a fazer do que esperar pela resposta. Em tais casos, uma solução muito melhor é reduzir a comunicação geral, por exemplo, movendo parte da computação que normalmente é feita no servidor para o processo cliente que solicita o serviço. Um caso típico em que essa abordagem funciona é acessar bancos de dados usando formulários.

O preenchimento de formulários pode ser feito enviando uma mensagem separada para cada campo e aguardando uma confirmação do servidor, conforme mostrado na Figura 1.6(a). Por exemplo, o servidor pode verificar se há erros sintáticos antes de aceitar uma entrada.

Uma solução muito melhor é enviar o código para preenchimento no formulário e, possivelmente, verificando as entradas, para o cliente, e fazer com que o cliente devolva um formulário preenchido formulário, conforme mostrado na Figura 1.6(b). Esta abordagem de código de envio é amplamente suportado pela Web através de JavaScript.

Particionamento e distribuição Outra técnica de dimensionamento importante é o particionamento e a distribuição, que envolve pegar um componente ou outro recurso, dividindo-o em partes menores e, posteriormente, espalhando essas partes o sistema. Um bom exemplo de particionamento e distribuição é a Internet Sistema de Nomes de Domínio (DNS). O espaço de nomes DNS é hierarquicamente organizado em uma árvore de **domínios**, que são divididos em **zonas não sobrepostas**, conforme mostrado para o DNS original na Figura 1.7. Os nomes em cada zona são manipulado por um único servidor de nomes. Sem entrar em muitos detalhes agora (retornaremos ao DNS extensivamente no Capítulo 6), pode-se pensar em cada nome de caminho sendo o nome de um host na Internet e, portanto, está associado a uma rede endereço desse host. Basicamente, resolver um nome significa retornar a rede endereço do host associado. Considere, por exemplo, o nome `yits.cs.vu.nl`. Para resolver esse nome, ele é primeiro passado para o servidor da zona Z1 (veja Figura 1.7) que retorna o endereço do servidor para a zona Z2, para o qual o restante do nome, `yits.cs.vu`, pode ser entregue. O servidor para Z2 retornará o endereço de o servidor para a zona Z3, que é capaz de manipular a última parte do nome, e retornará o endereço do host associado.

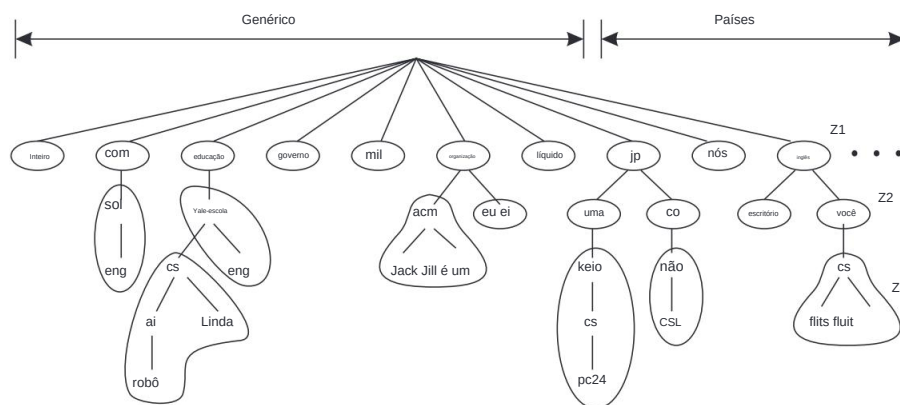


Figura 1.7: Um exemplo de divisão do espaço de nomes DNS (original) em zonas.

Esses exemplos ilustram como o **serviço de nomenclatura** fornecido pelo DNS é distribuídos em várias máquinas, evitando assim que um único servidor tenha que lidar com todas as solicitações de resolução de nomes.

Como outro exemplo, considere a World Wide Web. Para a maioria dos usuários, a Web parece ser um enorme sistema de informação baseado em documentos, no qual cada documento tem seu próprio nome exclusivo na forma de uma URL. Conceitualmente,

pode até parecer que há apenas um único servidor. No entanto, a Web é fisicamente particionada e distribuída por algumas centenas de milhões de servidores, cada um manipulando frequentemente um número de sites, ou partes de sites. O nome do servidor que manipula um documento é codificado na URL desse documento. É somente por causa dessa distribuição de documentos que a Web foi capaz de escalar para seu tamanho atual. No entanto, observe que descobrir quantos servidores fornecem serviços baseados na Web é virtualmente impossível: um site hoje é muito mais do que alguns documentos estáticos da Web.

Replicação Considerando que problemas de escalabilidade frequentemente aparecem na forma de degradação de desempenho, geralmente é uma boa ideia realmente **replicar** componentes ou recursos, etc., em um sistema distribuído. A replicação não apenas aumenta a disponibilidade, mas também ajuda a equilibrar a carga entre os componentes, levando a um melhor desempenho. Além disso, em sistemas geograficamente amplamente dispersos, ter uma cópia por perto pode esconder muitos dos problemas de latência de comunicação mencionados antes.

O **cache** é uma forma especial de replicação, embora a distinção entre os dois seja frequentemente difícil de fazer ou mesmo artificial. Como no caso da replicação, o cache resulta em fazer uma cópia de um recurso, geralmente na proximidade do cliente que acessa esse recurso. No entanto, em contraste com a replicação, o cache é uma decisão tomada pelo cliente de um recurso e não pelo proprietário de um recurso.

Há uma séria desvantagem no cache e na replicação que pode afetar adversamente a escalabilidade. Como agora temos várias cópias de um recurso, modificar uma cópia torna essa cópia diferente das outras. Consequentemente, o cache e a replicação levam a problemas **de consistência**.

Até que ponto as inconsistências podem ser toleradas depende do uso de um recurso. Por exemplo, muitos usuários da Web acham aceitável que seu navegador retorne um documento em cache cuja validade não foi verificada nos últimos minutos. No entanto, também há muitos casos em que garantias de consistência fortes precisam ser atendidas, como no caso de bolsas de valores eletrônicas e leilões. O problema com consistência forte é que uma atualização deve ser imediatamente propagada para todas as outras cópias. Além disso, se duas atualizações acontecem simultaneamente, geralmente também é necessário que as atualizações sejam processadas na mesma ordem em todos os lugares, introduzindo um problema de ordenação global. Para piorar a situação, combinar consistência com propriedades desejáveis, como disponibilidade, pode ser simplesmente impossível, como discutimos no Capítulo 8.

A replicação, portanto, frequentemente requer algum mecanismo de sincronização global. Infelizmente, tais mecanismos são extremamente difíceis ou mesmo impossíveis de implementar de forma escalável, mesmo porque as latências de rede têm um limite inferior natural. Consequentemente, o escalonamento por replicação pode introduzir outras soluções inerentemente não escaláveis. Retornaremos à replicação e à consistência extensivamente no Capítulo 7.

Discussão Ao considerar essas técnicas de dimensionamento, pode-se argumentar que a escalabilidade de tamanho é a menos problemática de uma perspectiva técnica. Frequentemente, aumentar a capacidade de uma máquina salvará o dia, embora talvez haja um alto custo monetário a pagar. A escalabilidade geográfica é um problema muito mais difícil, pois as latências de rede são naturalmente limitadas de baixo para cima. Como consequência, podemos ser forçados a copiar dados para locais próximos de onde os clientes estão, levando a problemas de manutenção de cópias consistentes. A prática mostra que combinar técnicas de distribuição, replicação e cache com diferentes formas de consistência geralmente leva a soluções aceitáveis. Finalmente, a escalabilidade administrativa parece ser o problema mais difícil de resolver, em parte porque precisamos lidar com questões não técnicas, como políticas de organizações e colaboração humana. A introdução e o uso agora generalizado da tecnologia ponto a ponto demonstraram com sucesso o que pode ser alcançado se os usuários finais forem colocados no controle [Lua et al., 2005; Oram, 2001]. Entretanto, redes ponto a ponto obviamente não são a solução universal para todos os problemas de escalabilidade administrativa.

1.3 Uma classificação simples de sistemas distribuídos

Discutimos sistemas distribuídos versus descentralizados, mas também é útil classificar sistemas distribuídos de acordo com o que eles estão sendo desenvolvidos e usados. Fazemos uma distinção entre sistemas que são desenvolvidos para computação (de alto desempenho), para processamento geral de informações, e aqueles que são desenvolvidos para computação pervasiva, ou seja, para a “Internet das Coisas”. Como acontece com muitas classificações, os limites entre esses três tipos não são rígidos e combinações podem ser facilmente pensadas.

1.3.1 Computação distribuída de alto desempenho

Uma classe importante de sistemas distribuídos é aquela usada para tarefas de computação de alto desempenho. Grosso modo, pode-se fazer uma distinção entre dois subgrupos. Na **computação em cluster**, o hardware subjacente consiste em uma coleção de nós de computação semelhantes, interconectados por uma rede de alta velocidade, frequentemente ao lado de uma rede local mais comum para controlar os nós. Além disso, cada nó geralmente executa o mesmo sistema operacional.

A situação se torna muito diferente no caso da **computação em grade**. Este subgrupo consiste em sistemas descentralizados que são frequentemente construídos como uma federação de sistemas de computadores, onde cada sistema pode cair sob um domínio administrativo diferente, e pode ser muito diferente quando se trata de hardware, software e tecnologia de rede implantada.

Nota 1.6 (Mais informações: Processamento paralelo)

A computação de alto desempenho começou mais ou menos com a introdução de **máquinas multiprocessadoras**. Neste caso, múltiplas CPUs são organizadas de tal forma que todas elas têm acesso à mesma memória física, como mostrado na Figura 1.8(a). Em contraste, em um **sistema multicomputador**, vários computadores são conectados por uma rede e não há compartilhamento de memória principal, como mostrado na Figura 1.8(b).

O modelo de memória compartilhada mostrou-se altamente conveniente para melhorar o desempenho dos programas e era relativamente fácil de programar.

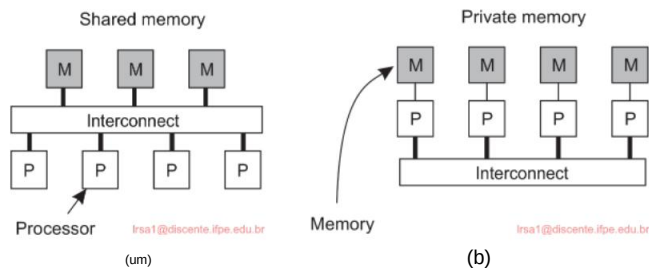


Figura 1.8: Uma comparação entre (a) arquiteturas multiprocessador e (b) multicomputador .

Sua essência é que vários threads de controle estão sendo executados ao mesmo tempo, enquanto todos os threads têm acesso a dados compartilhados. O acesso a esses dados é controlado por meio de mecanismos de sincronização bem compreendidos, como semáforos (veja Ben- Ari [2006] ou Herlihy et al. [2021] para mais informações sobre o desenvolvimento de programas paralelos). Infelizmente, o modelo não escala facilmente: até agora, foram desenvolvidas máquinas nas quais apenas algumas dezenas (e às vezes centenas) de CPUs têm acesso eficiente à memória compartilhada. Até certo ponto, estamos vendo as mesmas limitações para processadores multicore.

Para superar as limitações dos sistemas de memória compartilhada, a computação de alto desempenho migrou para sistemas de memória distribuída. Essa mudança também significou que muitos programas tiveram que fazer uso da passagem de mensagens em vez de modificar dados compartilhados como um meio de comunicação e sincronização entre threads. Infelizmente, os modelos de passagem de mensagens provaram ser muito mais difíceis e propensos a erros em comparação aos modelos de programação de memória compartilhada. Por esse motivo, houve uma pesquisa significativa na tentativa de construir os chamados multicomputadores de memória **compartilhada distribuída**, ou simplesmente **sistemas DSM** [Amza et al., 1996].

Em essência, um sistema DSM permite que um processador enderece um local de memória em outro computador como se fosse memória local. Isso pode ser alcançado usando técnicas existentes disponíveis para o sistema operacional, por exemplo, mapeando todas as páginas de memória principal dos vários processadores em um único espaço de endereço virtual. Sempre que um processador A endereça uma página localizada em outro processador B, ocorre uma falha de página em A, permitindo que o sistema operacional em A busque o conteúdo da página referenciada em B da mesma forma que normalmente o buscaria localmente do disco. Ao mesmo tempo, o processador B seria informado de que a página não está acessível no momento.

A imitação de sistemas de memória compartilhada usando multicomputadores eventualmente teve que ser abandonada porque o desempenho nunca poderia atender às expectativas dos programadores, que prefeririam recorrer a modelos de passagem de mensagens muito mais intrincados, mas com melhor desempenho (previsivelmente). Um efeito colateral importante da exploração dos limites hardware-software do processamento paralelo é uma compreensão completa dos modelos de consistência, aos quais retornamos extensivamente no Capítulo 7.

Computação em cluster

Os sistemas de computação em cluster se tornaram populares quando a relação preço/desempenho de computadores pessoais e estações de trabalho melhorou. Em um certo ponto, tornou-se financeiramente e tecnicamente atraente construir um supercomputador usando tecnologia pronta para uso simplesmente conectando uma coleção de computadores relativamente simples em uma rede de alta velocidade. Em praticamente todos os casos, a computação em cluster é usada para programação paralela, na qual um único programa (computação intensiva) é executado em paralelo em várias máquinas. O princípio dessa organização é mostrado na Figura 1.9.

Este tipo de computação de alto desempenho evoluiu consideravelmente. Conforme discutido extensivamente por Geroÿ et al. [2019], os desenvolvimentos de supercomputadores organizados como clusters atingiram um ponto em que vemos clusters com mais de 100.000 CPUs, com cada CPU tendo 8 ou 16 núcleos. Existem várias redes. O mais importante é uma rede formada por interconexões dedicadas de alta velocidade entre os vários nós (em outras palavras, geralmente não existe uma rede compartilhada de alta velocidade para computações). Uma rede de gerenciamento separada, bem como nós, são usados para monitorar e controlar

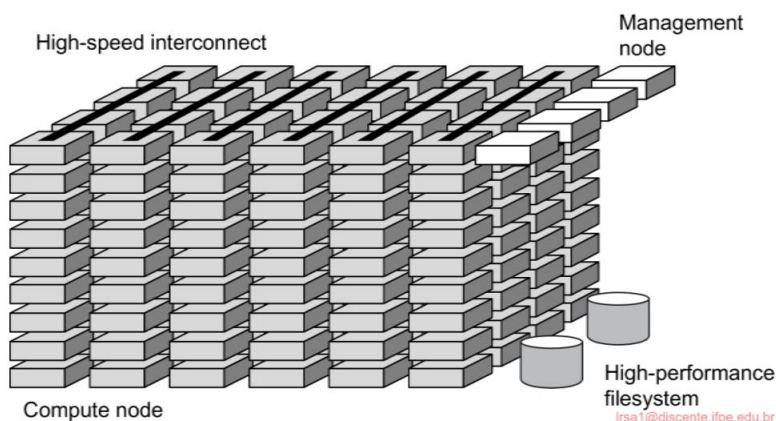


Figura 1.9: Um exemplo de um sistema de computação em cluster (adaptado de [Geroÿ et al., 2019].)

a organização e o desempenho do sistema como um todo. Além disso, um sistema de arquivos ou banco de dados especial de alto desempenho é usado, novamente com sua própria rede local dedicada. A Figura 1.9 não mostra equipamentos adicionais, notavelmente E/S de alta velocidade, bem como recursos de rede para acesso remoto e comunicação.

Um nó de gerenciamento é geralmente responsável por coletar trabalhos de usuários, para posteriormente distribuir as tarefas associadas entre os vários nós de computação. Na prática, vários nós de gerenciamento são usados ao lidar com clusters muito grandes. Como tal, um nó de gerenciamento realmente executa o software necessário para a execução de programas e gerenciamento do cluster, enquanto os nós de computação são equipados com um sistema operacional padrão estendido com funções típicas para comunicação, armazenamento, tolerância a falhas e assim por diante.

Um desenvolvimento interessante, conforme explicado em Geroÿ et al. [2019], é o papel do sistema operacional. Houve uma tendência clara de minimizar o sistema operacional para kernels leves, essencialmente garantindo a menor sobrecarga possível. Uma desvantagem é que tais sistemas operacionais se tornam altamente especializados e ajustados para o hardware subjacente. Essa especialização afeta a compatibilidade ou abertura. Para compensar, agora estamos vendo gradualmente as chamadas abordagens multikernel, nas quais um sistema operacional de ponta completa opera ao lado de um kernel leve, alcançando assim o melhor dos dois mundos. Essa combinação também é necessária, pois cada vez mais, um nó de computação de alto desempenho é necessário para executar vários trabalhos independentes simultaneamente. Atualmente, 95% de todos os computadores de alto desempenho executam sistemas baseados em Linux; abordagens multikernel são desenvolvidas para CPUs multicore, com a maioria dos núcleos executando um kernel leve e o outro executando um sistema Linux regular. Dessa forma, novos desenvolvimentos, como contêineres (que discutimos no Capítulo 3), também podem ser suportados. Os efeitos no desempenho da computação ainda precisam ser observados.

Computação em grade

Uma característica da computação em cluster tradicional é sua homogeneidade. Na maioria dos casos, os computadores em um cluster são basicamente os mesmos, têm o mesmo sistema operacional e estão todos conectados pela mesma rede. No entanto, como acabamos de discutir, há uma tendência contínua em direção a arquiteturas mais híbridas nas quais os nós são especificamente configurados para certas tarefas. Essa diversidade é ainda mais prevalente em **sistemas de computação em grade**: nenhuma suposição é feita sobre similaridade de hardware, sistemas operacionais, redes, domínios administrativos, políticas de segurança, etc. [Rajaraman, 2016].

Uma questão fundamental em um sistema de computação em grade é que recursos de diferentes organizações são reunidos para permitir a colaboração de um grupo de pessoas de diferentes instituições, formando de fato uma federação de sistemas. Tal colaboração é realizada na forma de uma **organização virtual**. Os processos pertencentes à mesma organização virtual têm direitos de acesso ao

recursos que são fornecidos para essa organização. Normalmente, os recursos consistem em servidores de computação (incluindo supercomputadores, possivelmente implementados como computadores de cluster), instalações de armazenamento e bancos de dados. Além disso, dispositivos especiais em rede, como telescópios, sensores, etc., também podem ser fornecidos.

Dada sua natureza, grande parte do software para realizar computação em grade envolve fornecer acesso a recursos de diferentes domínios administrativos e apenas para aqueles usuários e aplicativos que pertencem a uma organização virtual específica. Por esse motivo, o foco geralmente está em questões arquitetônicas. Uma arquitetura inicialmente proposta por Foster et al. [2001] é mostrada na Figura 1.10, que ainda forma a base para muitos sistemas de computação em grade.

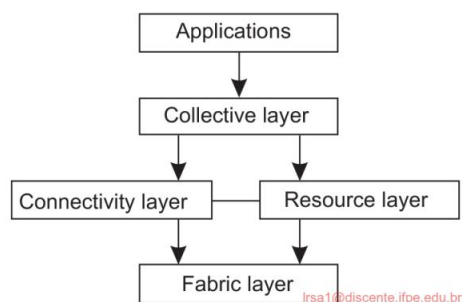


Figura 1.10: Uma arquitetura em camadas para sistemas de computação em grade.

A arquitetura consiste em quatro camadas. A **camada de fabric** mais baixa fornece interfaces para recursos locais em um site específico. Observe que essas interfaces são adaptadas para permitir o compartilhamento de recursos dentro de uma organização virtual. Normalmente, elas fornecerão funções para consultar o estado e as capacidades de um recurso, juntamente com funções para gerenciamento de recursos reais (por exemplo, bloqueio de recursos).

A **camada de conectividade** consiste em protocolos de comunicação para dar suporte a transações de grade que abrangem o uso de vários recursos. Por exemplo, protocolos são necessários para transferir dados entre recursos ou simplesmente acessar um recurso de um local remoto. Além disso, a camada de conectividade conterá protocolos de segurança para autenticar usuários e recursos. Observe que, em muitos casos, usuários humanos não são autenticados; em vez disso, programas agindo em nome dos usuários são autenticados. Nesse sentido, delegar direitos de um usuário para programas é uma função importante que precisa ser suportada na camada de conectividade. Retornamos à delegação ao discutir segurança em sistemas distribuídos no Capítulo 9.

A **camada de recursos** é responsável por gerenciar um único recurso. Ela usa as funções fornecidas pela camada de conectividade e chama diretamente as interfaces disponibilizadas pela camada de fabric. Por exemplo, esta camada oferecerá funções para obter informações de configuração sobre um recurso específico ou, em geral, para executar operações específicas, como criar um processo ou ler dados.

A camada de recursos é vista como responsável pelo controle de acesso e, portanto, dependerá da autenticação realizada como parte da camada de conectividade.

A próxima camada na hierarquia é a **camada coletiva**. Ela lida com o manuseio do acesso a múltiplos recursos e normalmente consiste em serviços para descoberta de recursos, alocação e agendamento de tarefas em múltiplos recursos, replicação de dados e assim por diante. Ao contrário da camada de conectividade e recursos, cada uma consistindo em uma coleção relativamente pequena e padrão de protocolos, a camada coletiva pode consistir em muitos protocolos que refletem o amplo espectro de serviços que ela pode oferecer a uma organização virtual.

Por fim, a **camada de aplicação** consiste nos aplicativos que operam dentro de uma organização virtual e que fazem uso do ambiente de computação em grade.

Normalmente, a camada coletiva, de conectividade e de recursos forma o coração do que poderia ser chamado de camada de middleware de grade. Essas camadas fornecem conjuntamente acesso e gerenciamento de recursos que são potencialmente dispersos em vários sites.

Uma observação importante de uma perspectiva de middleware é que na computação em grade, a noção de um site (ou unidade administrativa) é comum. Essa prevalência é enfatizada pela mudança gradual em direção a uma **arquitetura orientada a serviços** na qual os sites oferecem acesso às várias camadas por meio de uma coleção de serviços da Web [Joseph et al., 2004]. Isso, agora, levou à definição de uma arquitetura alternativa conhecida como **Open Grid Services Architecture (OGSA)** [Foster et al., 2006]. A OGSA é baseada nas ideias originais formuladas por Foster et al. [2001], mas ter passado por um processo de padronização a torna complexa, para dizer o mínimo. As implementações da OGSA geralmente seguem os padrões de serviços da Web.

1.3.2 Sistemas de informação distribuídos

Outra classe importante de sistemas distribuídos é encontrada em organizações que foram confrontadas com uma riqueza de aplicativos em rede, mas para as quais a interoperabilidade acabou sendo uma experiência dolorosa. Muitas das soluções de middleware existentes são o resultado de trabalhar com uma infraestrutura na qual era mais fácil integrar aplicativos em um sistema de informações de toda a empresa [Alonso et al., 2004; Bernstein, 1996; Hohpe e Woolf, 2004].

Podemos distinguir vários níveis nos quais a integração pode ocorrer. Frequentemente, um aplicativo em rede consiste simplesmente em um servidor executando esse aplicativo (frequentemente incluindo um banco de dados) e disponibilizando-o para programas remotos, chamados **clientes**. Esses clientes enviam uma solicitação ao servidor para executar uma operação específica, após a qual uma resposta é enviada de volta. A integração no nível mais baixo permite que os clientes envolvam várias solicitações, possivelmente para servidores diferentes, em uma única solicitação maior e a executem como uma **transação distribuída**. A ideia principal é que todas ou nenhuma das solicitações sejam executadas.

À medida que os aplicativos se tornaram mais sofisticados e foram gradualmente separados em componentes independentes (notavelmente distinguindo componentes de banco de dados de componentes de processamento), ficou claro que a integração também deveria ocorrer permitindo que os aplicativos se comunicassem diretamente entre si. Isso agora levou a uma indústria de **integração de aplicativos corporativos (EAI)**.

Processamento de transações distribuídas

Para esclarecer nossa discussão, nos concentramos em aplicações de banco de dados. Na prática, operações em um banco de dados são realizadas na forma de **transações**. A programação usando transações requer primitivas especiais que devem ser fornecidas pelo sistema distribuído subjacente ou pelo sistema de tempo de execução da linguagem. Exemplos típicos de primitivas de transação são mostrados na Figura 1.11. A lista exata de primitivas depende de quais tipos de objetos estão sendo usados na transação [Gray e Reuter, 1993; Bernstein e Newcomer, 2009]. Em um sistema de correio, pode haver primitivas para enviar, receber e encaminhar correio. Em um sistema de contabilidade, eles podem ser bem diferentes. READ e WRITE são exemplos típicos, no entanto. Instruções comuns, chamadas de procedimento e assim por diante, também são permitidas dentro de uma transação. Em particular, **chamadas de procedimento remoto (RPC)**, ou seja, chamadas de procedimento para servidores remotos, geralmente também são encapsuladas em uma transação, levando ao que é conhecido como **RPC transacional**. Discutimos RPCs extensivamente na Seção 4.2.

Primitivo	Descrição
BEGIN_TRANSACTION	Marca o início de uma transação
COMMIT_TRANSACTION	Encerre a transação e tente confirmar
ABORT_TRANSACTION	Mate a transação e restaure os valores antigos
READ	Ler dados de um arquivo, uma tabela ou de outra forma
WRITE	Grave dados em um arquivo, uma tabela ou de outra forma

Figura 1.11: Exemplo de primitivas para transações.

BEGIN_TRANSACTION e END_TRANSACTION são usados para delimitar o escopo de uma transação. As operações entre elas formam o corpo da transação. A característica de uma transação é que todas essas operações são executadas ou nenhuma é executada. Elas podem ser chamadas de sistema, procedimentos de biblioteca ou instruções de colchetes em uma linguagem, dependendo da implementação.

Essa propriedade tudo-ou-nada das transações é uma das quatro propriedades características que as transações têm. Mais especificamente, as transações aderem às chamadas propriedades **ACID** :

- **Atômico:** Para o mundo externo, a transação acontece de forma indivisível.
- **Consistente:** A transação não viola as invariantes do sistema.

- **Isolado:** As transações simultâneas não interferem umas nas outras
- **Durável:** uma vez que uma transação é confirmada, as alterações são permanentes

Em sistemas distribuídos, as transações são frequentemente construídas como uma série de subtransações, formando em conjunto uma **transação aninhada**, conforme mostrado na Figura 1.12. A transação de nível superior pode bifurcar filhos que são executados em paralelo uns com os outros, em máquinas diferentes, para ganhar desempenho ou simplificar a programação. Cada um desses filhos também pode executar uma ou mais subtransações ou bifurcar seus próprios filhos.

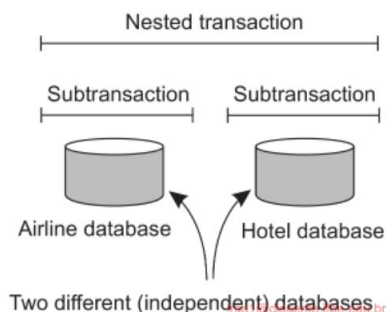


Figura 1.12: Uma transação aninhada.

Subtransações dão origem a um problema sutil, mas importante. Imagine que uma transação inicia várias subtransações em paralelo, e uma delas confirma, tornando seus resultados visíveis para a transação pai. Após mais cálculos, o pai aborta, restaurando todo o sistema ao estado em que estava antes do início da transação de nível superior. Consequentemente, os resultados da subtransação que confirmou devem, no entanto, ser desfeitos. Assim, a permanência mencionada acima se aplica apenas a transações de nível superior.

Como as transações podem ser aninhadas arbitrariamente profundamente, uma administração considerável é necessária para fazer tudo certo. A semântica é clara, no entanto. Quando qualquer transação ou subtransação começa, ela recebe conceitualmente uma cópia privada de todos os dados em todo o sistema para manipular como desejar. Se ela abortar, seu universo privado simplesmente desaparece, como se nunca tivesse existido. Se ela fizer commit, seu universo privado substitui o universo do pai. Assim, se uma subtransação fizer commit e depois uma nova subtransação for iniciada, a segunda verá os resultados produzidos pela primeira. Da mesma forma, se uma transação envolvente (nível mais alto) for abortada, todas as suas subtransações subjacentes também terão que ser abortadas. E se várias transações forem iniciadas simultaneamente, o resultado é como se elas fossem executadas sequencialmente em alguma ordem não especificada.

Transações aninhadas são importantes em sistemas distribuídos, pois fornecem uma maneira natural de distribuir uma transação entre várias máquinas. Elas seguem uma divisão lógica do trabalho da transação original. Por exemplo, uma transação para planejar uma viagem pela qual três voos diferentes precisam ser

reserved pode ser logicamente dividido em três subtransações. Cada uma dessas subtransações pode ser gerenciada separadamente e independentemente.

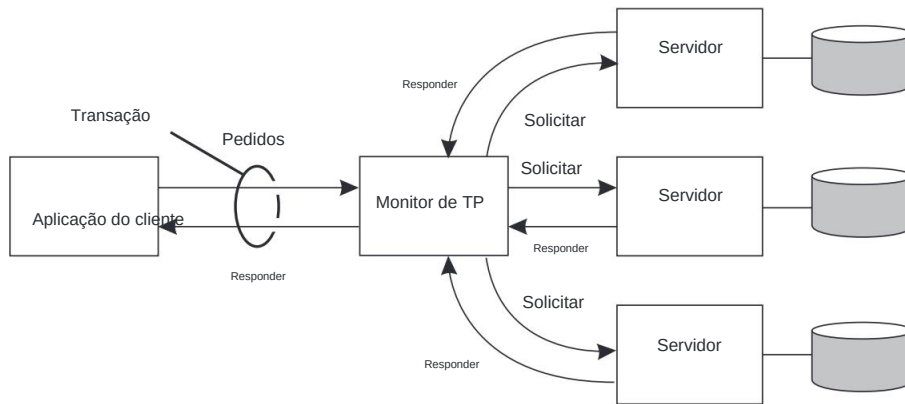


Figura 1.13: O papel de um monitor TP em sistemas distribuídos.

Desde os primeiros dias dos sistemas de middleware empresarial, o componente que manipula transações distribuídas (ou aninhadas) pertence ao núcleo para integração de aplicativos no nível do servidor ou banco de dados. Este componente é chamado de **monitor de processamento de transações** ou **monitor TP** para abreviar. Sua principal tarefa é permitir que um aplicativo acesse vários servidores/bancos de dados, oferecendo a ele um modelo de programação transacional, conforme mostrado na Figura 1.13. Essencialmente, o monitor TP coordena o comprometimento de subtransações seguindo um protocolo padrão conhecido como **comprometimento distribuído**, que discutimos em detalhes na Seção 8.5.

Uma observação importante é que aplicativos que desejam coordenar várias subtransações em uma única transação não precisam implementar essa coordenação eles mesmos. Simplesmente fazendo uso de um monitor TP, essa coordenação é feita para eles. É precisamente aí que o middleware entra em cena: ele implementa serviços que são úteis para muitos aplicativos, evitando que tais serviços tenham que ser reimplementados repetidamente pelos desenvolvedores de aplicativos.

Integração de aplicativos empresariais

Conforme mencionado, quanto mais os aplicativos se desacoplavam dos bancos de dados nos quais eram construídos, mais evidente se tornava que eram necessárias facilidades para integrar aplicativos independentemente de seus bancos de dados. Em particular, os componentes do aplicativo deveriam ser capazes de se comunicar diretamente entre si e não meramente por meio do comportamento de solicitação/resposta que era suportado pelos sistemas de processamento de transações.

Essa necessidade de comunicação entre aplicações levou a muitos modelos de comunicação. A ideia principal era que as aplicações existentes pudessem trocar informações diretamente, como mostrado na Figura 1.14.

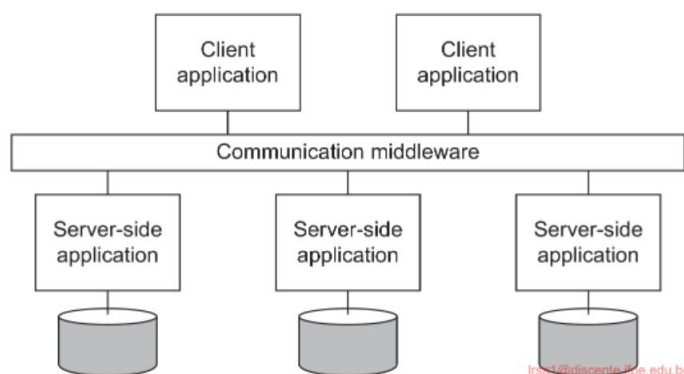


Figura 1.14: Middleware como facilitador de comunicação na integração de aplicativos corporativos.

Existem vários tipos de middleware de comunicação. Com **chamadas de procedimento remoto (RPC)**, um componente de aplicativo pode efetivamente enviar uma solicitação para outro componente de aplicativo fazendo uma chamada de procedimento local, o que resulta na solicitação sendo empacotada como uma mensagem e enviada ao chamado. Da mesma forma, o resultado será enviado de volta e retornado ao aplicativo como resultado da chamada de procedimento.

À medida que a popularidade da tecnologia de objetos aumentou, técnicas foram desenvolvidas para permitir chamadas para objetos remotos, levando ao que é conhecido como **invocações de métodos remotos (RMI)**. Um RMI é essencialmente o mesmo que um RPC, exceto que ele opera em objetos em vez de funções.

RPC e RMI têm a desvantagem de que o chamador e o chamado precisam estar ativos e funcionando no momento da comunicação. Além disso, eles precisam saber exatamente como se referir um ao outro. Esse acoplamento rígido é frequentemente experimentado como uma desvantagem séria e levou ao que é conhecido como **middleware orientado a mensagens**, ou simplesmente **MOM**. Nesse caso, os aplicativos enviam mensagens para pontos de contato lógicos, geralmente descritos por um assunto. Da mesma forma, os aplicativos podem indicar seu interesse por um tipo específico de mensagem, após o qual o middleware de comunicação cuidará para que essas mensagens sejam entregues a esses aplicativos. Esses chamados sistemas **de publicação-assinatura** formam uma classe importante e em expansão de sistemas distribuídos.

Nota 1.7 (Mais informações: Sobre integração de aplicações)

Dar suporte à integração de aplicativos corporativos é uma meta importante para muitos produtos de middleware. Em geral, há quatro maneiras de integrar aplicativos [Hohpe e Woolf, 2004]:

Transferência de arquivos: A essência da integração por meio da transferência de arquivos é que um aplicativo produz um arquivo contendo dados compartilhados que são posteriormente lidos por

outras aplicações. A abordagem é tecnicamente simples, o que a torna atraente. A desvantagem, no entanto, é que há inúmeras coisas que precisam ser acordadas:

- Formato e layout de arquivo: texto, binário, sua estrutura e assim por diante. Hoje em dia, a **linguagem de marcação estendida (XML)** se tornou popular, pois seus arquivos são, em princípio, autodescritivos.
- Gerenciamento de arquivos: onde eles são armazenados, como são nomeados, quem é responsável
- por excluir os arquivos?
- Propagação de atualização: quando um aplicativo produz um arquivo, pode haver vários aplicativos que precisam ler esse arquivo para fornecer a visualização de um único sistema coerente. Como consequência, às vezes programas separados precisam ser implementados para notificar os aplicativos sobre atualizações de arquivo.

Banco de dados compartilhado: Muitos dos problemas associados à integração por meio de arquivos são aliviados ao usar um banco de dados compartilhado. Todos os aplicativos terão acesso aos mesmos dados, e frequentemente por meio de uma linguagem de banco de dados de alto nível, como SQL. Além disso, é fácil notificar os aplicativos quando ocorrem alterações, pois os gatilhos geralmente fazem parte dos bancos de dados modernos. Existem, no entanto, duas grandes desvantagens. Primeiro, ainda há a necessidade de projetar um esquema de dados comum, o que pode estar longe de ser trivial se o conjunto de aplicativos que precisam ser integrados não for completamente conhecido com antecedência. Segundo, quando há muitas leituras e atualizações, um banco de dados compartilhado pode facilmente se tornar um gargalo de desempenho.

Chamada de procedimento remoto: A integração por meio de arquivos ou um banco de dados pressupõe implicitamente que alterações por um aplicativo podem facilmente acionar outros aplicativos para agir. No entanto, a prática mostra que, às vezes, pequenas alterações devem, na verdade, acionar muitos aplicativos para tomar ações. Em tais casos, não é realmente a alteração de dados que é importante, mas a execução de uma série de ações.

Séries de ações são melhor capturadas por meio da execução de um procedimento (que pode, por sua vez, levar a todos os tipos de alterações em dados compartilhados). Para evitar que cada aplicativo precise saber todos os detalhes internos dessas ações (conforme implementado por outro aplicativo), técnicas de encapsulamento padrão devem ser usadas, conforme implementadas com chamadas de procedimento tradicionais ou invocações de objeto. Para tais situações, um aplicativo pode oferecer melhor um procedimento a outros aplicativos na forma de uma chamada de procedimento remoto, ou RPC. Em essência, um RPC permite que um aplicativo A faça uso das informações disponíveis apenas para o aplicativo B, sem dar a A acesso direto a essas informações. Existem muitas vantagens e desvantagens nas chamadas de procedimento remoto, que são discutidas em profundidade no Capítulo 4.

Mensagens: Uma desvantagem principal dos RPCs é que o chamador e o chamado precisam estar ativos e funcionando ao mesmo tempo para que a chamada seja bem-sucedida. No entanto, em muitos cenários, essa atividade simultânea é frequentemente difícil ou impossível

para garantir. Em tais casos, oferecer um sistema de mensagens que carregue solicitações do aplicativo A para executar uma ação no aplicativo B é o que é necessário. O sistema de mensagens garante que, eventualmente, a solicitação seja entregue e, se necessário, que uma resposta seja eventualmente retornada também. Obviamente, mensagens não são a panaceia para integração de aplicativos: elas também introduzem problemas relacionados à formatação e layout de dados, exigem que um aplicativo saiba para onde enviar uma mensagem, é preciso haver cenários para lidar com mensagens perdidas, e assim por diante. Assim como RPCs, discutiremos essas questões extensivamente no Capítulo 4.

O que essas quatro abordagens nos dizem é que a integração de aplicativos geralmente não será simples. O middleware (na forma de um sistema distribuído), no entanto, pode ajudar significativamente na integração ao fornecer os recursos certos, como suporte para RPCs ou mensagens. Como dito, a integração de aplicativos corporativos é um campo-alvo importante para muitos produtos de middleware.

1.3.3 Sistemas pervasivos

Os sistemas distribuídos discutidos até agora são amplamente caracterizados por sua estabilidade: os nós são fixos e têm uma conexão mais ou menos permanente e de alta qualidade com uma rede. Até certo ponto, essa estabilidade é realizada por meio de várias técnicas para atingir a transparência da distribuição. Por exemplo, há muitas maneiras de criar a ilusão de que apenas ocasionalmente os componentes podem falhar. Da mesma forma, há todos os tipos de meios para ocultar a localização real da rede de um nó, permitindo efetivamente que usuários e aplicativos acreditem que os nós permanecem no lugar.

No entanto, as coisas mudaram desde a introdução de dispositivos de computação móveis e embarcados, levando ao que geralmente é chamado de **sistemas pervasivos**. Como o nome sugere, os sistemas pervasivos são destinados a se misturar ao nosso ambiente naturalmente. Muitos de seus componentes são necessariamente espalhados por vários computadores, tornando-os indiscutivelmente um tipo de sistema descentralizado em nossa visão. Ao mesmo tempo, a maioria dos sistemas pervasivos tem muitos componentes que são suicientemente espalhados por todo o sistema, por exemplo, para lidar com falhas e coisas assim. Nesse sentido, eles também são indiscutivelmente sistemas distribuídos. A separação aparentemente estrita entre sistemas descentralizados e distribuídos é, portanto, vista como menos estrita do que se poderia imaginar inicialmente.

O que os torna únicos, em comparação aos sistemas de computação e informação descritos até agora, é que a separação entre usuários e componentes do sistema é muito mais tênue. Muitas vezes, não há uma única interface dedicada, como uma combinação de tela/teclado. Em vez disso, um sistema pervasivo é frequentemente equipado com muitos **sensores** que captam vários aspectos do comportamento de um usuário. Da mesma forma, ele pode ter uma miríade de **atuadores** para fornecer informações e feedback, muitas vezes até mesmo visando propositalmente orientar o comportamento.

Muitos dispositivos em sistemas pervasivos são caracterizados por serem pequenos, alimentados por bateria, móveis e terem apenas uma conexão sem fio, embora nem todas essas características se apliquem a todos os dispositivos. Essas não são necessariamente características restritivas, como é ilustrado pelos smartphones [Roussos et al., 2005] e seu papel no que agora é cunhado como a **Internet das Coisas** [Mattern e Floerkemeier, 2010; Stankovic, 2014]. No entanto, notavelmente, o fato de que muitas vezes precisamos lidar com as complexidades da comunicação sem fio e móvel exigirá soluções especiais para tornar um sistema pervasivo o mais transparente ou discreto possível.

A seguir, faremos uma distinção entre três tipos diferentes de sistemas difundidos, embora haja uma sobreposição considerável entre os três tipos: sistemas de computação ubíquos, sistemas móveis e redes de sensores.

Essa distinção nos permite focar em diferentes aspectos dos sistemas generalizados.

Sistemas de computação ubíquos

Até agora, falamos sobre sistemas pervasivos para enfatizar que seus elementos se espalharam por muitas partes do nosso ambiente. Em um sistema de computação ubíquo, damos um passo além: o sistema é pervasivo e continuamente presente. O último significa que um usuário estará continuamente interagindo com o sistema, muitas vezes nem mesmo estando ciente de que a interação está ocorrendo. Poslad [2009] descreve os principais requisitos para um **sistema de computação ubíquo** aproximadamente da seguinte forma:

1. **(Distribuição)** Os dispositivos são interligados, distribuídos e acessíveis por meio de transmissão.

Parentalmente

2. **(Interação)** A interação entre usuários e dispositivos é altamente discreta.

(Consciência do contexto) O sistema está ciente do contexto do usuário para otimizar interação

4. **(Autonomia)** Os dispositivos operam de forma autônoma sem intervenção humana e, portanto, são altamente autogerenciados.

5. **(Inteligência)** O sistema como um todo pode lidar com uma ampla gama de ações e interações dinâmicas.

Vamos considerar esses requisitos de uma perspectiva de sistemas distribuídos.

Ad. 1: Distribuição Como mencionado, um sistema de computação ubíquo é um exemplo de um sistema distribuído: os dispositivos e outros computadores que formam os nós de um sistema são simplesmente conectados em rede e trabalham juntos para formar a ilusão de um sistema único e coerente. A distribuição também vem naturalmente: haverá dispositivos próximos aos usuários (como sensores e atuadores), conectados a computadores escondidos da vista e talvez até operando remotamente em um

nuvem. A maioria, se não todos, dos requisitos relativos à transparência de distribuição mencionados na Seção 1.2.2, devem, portanto, ser mantidos.

Ad. 2: Interação Quando se trata de interação com usuários, os sistemas de computação ubíqua diferem muito em comparação aos sistemas que discutimos até agora. Os usuários finais desempenham um papel proeminente no design de sistemas ubíquos, o que significa que atenção especial precisa ser dada a como a interação entre usuários e o sistema central ocorre. Para sistemas de computação ubíqua, grande parte da interação por humanos será implícita, com uma **ação implícita** sendo definida como aquela “que não tem como objetivo principal interagir com um sistema computadorizado, mas que tal sistema entende como entrada” [Schmidt, 2000]. Em outras palavras, um usuário pode estar praticamente inconsciente do fato de que a entrada está sendo fornecida a um sistema de computador. De uma certa perspectiva, pode-se dizer que a computação ubíqua aparentemente esconde interfaces.

Um exemplo simples é onde as configurações do assento do motorista, volante e espelhos de um carro são totalmente personalizadas. Se Bob se sentar, o sistema reconhecerá que está lidando com Bob e, subsequentemente, fará os ajustes apropriados. O mesmo acontece quando Alice usa o carro, enquanto um usuário desconhecido será direcionado a fazer seus próprios ajustes (para ser lembrado mais tarde). Este exemplo já ilustra um papel importante dos sensores na computação ubíqua, ou seja, como dispositivos de entrada que são usados para identificar uma situação (uma pessoa específica aparentemente querendo dirigir), cuja análise de entrada leva a ações (fazer ajustes). Por sua vez, as ações podem levar a reações naturais, por exemplo, que Bob altere ligeiramente as configurações do assento. O sistema terá que levar em consideração todas as ações (implícitas e explícitas) do usuário e reagir de acordo.

Ad. 3: Consciência de contexto Reagir à entrada sensorial, mas também à entrada explícita dos usuários, é mais fácil dizer do que fazer. O que um sistema de computação ubíqua precisa fazer é levar em conta o **contexto** em que as interações ocorrem. A consciência de contexto também diferencia os sistemas de computação ubíqua dos sistemas mais tradicionais que discutimos antes, e é descrita por Dey e Aboowd [2000] como “qualquer informação que pode ser usada para caracterizar a situação de entidades (ou seja, seja uma pessoa, lugar ou objeto) que são consideradas relevantes para a interação entre um usuário e um aplicativo, incluindo o usuário e o próprio aplicativo”. Na prática, o contexto é frequentemente caracterizado por localização, identidade, tempo e atividade: onde, quem, quando e o quê. Um sistema precisará ter a entrada (sensorial) necessária para determinar um ou vários desses tipos de contexto. Conforme discutido por Alegre et al. [2016], desenvolver sistemas sensíveis ao contexto é difícil, mesmo que seja apenas porque a noção de contexto é difícil de compreender.

O que é importante, de uma perspectiva de sistemas distribuídos, é que os dados brutos coletados por vários sensores são elevados a um nível de abstração que pode ser usado por aplicativos. Um exemplo concreto é detectar onde uma pessoa está,

por exemplo, em termos de coordenadas de GPS, e subsequentemente mapeando essas informações para um local real, como a esquina de uma rua, ou uma loja específica ou outra instalação conhecida. A questão é onde esse processamento de entrada sensorial ocorre: todos os dados são coletados em um servidor central conectado a um banco de dados com informações detalhadas sobre uma cidade, ou é no smartphone do usuário que o mapeamento é feito? Claramente, há compensações a serem consideradas.

Dey [2010] discute abordagens mais gerais para a construção de aplicativos sensíveis ao contexto. Quando se trata de combinar flexibilidade e distribuição potencial, os chamados **espaços de dados compartilhados** nos quais os processos são desacoplados no tempo e no espaço são atraentes, mas, como veremos em capítulos posteriores, sofrem de problemas de escalabilidade. Uma pesquisa sobre consciência de contexto e sua relação com middleware e sistemas distribuídos é fornecida por Baldauf et al. [2007].

Ad. 4: Autonomia Um aspecto importante da maioria dos sistemas de computação ubíquos é que o gerenciamento explícito de sistemas foi reduzido ao mínimo. Em um ambiente de computação ubíquo, simplesmente não há espaço para um administrador de sistemas manter tudo funcionando. Como consequência, o sistema como um todo deve ser capaz de agir de forma autônoma e reagir automaticamente a mudanças. Isso requer uma miríade de técnicas, das quais várias serão discutidas ao longo deste livro. Para dar alguns exemplos simples, pense no seguinte:

Alocação de endereços: para que dispositivos em rede se comuniquem, eles precisam de um endereço IP. Endereços podem ser alocados automaticamente usando protocolos como o **Dynamic Host Configuration Protocol (DHCP)** [Droms, 1997] (que requer um servidor) ou **Zeroconf** [Guttman, 2001].

Adicionar dispositivos: Deve ser fácil adicionar dispositivos a um sistema existente. Um passo em direção à configuração automática é realizado pelo **protocolo Universal Plug and Play (UPnP)** [UPnP Forum, 2008]. Usando o UPnP, os dispositivos podem descobrir uns aos outros e certificar-se de que podem configurar canais de comunicação entre eles.

Atualizações automáticas: Muitos dispositivos em um sistema de computação ubíquo devem ser capazes de verificar regularmente pela Internet se seu software deve ser atualizado. Se for o caso, eles podem baixar novas versões de seus componentes e, idealmente, continuar de onde pararam.

É verdade que esses são exemplos simples, mas a imagem deve deixar claro que a intervenção manual deve ser mantida no mínimo. Discutiremos muitas técnicas relacionadas à autogestão em detalhes ao longo do livro.

Ad. 5: Inteligência Finalmente, Poslad [2009] menciona que os sistemas de computação ubíquos frequentemente usam métodos e técnicas do campo da inteligência artificial.

inteligência. O que isso significa é que, frequentemente, uma ampla gama de algoritmos e modelos avançados precisam ser implantados para lidar com entradas incompletas, reagir rapidamente a um ambiente em mudança, lidar com eventos inesperados e assim por diante. A extensão em que isso pode ou deve ser feito de forma distribuída é crucial.

cial da perspectiva de sistemas distribuídos. Infelizmente, soluções distribuídas para muitos problemas no campo da inteligência artificial ainda não foram encontradas, o que significa que pode haver uma tensão natural entre o primeiro requisito de dispositivos em rede e distribuídos e o processamento avançado de informações distribuídas.

Sistemas de computação móvel

Conforme mencionado, a mobilidade frequentemente forma um componente importante de sistemas pervasivos, e muitos, se não todos os aspectos que acabamos de discutir também se aplicam à **computação móvel**. Existem várias questões que colocam a computação móvel de lado em relação aos sistemas pervasivos em geral (veja também Adelstein et al. [2005] e Tarkoma e Kangasharju [2009]).

Primeiro, os dispositivos que fazem parte de um sistema móvel (distribuído) podem variar muito. Normalmente, a computação móvel é feita com dispositivos como smartphones e tablets. No entanto, tipos totalmente diferentes de dispositivos agora estão usando o Protocolo de Internet (IP) para se comunicar, colocando a computação móvel em uma perspectiva diferente. Esses dispositivos incluem controles remotos, pagers, emblemas ativos, equipamentos de carro, vários dispositivos habilitados para GPS e assim por diante. Uma característica de todos esses dispositivos é que eles usam comunicação sem fio. Móvel implica sem fio, ao que parece (embora haja exceções às regras).

Em segundo lugar, na computação móvel, presume-se que a localização de um dispositivo muda ao longo do tempo. Uma localização em mudança tem seus efeitos em muitas questões. Por exemplo, se a localização de um dispositivo muda regularmente, talvez os serviços que estão disponíveis localmente também mudem. Como consequência, podemos precisar prestar atenção especial à descoberta dinâmica de serviços, mas também deixar que os serviços anunciem sua presença. De forma semelhante, muitas vezes também queremos saber onde um dispositivo realmente está. Isso pode significar que precisamos saber as coordenadas geográficas reais de um dispositivo, como em aplicativos de rastreamento e localização, mas também pode exigir que possamos simplesmente detectar sua posição na rede (como em IP móvel [Perkins, 2010; Perkins et al., 2011]).

Mudar de local também pode ter um efeito profundo na comunicação. Por algum tempo, pesquisadores em computação móvel têm se concentrado no que é conhecido como **redes ad hoc móveis**, também conhecidas como **MANETs**. A ideia básica era que um grupo de computadores móveis locais criaria em conjunto uma rede local sem fio e, posteriormente, compartilharia recursos e serviços. A ideia nunca se tornou realmente popular. Na mesma linha, há pesquisadores que acreditam que os usuários finais estão dispostos a compartilhar seus recursos locais para os requisitos de computação, armazenamento ou comunicação de outro usuário (veja, por exemplo, Ferrer

et al. [2019]). A prática tem mostrado repetidamente que disponibilizar recursos voluntariamente não é algo que os usuários estão dispostos a fazer, mesmo que tenham recursos em abundância. O efeito é que, no caso da computação móvel, geralmente vemos dispositivos móveis individuais configurando conexões com servidores estacionários. Mudar de local significa simplesmente que essas conexões precisam ser entregues por roteadores no caminho do dispositivo móvel para o servidor. A computação móvel é então trazida de volta à sua essência: um dispositivo móvel conectado a um servidor (e nada mais). Na prática, isso significa que a computação móvel tem tudo a ver com dispositivos móveis que fazem uso de serviços baseados em nuvem, conforme esboçado na Figura 1.15(a).

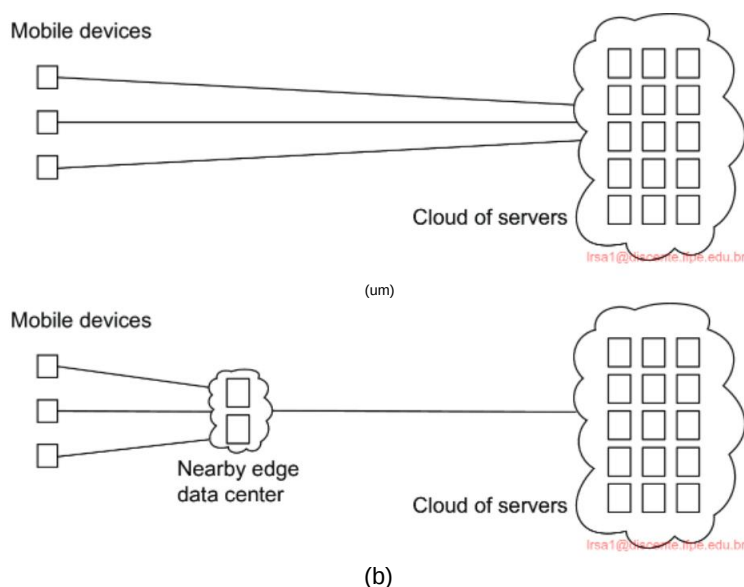


Figura 1.15: (a) Computação em nuvem móvel versus (b) Computação de ponta móvel.

Apesar da simplicidade conceitual deste modelo de computação móvel, o simples fato de tantos dispositivos fazerem uso de serviços remotos levou ao que é conhecido como **Mobile Edge Computing**, ou simplesmente **MEC** [Abbas et al., 2018], em contraste com **Mobile Cloud Computing (MCC)**. Como discutiremos mais adiante no Capítulo 2, a computação de ponta (móvel), conforme esboçado na Figura 1.15(b), está se tornando cada vez mais importante nos casos em que a latência, mas também questões computacionais, desempenham um papel para o dispositivo móvel. Exemplos típicos de aplicativos que exigem latências curtas e recursos computacionais incluem realidade aumentada, jogos interativos, monitoramento de esportes em tempo real e vários aplicativos de saúde [Dimou et al., 2022]. Nestes exemplos, a combinação de monitoramento, análises e feedback imediato em geral torna difícil confiar em servidores que podem estar localizados a milhares de quilômetros dos dispositivos móveis.

Redes de sensores

Nosso último exemplo de sistemas pervasivos são **as redes de sensores**. Essas redes, em muitos casos, fazem parte da tecnologia habilitadora para a pervasividade, e vemos que muitas soluções para redes de sensores retornam em aplicações pervasivas. O que torna as redes de sensores interessantes da perspectiva de um sistema distribuído é que elas são mais do que apenas uma coleção de dispositivos de entrada. Em vez disso, como veremos, os nós de sensores frequentemente colaboram para processar os dados detectados de forma eficiente de uma maneira específica da aplicação, tornando-os muito diferentes, por exemplo, das redes de computadores tradicionais. Akyildiz et al. [2002] e Akyildiz et al. [2005] fornecem uma visão geral de uma perspectiva de rede. Uma introdução mais orientada a sistemas para redes de sensores é dada por Zhao e Guibas [2004], mas também [Hahmy, 2021] mostrará ser útil.

Uma rede de sensores geralmente consiste de dezenas a centenas ou milhares de nós relativamente pequenos, cada um equipado com um ou mais dispositivos de detecção. Além disso, os nós podem frequentemente atuar como atuadores [Akyildiz e Kasimoglu, 2004], um exemplo típico sendo a ativação automática de sprinklers quando um incêndio é detectado. Muitas redes de sensores usam comunicação sem fio, e os nós são frequentemente alimentados por bateria. Seus recursos limitados, capacidades de comunicação restritas e consumo de energia restrito exigem que a eficiência esteja no topo da lista de critérios de projeto.

Ao ampliar um nó individual, vemos que, conceitualmente, eles não diferem muito dos computadores “normais”: acima do hardware, há uma camada de software semelhante ao que os sistemas operacionais tradicionais oferecem, incluindo acesso de rede de baixo nível, acesso a sensores e atuadores, gerenciamento de memória e assim por diante. Normalmente, o suporte para serviços específicos é incluído, como localização, armazenamento local (pense em dispositivos *flash* adicionais) e recursos de comunicação convenientes, como mensagens e roteamento. No entanto, semelhante a outros sistemas de computadores em rede, suporte adicional é necessário para implantar efetivamente aplicativos de rede de sensores. Em sistemas distribuídos, isso assume a forma de *middleware*. Para redes de sensores, podemos, em princípio, seguir uma abordagem semelhante nos casos em que os nós de sensores são suficientemente poderosos e que o consumo de energia não é um obstáculo para executar uma pilha de software mais elaborada. Várias abordagens são possíveis (consulte também [Zhang et al., 2021b]).

De uma perspectiva de programação, e extensivamente pesquisado por Mottola e Picco [2011], é importante levar em conta o escopo das primitivas de comunicação. Esse escopo pode variar entre endereçar a vizinhança física de um nó e fornecer primitivas para comunicação em todo o sistema.

Além disso, também pode ser possível abordar um grupo específico de nós.

Da mesma forma, as computações podem ser restritas a um nó individual, a um grupo de nós ou afetar todos os nós. Para ilustrar, Welsh e Mainland [2004] usam as chamadas **regiões abstratas**, permitindo que um nó identifique uma vizinhança de onde ele pode, por exemplo, coletar informações:

```

1 região = k_nearest_region.create(8); leitura
2 = get_sensor_reading();
3 região.putvar(chave_de_leitura, leitura);
4 max_id = região.reduce(OP_MAXID, chave_de_leitura);

```

Na linha 1, um nó primeiro cria uma região de seus oito vizinhos mais próximos, depois do qual ele busca um valor de seu(s) sensor(es). Essa leitura é subsequentemente escrita na região previamente deŷnida para ser deŷnida usando a chave `reading_key`. Na linha 4, o nó verifica qual leitura do sensor na região deŷnida foi a maior, que é retornada na variável `max_id`.

Ao considerar que as redes de sensores produzem dados, também é possível focar no modelo de acesso a dados. Isso pode ser feito diretamente enviando mensagens para e entre nós, ou movendo código entre nós para acessar dados localmente. Mais avançado é tornar dados remotos diretamente acessíveis, como se variáveis e coisas assim estivessem disponíveis em um espaço de dados compartilhado. Finalmente, e também bastante popular, é deixar a rede de sensores fornecer uma visão de um único banco de dados. Tal visão é fácil de entender ao perceber que muitas redes de sensores são implantadas para aplicações de medição e vigilância [Bonnet et al., 2002]. Nesses casos, um operador gostaria de extrair informações de (uma parte de) a rede simplesmente emitindo consultas como "Qual é a carga de tráfego em direção ao norte na rodovia 1 em Santa Cruz?" Essas consultas se assemelham às de bancos de dados tradicionais. Nesse caso, a resposta provavelmente precisará ser fornecida por meio da colaboração de muitos sensores ao longo da rodovia 1, deixando outros sensores intocados.

Para organizar uma rede de sensores como um banco de dados distribuído, há essencialmente dois extremos, como mostrado na Figura 1.16. Primeiro, os sensores não cooperam, mas simplesmente enviam seus dados para um banco de dados centralizado localizado no site do operador. O outro extremo é encaminhar consultas para sensores relevantes e deixar que cada um calcule uma resposta, exigindo que o operador agregue as respostas.

Nenhuma dessas soluções é muito atraente. A primeira requer que os sensores enviem todos os seus dados medidos pela rede, o que pode desperdiçar recursos e energia da rede. A segunda solução também pode ser um desperdício, pois descarta as capacidades de agregação dos sensores, o que permitiria que muito menos dados fossem retornados ao operador. O que é necessário são facilidades para **processamento de dados na rede**, semelhante ao exemplo anterior de regiões abstratas.

O processamento na rede pode ser feito de várias maneiras. Uma maneira óbvia é encaminhar uma consulta para todos os nós sensores ao longo de uma árvore que abrange todos os nós e, subsequentemente, agregar os resultados à medida que são propagados de volta para a raiz, onde o iniciador está localizado. A agregação ocorrerá onde dois ou mais ramos da árvore se juntam. Por mais simples que esse esquema possa parecer, ele introduz perguntas difíceis:

- Como configuramos (dinamicamente) uma árvore eficiente em uma rede de sensores?
- Como ocorre a agregação de resultados? Pode ser controlada?

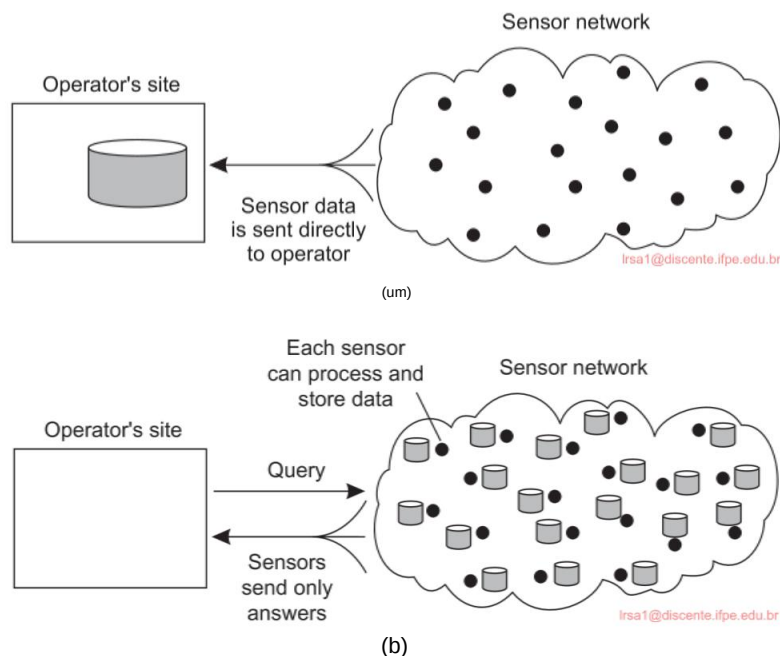


Figura 1.16: Organizando um banco de dados de rede de sensores, enquanto armazena e processa dados (a) somente no local do operador ou (b) somente nos sensores.

- O que acontece quando os links de rede falham?

Essas questões foram parcialmente abordadas no TinyDB, que implementa uma interface declarativa (banco de dados) para redes de sensores sem fio [Madden et al., 2005]. Em essência, o TinyDB pode usar qualquer algoritmo de roteamento baseado em árvore. Um nó intermediário coletará e agregará os resultados de seus filhos, juntamente com suas próprias descobertas, e os enviará para a raiz. Para tornar as coisas eficientes, as consultas abrangem um período de tempo, permitindo um agendamento cuidadoso de operações para que os recursos de rede e a energia sejam consumidos de forma otimizada.

Entretanto, quando as consultas podem ser iniciadas de diferentes pontos na rede, usar árvores de raiz única, como no TinyDB, pode não ser eficiente o suficiente. Como alternativa, as redes de sensores podem ser equipadas com nós especiais para onde os resultados são encaminhados, bem como as consultas relacionadas a esses resultados. Para dar um exemplo simples, consultas e resultados relacionados a leituras de temperatura podem ser coletados em um local diferente daqueles relacionados a medições de umidade. Essa abordagem corresponde diretamente à noção de sistemas de publicação-assinatura.

O aspecto interessante das redes de sensores, conforme discutido ao longo destas linhas, é que realmente precisamos nos concentrar na organização dos nós sensores, e não nos sensores em si. Da mesma forma, muitos nós sensores serão equipados com **atuadores**, ou seja, dispositivos que influenciam diretamente um ambiente. Um típico

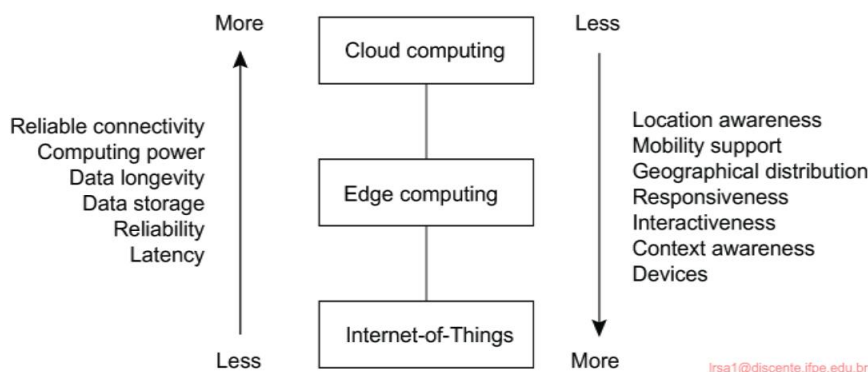


Figura 1.17: Uma visão hierárquica de nuvens para dispositivos (adaptado de Yousef - pour et al. [2019]).

atuador é aquele que controla a temperatura em uma sala, ou liga ou desliga dispositivos. Ao visualizar e organizar a rede como um sistema distribuído, um operador recebe um nível mais alto de abstração para monitorar e controlar uma situação.

Nuvem, borda, coisas

Como já deve ter ficado claro, sistemas distribuídos abrangem uma enorme variedade de diferentes sistemas de computadores em rede. Muitos desses sistemas operam em um ambiente no qual os vários computadores são conectados por meio de uma rede local. No entanto, com o crescimento da Internet das Coisas e a conectividade com serviços remotos oferecidos por meio de sistemas baseados em nuvem, novas organizações em redes de longa distância estão surgindo. A Figura 1.17 apresenta essa abordagem mais hierárquica.

Normalmente, mais acima na hierarquia, vemos que as qualidades típicas dos sistemas distribuídos melhoram: eles se tornam mais confiáveis, têm mais capacidade e, em geral, têm melhor desempenho. Mais abaixo na hierarquia, vemos que os aspectos relacionados à localização são mais facilmente facilitados, assim como as qualidades de desempenho relacionadas às latências. Ao mesmo tempo, as partes mais baixas mostram um aumento no número de dispositivos e computadores, enquanto mais acima, o número de computadores se torna menor.

1.4 Armadilhas

Deve estar claro agora que desenvolver um sistema distribuído é uma tarefa formidável. Como veremos muitas vezes ao longo deste livro, há muitas questões a serem consideradas, enquanto parece que apenas a complexidade pode ser o resultado.

No entanto, seguindo vários princípios de design, é possível desenvolver sistemas distribuídos que aderem fortemente aos objetivos que definimos neste capítulo.

Os sistemas distribuídos diferem do software tradicional porque os componentes são dispersos por uma rede. Não levar essa dispersão em consideração durante o tempo de design é o que torna tantos sistemas desnecessariamente complexos e resulta em falhas que precisam ser corrigidas mais tarde. Peter Deutsch, na época trabalhando na Sun Microsystems, formulou essas falhas como as seguintes suposições falsas que muitos fazem ao desenvolver um aplicativo distribuído pela primeira vez:

- A rede é confiável
- A rede é segura
- A rede é homogênea • A topologia não muda • A latência é zero • A largura de banda é infinita
- O custo do transporte é zero
- Existe um administrador

Observe como essas suposições se relacionam com propriedades que são exclusivas de sistemas dis-distribuídos: confiabilidade, segurança, heterogeneidade e topologia da rede; latência e largura de banda; custos de transporte; e, finalmente, domínios administrativos. Ao desenvolver aplicativos não distribuídos, a maioria desses problemas provavelmente não aparecerá.

A maioria dos princípios que discutimos neste livro se relacionam imediatamente a essas suposições. Em todos os casos, discutiremos soluções para problemas que são causados pelo fato de uma ou mais suposições serem falsas. Por exemplo, redes confiáveis simplesmente não existem e levam à impossibilidade de atingir a transparência de falhas. Dedicamos um capítulo inteiro para lidar com o fato de que a comunicação em rede é inerentemente insegura. Já argumentamos que os sistemas distribuídos precisam ser abertos e levar a heterogeneidade em consideração. Da mesma forma, ao discutir replicação para resolver problemas de escalabilidade, estamos essencialmente lidando com problemas de latência e largura de banda. Também abordaremos questões de gerenciamento em vários pontos ao longo deste livro.

1.5 Resumo

Um sistema distribuído é uma coleção de sistemas de computadores em rede nos quais processos e recursos são espalhados por diferentes computadores. Fazemos uma distinção entre suicientemente e necessariamente espalhados, onde o último se relaciona a sistemas descentralizados. Essa distinção é importante de se fazer, pois espalhar processos e recursos não pode ser considerado um objetivo por si só. Em vez disso,

a maioria das escolhas para chegar a um sistema distribuído vem da necessidade de melhorar o desempenho de um único sistema de computador em termos de, por exemplo, confiabilidade, escalabilidade e eficiência. No entanto, considerando que a maioria dos sistemas centralizados ainda são muito mais fáceis de gerenciar e manter, deve-se pensar duas vezes antes de decidir espalhar processos e recursos. Também há casos em que simplesmente não há escolha, por exemplo, ao conectar sistemas pertencentes a diferentes organizações, ou quando os computadores simplesmente operam de diferentes locais (como na computação móvel).

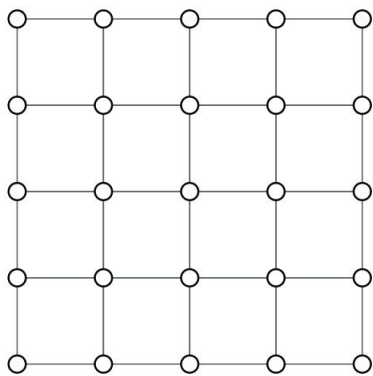
Os objetivos de design para sistemas distribuídos incluem compartilhar recursos e garantir abertura. Cada vez mais importante é projetar sistemas distribuídos seguros. Além disso, os designers visam esconder muitas das complexidades relacionadas à distribuição de processos, dados e controle. No entanto, essa transparência de distribuição não só tem um preço de desempenho, em situações práticas, ela nunca pode ser totalmente alcançada. O fato de que compensações precisam ser feitas entre alcançar várias formas de transparência de distribuição é inerente ao design de sistemas distribuídos e pode facilmente complicar sua compreensão. Uma meta de design difícil específica que nem sempre combina bem com a obtenção de transparência de distribuição é a escalabilidade. Isso é particularmente verdadeiro para a escalabilidade geográfica, caso em que ocultar latências e restrições de largura de banda pode acabar sendo difícil. Da mesma forma, a escalabilidade administrativa, pela qual um sistema é projetado para abranger vários domínios administrativos, pode facilmente entrar em conflito com as metas para alcançar a transparência de distribuição.

Existem diferentes tipos de sistemas distribuídos que podem ser classificados como orientados para dar suporte a computações, processamento de informações e difusão. Os sistemas de computação distribuída são normalmente implantados para aplicativos de alto desempenho, geralmente originários do campo da computação paralela. Um campo que surgiu do processamento paralelo foi inicialmente a computação em grade com um forte foco no compartilhamento mundial de recursos, levando ao que agora é conhecido como computação em nuvem. A computação em nuvem vai além da computação de alto desempenho e também dá suporte a sistemas distribuídos encontrados em ambientes de escritório tradicionais, onde vemos bancos de dados desempenhando um papel importante. Normalmente, os sistemas de processamento de transações são implantados nesses ambientes. Finalmente, uma classe emergente de sistemas distribuídos é onde os componentes são pequenos, o sistema é composto de forma ad hoc, mas acima de tudo não é mais gerenciado por um administrador de sistema. Esta última classe é normalmente representada por ambientes de computação difundidos, incluindo sistemas de computação móvel, bem como ambientes ricos em sensores.

As coisas são ainda mais complicadas pelo fato de que muitos desenvolvedores inicialmente fazem suposições sobre a rede subjacente que são fundamentalmente erradas. Mais tarde, quando as suposições são descartadas, pode se tornar difícil mascarar comportamento indesejado. Um exemplo típico é assumir que a latência da rede não é significativa. Outras armadilhas incluem assumir que a rede é confiável, estática, segura e homogênea.

02

ARQUITETURAS



Sistemas distribuídos são frequentemente peças complexas de software, cujos componentes são por definição dispersos em múltiplas máquinas. Para dominar sua complexidade, é crucial que esses sistemas sejam organizados adequadamente. Existem diferentes maneiras de visualizar a organização de um sistema distribuído, mas uma óbvia é fazer uma distinção entre, por um lado, a organização lógica da coleção de componentes de software e, por outro lado, a realização física real.

A organização de sistemas distribuídos é principalmente sobre os componentes de software que constituem o sistema. Essas **arquiteturas de software** nos dizem como os vários componentes de software devem ser organizados e como eles devem interagir. Neste capítulo, primeiro daremos atenção a alguns estilos de arquitetura comumente aplicados para organizar sistemas de computadores (distribuídos).

Um objetivo importante dos sistemas distribuídos é separar os aplicativos das plataformas subjacentes, fornecendo uma chamada camada de middleware. Adotar tal camada é uma decisão arquitetônica importante, e seu principal propósito é fornecer transparência de distribuição. No entanto, compensações precisam ser feitas para atingir a transparência, o que levou a várias técnicas para ajustar o middleware às necessidades dos aplicativos que o utilizam. Discutimos algumas das técnicas mais comumente aplicadas, pois elas afetam a organização do próprio middleware.

A realização real de um sistema distribuído requer que instanciemos e coloquemos componentes de software em máquinas reais. Há muitas escolhas que podem ser feitas ao fazer isso. A instanciação final de uma arquitetura de software também é chamada de **arquitetura de sistema**. Neste capítulo, examinaremos arquiteturas centralizadas tradicionais nas quais um único servidor implementa a maioria dos componentes de software (e, portanto, a funcionalidade), enquanto clientes remotos podem acessar esse servidor usando meios de comunicação simples. Além disso, consideramos arquiteturas ponto a ponto descentralizadas nas quais todos os nós desempenham mais ou menos papéis iguais. Muitos sistemas distribuídos do mundo real são frequentemente organizados de forma híbrida, combinando elementos de arquiteturas centralizadas e descentralizadas. Discutimos vários exemplos que ilustram a complexidade de muitos sistemas distribuídos do mundo real.

2.1 Estilos arquitetônicos

Começamos nossa discussão sobre arquiteturas considerando primeiro a organização lógica de um sistema distribuído em componentes de software, também conhecida como **arquitetura de software** [Bass et al., 2021; Richards e Ford, 2020].

A pesquisa sobre arquiteturas de software amadureceu consideravelmente, e agora é comumente aceito que projetar ou adotar uma arquitetura é crucial para o desenvolvimento bem-sucedido de grandes sistemas de software.

Para nossa discussão, a noção de um **estilo arquitetônico** é importante. Tal estilo é formulado em termos de componentes, a maneira como os componentes são

conectados entre si, os dados trocados entre componentes e, por fim, como esses elementos são conÿgurados conjuntamente em um sistema. Um **componente** é uma unidade modular com **interfaces** necessárias e fornecidas bem deÿnidas que é substituível dentro de seu ambiente [OMG, 2004]. Que um componente possa ser substituído e, em particular, enquanto um sistema continua a operar, é importante.

Isso ocorre porque muitas vezes não é uma opção desligar um sistema para manutenção. Na melhor das hipóteses, apenas partes dele podem ser colocadas temporariamente fora de ordem. A substituição de um componente pode ser feita apenas se suas interfaces permanecerem intocadas. Na prática, vemos que substituir ou atualizar um componente significa que uma parte de um sistema (como um servidor) executa uma atualização regular e alterna para os componentes atualizados assim que sua instalação for concluída. Medidas especiais podem precisar ser tomadas quando uma parte do sistema distribuído precisa ser reiniciada para permitir que as atualizações entrem em vigor. Essas medidas podem incluir ter standbys replicados que assumem o controle enquanto a reinicialização parcial está ocorrendo.

Um conceito um pouco mais difícil de entender é o de um **conector**, que é geralmente descrito como um mecanismo que media a comunicação, coordenação ou cooperação entre componentes [Bass et al., 2021]. Por exemplo, um conector pode ser formado pelas facilidades para chamadas de procedimento (remotas), passagem de mensagens ou streaming de dados. Em outras palavras, um conector permite o ÿuxo de controle e dados entre componentes.

Usando componentes e conectores, podemos chegar a várias conÿgurações, que, por sua vez, foram classiÿcadas em estilos arquitetônicos. Vários estilos já foram identiÿcados, dos quais os mais importantes para sistemas distribuídos são:

- Arquiteturas em camadas
- Arquiteturas orientadas a serviços
- Arquiteturas de publicação-assinatura

A seguir, discutiremos cada um desses estilos separadamente. Notamos antecipadamente que na maioria dos sistemas distribuídos do mundo real, muitos estilos são combinados.

Notavelmente, seguir uma abordagem pela qual um sistema é subdividido em várias camadas (lógicas) é um princípio tão universal que geralmente é combinado com a maioria dos outros estilos arquitetônicos.

2.1.1 Arquiteturas em camadas

A ideia básica para o estilo em camadas é simples: os componentes são organizados de **forma em camadas**, onde um componente na camada L_j pode fazer uma **chamada descendente** para um componente em uma camada de nível inferior L_i (com $i < j$) e geralmente espera uma resposta. Somente em casos excepcionais uma **chamada ascendente** será feita para um componente de nível superior. Os três casos comuns são mostrados na Figura 2.1.

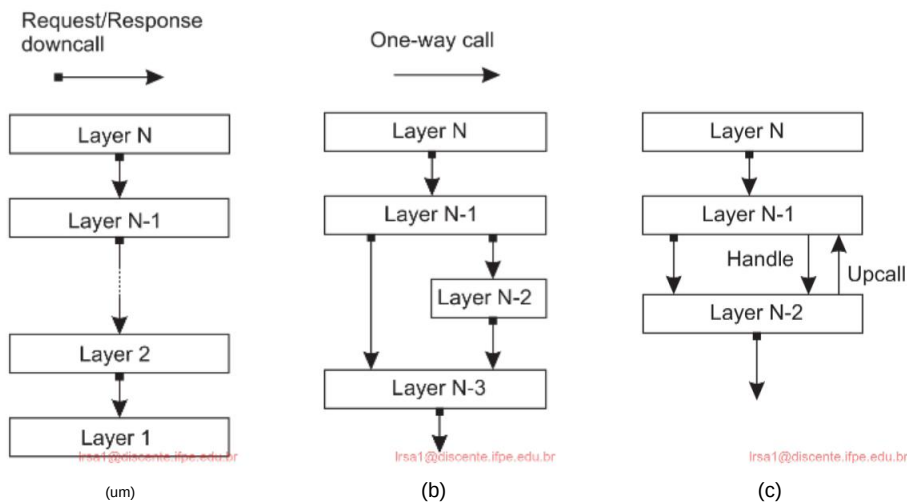


Figura 2.1: (a) Organização em camadas pura. (b) Organização em camadas mista. (c) Organização em camadas com upcalls (adotado de [Krakowiak, 2009]).

A Figura 2.1(a) mostra uma organização padrão na qual apenas downcalls para a próxima camada inferior são feitos. Essa organização é comumente implantada no caso de comunicação de rede.

Em muitas situações, também encontramos a organização mostrada na Figura 2.1 (b). Considere, por exemplo, um aplicativo A que faz uso de uma biblioteca LOS para fazer interface com um sistema operacional. Ao mesmo tempo, o aplicativo usa uma biblioteca matemática especializada Lmath que foi implementada também fazendo uso de LOS. Neste caso, referindo-se à Figura 2.1(b), A é implementado na camada N e 1, Lmath na camada N e 2 e LOS que é comum a ambos, na camada N e 3.

Finalmente, uma situação especial é mostrada na Figura 2.1(c). Em alguns casos, é conveniente ter uma camada inferior fazendo uma chamada ascendente para sua próxima camada superior. Um exemplo típico é quando um sistema operacional sinaliza a ocorrência de um evento, para o qual ele chama uma operação definida pelo usuário para a qual um aplicativo havia passado anteriormente uma referência (tipicamente chamada de handle).

Protocolos de comunicação em camadas

Uma arquitetura em camadas bem conhecida e aplicada ubiqüamente é a das pilhas de protocolos de **comunicação**. Vamos nos concentrar aqui apenas no quadro global e adiar uma discussão detalhada para a Seção 4.1.1.

Em pilhas de protocolos de comunicação, cada camada implementa um ou vários **serviços de comunicação** permitindo que dados sejam enviados de um destino para um ou vários alvos. Para esse fim, cada camada oferece uma **interface** especificando o

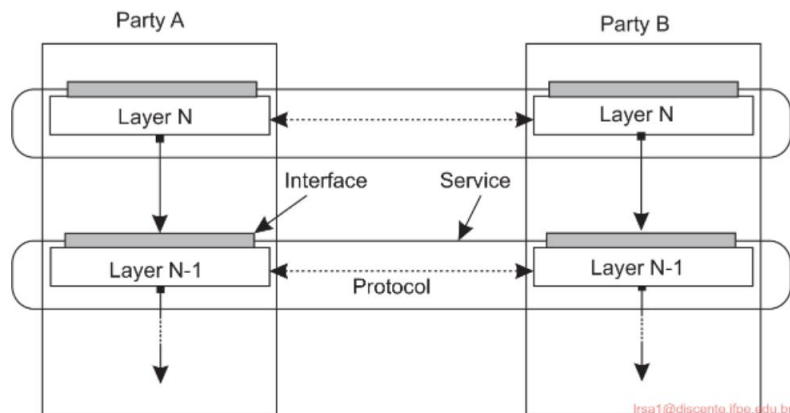


Figura 2.2: Uma pilha de protocolo de comunicação em camadas, mostrando a diferença entre um serviço, sua interface e o protocolo que ele implementa.

funções que podem ser chamadas. Em princípio, a interface deve ocultar completamente a implementação real de um serviço. Outro conceito importante no caso de comunicação é o de um **protocolo (de comunicação)**, que descreve as regras que as partes seguirão para trocar informações. É importante entender a diferença entre um serviço oferecido por uma camada, a interface pela qual esse serviço é disponibilizado e o protocolo que uma camada implementa para estabelecer comunicação. Essa distinção é mostrada na Figura 2.2.

Para deixar essa distinção clara, considere um serviço confiável e orientado à conexão, que é fornecido por muitos sistemas de comunicação. Nesse caso, uma parte comunicante precisa primeiro configurar uma conexão com outra parte antes que as duas possam enviar e receber mensagens. Ser confiável significa que garantias fortes serão dadas de que as mensagens enviadas serão de fato entregues ao outro lado, mesmo quando houver um alto risco de que as mensagens possam ser perdidas (como, por exemplo, pode ser o caso ao usar um meio sem fio). Além disso, esses serviços geralmente também garantem que as mensagens sejam entregues na mesma ordem em que foram enviadas.

Esse tipo de serviço é realizado na Internet pelo **Transmission Control Protocol (TCP)**. O protocolo especifica quais mensagens devem ser trocadas para configurar ou interromper uma conexão, o que precisa ser feito para preservar a ordem dos dados transferidos e o que ambas as partes precisam fazer para detectar e corrigir dados perdidos durante a transmissão. O serviço é disponibilizado na forma de uma interface de programação relativamente simples, contendo chamadas para configurar uma conexão, enviar e receber mensagens e interromper a conexão novamente. Na verdade, há diferentes interfaces disponíveis, geralmente dependentes do sistema operacional ou da linguagem de programação usada. Da mesma forma, há muitas implementações do protocolo e suas interfaces. (Todos os detalhes sangrentos podem ser encontrados em [Stevens, 1994; Wright e Stevens, 1995].)

Nota 2.1 (Exemplo: Duas partes comunicantes)

Para tornar essa distinção entre serviço, interface e protocolo mais concreta, considere as duas partes comunicantes a seguir, também conhecidas como **cliente** e **servidor**, respectivamente, expressas em Python (observe que parte do código foi removido para maior clareza).

```

1 de importação de soquete *
2
3 s = socket(AF_INET, SOCK_STREAM) 4
(conn, addr) = s.accept() # retorna novo socket e endereço. cliente 5 while True:
# para sempre
6 data = conn.recv(1024) # receber dados do cliente 7 if not data: break #
parar se o cliente parou
8 msg = data.decode()+"*" # processa os dados recebidos em uma resposta 9
conn.send(msg.encode()) # retorna a resposta
10 conexão.fechar() # feche a conexão

```

(a) Um servidor simples

```

1 de importação de soquete *
2
3 s = socket(AF_INET, SOCK_STREAM) 4
s.connect((HOST, PORT)) # conectar ao servidor (bloquear até ser aceito) 5 msg = "Olá
Mundo" 6
# compor uma mensagem
s.send(msg.encode()) 7 data
# enviar a mensagem #
= s.recv(1024) 8
receber a resposta # imprimir
print(data.decode()) 9 s.close()
o resultado # fechar a
conexão

```

(b) Um cliente

Figura 2.3: Duas partes comunicantes.

Neste exemplo, é criado um servidor que faz uso de um **serviço orientado a conexão**, como oferecido pela biblioteca de socket disponível em Python. Este serviço permite que duas partes comunicantes enviem e recebam dados de forma confiável por uma conexão. As principais funções disponíveis em sua interface são:

- `socket()`: para criar um objeto que representa a conexão
- `accept()`: uma chamada de bloqueio para aguardar solicitações de conexão de entrada; se bem-sucedida, a chamada retorna um novo soquete para uma conexão separada
- `connect()`: para estabelecer uma conexão com uma parte específica
- `close()`: para interromper uma conexão
- `send()`, `recv()`: para enviar e receber dados por meio de uma conexão, respectivamente

A combinação das constantes `AF_INET` e `SOCK_STREAM` é usada para especificar que o protocolo TCP deve ser usado na comunicação entre as duas partes. Essas duas constantes podem ser vistas como parte da interface, enquanto fazer uso do TCP é parte do serviço oferecido. Como o TCP é implementado, ou, nesse caso, qualquer parte do serviço de comunicação, é completamente oculto dos aplicativos.

Por fim, observe também que esses dois programas aderem implicitamente a um protocolo de nível de aplicação: aparentemente, se o cliente enviar alguns dados, o servidor os retornará. Na verdade, ele opera como um servidor de eco, onde o servidor adiciona um asterisco aos dados enviados pelo cliente.

Camadas de aplicação

Vamos agora voltar nossa atenção para a estratificação lógica de aplicativos. Considerando que uma grande classe de aplicativos distribuídos é direcionada para dar suporte ao acesso de usuários ou aplicativos a bancos de dados, muitas pessoas têm defendido uma distinção entre três níveis lógicos, essencialmente seguindo um estilo arquitetônico em camadas :

- O nível da interface de aplicação • O
- nível de processamento
- O nível de dados

Em linha com essa estratificação, vemos que os aplicativos podem frequentemente ser construídos a partir de aproximadamente três partes diferentes: uma parte que lida com a interação com um usuário ou algum aplicativo externo, uma parte que opera em um banco de dados ou sistema de arquivos e uma parte do meio que geralmente contém a funcionalidade principal do aplicativo. Essa parte do meio é logicamente colocada no nível de processamento. Em contraste com interfaces de usuário e bancos de dados, não há muitos aspectos comuns ao nível de processamento. Portanto, daremos vários exemplos para tornar esse nível mais claro.

Como primeiro exemplo, considere um mecanismo de busca simples na Internet, por exemplo, um dedicado à compra de casas. Esses mecanismos de busca aparecem como sites aparentemente simples por meio dos quais alguém pode fornecer descritores como uma cidade ou região, uma faixa de preço, o tipo de casa, etc. O back-end consiste em um enorme banco de dados de casas atualmente à venda. A camada de processamento não faz nada além de transformar os descritores fornecidos em uma coleção de consultas de banco de dados, recupera as respostas e pós-processa essas respostas, por exemplo, classificando a saída por relevância e, subsequentemente, gerando uma página HTML. A Figura 2.4 mostra essa organização.

Como outro exemplo, considere a organização do site deste livro, em particular, a interface que permite que alguém obtenha uma cópia digital personalizada do livro em PDF. Neste caso, a interface consiste em um servidor Web baseado em WordPress que meramente coleta o endereço de e-mail do usuário (e algumas informações sobre exatamente qual versão do livro está sendo solicitada). Essas informações são anexadas internamente a um arquivo requests.txt. A camada de dados é simples: ela meramente consiste em uma coleção de arquivos e figuras LATEX que, em conjunto, constituem o livro inteiro.

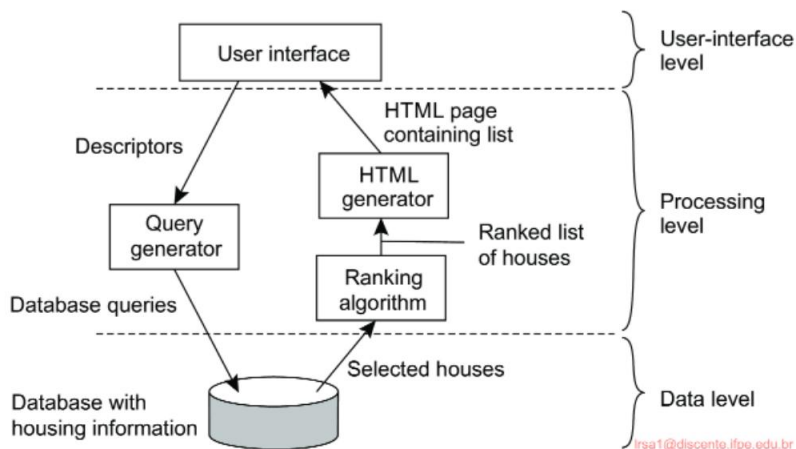


Figura 2.4: A organização simplificada de um mecanismo de busca na Internet para habitação, em três camadas diferentes.

Fazer uma cópia personalizada consiste em incorporar o endereço de e-mail do usuário em cada uma das páginas. Para esse fim, uma vez a cada cinco minutos, um processo separado é iniciado, que pega a lista de solicitações e, uma por uma, adiciona o endereço de e-mail do solicitante em cada página bitmap, gera uma nova cópia do livro, armazena o PDF gerado em um local especial (acessível por meio de uma URL exclusiva) e envia ao solicitante um e-mail informando que a cópia está disponível para download. Esse processo continua até que todas as solicitações tenham sido tratadas. Nesse exemplo, vemos que a camada de processamento supera a camada de dados ou a camada de interface de aplicativo em termos de esforços computacionais e ações a serem tomadas.

2.1.2 Arquiteturas orientadas a serviços

Embora o estilo arquitetônico em camadas seja popular, uma de suas principais desvantagens é a dependência frequentemente forte entre diferentes camadas. Bons exemplos em que essas dependências potenciais foram cuidadosamente consideradas são encontrados no design de pilhas de protocolos de comunicação. Exemplos ruins incluem aplicativos que foram essencialmente projetados e desenvolvidos como composições de componentes existentes sem muita preocupação com a estabilidade das interfaces ou dos próprios componentes, muito menos com a sobreposição de funcionalidade entre diferentes componentes. (Um exemplo convincente é dado por Kucharski [2020], que descreve a dependência de um componente simples que preenche uma determinada string com zeros ou espaços. O autor retirou o componente da biblioteca NPM, deixando milhares de programas afetados.)

Essas dependências diretas de componentes específicos levaram a um estilo arquitetônico que reflete uma organização mais flexível em uma coleção de componentes separados,

entidades independentes. Cada entidade encapsula um serviço. Sejam chamadas de **serviços**, **objetos** ou **microserviços**, elas têm em comum que o serviço é executado como um processo separado (ou thread). Claro, executar entidades separadas não necessariamente reduz dependências em comparação a um estilo arquitetônico em camadas.

Estilo arquitetônico baseado em objetos

Tomando a abordagem baseada em objeto como exemplo, temos uma organização lógica como mostrado na Figura 2.5. Em essência, cada objeto corresponde ao que deñimos como um componente, e esses componentes são conectados por meio de um mecanismo de chamada de procedimento. No caso de sistemas distribuídos, uma chamada de procedimento também pode ocorrer em uma rede, ou seja, o objeto chamador não precisa ser executado na mesma máquina que o objeto chamado. De fato, onde exatamente o objeto chamado está localizado pode ser transparente para o chamador: o objeto chamado pode igualmente ser executado como um processo separado na mesma máquina.

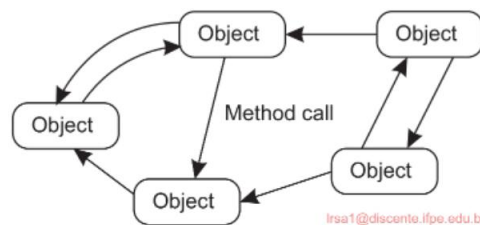


Figura 2.5: Um estilo arquitetônico baseado em objetos.

Arquiteturas baseadas em objetos são atraentes porque fornecem uma maneira natural de **encapsular** dados (chamados de **estado de um objeto**) e as operações que podem ser executadas nesses dados (que são chamadas de **métodos de um objeto**) em uma única entidade. A **interface** oferecida por um objeto oculta detalhes de implementação, significando essencialmente que nós, em princípio, podemos considerar um objeto completamente independente de seu ambiente. Assim como com componentes, isso também significa que se a interface for claramente deñnida e deixada intocada de outra forma, um objeto deve ser substituível por um que tenha a mesma interface.

Essa separação entre interfaces e os objetos que implementam essas interfaces nos permite colocar uma interface em uma máquina, enquanto o objeto em si reside em outra máquina. Essa organização, que é mostrada na Figura 2.6, é comumente referida como um **objeto distribuído**, ou de vez em quando um **objeto remoto**.

Quando um cliente **se vincula** a um objeto distribuído, uma implementação da interface do objeto, chamada **proxy**, é carregada no espaço de endereço do cliente. Um proxy é análogo a um chamado **stub de cliente** em sistemas RPC. A única coisa que ele faz é empacotar invocações de método em mensagens e descompactar mensagens de resposta para retornar o resultado da invocação de método ao cliente. O objeto real

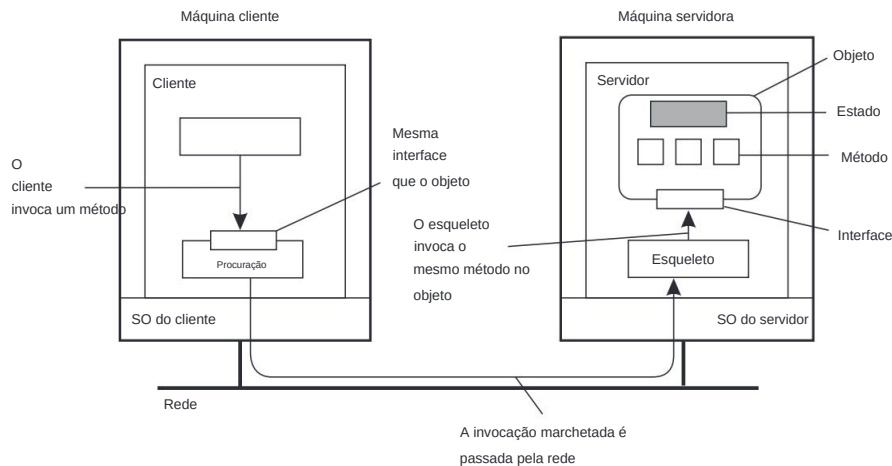


Figura 2.6: Organização comum de um objeto remoto com proxy do lado do cliente.

reside em uma máquina servidora, onde oferece a mesma interface que oferece na máquina cliente. As solicitações de invocação recebidas são primeiro passadas para um stub do servidor, que as descompacta para fazer invocações de método na interface do objeto no servidor. O stub do servidor também é responsável por empacotar valores de retorno em uma mensagem e encaminhar essas mensagens de resposta para o proxy do lado do cliente.

O stub do lado do servidor é frequentemente chamado de **esqueleto**, pois fornece os meios básicos para permitir que o middleware do servidor acesse os objetos definidos pelo usuário. Na prática, ele geralmente contém código incompleto na forma de uma classe específica da linguagem que precisa ser mais especializada pelo desenvolvedor.

Uma característica, mas um tanto contraintuitiva, da maioria dos objetos distribuídos é que seu estado não é distribuído: ele reside em uma única máquina. Apenas as interfaces implementadas pelo objeto são disponibilizadas em outras máquinas. Tais objetos também são chamados de **objetos remotos**. Em um objeto distribuído geral, o próprio estado pode ser fisicamente distribuído por várias máquinas, mas essa distribuição também é ocultada dos clientes por trás das interfaces do objeto.

Estilo arquitetônico de microserviço

Pode-se argumentar que arquiteturas baseadas em objetos formam a base do encapsulamento de serviços em unidades independentes. **Encapsulamento** é a palavra-chave aqui: o serviço como um todo é percebido como uma entidade autocontida, embora possa possivelmente fazer uso de outros serviços. Ao separar claramente vários serviços de modo que eles possam operar independentemente, estamos pavimentando o caminho em direção a **arquiteturas orientadas a serviços**, geralmente abreviadas como **SOAs**.

Estimulados por projetos orientados a objetos e inspirados pela abordagem Unix, na qual muitos, muitos programas pequenos e mutuamente independentes podem ser facilmente

compostos para formar programas maiores, arquitetos de software têm trabalhado no que são chamados de **microsserviços**. O essencial é que cada microsserviço seja executado como um processo separado (rede). A implementação de um microsserviço pode ser na forma de um objeto remoto, mas isso não é um requisito. Além disso, apesar de as pessoas falarem de microsserviços, não há um acordo comum sobre qual deve ser o tamanho de tal serviço. O mais importante, no entanto, é que um microsserviço realmente represente um serviço separado e independente. Em outras palavras, a modularização é a chave para projetar microsserviços [Wolff, 2017].

No entanto, o tamanho importa. Ao já declarar que os microsserviços são executados como processos em rede separados, também nos é dada uma escolha de onde colocar um microsserviço. Como veremos mais adiante neste capítulo, no advento das infraestruturas edge e fog, as discussões começaram sobre a **orquestração** da implantação de aplicativos distribuídos em diferentes camadas. Em outras palavras, onde colocamos o quê.

Composição de serviço de granulação grossa

Em uma arquitetura orientada a serviços, um aplicativo ou sistema distribuído é essencialmente construído como uma composição de muitos serviços. Uma diferença (embora não estrita) com microsserviços é que nem todos esses serviços podem pertencer à mesma organização administrativa. Já nos deparamos com esse fenômeno ao discutir computação em nuvem: pode muito bem ser que uma organização executando seu aplicativo de negócios faça uso de serviços de armazenamento oferecidos por um provedor de nuvem. Esses serviços de armazenamento são logicamente completamente encapsulados em uma única unidade, da qual uma interface é disponibilizada aos clientes.

Claro, o armazenamento é um serviço bastante básico, mas situações mais sofisticadas facilmente vêm à mente. Considere, por exemplo, uma loja virtual que vende produtos como e-books. Uma implementação simples, seguindo a camada de aplicação que discutimos anteriormente, pode consistir em uma aplicação para processamento de pedidos, que, por sua vez, opera em um banco de dados local contendo os e-books. O processamento de pedidos normalmente envolve selecionar itens, registrar e verificar o canal de entrega (talvez usando e-mail), mas também certificar-se de que um pagamento seja feito. Este último pode ser tratado por um serviço separado, executado por uma organização diferente, para o qual um cliente comprador é redirecionado para o pagamento, após o qual a organização de e-books é notificada para que possa concluir a transação. Este exemplo também ilustra que, onde os microsserviços são considerados relativamente pequenos, pode-se esperar que um serviço geral seja relativamente grande. Na verdade, não é incomum implementar um serviço como uma coleção de microsserviços.

Dessa forma, vemos que o problema de desenvolver um sistema distribuído é, em parte, um problema de composição de serviços e de garantir que esses serviços operem em harmonia. De fato, esse problema é completamente análogo aos problemas de integração de aplicativos corporativos discutidos na Seção 1.3.2. Crucial é, e continua sendo, que cada serviço ofereça uma interface (de programação) bem definida. Em

Na prática, isso também significa que cada serviço oferece sua própria interface, o que pode tornar a composição dos serviços longe de ser trivial.

Arquiteturas baseadas em recursos

À medida que um número crescente de serviços se tornou disponível pela Web e o desenvolvimento de sistemas distribuídos por meio da composição de serviços se tornou mais importante, os pesquisadores começaram a repensar a arquitetura de sistemas distribuídos, principalmente baseados na Web. Um dos problemas com a composição de serviços é que conectar vários componentes pode facilmente se transformar em um pesadelo de integração.

Como alternativa, também se pode ver um sistema distribuído como uma enorme coleção de recursos que são gerenciados individualmente por componentes. Os recursos podem ser adicionados ou removidos por aplicativos (remotos) e, da mesma forma, podem ser recuperados ou modificados. Essa abordagem agora foi amplamente adotada para a Web e é conhecida como **Representational State Transfer (REST)** [Fielding, 2000]. Existem quatro características principais do que são conhecidas como **arquiteturas RESTful** [Pautasso et al., 2008]:

- 1. Os recursos são identificados por meio de um único esquema de nomenclatura
- 2. Todos os serviços oferecem a mesma interface, consistindo em no máximo quatro operações, conforme mostrado na Figura 2.7
- 3. As mensagens enviadas para ou de um serviço são totalmente autodescritas
- 4. Após executar uma operação em um serviço, esse componente esquece tudo sobre o chamador

A última propriedade também é chamada de **execução sem estado**, um conceito ao qual retornaremos no Capítulo 3.

Descrição da operação	
COLOCAR	Modificar um recurso transferindo um novo estado
PUBLICAR	Criar um novo recurso
PEGAR	Recuperar o estado de um recurso em alguma representação
EXCLUIR	Excluir um recurso

Figura 2.7: As quatro operações disponíveis em arquiteturas RESTful.

Para ilustrar como o RESTful pode funcionar na prática, considere um serviço de armazenamento em nuvem, como o **Simple Storage Service** da Amazon (**Amazon S3**). O Amazon S3, descrito em [Murty, 2008] e mais recentemente em [Culkin e Zazon, 2022], oferece suporte a dois recursos: objetos, que são essencialmente o equivalente a arquivos, e buckets, o equivalente a diretórios. Não há conceito de colocar buckets em buckets. Um objeto chamado `ObjectName` contido em um bucket `BucketName` é referenciado pelo seguinte **Uniform Resource Identifier (URI)**:

`https://s3.amazonaws.com/BucketName/ObjectName`

Para criar um bucket, ou um objeto, um aplicativo basicamente envie uma solicitação PUT com o URI do bucket/objeto. Em princípio, o protocolo que é usado com o serviço é HTTP. Em outras palavras, é apenas outro HTTP solicitação, que posteriormente será interpretada corretamente pelo S3. Se o bucket ou o objeto já existe, uma mensagem de erro HTTP é retornada.

Da mesma forma, para saber quais objetos estão contidos em um bucket, um aplicativo enviaria uma solicitação GET com o URI desse bucket. O S3 retornará uma lista de nomes de objetos, novamente como uma resposta HTTP comum.

A arquitetura RESTful se tornou popular devido à sua simplicidade. Pautasso et al. [2008] compararam serviços RESTful a serviços específicos interfaces e, como era de se esperar, ambas têm suas vantagens e desvantagens. Em particular, a simplicidade das arquiteturas RESTful pode facilmente proibir soluções fáceis para esquemas de comunicação intrincados. Um exemplo é onde transações distribuídas são necessárias, o que geralmente requer que os serviços acompanhem o estado da execução. Por outro lado, há muitos exemplos em que arquiteturas RESTful combinam perfeitamente com uma integração simples esquema de serviços, mas onde a miríade de interfaces de serviço complicará assuntos.

Nota 2.2 (Avançado: Sobre interfaces)
É claro que um serviço não pode ser facilitado ou dificultado apenas por causa da interface específica que oferece. Um serviço oferece funcionalidade e, na melhor das hipóteses, a maneira que o serviço é acessado é determinado pela interface. Na verdade, pode-se argumentar que a discussão sobre interfaces RESTful versus interfaces específicas de serviço é muito sobre transparência de acesso. Para entender melhor por que tantas pessoas estão pagando atenção a esta questão, vamos focar-nos no serviço **Amazon S3**, que oferece uma Interface REST, bem como uma interface mais tradicional (chamada de SOAP interface). O fato de que este último foi descontinuado diz muito.

Operações de balde	Operações de objeto
ListarTodosOsMeusBuckets	ColocarObjetoEmLinha
CriarBucket	ColocarObjeto
ExcluirBalde	CopiarObjeto
ListaBucket	ObterObjeto
ObterBucketAccessControlPolicy	ObterObjectExtended
DefinirBucketAccessControlPolicy	ExcluirObjeto
Obter status de registro do Bucket	ObterPolítica de Controle de Acesso a Objetos
Definir status de registro do bucket	DefinirPolítica de Controle de Acesso a Objetos

Figura 2.8: As operações na interface SOAP S3 da Amazon, até agora obsoleto.

A interface SOAP consiste em aproximadamente 16 operações, listadas na Figura 2.8. No entanto, se acessássemos o Amazon S3 usando a biblioteca Python boto3, teríamos mais de 100 operações disponíveis. Em contraste, a interface REST

oferece apenas pouquíssimas operações, essencialmente aquelas listadas na Figura 2.7. De onde vêm essas diferenças? A resposta está, é claro, no espaço de parâmetros. No caso de arquiteturas RESTful, um aplicativo precisará fornecer tudo o que deseja por meio dos parâmetros que passa por uma das operações. Na interface SOAP da Amazon, o número de parâmetros por operação é geralmente limitado, e esse certamente é o caso se usássemos a biblioteca Python boto3.

Seguindo os princípios (para que possamos evitar as complexidades do código real), suponha que temos um bucket de interface que oferece uma operação create, exigindo uma string de entrada como mybucket, para criar um bucket com o nome "mybucket".

Normalmente, a operação seria chamada aproximadamente da seguinte forma:

```
balde de importação
bucket.create("mybucket")
```

Entretanto, em uma arquitetura RESTful, a chamada precisaria ser essencialmente codificada como uma única string, como

```
COLOQUE "https://mybucket.s3.amazonaws.com/"
```

A diferença é impressionante. Por exemplo, no primeiro caso, muitos erros sintáticos podem frequentemente já ser capturados durante o tempo de compilação, enquanto no segundo caso, a verificação precisa ser adiada até o tempo de execução. Em segundo lugar, pode-se argumentar que especificar a semântica de uma operação é muito mais fácil com interfaces específicas do que com aquelas que oferecem apenas operações genéricas. Por outro lado, com operações genéricas, as mudanças são muito mais fáceis de acomodar, pois geralmente envolveriam a alteração do layout de strings que codificam o que é realmente necessário.

2.1.3 Arquiteturas de publicação-assinatura

À medida que os sistemas continuam a crescer e os processos podem entrar ou sair mais facilmente, torna-se importante ter uma arquitetura na qual as dependências entre os processos se tornem o mais soltas possível. Uma grande classe de sistemas distribuídos adotou uma arquitetura na qual há uma forte separação entre processamento e coordenação. A ideia é visualizar um sistema como uma coleção de processos operando de forma autônoma. Neste modelo, a **coordenação** abrange a comunicação e a cooperação entre os processos. Ela forma a cola que une as atividades realizadas pelos processos em um todo [Gelernter e Carriero, 1992].

Cabri et al. [2000] fornecem uma taxonomia de modelos de coordenação que podem ser aplicados igualmente a muitos tipos de sistemas distribuídos. Adaptando ligeiramente sua terminologia, fazemos uma distinção entre modelos ao longo de duas dimensões diferentes, temporal e referencial, conforme mostrado na Figura 2.9.

Quando os processos são acoplados temporal e referencialmente, a coordenação ocorre diretamente, referida como **coordenação direta**. O acoplamento referencial geralmente aparece na forma de referência explícita na comunicação. Por exemplo, um processo pode se comunicar somente se souber o nome ou identificador de

	Temporalmente acoplado	Temporalmente desacoplado
Acoplado referencialmente	Direto	Caixa de correio
Desacoplado referencialmente	Baseado em eventos	Compartilhado espaço de dados

Figura 2.9: Exemplos de diferentes formas de coordenação.

os outros processos com os quais deseja trocar informações. Acoplamento temporal significa que os processos que estão se comunicando terão que estar ativos e correndo. Na vida real, falar por celulares (e assumindo que um celular tem apenas um dono), é um exemplo de comunicação direta.

Um tipo diferente de coordenação ocorre quando os processos são desacoplados temporalmente, mas acoplados referencialmente, o que chamamos de **coordenação de caixa de correio**. Neste caso, não há necessidade de dois processos comunicantes estarem ativos ao mesmo tempo para permitir que a comunicação ocorra. Em vez disso, a comunicação ocorre colocando mensagens em uma caixa de correio (possivelmente compartilhada). Porque é necessário endereçar explicitamente a caixa de correio que conterá as mensagens que devem ser trocados, há um acoplamento referencial.

A combinação de sistemas referencialmente desacoplados e temporalmente acoplados formam o grupo de modelos para **coordenação baseada em eventos**. Em referencialmente sistemas desacoplados, os processos não se conhecem explicitamente. A única coisa que um processo pode fazer é **publicar** uma **notificação** descrevendo a ocorrência de um evento (por exemplo, que deseja coordenar atividades ou que apenas produziu algum resultados interessantes). Assumindo que as notificações vêm em todos os tipos e espécies, os processos podem **subscriver** um tipo específico de notificação (ver também [Mühl et al., [2006]]). Em um modelo de coordenação baseado em eventos ideal, uma notificação publicada será entregue exatamente aos processos que o inscreveram. No entanto, geralmente é necessário que o assinante esteja ativo e funcionando no momento em que o a notificação foi publicada.

Um modelo de coordenação bem conhecido é a combinação de referencial e processos temporariamente desacoplados, levando ao que é conhecido como um conjunto **de dados compartilhados espaço**. A ideia-chave é que os processos se comunicam inteiramente por meio de **tuplas**, que são registros de dados estruturados compostos por vários campos, muito semelhantes a um linha em uma tabela de banco de dados. Os processos podem colocar qualquer tipo de tupla na tabela compartilhada espaço de dados. Para recuperar uma tupla, um processo fornece um padrão de pesquisa que é correspondido com as tuplas. Qualquer tupla que corresponda é retornada.

Os espaços de dados compartilhados são, portanto, vistos como implementadores de uma pesquisa associativa mecanismo para tuplas. Quando um processo deseja extrair uma tupla dos dados espaço, ele especifica (alguns dos) valores dos campos nos quais está interessado. Qualquer a tupla que corresponde a essa especificação é então removida do espaço de dados e passado para o processo.

Espaços de dados compartilhados são frequentemente combinados com coordenação baseada em eventos: um processo assina certas tuplas fornecendo um padrão de pesquisa; quando um processo insere uma tupla no espaço de dados, assinantes correspondentes são notificados. Em ambos os casos, estamos lidando com uma arquitetura de **publicação-assinatura** e, de fato, a principal característica é que os processos não têm referência explícita uns aos outros. A diferença entre um estilo arquitetônico puramente baseado em eventos e o de um espaço de dados compartilhado é mostrada na Figura 2.10. Também mostramos uma abstração do mecanismo pelo qual publicadores e assinantes são correspondidos, conhecido como **barramento de eventos**.

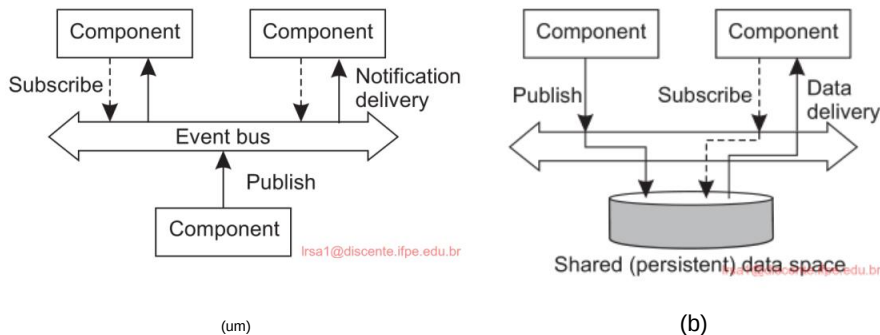


Figura 2.10: O estilo arquitetônico (a) baseado em eventos e (b) de espaço de dados compartilhado.

Nota 2.3 (Exemplo: Espaços de tuplas de Linda)

Para tornar as coisas um pouco mais concretas, daremos uma olhada mais de perto em **Linda**, um modelo de programação desenvolvido na década de 1980 [Carriero e Gelernter, 1989]. O espaço de dados compartilhado em Linda é conhecido como um **espaço de tupla**, que suporta três operações:

- $\text{in}(t)$: remove uma tupla que corresponde ao modelo t
- $\text{rd}(t)$: obtém uma cópia de uma tupla que corresponde ao modelo t
- $\text{out}(t)$: adiciona a tupla t ao espaço da tupla

Note que se um processo chamasse $\text{out}(t)$ duas vezes seguidas, descobriríamos que duas cópias da tupla t teriam sido armazenadas. Formalmente, um espaço de tupla é, portanto, sempre modelado como um multiconjunto. Tanto in quanto rd são operações de bloqueio: o chamador será bloqueado até que uma tupla correspondente seja encontrada ou se torne disponível.

Considere um aplicativo de microblog simples no qual as mensagens são marcadas com o nome do autor da postagem e um tópico, seguido por uma string curta. Cada mensagem é modelada como uma tupla $\langle \text{string}, \text{string}, \text{string} \rangle$ onde a primeira string nomeia o autor da postagem, a segunda string representa o tópico e a terceira é o conteúdo real. Supondo que criamos um espaço de dados compartilhado chamado MicroBlog, a Figura 2.11 mostra como Alice e Bob podem postar mensagens nesse espaço e como

Chuck pode escolher uma mensagem (selecionada aleatoriamente). Omitimos algum código para maior clareza. Note que nem Alice nem Bob sabem quem lerá suas postagens.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("bob","distsys","Estou estudando o capítulo 2")) 4
blog._out(("bob","distsys","O exemplo da Linda é bem simples")) 5
blog._out(("bob","gtcn","Livro legal!"))
```

(a) Código de Bob para criar um microblog e postar três mensagens.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("alice","gtcn","Essa coisa de teoria dos grafos não é fácil")) 4
blog._out(("alice","distsys","Gosto mais de sistemas do que de grafos"))
```

(b) Código de Alice para criar um microblog e postar duas mensagens.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 t1 = blog._rd(("bob","distsys",str)) 4 t2 =
blog._rd(("alice","gtcn",str)) 5 t3 =
blog._rd(("bob","gtcn",str))
```

(c) Chuck lendo uma mensagem do microblog de Bob e Alice.

Figura 2.11: Um exemplo simples de uso de um espaço de dados compartilhado.

Na primeira linha de cada fragmento de código, um processo procura o espaço de tupla chamado "MicroBlog". Bob posta três mensagens: duas no tópico distsys e uma no gtcn. Alice posta duas mensagens, uma em cada tópico. Chuck, finalmente, lê três mensagens: uma de Bob no distsys e uma no gtcn, e uma de Alice no gtcn.

Obviamente, há muito espaço para melhorias. Por exemplo, devemos garantir que Alice não possa postar mensagens sob o nome de Bob. No entanto, a questão importante a ser observada agora é que, ao fornecer apenas tags, um leitor como Chuck poderá pegar mensagens sem precisar referenciar diretamente o autor da postagem. Em particular, Chuck também poderia ler uma mensagem selecionada aleatoriamente no tópico distsys por meio da declaração

```
t = blog._rd((str,"distsys",str))
```

Deixamos como exercício para o leitor estender os fragmentos de código de modo que uma próxima mensagem seja selecionada em vez de uma aleatória.

Um aspecto importante dos sistemas de publicação-assinatura é que a comunicação ocorre descrevendo os eventos nos quais um assinante está interessado. Como consequência, a nomenclatura desempenha um papel crucial. Retornaremos à nomenclatura mais tarde, mas, por enquanto, a questão importante é que, frequentemente, os itens de dados não são explicitamente identificados por remetentes e destinatários.

Vamos primeiro supor que os eventos são descritos por uma série de **atributos**. Diz-se que uma notificação que descreve um evento é **publicada** quando é feita

disponível para outros processos lerem. Para esse fim, uma **assinatura** precisa ser passada para o middleware, contendo uma descrição do evento no qual o assinante está interessado. Tal descrição normalmente consiste em alguns pares (atributo, valor), o que é comum para os chamados sistemas **de publicação-assinatura baseados em tópicos**.

Como alternativa, em **sistemas de publicação-assinatura baseados em conteúdo**, uma assinatura também pode consistir em pares (atributo, intervalo). Nesse caso, espera-se que o atributo especificado assuma valores dentro de um intervalo especificado. Às vezes, as descrições podem ser fornecidas usando todos os tipos de predicados formulados sobre os atributos, muito semelhantes em natureza às consultas do tipo SQL no caso de bancos de dados relacionais. Obviamente, quanto mais expressiva uma descrição puder ser, mais difícil será testar se um evento corresponde a uma descrição.

Agora, somos confrontados com uma situação em que as assinaturas precisam ser **correspondidas** com as notificações, conforme mostrado na Figura 2.12. Frequentemente, um evento corresponde, na verdade, a dados se tornando disponíveis. Nesse caso, quando a correspondência é bem-sucedida, há dois cenários possíveis. No primeiro caso, o middleware pode decidir encaminhar a notificação publicada, junto com os dados associados, para seu conjunto atual de assinantes, ou seja, processos com uma assinatura correspondente. Como alternativa, o middleware também pode encaminhar apenas uma notificação, em que ponto os assinantes podem executar uma operação de leitura para recuperar o item de dados.

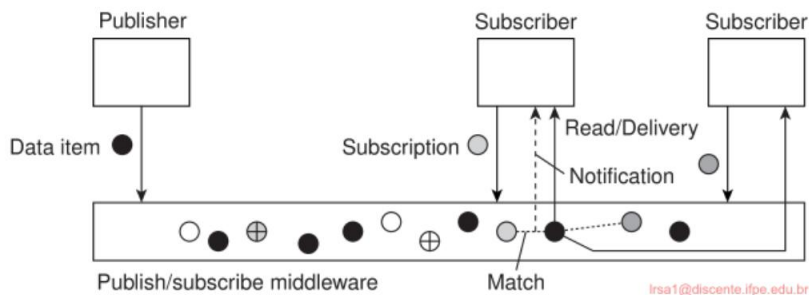


Figura 2.12: O princípio de troca de itens de dados entre publicadores e assinantes.

Nesses casos, nos quais os dados associados a um evento são imediatamente encaminhados aos assinantes, o middleware geralmente não oferecerá armazenamento de dados. O armazenamento é explicitamente manipulado por um serviço separado ou é de responsabilidade dos assinantes. Em outras palavras, temos um sistema referencialmente desacoplado, mas temporalmente acoplado.

Essa situação é diferente quando as notificações são enviadas para que os assinantes precisem ler explicitamente os dados associados. Necessariamente, o middleware terá que armazenar itens de dados. Nessas situações, há operações adicionais para gerenciamento de dados. Também é possível anexar um **lease** a um item de dados de forma que, quando o lease expirar, o item de dados seja excluído automaticamente.

Eventos podem facilmente complicar o processamento de assinaturas. Para ilustrar, considere uma assinatura como “notificar quando o quarto Z1.1060 estiver desocupado e a porta estiver destrancada”. Normalmente, um sistema distribuído que suporte tais assinaturas pode ser implementado colocando sensores independentes para monitorar a ocupação do quarto (por exemplo, sensores de movimento) e aqueles para registrar o status de uma fechadura de porta. Seguindo a abordagem esboçada até agora, precisaríamos compor tais eventos primitivos em um item de dados publicável, ao qual os processos podem então se inscrever. A composição de eventos acaba sendo uma tarefa difícil, notavelmente quando os eventos primitivos são gerados a partir de fontes dispersas pelo sistema distribuído.

Claramente, em sistemas de publicação-assinatura como esses, a questão crucial é a implementação eficiente e escalável de assinaturas correspondentes a notificações. De fora, a arquitetura de publicação-assinatura fornece muito potencial para construir sistemas distribuídos de larga escala devido ao forte desacoplamento de processos. Por outro lado, conceber implementações escaláveis sem perder essa independência não é um exercício trivial, notavelmente no caso de sistemas de publicação-assinatura baseados em conteúdo. Nesse sentido, embora muitos afirmem que o estilo de publicação-assinatura oferece o caminho para arquiteturas escaláveis, o fato é que as implementações podem facilmente formar um gargalo, certamente quando a segurança e a privacidade estão em jogo, como discutiremos no Capítulo 9 e mais tarde no Capítulo 5.

2.2 Middleware e sistemas distribuídos

Para auxiliar o desenvolvimento de aplicações distribuídas, os sistemas distribuídos são frequentemente organizados para ter uma camada separada de software que é logicamente colocada sobre os respectivos sistemas operacionais dos computadores que fazem parte do

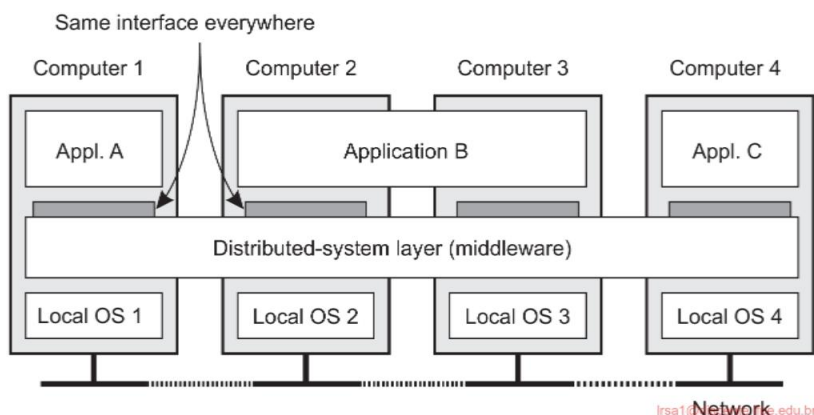


Figura 2.13: Um sistema distribuído organizado em uma camada de middleware, que se estende por várias máquinas, oferecendo a cada aplicação a mesma interface.

sistema. Essa organização é mostrada na Figura 2.13, levando ao que é conhecido como **middleware** [Bernstein, 1996].

A Figura 2.13 mostra quatro computadores em rede e três aplicativos, dos quais o aplicativo B é distribuído pelos computadores 2 e 3. Cada aplicativo recebe a mesma interface. O sistema distribuído fornece os meios para que os componentes de um único aplicativo distribuído se comuniquem entre si, mas também para permitir que diferentes aplicativos se comuniquem. Ao mesmo tempo, ele oculta, da melhor forma e razoavelmente possível, as diferenças em hardware e sistemas operacionais de cada aplicativo.

Em certo sentido, o middleware é o mesmo para um sistema distribuído que um sistema operacional é para um computador: um gerenciador de recursos que oferece seus aplicativos para compartilhar e implantar esses recursos de forma eficiente em uma rede.

Além do gerenciamento de recursos, ele oferece serviços que também podem ser encontrados na maioria dos sistemas operacionais, incluindo:

- Facilidades para comunicação entre aplicações.
- Serviços de segurança.
- Serviços de contabilidade.
- Mascaramento e recuperação de falhas.

A principal diferença com seus equivalentes de sistema operacional é que os serviços de middleware são oferecidos em um ambiente de rede. Observe também que a maioria dos serviços é útil para muitos aplicativos. Nesse sentido, o middleware também pode ser visto como um contêiner de componentes e funções comumente usados que agora não precisam mais ser implementados por aplicativos separadamente.

Nota 2.4 (Nota histórica: O termo middleware)

Embora o termo middleware tenha se tornado popular em meados da década de 1990, ele provavelmente foi mencionado pela primeira vez em um relatório sobre uma conferência de engenharia de software da OTAN, editado por Peter Naur e Brian Randell em outubro de 1968 [Naur e Randell, 1968]. De fato, o middleware foi colocado precisamente entre aplicativos e rotinas de serviço (o equivalente a sistemas operacionais).

2.2.1 Organização do middleware

Vamos agora dar um zoom na organização real do middleware. Há dois tipos importantes de padrões de design que são frequentemente aplicados à organização do middleware: wrappers e interceptors. Cada um tem como alvo problemas diferentes, mas aborda o mesmo objetivo para o middleware: atingir a abertura (como discutimos na Seção 1.2.3).

Embalagens

Ao construir um sistema distribuído a partir de componentes existentes, imediatamente nos deparamos com um problema fundamental: as interfaces oferecidas pelo componente legado provavelmente não são adequadas para todos os aplicativos. Na Seção 1.3.2, discutimos como a integração de aplicativos corporativos poderia ser estabelecida por meio de middleware como um facilitador de comunicação, mas ainda assim assumimos implicitamente que, no final, os componentes poderiam ser acessados por meio de suas interfaces nativas.

Um **wrapper** ou **adaptador** é um componente especial que oferece uma interface aceitável para um aplicativo cliente, cujas funções são transformadas naquelas disponíveis no componente. Em essência, ele resolve o problema de interfaces incompatíveis (veja também Gamma et al. [1994]).

Embora originalmente deñidas de forma restrita no contexto da programação orientada a objetos, no contexto de sistemas distribuídos, wrappers são muito mais do que simples transformadores de interface. Por exemplo, um **adaptador de objeto** é um componente que permite que aplicativos invoquem objetos remotos, embora esses objetos possam ter sido implementados como uma combinação de funções de biblioteca operando nas tabelas de um banco de dados relacional.

Como outro exemplo, reconsidere o serviço de armazenamento S3 da Amazon. Conforme mencionado, há dois tipos de interfaces disponíveis, uma aderindo a uma arquitetura RESTful, outra seguindo uma abordagem mais tradicional. Para a interface RESTful, os clientes usarão o protocolo HTTP, essencialmente se comunicando com um servidor Web tradicional que agora atua como um adaptador para o serviço de armazenamento real, dissecando parcialmente as solicitações recebidas e, subsequentemente, entregando -as a servidores especializados internos ao S3.

Os wrappers sempre desempenham um papel importante na extensão de sistemas com componentes existentes. A extensibilidade, que é crucial para atingir a abertura, costumava ser abordada adicionando wrappers conforme necessário. Em outras palavras, se um aplicativo A gerenciasse dados que eram necessários para um aplicativo B, uma abordagem seria desenvolver um wrapper específico para B para que ele pudesse ter acesso aos dados de A. Claramente, essa abordagem não escala bem: com N aplicativos, precisaríamos, em teoria, desenvolver $N \times (N - 1) = O(N^2)$ invólucros.

Novamente, facilitar uma redução do número de wrappers é tipicamente feito por meio de middleware. Uma maneira de fazer isso é implementar um chamado **broker**, que é logicamente um componente centralizado que manipula todos os acessos entre diferentes aplicativos. Um tipo frequentemente usado é um **message broker**, do qual discutimos os detalhes técnicos na Seção 4.3.3. No caso de um message broker, os aplicativos simplesmente enviam solicitações ao broker contendo informações sobre o que eles precisam. O broker, tendo conhecimento de todos os aplicativos relevantes, contata os aplicativos apropriados, possivelmente combina e transforma as respostas e retorna o resultado para o aplicativo inicial. Em princípio, como um broker oferece uma única interface para cada aplicativo, nós

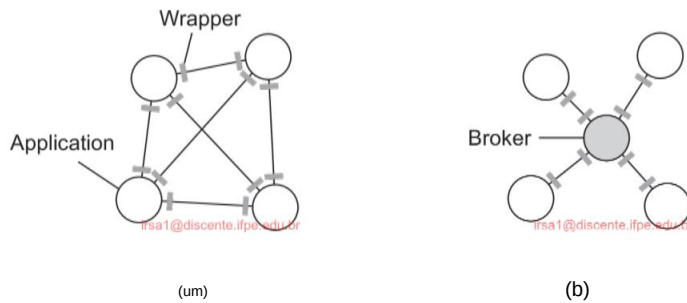


Figura 2.14: (a) Exigindo que cada aplicativo tenha um wrapper para cada outro aplicativo. (b) Redução do número de wrappers por meio do uso de um broker.

agora precisa de no máximo $2N = O(N)$ wrappers em vez de $O(N^2)$. Esta situação é esboçada na Figura 2.14.

Interceptadores

Conceitualmente, um **interceptor** nada mais é do que uma construção de software que quebrará o fluxo de controle usual e permitirá que outro código (específico do aplicativo) seja executado. Os interceptores são um meio primário para adaptar o middleware às necessidades específicas de um aplicativo. Como tal, eles desempenham um papel importante em tornar o middleware aberto. Tornar os interceptores genéricos pode exigir um esforço substancial de implementação, conforme ilustrado por Schmidt et al. [2000], e não está claro se, em tais casos, a generalidade deve ser preferida em vez da aplicabilidade e simplicidade restritas. Além disso, muitas vezes, ter apenas recursos de interceptação limitados melhorará o gerenciamento do software e do sistema distribuído como um todo.

Para tornar as coisas concretas, considere a interceptação como suportada em muitos sistemas distribuídos baseados em objetos. A ideia básica é simples: um objeto A pode chamar um método que pertence a um objeto B, enquanto este último reside em uma máquina diferente de A. Como explicamos em detalhes mais adiante no livro, tal invocação de objeto remoto é realizada em três etapas:

1. Ao objeto A é oferecida uma interface local que é a mesma que a interface oferecida pelo objeto B. A chama o método disponível nessa interface.
2. A chamada de A é transformada em uma invocação de objeto genérica, possibilitada por meio de uma interface geral de invocação de objeto oferecida pelo middleware na máquina onde A reside.
3. Finalmente, a invocação do objeto genérico é transformada em uma mensagem que é enviada através da interface de rede de nível de transporte, conforme oferecida pelo sistema operacional local de A.

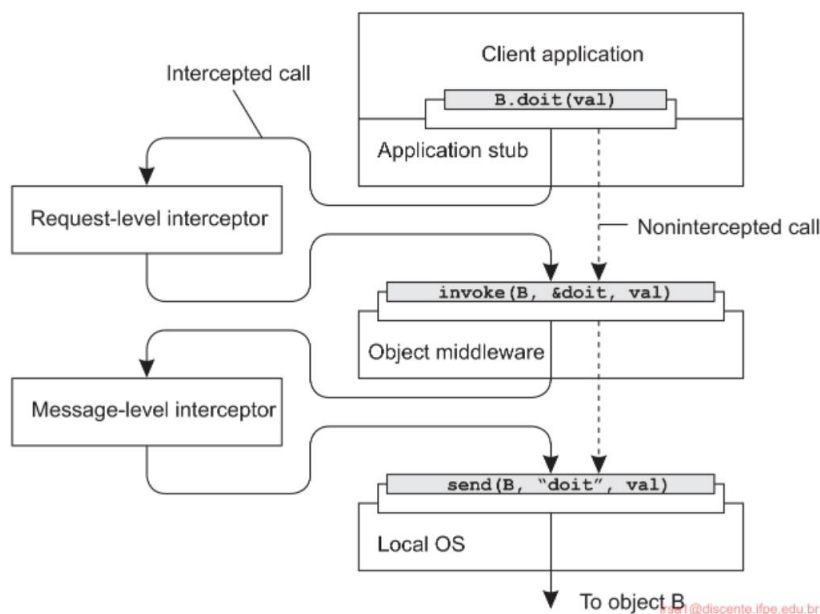


Figura 2.15: Usando interceptadores para manipular invocações de objetos remotos.

Este esquema é mostrado na Figura 2.15. Após o primeiro passo, a chamada `B.doit(val)` é transformada em uma chamada genérica, como `invoke(B,&doit,val)` com uma referência ao método de `B` e os parâmetros que acompanham a chamada. Agora imagine que o objeto `B` é replicado. Nesse caso, cada réplica deve realmente ser invocada. Este é um ponto claro onde a interceptação pode ajudar. O que o **interceptor de nível de solicitação** fará é simplesmente chamar `invoke(B,&doit,val)` para cada uma das réplicas. A beleza de tudo isso é que o objeto `A` não precisa estar ciente da replicação de `B`, mas também o middleware do objeto não precisa ter componentes especiais que lidem com essa chamada replicada. Apenas o interceptor de nível de solicitação, que pode ser adicionado ao middleware, precisa saber sobre a replicação de `B`.

No final, uma chamada para um objeto remoto terá que ser enviada pela rede. Na prática, isso significa que a interface de mensagens oferecida pelo sistema operacional local precisará ser invocada. Nesse nível, um **interceptor de nível de mensagem** pode auxiliar na transferência da invocação para o objeto de destino. Por exemplo, imagine que o parâmetro `val` realmente corresponde a uma enorme matriz de dados. Nesse caso, pode ser sensato fragmentar os dados em partes menores para montá-los novamente no destino. Essa fragmentação pode melhorar o desempenho ou a confiabilidade. Novamente, o middleware não precisa estar ciente dessa fragmentação; o interceptor de nível inferior manipulará de forma transparente o restante da comunicação com o sistema operacional local.

2.2.2 Middleware modificável

O que wrappers e interceptors oferecem são meios para estender e adaptar o middleware. A necessidade de adaptação vem do fato de que o ambiente no qual os aplicativos distribuídos são executados muda continuamente. As mudanças incluem aquelas resultantes de mobilidade, uma forte variação na qualidade de serviço das redes, hardware com falha e drenagem de bateria, entre outras.

Em vez de tornar os aplicativos responsáveis por reagir a mudanças, essa tarefa é colocada no middleware. Além disso, conforme o tamanho de um sistema distribuído aumenta, a mudança de suas partes raramente pode ser feita desligando-o temporariamente. O que é necessário é ser capaz de fazer mudanças no momento.

Essas fortes influências do ambiente levaram muitos projetistas de middleware a considerar a construção de software adaptável. Seguimos Parlavantzas e Coulson [2007] ao falar de **middleware modifiável** para expressar que o middleware pode não apenas precisar ser adaptável, mas que devemos ser capazes de modificá-lo propositalmente sem derrubá-lo. Nesse contexto, os interceptores podem ser pensados como oferecendo um meio de adaptar o fluxo de controle padrão. Substituir componentes de software em tempo de execução é um exemplo de modificação de um sistema. E, de fato, talvez uma das abordagens mais populares em relação ao middleware modifiável seja a de construir dinamicamente o middleware a partir de componentes.

O design baseado em componentes foca em dar suporte à modificabilidade por meio da composição. Um sistema pode ser configurado estaticamente no tempo de design ou dinamicamente no tempo de execução. Este último requer suporte para ligação tardia, uma técnica que foi aplicada com sucesso em ambientes de linguagem de programação, mas também para sistemas operacionais onde os módulos podem ser carregados e descarregados à vontade. Selecionar automaticamente a melhor implementação de um componente durante o tempo de execução é agora bem compreendido [Yellin, 2003], mas, novamente, o processo continua complexo para sistemas distribuídos, especialmente quando se considera que a substituição de um componente requer saber exatamente qual será o efeito dessa substituição em outros componentes. Frequentemente, os componentes são menos independentes do que se pode pensar.

O ponto principal é que, para acomodar mudanças dinâmicas no software que compõe o middleware, precisamos de pelo menos suporte básico para carregar e descarregar componentes em tempo de execução. Além disso, para cada componente, são necessárias especificações explícitas das interfaces que ele oferece, bem como das interfaces que ele requer. Se o estado for mantido entre chamadas para um componente, então medidas especiais adicionais são necessárias. De modo geral, deve ficar claro que organizar o middleware para ser modifiável requer atenção especial.

2.3 Arquiteturas de sistemas em camadas

Vamos agora dar uma olhada em quantos sistemas distribuídos são realmente organizados, considerando onde os componentes de software são colocados. Decidir sobre software

componentes, sua interação e seu posicionamento levam a uma instância de uma arquitetura de software, também conhecida como **arquitetura de sistema** [Bass et al., 2021].

Começamos discutindo arquiteturas em camadas. Outras formas seguem depois.

Apesar da falta de consenso sobre muitas questões de sistemas distribuídos, há uma questão com a qual muitos pesquisadores e profissionais concordam: pensar em termos de clientes que solicitam serviços de servidores ajuda a entender e gerenciar a complexidade dos sistemas distribuídos [Saltzer e Kaashoek, 2009]. A seguir, consideraremos primeiro uma organização em camadas simples e, em seguida, analisaremos organizações com várias camadas.

2.3.1 Arquitetura cliente-servidor simples

No modelo básico cliente-servidor, os processos em um sistema distribuído são divididos em dois grupos (possivelmente sobrepostos). Um **servidor** é um processo que implementa um serviço específico, por exemplo, um serviço de sistema de arquivos ou um serviço de banco de dados. Um **cliente** é um processo que solicita um serviço de um servidor enviando a ele uma solicitação e, subsequentemente, aguardando a resposta do servidor. Essa interação cliente-servidor, também conhecida como **comportamento de solicitação-resposta**, é mostrada na Figura 2.16.

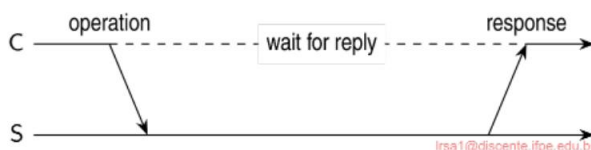


Figura 2.16: Interação geral entre um cliente C e um servidor S. C envia a operação oper e aguarda a resposta de S.

A comunicação entre um cliente e um servidor pode ser implementada por um protocolo simples sem conexão quando a rede subjacente é razoavelmente confiável, como em muitas redes locais. Nesses casos, quando um cliente solicita um serviço, ele simplesmente empacota uma mensagem para o servidor, identificando o serviço que ele quer, junto com os dados de entrada necessários. A mensagem é então enviada ao servidor. Este último, por sua vez, sempre aguardará uma solicitação recebida, posteriormente a processará e empacotará os resultados em uma mensagem de resposta que será então enviada ao cliente.

Usar um protocolo sem conexão tem a vantagem óbvia de ser eficiente. Contudo que as mensagens não sejam perdidas ou corrompidas, o protocolo de solicitação/resposta que acabamos de esboçar funciona bem. Infelizmente, tornar o protocolo resistente a falhas ocasionais de transmissão não é trivial. A única coisa que podemos fazer é possivelmente deixar o cliente reenviar a solicitação quando nenhuma mensagem de resposta chegar. O problema, no entanto, é que o cliente não consegue detectar se a mensagem de solicitação original foi perdida ou se a transmissão da resposta falhou. Se a resposta foi perdida, o reenvio de uma solicitação pode resultar na execução da operação duas vezes. Se

a operação fosse algo como “transferir US\$ 10.000 da minha conta bancária”, então, claramente, teria sido melhor simplesmente relatar um erro.

Por outro lado, se a operação fosse “diga-me quanto dinheiro me resta”, seria perfeitamente aceitável reenviar a solicitação. Quando uma operação pode ser repetida várias vezes sem danos, diz-se que é **idempotente**.

Como algumas requisições são idempotentes e outras não, deve ficar claro que não há uma solução única para lidar com mensagens perdidas. Adiamos uma discussão detalhada sobre como lidar com falhas de transmissão para a Seção 8.3.

Como alternativa, muitos sistemas cliente-servidor usam um protocolo confiável orientado a conexão. Embora essa solução não seja totalmente apropriada em uma rede local devido ao desempenho relativamente baixo, ela funciona perfeitamente bem em sistemas de área ampla nos quais a comunicação é inerentemente não confiável. Por exemplo, praticamente todos os protocolos de aplicativos da Internet são baseados em conexões TCP/IP confiáveis. Nesse caso, sempre que um cliente solicita um serviço, ele primeiro configura uma conexão com o servidor antes de enviar a solicitação. O servidor geralmente usa essa mesma conexão para enviar a mensagem de resposta, após a qual a conexão é interrompida. O problema pode ser que configurar e interromper uma conexão seja relativamente caro, especialmente quando as mensagens de solicitação e resposta são pequenas.

O modelo cliente-servidor tem sido objeto de muitos debates e controvérsias ao longo dos anos. Uma das principais questões era como traçar uma distinção clara entre um cliente e um servidor. Não surpreendentemente, muitas vezes não há uma distinção clara. Por exemplo, um servidor para um banco de dados distribuído pode atuar continuamente como um cliente porque está encaminhando solicitações para diferentes servidores de arquivo responsáveis por implementar as tabelas do banco de dados. Nesse caso, o próprio servidor de banco de dados processa apenas as consultas.

2.3.2 Arquiteturas multicamadas

A distinção em três níveis lógicos, como discutido até agora, sugere várias possibilidades para distribuir fisicamente um aplicativo cliente-servidor em várias máquinas. A organização mais simples é ter apenas dois tipos de máquinas:

1. Uma máquina cliente contendo apenas os programas que implementam (parte de) o nível da interface do usuário
2. Uma máquina servidora contendo o restante, ou seja, os programas que implementam o processamento e o nível de dados

Nesta organização, tudo é manipulado pelo servidor, enquanto o cliente é essencialmente nada mais do que um terminal burro, possivelmente com apenas uma interface gráfica conveniente. Existem, no entanto, muitas outras possibilidades. Conforme explicado na Seção 2.1.1, muitos aplicativos distribuídos são divididos em três camadas: (1) uma camada de interface do usuário, (2) uma camada de processamento e (3) uma camada de dados. Uma

A abordagem para organizar clientes e servidores é então distribuir essas camadas em diferentes máquinas, conforme mostrado na Figura 2.17 (veja também Umar [1997]). Como primeiro passo, faremos uma distinção entre apenas dois tipos de máquinas: máquinas clientes e máquinas servidores, levando ao que também é conhecido como **(fisicamente) arquitetura de duas camadas**.

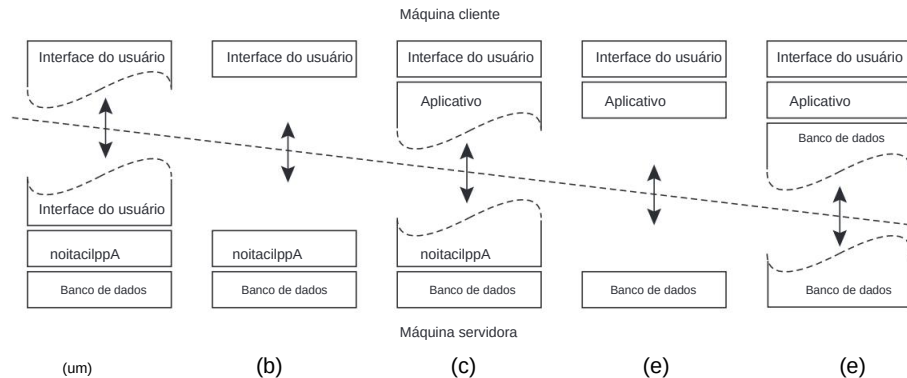


Figura 2.17: Organizações cliente-servidor em uma arquitetura de duas camadas.

Uma organização possível é ter apenas a parte dependente do terminal da interface do usuário na máquina cliente, conforme mostrado na Figura 2.17(a), e dar aos aplicativos controle remoto sobre a apresentação de seus dados. Um a alternativa é colocar todo o software de interface do usuário no lado do cliente, como mostrado na Figura 2.17(b). Nesses casos, essencialmente dividimos a aplicação em um front-end gráfico, que se comunica com o resto do aplicativo (residindo no servidor) por meio de um protocolo específico do aplicativo. Neste modelo, o front end (o software cliente) não realiza nenhum processamento além do necessário para apresentando a interface do aplicativo.

Continuando nesta linha de raciocínio, podemos também deslocar parte do aplicação ao front-end, conforme mostrado na Figura 2.17(c). Um exemplo onde isso faz sentido é onde o aplicativo faz uso de um formulário que precisa ser preenchido completamente antes de poder ser processado. O front end pode então verificar a correção e a consistência da forma e, quando necessário, interagir com o usuário. Outro exemplo da organização da Figura 2.17(c), é o de um processador de texto no qual as funções básicas de edição são executadas no cliente lado onde operam em cache localmente, ou dados na memória, mas onde o ferramentas de suporte avançadas, como verificação ortográfica e gramatical, executam no lado do servidor.

Em muitos ambientes cliente-servidor, as organizações mostradas na Figura 2.17 (d) e Figura 2.17(e) são particularmente populares. Essas organizações são usados onde a máquina cliente é um PC ou estação de trabalho, conectado através de uma rede para um sistema de arquivos distribuído ou banco de dados. Essencialmente, a maioria dos

o aplicativo está sendo executado na máquina cliente, mas todas as operações em arquivos ou entradas de banco de dados vão para o servidor. Por exemplo, muitos aplicativos bancários são executados na máquina de um usuário final, onde o usuário prepara transações e tal. Uma vez finalizado, o aplicativo contata o banco de dados no servidor do banco e carrega as transações para processamento posterior. A Figura 2.17(e) representa a situação em que o disco local do cliente contém parte dos dados. Por exemplo, ao navegar na Web, um cliente pode gradualmente construir um enorme cache no disco local das páginas da Web inspecionadas mais recentemente.

Nota 2.5 (Mais informações: Existe algo como a melhor organização?)

Notamos que houve uma forte tendência de se afastar das configurações mostradas na Figura 2.17(d) e Figura 2.17(e) nesses casos, em que o software cliente é colocado nas máquinas do usuário final. Em vez disso, a maior parte do processamento e armazenamento de dados é tratada no lado do servidor. A razão para isso é simples: embora as máquinas clientes façam muito, elas também são mais problemáticas de gerenciar. Ter mais funcionalidade na máquina cliente significa que uma ampla gama de usuários finais precisará ser capaz de lidar com esse software. Isso implica que mais esforço precisa ser gasto para tornar o software resiliente ao comportamento do usuário final. Além disso, o software do lado do cliente depende da plataforma subjacente do cliente (ou seja, sistema operacional e recursos), o que pode facilmente significar que várias versões precisarão ser mantidas. De uma perspectiva de gerenciamento de sistemas, ter o que são chamados de **clientes gordos** não é o ideal. Em vez disso, os **clientes finos**, conforme representados pelas organizações mostradas na Figura 2.17(a)–(c), são muito mais fáceis, talvez ao custo de interfaces de usuário menos sofisticadas e desempenho percebido pelo cliente.

Isso significa o fim dos fat clients? Nem um pouco. Por um lado, há muitas aplicações para as quais uma organização fat-client ainda é frequentemente a melhor. Já mencionamos suítes de escritório, mas também muitas aplicações multimídia exigem que o processamento seja feito no lado do cliente. Além disso, quando os usuários finais precisam operar off-line, vemos que a instalação de aplicações será necessária. Segundo, com o advento da tecnologia avançada de navegação na Web, agora é muito mais fácil colocar e gerenciar dinamicamente o software do lado do cliente simplesmente carregando scripts (às vezes muito sofisticados) para o cliente. Combinado com o fato de que esse tipo de software do lado do cliente é executado em ambientes bem definidos e comumente implantados, e, portanto, que a dependência de plataforma é muito menos problemática, vemos que o contra-argumento da complexidade de gerenciamento geralmente não é mais válido. Isso levou à implantação de **ambientes de desktop virtual**, que discutiremos mais detalhadamente no Capítulo 3.

Por fim, observe que se afastar de fat clients não significa que não precisamos mais de sistemas distribuídos. Pelo contrário, o que continuamos a ver é que as soluções do lado do servidor estão se tornando cada vez mais distribuídas, pois um único servidor está sendo substituído por vários servidores em execução em máquinas diferentes. A computação em nuvem é um bom exemplo neste caso: o lado completo do servidor está sendo executado em data centers e, geralmente, em vários servidores.

Ao distinguir apenas máquinas cliente e servidor, como fizemos até agora, perdemos o ponto de que um servidor pode às vezes precisar atuar como um cliente, conforme mostrado

na Figura 2.18, levando a uma **arquitetura (fisicamente) de três camadas**.

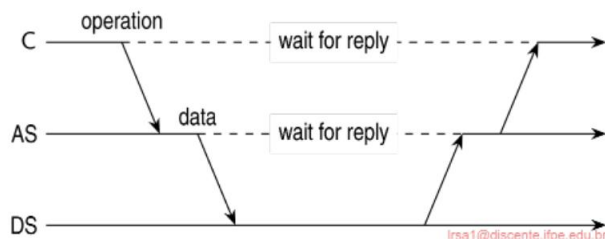


Figura 2.18: Um exemplo de um servidor de aplicativos AS atuando como cliente para um servidor de banco de dados DS.

Nessa arquitetura, tradicionalmente os programas que fazem parte da camada de processamento são executados por um servidor separado, mas podem adicionalmente ser parcialmente distribuídos entre as máquinas cliente e servidor. Um exemplo típico de onde uma arquitetura de três camadas é usada é no processamento de transações. Um processo separado, chamado de monitor de processamento de transações, coordena todas as transações entre servidores de dados possivelmente diferentes.

Outro exemplo, mas muito diferente, em que frequentemente vemos uma arquitetura de três camadas é na organização de Websites. Neste caso, um servidor Web atua como um ponto de entrada para um site, passando solicitações para um servidor de aplicativos onde o processamento real ocorre. Este servidor de aplicativos, por sua vez, interage com um servidor de banco de dados. Já nos deparamos com tal organização ao discutir o Website deste livro e as facilidades para gerar e baixar uma cópia personalizada.

2.3.3 Exemplo: O Sistema de Arquivos de Rede

Muitos sistemas de arquivos distribuídos são organizados como arquiteturas cliente-servidor, sendo o **Network File System (NFS)** da Sun Microsystems um dos mais amplamente implantados para sistemas Unix [Callaghan, 2000; Haynes, 2015; Noveck e Lever, 2020].

A ideia básica por trás do NFS é que cada servidor de arquivos fornece uma visão padronizada de seu sistema de arquivos local. Em outras palavras, não deve importar como esse sistema de arquivos local é implementado; cada servidor NFS suporta o mesmo modelo. Essa abordagem também foi adotada para outros sistemas de arquivos distribuídos. O NFS vem com um protocolo de comunicação que permite que os clientes acessem os arquivos armazenados em um servidor, permitindo assim que uma coleção heterogênea de processos, possivelmente em execução em diferentes sistemas operacionais e máquinas, compartilhem um sistema de arquivos comum.

O modelo subjacente ao NFS e sistemas similares é o de um **serviço de arquivo remoto**. Neste modelo, os clientes recebem acesso transparente a um sistema de arquivo que é gerenciado por um servidor remoto. No entanto, os clientes normalmente não têm conhecimento

da localização real dos arquivos. Em vez disso, é oferecida a eles uma interface para um sistema de arquivos que é semelhante à interface oferecida por um sistema de arquivos local convencional. Em particular, é oferecida ao cliente apenas uma interface contendo várias operações de arquivo, mas o servidor é responsável por implementar essas operações. Este modelo é, portanto, também chamado de **modelo de acesso remoto**. Ele é mostrado na Figura 2.19(a).

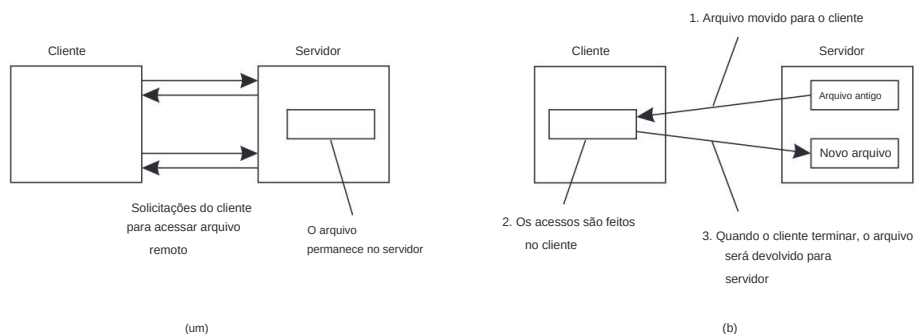


Figura 2.19: (a) O modelo de acesso remoto. (b) O modelo de upload/download.

Em contraste, no **modelo de upload/download**, um cliente acessa um arquivo localmente após tê-lo baixado do servidor, como mostrado na Figura 2.19(b). Quando o cliente termina de usar o arquivo, ele é carregado de volta para o servidor novamente para que possa ser usado por outro cliente. O serviço FTP da Internet pode ser usado dessa forma quando um cliente baixa um arquivo completo, o modifica e o coloca de volta.

O NFS foi implementado para vários sistemas operacionais, embora as versões Unix sejam predominantes. Para praticamente todos os sistemas Unix modernos, o NFS é geralmente implementado seguindo a arquitetura mostrada na Figura 2.20.

Um cliente acessa o sistema de arquivos usando as chamadas de sistema fornecidas por seu sistema operacional local. No entanto, a interface do sistema de arquivos Unix local é substituída por uma interface para o **Virtual File System (VFS)**, que agora é um padrão de fato para interface com diferentes sistemas de arquivos (distribuídos) [Kleiman, 1986].

Praticamente todos os sistemas operacionais modernos fornecem VFS, e não fazê-lo mais ou menos força os desenvolvedores a reimplementar amplamente grandes partes de um sistema operacional ao adotar uma nova estrutura de sistema de arquivos. Com o NFS, as operações na interface VFS são passadas para um sistema de arquivos local ou para um componente separado conhecido como **cliente NFS**, que cuida do manuseio do acesso aos arquivos armazenados em um servidor remoto. No NFS, toda a comunicação cliente-servidor é feita por meio das chamadas **chamadas de procedimento remoto (RPCs)**. Como mencionado antes, um RPC é essencialmente uma maneira padronizada de permitir que um cliente em uma máquina A faça uma chamada comum para um procedimento que é implementado em outra máquina B. Discutimos RPCs extensivamente no Capítulo 4. O cliente NFS implementa as operações do sistema de arquivos NFS como chamadas de procedimento remoto para o servidor. Observe que as operações oferecidas pela interface VFS podem ser diferentes daquelas oferecidas por

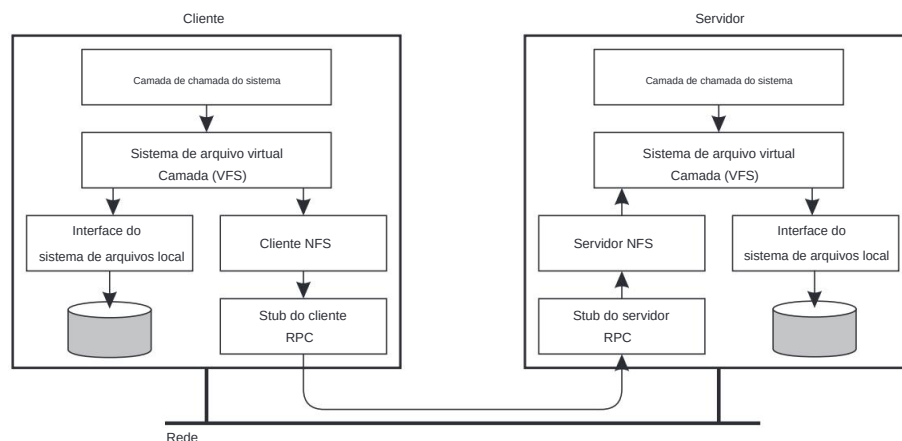


Figura 2.20: Arquitetura básica do NFS para sistemas Unix.

o cliente NFS. A ideia do VFS é esconder as diferenças entre vários sistemas de arquivos.

No lado do servidor, vemos uma organização semelhante. O **servidor NFS** é responsável por manipular solicitações de clientes de entrada. O componente RPC no servidor converte solicitações de entrada em operações regulares de arquivo VFS que são posteriormente passadas para a camada VFS. Novamente, o VFS é responsável por implementar um sistema de arquivo local no qual os arquivos reais são armazenados.

Uma vantagem importante desse esquema é que o NFS é amplamente independente dos sistemas de arquivos locais. Não importa se o sistema operacional no cliente ou servidor usa um sistema de arquivos Unix, um sistema de arquivos Windows ou mesmo um antigo sistema de arquivos MS-DOS. A única questão importante é que esses sistemas de arquivos são compatíveis com o modelo de sistema de arquivos oferecido pelo NFS. Por exemplo, o MS-DOS com seus nomes de arquivos curtos não pode ser usado para implementar um servidor NFS de forma totalmente transparente.

2.3.4 Exemplo: A Web

A arquitetura de sistemas distribuídos baseados na Web não é fundamentalmente diferente de outros sistemas distribuídos. No entanto, é interessante ver como a ideia inicial de dar suporte a documentos distribuídos evoluiu desde seu início na década de 1990. Os documentos deixaram de ser puramente estáticos e passivos para se tornarem conteúdo gerado dinamicamente. Além disso, recentemente, muitas organizações começaram a dar suporte a serviços em vez de apenas documentos.

Sistemas simples baseados na Web

Muitos sistemas baseados na Web ainda são organizados como arquiteturas cliente-servidor relativamente simples. O núcleo de um site é formado por um processo que tem acesso a um

sistema de arquivos local armazenando documentos. A maneira mais simples de se referir a um documento é por uma referência chamada **localizador uniforme de recursos (URL)**. Ele especifica onde um documento está localizado incorporando o nome DNS de seu servidor associado junto com um nome de arquivo pelo qual o servidor pode procurar o documento em seu sistema de arquivos local. Além disso, uma URL especifica o protocolo de nível de aplicativo para transferir o documento pela rede.

Um cliente interage com servidores Web por meio de um **navegador**, que é responsável por exibir corretamente um documento. Além disso, um navegador aceita entrada de um usuário principalmente permitindo que o usuário selecione uma referência a outro documento, que ele então busca e exibe. A comunicação entre um navegador e um servidor Web é padronizada: ambos aderem ao **HyperText Transfer Protocol (HTTP)**. Isso leva à organização geral mostrada na Figura 2.21.

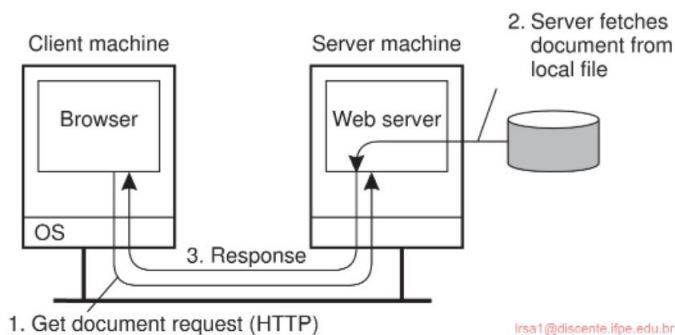


Figura 2.21: A organização geral de um site tradicional.

Vamos ampliar um pouco o que um documento realmente é. Talvez a forma mais simples seja um arquivo de texto padrão. Nesse caso, o servidor e o navegador não têm quase nada a fazer: o servidor copia o arquivo do sistema de arquivos local e o transfere para o navegador. Este último, por sua vez, apenas exibe o conteúdo do arquivo ad verbatim sem mais delongas.

Mais interessantes são os documentos da Web que foram marcados, o que geralmente é feito na **HyperText Markup Language**, ou simplesmente **HTML**. Nesse caso, o documento inclui várias instruções expressando como seu conteúdo deve ser exibido, semelhante ao que se pode esperar de qualquer sistema de processamento de texto decente (embora essas instruções normalmente estejam ocultas do usuário final). Por exemplo, instruir o texto a ser enfatizado é feito pela seguinte marcação:

```
<emph>Dê ênfase a este texto</emph>
```

Há muito mais dessas instruções de marcação. O ponto é que o navegador entende essas instruções e agirá de acordo ao exibir o texto.

Documentos podem conter muito mais do que apenas instruções de marcação. Em particular, eles podem ter programas completos incorporados, dos quais **JavaScript** é o mais frequentemente implantado. Neste caso, o navegador é avisado de que há algum código para executar como em:

```
<script type="text/javascript">....</script>
```

e enquanto o navegador tiver um interpretador incorporado apropriado para a linguagem especiçada, tudo entre “<script>” e “</script>” será executado como qualquer outro programa. O principal benefício de incluir scripts é que ele permite uma interação muito melhor com o usuário final, incluindo o envio de informações de volta para o servidor. (Este último, a propósito, sempre foi suportado em HTML por meio de **formulários**.)

Muito mais pode ser dito sobre documentos da Web, mas este não é o lugar para fazê-lo. Uma boa introdução sobre como construir aplicativos baseados na Web pode ser encontrada em [Sebesta, 2015].

Arquiteturas multicamadas

A Web começou como um sistema cliente-servidor de duas camadas relativamente simples, mostrado na Figura 2.21. Agora, essa arquitetura simples foi estendida para suportar meios muito mais sofisticados de documentos. Na verdade, alguém poderia argumentar justificadamente que o termo “documento” não é mais apropriado. Por um lado, a maioria das coisas que vemos em nosso navegador foi gerada no local como resultado do envio de uma solicitação a um servidor Web. O conteúdo é armazenado em um banco de dados no lado do servidor, junto com scripts do lado do cliente e coisas assim, para ser composto no momento em um documento que é então enviado posteriormente ao navegador do cliente. Os documentos se tornaram, portanto, completamente dinâmicos.

Uma das primeiras melhorias na arquitetura básica foi o suporte para interação simples do usuário pela **Common Gateway Interface** ou simplesmente **CGI**.

CGI define uma maneira padrão pela qual um servidor Web pode executar um programa tomando dados do usuário como entrada. Normalmente, os dados do usuário vêm de um formulário HTML; ele especiça o programa que deve ser executado no lado do servidor, junto com valores de parâmetros que são preenchidos pelo usuário. Uma vez que o formulário tenha sido preenchido, o nome do programa e os valores de parâmetros coletados são enviados ao servidor, como mostrado na Figura 2.22.

Quando o servidor vê a solicitação, ele inicia o programa nomeado na solicitação e passa os valores de parâmetros. Nesse ponto, o programa simplesmente faz seu trabalho e geralmente retorna os resultados na forma de um documento que é enviado de volta ao navegador do usuário para ser exibido.

Os programas CGI podem ser tão sofisticados quanto um desenvolvedor quiser. Por exemplo, como mostrado na Figura 2.22, muitos programas operam em um banco de dados local para o servidor Web. Após processar os dados, o programa gera um documento HTML e retorna esse documento para o servidor. O servidor então passará o documento para o cliente. Uma observação interessante é que para o servidor,

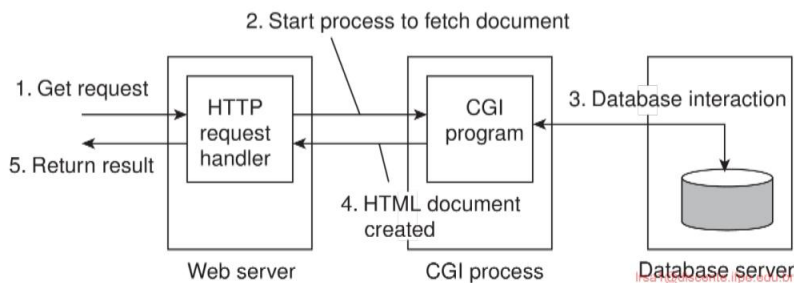


Figura 2.22: O princípio de uso de programas CGI do lado do servidor.

parece que está pedindo ao programa CGI para buscar um documento. Em outras palavras, o servidor não faz nada além de delegar a busca de um documento para um programa externo.

A principal tarefa de um servidor costumava ser lidar com solicitações de clientes simplesmente buscando documentos. Com programas CGI, buscar um documento poderia ser delegado de tal forma que o servidor permaneceria sem saber se um documento havia sido gerado no voo ou realmente lido do sistema de arquivos local. Observe que acabamos de descrever uma organização de dois níveis de software do lado do servidor.

No entanto, hoje em dia, os servidores fazem muito mais do que apenas buscar documentos. Uma das melhorias mais importantes é que os servidores também podem processar um documento antes de passá-lo para o cliente. Em particular, um documento pode conter um **script do lado do servidor**, que é executado pelo servidor quando o documento é buscado localmente. O resultado da execução de um script é enviado junto com o restante do documento para o cliente. O script em si não é enviado. Em outras palavras, usar um script do lado do servidor altera um documento essencialmente substituindo o script pelos resultados de sua execução. Para tornar as coisas concretas, dê uma olhada em um exemplo simples de geração dinâmica de um documento. Suponha que um arquivo seja armazenado no servidor com o seguinte conteúdo:

```
<strong> <?php echo $_SERVER['REMOTE_ADDR']; ?> </strong>
```

O servidor examinará o arquivo e, subsequentemente, processará o código PHP (entre “<?php” e “?>”) substituindo o código pelo endereço do cliente solicitante. Configurações muito mais sofisticadas são possíveis, como acessar um banco de dados local e, subsequentemente, buscar conteúdo desse banco de dados para ser combinado com outro conteúdo gerado dinamicamente.

2.4 Arquiteturas de sistemas simetricamente distribuídos

As arquiteturas cliente-servidor multicamadas são uma consequência direta da divisão de aplicativos distribuídos em uma interface de usuário, componentes de processamento e

componentes de gerenciamento de dados. As diferentes camadas correspondem diretamente à organização lógica dos aplicativos. Em muitos ambientes de negócios, o processamento distribuído é equivalente a organizar um aplicativo cliente-servidor como uma arquitetura multicamadas. Nós nos referimos a esse tipo de distribuição como distribuição **vertical**. A característica da distribuição vertical é que ela é obtida colocando componentes logicamente diferentes em máquinas diferentes. O termo está relacionado ao conceito de fragmentação vertical, conforme usado em bancos de dados relacionais distribuídos, onde significa que as tabelas são divididas em colunas e, posteriormente, distribuídas em várias máquinas [Özsu e Valduriez, 2020].

Novamente, de uma perspectiva de gerenciamento de sistemas, ter uma distribuição vertical pode ajudar: as funções são divididas lógica e fisicamente em várias máquinas, onde cada máquina é adaptada a um grupo específico de funções. No entanto, a distribuição vertical é apenas uma maneira de organizar aplicativos cliente-servidor. Em arquiteturas modernas, geralmente é a distribuição dos clientes e servidores que conta, o que chamamos de **distribuição horizontal**. Nesse tipo de distribuição, um cliente ou servidor pode ser fisicamente dividido em partes logicamente equivalentes, mas cada parte está operando em sua própria parcela do conjunto de dados completo, equilibrando assim a carga. Nesta seção, daremos uma olhada em uma classe de arquiteturas de sistemas modernas que suportam distribuição horizontal, conhecidas como **sistemas peer-to-peer**.

De uma perspectiva de alto nível, os processos que constituem um sistema peer-to-peer são todos iguais. Isso significa que as funções que precisam ser executadas são representadas por cada processo que constitui o sistema distribuído. Como consequência, grande parte da interação entre processos é simétrica: cada processo atuará como cliente e servidor ao mesmo tempo (o que também é chamado de atuação como **servo**).

Dado esse comportamento simétrico, as arquiteturas peer-to-peer giram em torno da questão de como organizar os processos em uma **rede overlay** [Tarkoma, 2010]: uma rede na qual os nós são formados pelos processos e os links representam os possíveis canais de comunicação (que geralmente são realizados como conexões TCP). Um nó pode não ser capaz de se comunicar diretamente com um outro nó arbitrário, mas é necessário enviar mensagens pelos canais de comunicação disponíveis. Existem dois tipos de redes overlay: aquelas que são estruturadas e aquelas que não são. Esses dois tipos são pesquisados extensivamente em Lua et al. [2005] junto com vários exemplos. Buford e Yu [2010] também incluem uma extensa lista de vários sistemas peer-to-peer. Aberer et al. [2005] fornecem uma arquitetura de referência que permite uma comparação mais formal dos diferentes tipos de sistemas peer-to-peer. Uma pesquisa feita da perspectiva da distribuição de conteúdo é fornecida por Androutsellis-Theotokis e Spinellis [2004]. Finalmente, Buford et al. [2009], Tarkoma [2010] e Vu et al. [2010] vão além do nível de pesquisas e formam livros didáticos adequados para estudo inicial ou posterior.

2.4.1 Sistemas peer-to-peer estruturados

Como o próprio nome sugere, em um **sistema peer-to-peer estruturado**, os nós (ou seja, processos) são organizados em uma sobreposição que adere a uma topologia específica e determinística: um anel, uma árvore binária, uma grade, etc. Essa topologia é usada para procurar dados de forma eficiente. Característica para sistemas peer-to-peer estruturados, é que eles são geralmente baseados no uso de um chamado índice semântico livre. O que isso significa é que cada item de dados que deve ser mantido pelo sistema é exclusivamente associado a uma chave, e que essa chave é subsequentemente usada como um índice. Para esse fim, é comum usar uma função hash para que tenhamos:

$$\text{chave}(\text{item de dados}) = \text{hash}(\text{valor do item de dados}).$$

O sistema peer-to-peer como um todo agora é responsável por armazenar pares (chave, valor). Para esse fim, cada nó recebe um identificador do mesmo conjunto de todos os valores de hash possíveis, e cada nó é responsabilizado por armazenar dados associados a um subconjunto específico de chaves. Em essência, o sistema é visto como implementando uma **tabela de hash distribuída**, geralmente abreviada para **DHT** [Balakrishnan et al., 2003].

Seguir essa abordagem agora reduz a essência dos sistemas peer-to-peer estruturados para ser capaz de procurar um item de dados por sua chave. Ou seja, o sistema fornece uma implementação eficiente de uma função lookup que mapeia uma chave para um nó existente:

$$\text{nó existente} = \text{lookup}(\text{chave}).$$

É aqui que a topologia de um sistema peer-to-peer estruturado desempenha um papel crucial. Qualquer nó pode ser solicitado a procurar uma determinada chave, o que então se resume a rotear eficientemente essa solicitação de pesquisa para o nó responsável por armazenar os dados associados à chave fornecida.

Para esclarecer essas questões, vamos considerar um sistema peer-to-peer simples com um número fixo de nós, organizados em um hipercubo. Um **hipercubo** é um cubo n -dimensional. O hipercubo mostrado na Figura 2.23 é quadridimensional. Pode ser pensado como dois cubos comuns, cada um com 8 vértices e 12 arestas. Para expandir o hipercubo para cinco dimensões, adicionaríamos outro conjunto de dois cubos interconectados à figura, conectaríamos as arestas correspondentes nas duas metades e assim por diante.

Para este sistema (reconhecidamente ingênuo), cada item de dados é associado a um dos 16 nós. Isso pode ser obtido por meio do hash do valor de um item de dados para um $k \in \{0, 1, 2, \dots, 24\}$. Agora, suponha que o nó com a chave identificadora 0111 solicitado a procurar os dados com a chave 14, correspondendo ao valor binário 1110. Neste exemplo, assumimos que o nó com o identificador 1110 é responsável por armazenar todos os itens de dados que têm a chave 14. O que o nó 0111 pode simplesmente fazer é encaminhar a solicitação para um vizinho que esteja mais próximo do nó 1110. Neste caso, este é o nó 0110 ou o nó 1111. Se ele escolher um nó 0110, isso

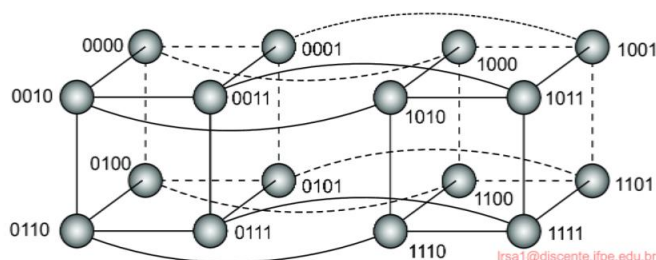


Figura 2.23: Um sistema ponto a ponto simples organizado como um hipercubo quadridimensional .

O nó então encaminhará a solicitação diretamente para um nó 1110 de onde os dados podem ser recuperados.

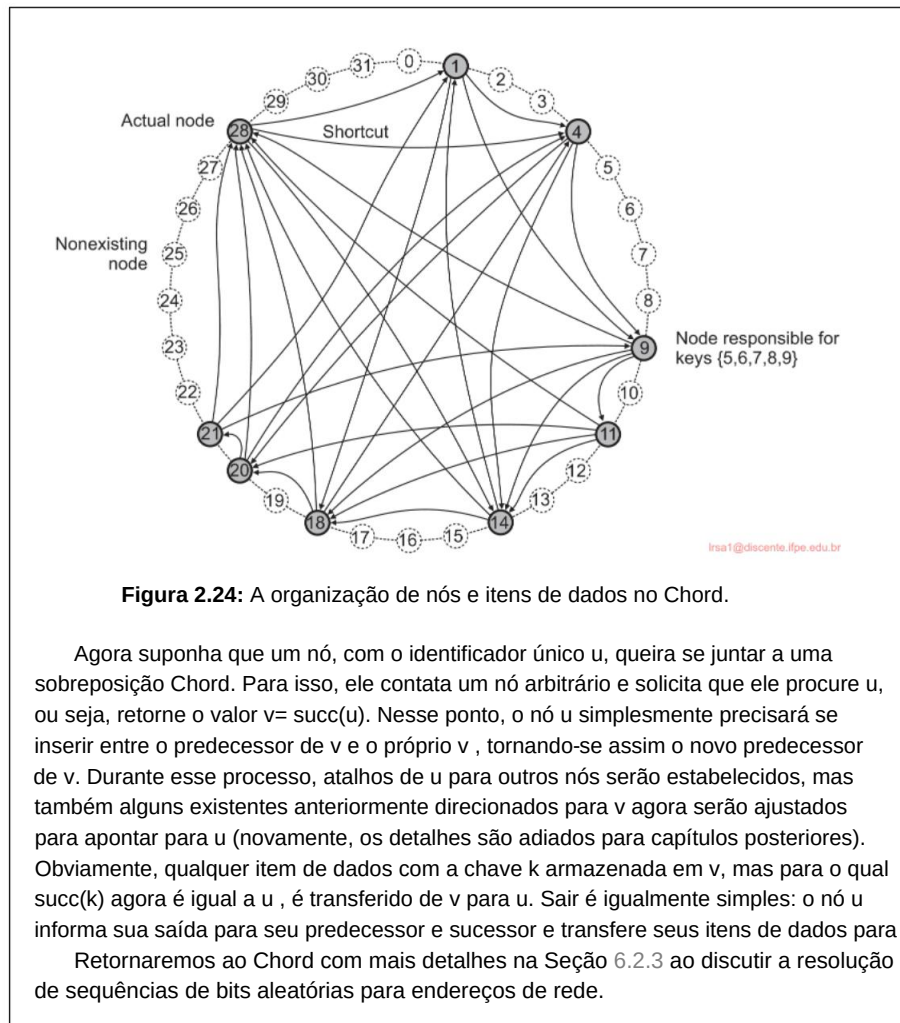
Nota 2.6 (Exemplo: O sistema de acordes)

O exemplo anterior ilustra duas coisas: (1) o uso de uma função de hash para identificar o nó responsável por armazenar algum item de dados e (2) o roteamento ao longo da topologia de um sistema ponto a ponto ao procurar um item de dados dada sua chave.

No entanto, não é um exemplo muito realista, mesmo que seja apenas pela razão de que assumimos que o conjunto total de nós é $\gamma \times \alpha$. Vamos, portanto, considerar um exemplo mais realista de um sistema peer-to-peer estruturado que é considerado como pertencente às fundações para muitos outros sistemas semelhantes.

No **sistema Chord** [Stoica et al., 2003] os nós são organizados logicamente em um anel de modo que um item de dados com uma chave de m bits k é mapeado para o nó com o menor (novamente, também m bits) identificador $id \gamma k$. Este nó é referido como o **sucessor** da chave k e denotado como $succ(k)$. Chaves e identificadores têm tipicamente 128 ou 160 bits de comprimento. A Figura 2.24 mostra um anel Chord muito menor, onde $m = 5$ e com nove nós $\gamma 1, 4, 9, 11, 14, 18, 20, 21, 28\gamma$. O sucessor da chave 7 é igual a 9. Da mesma forma, $succ(5) = 9$, mas também $succ(9) = 9$. No Chord, cada nó mantém atalhos para outros nós. Um atalho aparece como uma aresta direcionada de um nó para outro. Como esses atalhos são construídos é explicado no Capítulo 6. A construção é feita de tal forma que o comprimento do caminho mais curto entre qualquer par de nós é da ordem $O(\log N)$, onde N é o número total de nós.

Para procurar uma chave, um nó tentará encaminhar a solicitação "o mais longe possível", mas sem passá-la além do nó responsável por essa chave. Para esclarecer, suponha que em nosso exemplo Chord ring, o nó 9 seja solicitado a procurar o nó responsável pela chave 3 (que é o nó 4). O nó 9 tem quatro atalhos: para os nós 11, 14, 18 e 28, respectivamente. Como o nó 28 é o nó mais distante que o 9 conhece e ainda precede o responsável pela chave 3, ele receberá a solicitação de pesquisa. O nó 28 tem três atalhos: para os nós 1, 4 e 14, respectivamente. Observe que o nó 28 não tem conhecimento sobre a existência de nós entre os nós 1 e 4. Por esse motivo, o melhor que ele pode fazer é encaminhar a solicitação para o nó 1. Este último sabe que seu sucessor no anel é o nó 4 e, portanto, que este é o nó responsável pela chave 3, para o qual ele encaminhará a solicitação posteriormente.



2.4.2 Sistemas peer-to-peer não estruturados

Sistemas peer-to-peer estruturados tentam manter uma rede de sobreposição específica e determinística. Em contraste, em um sistema peer-to-peer não estruturado, cada nó mantém uma lista ad hoc de vizinhos. A sobreposição resultante se assemelha ao que é conhecido como um **gráfico aleatório**: um gráfico no qual uma aresta yu, v entre dois nós u e v existe apenas com uma certa probabilidade $P[yu, v]$. Idealmente, essa probabilidade é a mesma para todos os pares de nós, mas na prática uma ampla gama de distribuições é observada.

Em um sistema peer-to-peer não estruturado, quando um nó se junta, ele frequentemente contata um nó bem conhecido para obter uma lista inicial de outros pares no sistema. Essa lista pode então ser usada para encontrar mais pares, e talvez ignorar outros, e assim

em. Na prática, um nó geralmente muda sua lista local quase continuamente.

Por exemplo, um nó pode descobrir que um vizinho não está mais respondendo e que ele precisa ser substituído. Pode haver outros motivos, que descreveremos em breve.

Diferentemente de sistemas peer-to-peer estruturados, a busca por dados não pode seguir uma rota predeterminada quando listas de vizinhos são construídas de forma ad hoc. Em vez disso, em sistemas peer-to-peer não estruturados, realmente precisamos recorrer à busca por dados [Risson e Moors, 2006]. Vamos olhar para dois extremos e considerar o caso em que somos solicitados a buscar por dados específicos (por exemplo, identificados por palavras-chave).

Inundação: No caso de **inundação**, um nó emissor u simplesmente passa uma solicitação de um item de dados para todos os seus vizinhos. Uma solicitação será ignorada quando seu nó receptor, digamos v, a tiver visto antes. Caso contrário, v pesquisa localmente o item de dados solicitado. Se v tiver os dados necessários, ele pode responder diretamente ao nó emissor u ou enviá-los de volta ao encaminhador original, que então os retornará ao seu encaminhador original, e assim por diante. Se v não tiver os dados solicitados, ele encaminha a solicitação para todos os seus próprios vizinhos.

Obviamente, o **flooding** pode ser caro, razão pela qual uma solicitação frequentemente tem um valor **de tempo de vida** ou **TTL** associado, dando o número máximo de saltos que uma solicitação pode ser encaminhada. Escolher o valor TTL correto é crucial: muito pequeno significa que uma solicitação ficará perto do emissor e pode, portanto, não alcançar um nó que tenha os dados. Muito grande incorre em altos custos de comunicação.

Como alternativa à configuração de valores TTL, um nó também pode iniciar uma pesquisa com um valor TTL inicial de 1, o que significa que ele primeiro consultará apenas seus vizinhos. Se nenhum resultado ou resultados insuficientes forem retornados, o TTL é aumentado e uma nova pesquisa é iniciada.

Caminhadas aleatórias: Na outra ponta do espectro de busca, um nó emissor u pode simplesmente tentar encontrar um item de dados perguntando a um vizinho escolhido aleatoriamente, digamos v. Se v não tiver os dados, ele encaminha a solicitação para um de seus vizinhos escolhidos aleatoriamente, e assim por diante. O resultado é conhecido como uma **caminhada aleatória** [Gkantsidis et al., 2006; Lv et al., 2002]. Obviamente, uma caminhada aleatória impõe muito menos tráfego de rede, mas pode levar muito mais tempo antes que um nó seja alcançado que tenha os dados solicitados. Para diminuir o tempo de espera, um emissor pode simplesmente iniciar n caminhadas aleatórias simultaneamente. De fato, estudos mostram que, neste caso, o tempo que leva antes de chegar a um nó que tem os dados cai aproximadamente por um fator n. Lv et al. [2002] relata que valores relativamente pequenos de n, como 16 ou 64, acabam sendo eficazes.

Uma caminhada aleatória também precisa ser interrompida. Para isso, podemos usar novamente um TTL ou, alternativamente, quando um nó recebe uma solicitação de pesquisa, verificar

com o emissor se ainda é necessário encaminhar a solicitação para outro vizinho selecionado aleatoriamente.

Note que nenhum dos métodos depende de uma técnica de comparação específica para decidir quando os dados solicitados foram encontrados. Para sistemas peer-to-peer estruturados, assumimos o uso de chaves para comparação; para as duas abordagens recém- descritas, qualquer técnica de comparação seria suficiente.

Entre as inundações e as caminhadas aleatórias estão **os métodos de busca baseados em políticas**. Por exemplo, um nó pode decidir manter o controle de pares que responderam positivamente, efetivamente transformando-os em vizinhos preferidos para consultas subsequentes. Da mesma forma, podemos querer restringir o *flooding* a menos vizinhos, mas em qualquer caso dar preferência a vizinhos que tenham muitos vizinhos.

Nota 2.7 (Avançado: Inundação versus caminhadas aleatórias)

Ao pensar um pouco sobre o assunto, pode ser uma surpresa que as pessoas tenham até considerado uma caminhada aleatória como uma forma alternativa de busca. À primeira vista, pareceria uma técnica semelhante à busca por uma agulha no palheiro.

No entanto, precisamos perceber que, na prática, estamos lidando com dados replicados e, mesmo para fatores de replicação mínimos e diferentes distribuições de replicação, estudos mostram que a implantação de caminhadas aleatórias não é apenas eficaz, mas também pode ser muito mais eficiente em comparação ao *flooding*.

Para ver o porquê, seguimos de perto o modelo descrito em Lv et al. [2002] e Cohen e Shenker [2002]. Suponha que haja um total de N nós e que cada item de dados seja replicado em r nós escolhidos aleatoriamente. Uma busca consiste em selecionar repetidamente um nó aleatoriamente até que o item seja encontrado. Se $P[k]$ for a probabilidade de que o item seja encontrado após k tentativas, temos

$$P[k] = \left(1 - \frac{r}{N}\right)^{k-1} \frac{r}{N}$$

Seja o tamanho médio da pesquisa S o número esperado de nós que precisam ser sondados antes de encontrar o item de dados solicitado:

$$S = \sum_{k=1}^{\infty} k \cdot P[k] = \sum_{k=1}^{\infty} k \cdot \left(1 - \frac{r}{N}\right)^{k-1} \frac{r}{N} = \frac{1}{r/N} = \frac{N}{r}$$

Ao simplesmente replicar cada item de dados para cada nó, $S = 1$, fica claro que uma caminhada aleatória sempre terá um desempenho melhor que o *flooding*, mesmo para valores de TTL de 1. Mais realisticamente, no entanto, é supor que r/N é relativamente baixo, como 0,1%, o que significa que o tamanho médio da pesquisa seria de aproximadamente 1.000 nós.

Para comparar isso com o *flooding*, suponha que cada nó, em média, encaminha uma solicitação para d vizinhos selecionados aleatoriamente. Após uma etapa, a solicitação terá chegado a d nós, cada um dos quais a encaminhará para outros $d-1$ nós (assumindo que o nó de onde a solicitação veio é pulado), e assim por diante. Em outras palavras, após k etapas, e considerando que um nó pode receber a solicitação mais de uma vez, teremos alcançado (no máximo) o seguinte número de nós:

$$R(k) = d(d-1)^{k-1}$$

Vários estudos mostram que $R(k)$ é uma boa estimativa para o número real de nós alcançados, desde que tenhamos apenas um pequeno número de etapas de inundação. Destes nós, podemos esperar que uma fração de r/N tenha o item de dados solicitado, o que significa que quando $R(k) \approx 1$, provavelmente teremos encontrado um nó que tem o item de dados.

Para ilustrar, deixe $r/N = 0,001 = 0,1\%$, o que significa que $S \approx 1000$. Com flooding para, em média, $d = 10$ vizinhos, precisaríamos de pelo menos 4 passos de flooding , atingindo cerca de 7290 nós, o que é consideravelmente mais do que os 1000 nós necessários ao usar uma caminhada aleatória. Somente com $d = 33$ precisaremos contatar aproximadamente também 1000 nós em $k = 2$ passos de flooding e tendo $r/N \cdot R(k) \approx 1$.

A desvantagem óbvia da implementação de caminhadas aleatórias é que pode levar muito tempo mais tempo antes que uma resposta seja retornada.

2.4.3 Redes peer-to-peer organizadas hierarquicamente

Notavelmente em sistemas peer-to-peer não estruturados, localizar itens de dados relevantes pode se tornar problemático conforme a rede cresce. A razão para esse problema de escalabilidade é simples: como não há uma maneira determinística de rotear uma solicitação de pesquisa para um item de dados específico, essencialmente a única técnica à qual um nó pode recorrer é procurar a solicitação por inundação ou caminhando aleatoriamente pela rede. Como alternativa, muitos sistemas peer-to-peer propuseram fazer uso de nós especiais que mantêm um índice de itens de dados.

Há outras situações em que abandonar a natureza simétrica dos sistemas peer-to-peer é sensato. Considere uma colaboração de nós que oferecem recursos uns aos outros. Por exemplo, em uma **Content Delivery Network** (CDN) colaborativa, os nós podem oferecer armazenamento para hospedar cópias de documentos da Web, permitindo que clientes da Web acessem páginas próximas e, assim, acessem-nas rapidamente. O que é necessário é um meio de descobrir onde os documentos podem ser melhor armazenados.

Nesse caso, utilizar um **broker** que coleta dados sobre o uso e a disponibilidade de recursos para vários nós que estão próximos uns dos outros permite selecionar rapidamente um nó com recursos suficientes.

Nós como aqueles que mantêm um índice ou agem como um corretor são geralmente chamados de **super peers**. Como o nome sugere, os super peers geralmente também são organizados em uma rede peer-to-peer, levando a uma organização hierárquica, conforme explicado em Yang e Garcia-Molina [2003]. Um exemplo simples de tal organização é mostrado na Figura 2.25. Nessa organização, cada peer regular, agora chamado de **peer fraco**, é conectado como um cliente a um super peer. Toda a comunicação de e para um peer fraco prossegue por meio do super peer associado a esse peer.

Frequentemente, a associação entre um peer fraco e seu super peer é fixa: sempre que um peer fraco se junta à rede, ele se conecta a um dos super peers e permanece conectado até que deixe a rede. Obviamente, espera-se que os super peers sejam processos de longa duração com alta disponibilidade. Para compensar o comportamento instável potencial de um super peer, esquemas de backup podem ser implantados,

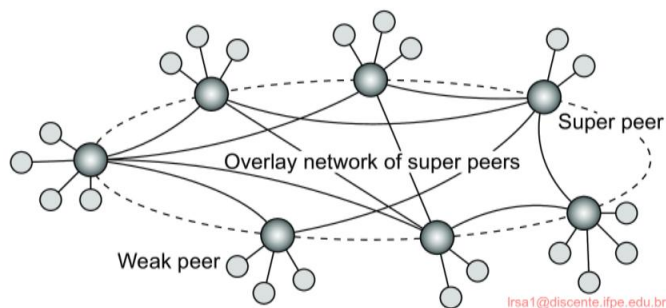


Figura 2.25: Uma organização hierárquica de nós em uma rede superpeer.

como parear cada superpar com outro e exigir que os pares fracos se conectem a ambos.

Ter uma associação *1:1* com um super peer pode não ser sempre a melhor solução. Por exemplo, no caso de redes de compartilhamento de arquivos, pode ser melhor para um peer fraco se conectar a um super peer que mantém um índice de arquivos nos quais o peer fraco está interessado no momento. Nesse caso, as chances são maiores de que, quando um peer fraco estiver procurando por um arquivo específico, seu super peer saberá onde encontrá-lo. Garbacki et al. [2010] descrevem um esquema relativamente simples no qual a associação entre peer fraco e peer forte pode mudar à medida que os peers fracos descobrem melhores super peers para se associar. Em particular, um super peer que retorna o resultado de uma operação de pesquisa recebe preferência sobre outros super peers.

Como vimos, redes peer-to-peer oferecem um meio *1:1* para nós se juntarem e saírem da rede. No entanto, com redes super-peer, um novo problema é introduzido, a saber, como selecionar os nós que são elegíveis para se tornarem super-peer. Esse problema está intimamente relacionado ao problema de **eleição de líder**, que discutimos na Seção 5.4.

2.4.4 Exemplo: BitTorrent

Vamos considerar o sistema de compartilhamento de arquivos **BitTorrent** amplamente popular [Cohen, 2003] como um exemplo de um sistema peer-to-peer (amplamente) não estruturado. O BitTorrent é um sistema de download de arquivos. Seu princípio de funcionamento é mostrado na Figura 2.26. A ideia básica é que quando um usuário final está procurando um arquivo, ele baixa pedaços do arquivo de outros usuários até que os pedaços baixados possam ser montados, produzindo o arquivo completo. Um objetivo importante do design era garantir a colaboração. Na maioria dos sistemas de compartilhamento de arquivos, uma fração significativa de participantes apenas baixa arquivos, mas, de outra forma, contribui com quase nada [Adar e Huberman, 2000; Saroiu et al., 2003; Yang et al., 2005], um fenômeno conhecido como **carona**. Para evitar essa situação, no BitTorrent um arquivo pode

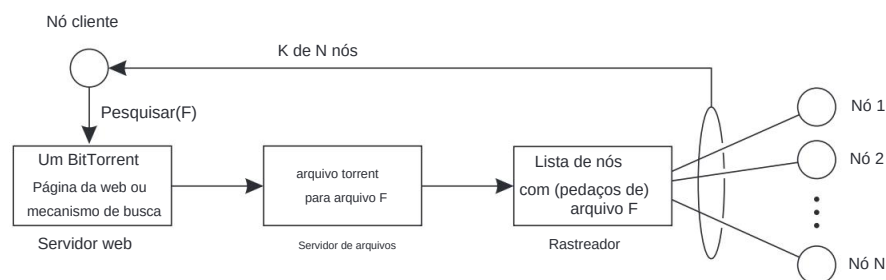


Figura 2.26: O princípio de funcionamento do BitTorrent [adaptado com permissão de Pouwelse et al. [2005].

ser baixado somente quando o cliente de download estiver fornecendo conteúdo para outra pessoa.

Para baixar um arquivo, um usuário precisa acessar um diretório global, que geralmente é apenas um dos poucos sites conhecidos. Esse diretório contém referências ao que são chamados de arquivos torrent. Um **arquivo torrent** contém as informações necessárias para baixar um arquivo específico. Em particular, ele contém um link para o que é conhecido como **rastreador**, que é um servidor que mantém uma conta precisa dos nós ativos que têm (pedaços de) o arquivo solicitado. Um nó ativo é aquele que está atualmente baixando o arquivo também. Obviamente, haverá muitos rastreadores, embora geralmente haja apenas um único rastreador por arquivo (ou coleção de arquivos).

Uma vez que os nós foram identificados de onde os pedaços podem ser baixados, o nó de download efetivamente se torna ativo. Nesse ponto, ele será forçado a ajudar os outros, por exemplo, fornecendo pedaços do arquivo que está baixando que outros ainda não têm. Essa imposição vem de uma regra simples: se um nó P percebe que um nó Q está baixando mais do que está carregando, P pode decidir diminuir a taxa na qual ele envia dados para Q.

Esse esquema funciona bem, desde que P tenha algo para baixar de Q. Por esse motivo, os nós geralmente recebem referências a muitos outros nós, o que os coloca em uma posição melhor para negociar dados.

Claramente, o BitTorrent combina soluções centralizadas com descentralizadas. Como se vê, o gargalo do sistema é facilmente formado pelos rastreadores.

Em uma implementação alternativa do BitTorrent, um nó também se junta a um sistema ponto a ponto estruturado separado (ou seja, um DHT) para auxiliar no rastreamento de downloads de arquivos.

Na verdade, a carga de um rastreador central agora é distribuída entre os nós participantes, com cada nó atuando como um rastreador para um conjunto relativamente pequeno de arquivos torrent. A função original do rastreador coordenando o download colaborativo de um arquivo é mantida. No entanto, notamos que em muitos sistemas BitTorrent usados hoje, a funcionalidade de rastreamento foi realmente minimizada para um provisionamento único de pares atualmente envolvidos no download. A partir desse momento, o novo participante comunicará apenas

com esses pares e não mais com o rastreador inicial. O rastreador inicial para o arquivo solicitado é pesquisado no DHT por meio de um chamado **link magnético**. Retornaremos às pesquisas baseadas em DHT na Seção 6.2.3.

2.5 Arquiteturas de sistemas híbridos

Os sistemas distribuídos do mundo real são complexos no sentido de que combinam uma miríade de arquiteturas: recursos centralizados são combinados com recursos peer-to-peer são combinados com organizações hierárquicas, etc. A complexidade é agravada pelo fato de que muitos sistemas distribuídos cruzam limites organizacionais, levando a soluções verdadeiramente descentralizadas nas quais nem mesmo uma única organização pode assumir a responsabilidade pela operação de um sistema. Nesta seção, examinaremos mais de perto essas arquiteturas de sistemas híbridas e complexas.

2.5.1 Computação em nuvem

As organizações responsáveis por administrar data centers têm buscado maneiras de abrir seus recursos para os clientes. Eventualmente, isso levou ao conceito de **computação utilitária**, pelo qual um cliente poderia carregar tarefas para um data center e ser cobrado por recurso. A computação utilitária formou a base para o que agora é comumente chamado de **computação em nuvem**.

Segundo Vaquero et al. [2008], a computação em nuvem é caracterizada por um conjunto de recursos virtualizados facilmente utilizáveis e acessíveis. Quais e como os recursos são usados podem ser configurados dinamicamente, fornecendo a base para a escalabilidade: se mais trabalho precisa ser feito, um cliente pode simplesmente adquirir mais recursos. O link para a computação utilitária é formado pelo fato de que a computação em nuvem é geralmente baseada em um modelo de pagamento por uso no qual as garantias são oferecidas por **acordos de nível de serviço (SLAs) personalizados**. Mantendo a simplicidade, as nuvens são organizadas em quatro camadas, conforme mostrado na Figura 2.27.

Hardware: A camada mais baixa é formada pelos meios para gerenciar o hardware necessário: processadores, roteadores, mas também sistemas de energia e resfriamento. Geralmente é implementada em data centers e contém os recursos que os clientes normalmente nunca conseguem ver diretamente.

Infraestrutura: Esta é uma camada importante que forma a espinha dorsal para a maioria das plataformas de computação em nuvem. Ela implementa técnicas de virtualização (discutidas na Seção 3.2) para fornecer aos clientes uma infraestrutura que consiste em armazenamento virtual e recursos de computação. De fato, nada é o que parece: a computação em nuvem evolui em torno da alocação e gerenciamento de dispositivos de armazenamento virtual e servidores virtuais.

Plataforma: Pode-se argumentar que a camada de plataforma fornece a um cliente de computação em nuvem o que um sistema operacional fornece aos desenvolvedores de aplicativos, ou seja, os meios para desenvolver e implantar aplicativos facilmente.

que precisam ser executados em uma nuvem. Na prática, um desenvolvedor de aplicativo recebe uma API específica do fornecedor, que inclui chamadas para carregar e executar um programa na nuvem desse fornecedor. Em certo sentido, isso é comparável à família Unix exec de chamadas de sistema, que pega um arquivo executável como parâmetro e o passa para o sistema operacional para ser executado.

Além disso, como os sistemas operacionais, a camada de plataforma fornece abstrações de nível mais alto para armazenamento e tal. Por exemplo, como discutimos, o **sistema de armazenamento Amazon S3** [Murty, 2008; Culkin e Zazon, 2022] é oferecido ao desenvolvedor de aplicativos na forma de uma API permitindo que arquivos (criados localmente) sejam organizados e armazenados em **buckets**. Ao armazenar um arquivo em um bucket, esse arquivo é automaticamente carregado para a nuvem da Amazon.

Aplicação: Os aplicativos reais são executados nesta camada e são oferecidos aos usuários para personalização adicional. Exemplos bem conhecidos incluem aqueles encontrados em suítes de escritório (processadores de texto, aplicativos de planilha, aplicativos de apresentação e assim por diante). É importante perceber que esses aplicativos são novamente executados na nuvem do fornecedor. Como antes, eles podem ser comparados ao conjunto tradicional de aplicativos que são enviados ao instalar um sistema operacional.

Os provedores de computação em nuvem oferecem essas camadas aos seus clientes por meio de várias interfaces (incluindo ferramentas de linha de comando, interfaces de programação e interfaces da Web), resultando em três tipos diferentes de serviços:

- **Infraestrutura como serviço (IaaS)** abrangendo a camada de hardware e infraestrutura.
- **Plataforma como serviço (PaaS)** cobrindo a camada de plataforma.
- **Software como serviço (SaaS)** no qual suas aplicações são cobertas.

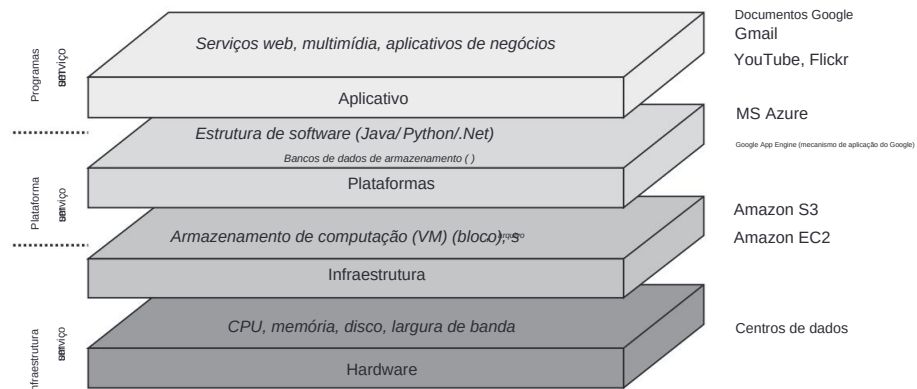


Figura 2.27: A organização das nuvens (adaptado de Zhang et al. [2010]).

Até agora, fazer uso de nuvens é relativamente fácil, e discutiremos em capítulos posteriores exemplos mais concretos de interfaces para provedores de nuvem. Como consequência, a computação em nuvem como um meio de terceirizar infraestruturas de computação locais se tornou uma opção séria para muitas empresas.

Da perspectiva de uma arquitetura de sistema, que lida com a configuração de (micro)serviços em alguma infraestrutura, pode-se argumentar que, no caso da computação em nuvem, estamos lidando com uma arquitetura cliente-servidor altamente avançada. No entanto, deve-se notar que a implementação real de um servidor é geralmente completamente oculta do cliente: muitas vezes não está claro onde o servidor realmente está, e até mesmo se o servidor é realmente implementado de uma maneira totalmente distribuída (o que geralmente é). Para ilustrar melhor esse ponto, a noção de uma **Função como Serviço**, ou simplesmente **FaaS**, permite que um cliente execute código sem se preocupar nem mesmo em iniciar um servidor para manipular o código (veja também Shahrade et al. [2019]).

2.5.2 A arquitetura edge-cloud

Com o advento de cada vez mais dispositivos conectados em rede e o surgimento da **Internet das Coisas (IoT)**, muitos se conscientizaram do fato de que podemos precisar de mais do que apenas computação em nuvem. **A computação de ponta** nasceu. Há muito a dizer sobre computação de ponta, e muito já foi dito. E como é o caso de tantos tópicos em sistemas distribuídos, leva apenas alguns anos para que as coisas se acalmem um pouco. Nesta seção, daremos uma olhada na computação de ponta de uma perspectiva arquitetônica e retornaremos a vários elementos ao longo do livro, muitas vezes sem nem mesmo mencionar explicitamente a computação de ponta. Uma excelente visão geral da computação de ponta é dada por Yousefpour et al. [2019] e o leitor interessado é encaminhado a esse artigo para entender melhor sua nomenclatura. Deliberadamente, adotamos uma visão simplificada e ampla da computação de ponta, usando-a como um termo geral para a maioria das coisas que ficam entre os dispositivos que compõem a Internet das Coisas e os serviços normalmente oferecidos por meio da computação em nuvem. Nesse sentido, seguimos a discussão apresentada por Horner [2021].

Como o próprio nome sugere, a computação de ponta lida com a colocação de serviços “na ponta” da rede. Essa ponta é frequentemente formada pelo limite entre redes corporativas e a Internet real, por exemplo, conforme fornecido por um **Provedor de Serviços de Internet (ISP)**. Por exemplo, muitas universidades residem em um campus que consiste em vários edifícios, cada um com sua própria rede local, por sua vez conectada por uma rede em todo o campus. Como parte do campus, pode haver vários serviços locais para armazenamento, computação, segurança, palestras e assim por diante. Local significa que o departamento de TI local é responsável por hospedar esses serviços em servidores conectados diretamente à rede do campus. Grande parte do tráfego relacionado a esses serviços nunca sairá da rede do campus, e a rede, juntamente com seus servidores e serviços, forma uma **infraestrutura de ponta** típica.

Ao mesmo tempo, tais servidores podem ser conectados aos de outras universidades e talvez até mesmo fazer uso de, novamente, outros servidores. Em outras palavras, em vez de configurar conexões entre universidades de forma peer-to-peer, também vemos configurações nas quais várias universidades compartilham serviços por meio de uma infraestrutura logicamente centralizada. Tal infraestrutura pode estar situada “na nuvem”, mas pode igualmente ter sido configurada por meio de uma infraestrutura regional usando data centers disponíveis localmente. À medida que nos aproximamos das infraestruturas de nuvem, o termo **computação em névoa** é frequentemente usado. Vemos, portanto, um quadro geral emergir como o mostrado na Figura 2.28, onde os limites entre nuvem e borda estão se tornando confusos.

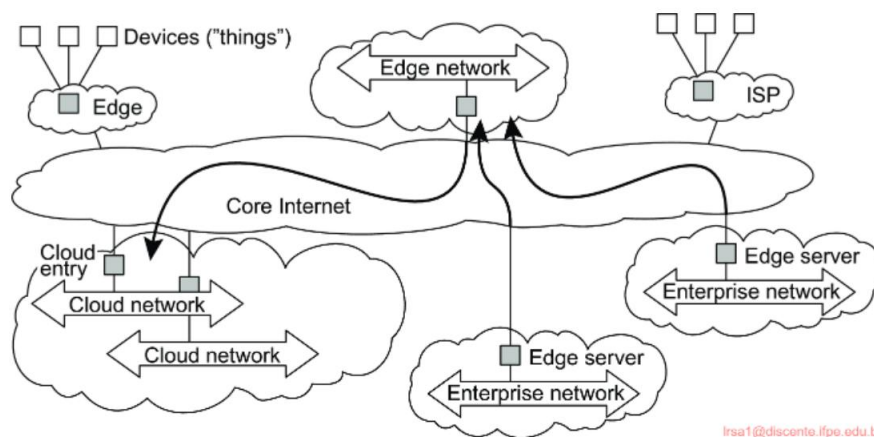


Figura 2.28: Uma coleção de infraestruturas envolvendo dispositivos de ponta, infraestruturas de ponta e infraestruturas de nuvem, e uma possível configuração entre duas infraestruturas de ponta corporativas, uma infraestrutura de ponta intermediária e uma infraestrutura de nuvem.

Muitas configurações para uma infraestrutura de ponta vêm facilmente à mente, variando de infraestruturas necessárias para manter o controle de suas atividades, infraestruturas de streaming de vídeo em camadas, infraestruturas de jogos, etc. O que todas elas têm em comum é que há algum dispositivo final inteligente que, de uma forma ou de outra, precisa (eventualmente) se conectar a um serviço hospedado em algum lugar na nuvem. A questão então surge: por que uma infraestrutura de ponta é necessária?

Logicamente, parece muito mais simples simplesmente conectar-se ao serviço de nuvem diretamente usando instalações de rede existentes e frequentemente excelentes. Vamos dar uma olhada crítica em alguns argumentos.

Latência e largura de banda O que deveria ter ficado claro em nossos exemplos é que as infraestruturas de ponta são consideradas próximas aos dispositivos finais. A proximidade pode ser medida em termos de latência e, muitas vezes, também de largura de banda. Ao longo das décadas, a largura de banda, ou na verdade a falta de largura de banda, tem

sempre foi usado como argumento para introduzir soluções próximas a dispositivos específicos. No entanto, se algo ficou claro durante todo esse tempo, é que a largura de banda disponível continua a aumentar, chegando agora ao ponto em que se deve questionar seriamente o quão problemática ela realmente é, e se instalar e manter infraestruturas de ponta por ter largura de banda insuŕciente é um bom motivo. No entanto, há situações em que a proximidade dos dispositivos finais é realmente necessária para garantir a qualidade do serviço. O exemplo canônico é formado pelos serviços de vídeo: quanto mais próximas as fontes de vídeo estiverem, melhores garantias de largura de banda podem ser dadas, reduzindo problemas como jitter.

Mais problemático é quando a Mãe Natureza entra no nosso caminho. Isso pode acontecer facilmente ao lidar com latência. Pode levar 100 ms para chegar a uma nuvem, tornando muitos aplicativos interativos bastante inúteis. Um desses aplicativos importantes é a direção (semi-)autônoma. Um carro precisará observar continuamente seu ambiente por meio de uma miríade de sensores e reagir de acordo.

Ter que coordenar seus movimentos pela nuvem não é aceitável apenas de um aspecto em tempo real. Este exemplo também ilustra que os carros podem precisar detectar uns aos outros além das capacidades de seus sensores, por exemplo, ao se dirigirem a um cruzamento com visibilidade clara. Em um sistema em tempo real, os carros podem ser capazes de fornecer sua posição atual a uma infraestrutura de ponta local e se revelarem uns aos outros ao se aproximarem do cruzamento.

Outros exemplos nos quais a latência desempenha um papel crucial vêm facilmente à mente. Superar a latência é um dos motivos mais convincentes para desenvolver infraestruturas de ponta.

Confiabilidade Muitos argumentam que a conectividade em nuvem simplesmente não é confiável o suficiente para muitas aplicações, razão pela qual as infraestruturas de ponta devem ser implantadas. Até que ponto esse é um argumento válido ainda está para ser visto. O fato é que para muitas aplicações em rede, a conectividade é geralmente boa e confiável, se não excelente. Claro, há situações em que confiar na confiabilidade 24/7 não é uma opção. Esse pode ser o caso de hospitais, fábricas e outros ambientes críticos em geral. No entanto, nesses casos, medidas já foram tradicionalmente tomadas e até que ponto a computação de ponta traz algo novo nem sempre está claro.

Segurança e privacidade Finalmente, muitos argumentam que as soluções de ponta aumentam a segurança e a privacidade. Tudo depende. Alguém poderia argumentar que se uma solução de nuvem não for segura, então não há razão para que uma solução de ponta seja. Uma suposição implícita que muitas pessoas fazem é que uma infraestrutura de ponta é de propriedade de uma organização específica e opera dentro dos limites de rede (protegidos) dessa organização. Nesse caso, muitas vezes se torna mais simples proteger dados e operações, mas deve-se perguntar se essa proteção é suŕciente. Como discutiremos no Capítulo 9, simplesmente tentar proteger ativos desenvolvendo um muro seguro ao redor de uma organização não ajuda contra

ataques internos, sejam eles intencionais ou não. O mesmo raciocínio vale para a privacidade: se não podemos proteger dados pessoais na nuvem, então por que uma infraestrutura de ponta seria suficiente para a privacidade? Uma discussão completa sobre o papel e a posição da privacidade na ponta e nos dispositivos de ponta é dada por Hong [2017]. A partir dessa discussão, ficou claro que ainda há muito trabalho a ser feito.

No entanto, pode haver outro motivo relacionado à segurança e privacidade pelo qual as infraestruturas de ponta são necessárias. Em muitos casos, as organizações simplesmente não têm permissão, por quaisquer razões regulatórias, de colocar dados na nuvem ou ter dados processados por um serviço de nuvem. Por exemplo, os registros médicos podem ter que ser mantidos no local em servidores certificados e com procedimentos de auditoria rigorosos em vigor. Nesse caso, uma organização terá que recorrer à manutenção de uma infraestrutura de ponta.

A introdução de camadas adicionais entre dispositivos finais e infraestruturas de nuvem abre uma lata de minhocas em comparação à situação relativamente simples de apenas ter que lidar com computação em nuvem. Para o último, pode-se argumentar que o provedor de nuvem, em grande medida, decide onde e como um serviço é realmente implementado. Na prática, estaremos lidando com um data center no qual os (micro)serviços que compõem todo o serviço são distribuídos em várias máquinas.

As questões se tornam mais intrincadas no caso da computação de ponta. Neste caso, a organização cliente agora terá que tomar decisões informadas sobre o que fazer e onde. Quais serviços precisam ser colocados no local em uma infraestrutura de ponta local e quais podem ser movidos para a nuvem? Até que ponto uma infraestrutura de ponta oferece facilidades para recursos virtuais, semelhantes às facilidades oferecidas na computação em nuvem? Além disso, onde podemos assumir que os recursos computacionais e de armazenamento são abundantes ao lidar com uma nuvem, este não é necessariamente o caso para uma infraestrutura de ponta. Na prática, esta última simplesmente tem menos recursos de hardware disponíveis, mas muitas vezes também oferece menos flexibilidade em termos de plataformas disponíveis.

Em geral, alocar recursos no caso de edge computing parece ser muito mais desafiador em comparação às nuvens. Conforme resumido por Hong e Varghese [2019], estamos lidando com limitações quando se trata de recursos, maiores graus de heterogeneidade de hardware e cargas de trabalho muito mais dinâmicas, que, quando tomadas em conjunto, levaram a uma maior demanda de **orquestração**. Além disso, onde da perspectiva de um cliente a nuvem parece estar escondendo muitas de suas complexidades internas, isso não é mais necessariamente o caso, tornando muito mais difícil fazer a orquestração [Bittencourt et al., 2018]. A orquestração se resume ao seguinte (veja também Taleb et al. [2017]):

- **Alocação de recursos:** serviços específicos exigem recursos específicos. A questão é então garantir a disponibilidade dos recursos necessários para executar um serviço. Normalmente, os recursos equivalem a CPU, armazenamento, memória e recursos de rede.

- **Posicionamento do serviço:** independentemente da disponibilidade de recursos, é importante decidir quando e onde colocar um serviço. Isso é notavelmente relevante para aplicativos móveis, pois nesse caso encontrar a infraestrutura de ponta mais próxima desse aplicativo pode ser crucial. Um caso de uso típico é o de videoconferência, para o qual a codificação geralmente não é feita no dispositivo móvel, mas em uma infraestrutura de ponta. Na prática, é preciso decidir em quais bordas o serviço deve ser instalado. Uma visão geral abrangente do posicionamento do serviço no caso da computação de ponta é fornecida por [Salaht et al. \[2020\]](#).
- **Seleção de ponta:** relacionado ao posicionamento do serviço está decidir qual

infraestrutura de ponta deve ser usada quando o serviço precisa ser oferecido. Pode parecer lógico usar a infraestrutura de ponta mais próxima do dispositivo final, mas todos os tipos de circunstâncias podem exigir uma solução alternativa, por exemplo, a conectividade dessa ponta ao provedor de nuvem.

Outras questões também desempenham um papel, mas já deve estar claro que a arquitetura edge-cloud é muito mais exigente do que se poderia pensar inicialmente. Além disso, as diferentes perspectivas sobre como o continuum entre dispositivos finais e a nuvem deve ser preenchido com componentes e soluções edge ainda precisam convergir [\[Antonini et al., 2019\]](#).

2.5.3 Arquiteturas de blockchain

Um tipo de sistema distribuído que está surgindo e é muito debatido é o dos chamados **blockchains**. Os sistemas blockchain permitem o registro de transações, razão pela qual também são chamados de **livros-razão distribuídos**. O último é, na verdade, mais preciso, com os blockchains formando uma das diferentes maneiras de implementar livros-razão distribuídos.

A questão-chave em sistemas de transação é que uma transação é validada, efetuada e subsequentemente armazenada para vários propósitos de auditoria. Por exemplo, Alice pode decidir criar uma transação afirmando que ela transfere \$ 10 para a conta de Bob. Normalmente, ela iria a um banco, onde teria que assinar a transação para provar que realmente quer que ela seja realizada. Se tudo isso acontece fisicamente ou digitalmente, não importa realmente. O banco verificará se ela tem crédito suficiente, se Bob é elegível para receber o dinheiro e, assumindo que tudo está bem, transferirá o dinheiro subsequentemente. Um registro da transação é mantido para todos os tipos de propósitos de auditoria. Observe que as transações em sistemas de blockchains são consideradas muito amplas. Além das transações monetárias, foram desenvolvidos sistemas para registrar documentos de identificação, registrar o uso e a alocação de recursos, votação eletrônica e compartilhamento de registros de saúde, para citar alguns.

O banco opera como um **terceiro confiável**. Uma importante suposição de design para blockchains é que as partes participantes podem, em princípio, não ser confiáveis. Isso também exclui ter um terceiro confiável que lida com todos

transações. Retornaremos à confiança na Seção 9.4 e nos concentraremos nas implicações que a falta de confiança tem para a arquitetura de um sistema.

No caso de blockchains, assumimos que há um conjunto (potencialmente muito grande) de participantes que registram transações conjuntamente entre eles em um livro-razão disponível publicamente. Dessa forma, qualquer participante pode ver o que aconteceu e também verificar a validade de uma transação. Por exemplo, em um sistema de blockchain para moedas digitais, cada uma tendo um ID único e infalsificável, qualquer participante deve ser capaz de verificar se uma moeda já foi gasta verificando todas as transações que ocorreram desde o início.

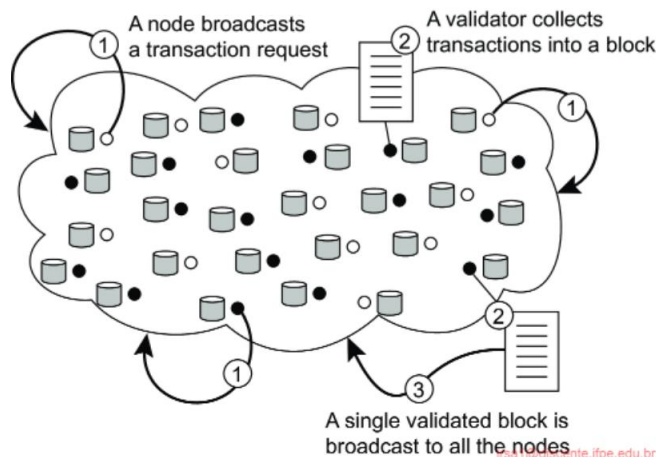


Figura 2.29: O princípio de operação de uma blockchain.

Para esse fim, quando Alice quer transferir \$ 10 para Bob, ela essencialmente conta a todos os participantes no sistema blockchain sobre essa intenção, permitindo assim que voluntários validem a transação pretendida. Isso é mostrado como Etapa 1 na Figura 2.29. Para evitar ter que verificar cada transação separadamente, uma por uma, conforme elas são enviadas, um validador agrupa várias transações em um bloco para aumentar a eficiência, mostrado como Etapa 2 na Figura 2.29. Se tudo correr bem, ou seja, as transações no bloco forem consideradas válidas, o validador protege o bloco com segurança contra quaisquer modificações e anexa o bloco agora imutável à cadeia de outros blocos com transações validadas. Ele faz isso transmitindo o bloco validado para todos os participantes, mostrado como Etapa 3 na Figura 2.29.

Uma observação importante é que há logicamente apenas uma única cadeia de blocos com transações validadas. Cada bloco é imutável, no sentido de que se um adversário decidir modificar qualquer transação de qualquer bloco naquela cadeia, a modificação nunca pode passar despercebida. Proteger com segurança blocos de transações contra modificações, mas também anexar com segurança um bloco a uma lista existente, são técnicas bem compreendidas, que explicaremos mais adiante.

na Seção 9.4.3. A imutabilidade de um bloco o torna um ótimo ideal para replicação massiva: ele nunca será alterado de qualquer forma, então alguém pode muito bem armazená-lo localmente para tornar a verificação o mais simples possível. Efetivamente, a cadeia logicamente única de blocos imutáveis pode ser fisicamente replicada massivamente pela Internet entre todas as partes participantes. É exatamente isso que acontece: cada bloco é transmitido para cada nó participante em um blockchain, como acabamos de explicar.

O que diferencia tantos sistemas de blockchain uns dos outros é decidir qual(is) nó(s) pode(m) realmente executar tarefas de validação. Em outras palavras, precisamos descobrir quem tem permissão para anexar um bloco de transações validadas à cadeia existente. Anexar tal bloco significa que há um consenso global sobre transações concluídas. Portanto, é importante que também cheguemos a um consenso sobre qual validador pode seguir em frente. Todos os outros terão que fazer sua validação novamente, pois suas transações podem ser afetadas pelas recém-anexadas e, portanto, podem precisar ser revisitadas.

Decidir qual validador pode seguir adiante requer **consenso (distribuído)**. Em princípio, há três opções:

1. Uma solução centralizada, na qual um terceiro confiável valida as transações como antes.
2. Uma solução distribuída, na qual um pequeno grupo pré-selecionado de processos assume o papel de um terceiro confiável.
3. Uma solução totalmente descentralizada, na qual, em princípio, todos os nós participantes do blockchain chegam a um consenso em conjunto, sem qualquer terceiro (distribuído).

Essas três opções são mostradas na Figura 2.30. Conforme mencionado, cada nó participante do blockchain é assumido como tendo uma cópia completa disponível localmente.

Obviamente, uma arquitetura centralizada para um blockchain não atende aos seus objetivos de design, que afirmam que essencialmente não há lugar para um terceiro confiável. A arquitetura distribuída é interessante. Neste caso, há um grupo relativamente pequeno de nós que têm permissão para validar transações.

Para blockchains, é importante perceber que nenhum desses nós permissionados é considerado confiável, mas eles são considerados como executando um protocolo de consenso que pode suportar comportamento malicioso. Especificamente, se houver n nós permissionados, então é assumido que no máximo k ($n \geq 1/3$) falhará e talvez agirá maliciosamente. Um problema com tais **blockchains** chamados **permissionados** é que n é bastante limitado, na prática, a menos de algumas dezenas de nós.

Finalmente, nos chamados **blockchains permissionless**, todos os nós participam coletivamente para validar transações. Na prática, isso significa que todos os nós que querem validar transações estão envolvidos em um processo chamado **eleição de líder**. O processo eleito como líder anexa um bloco à cadeia atual (e é

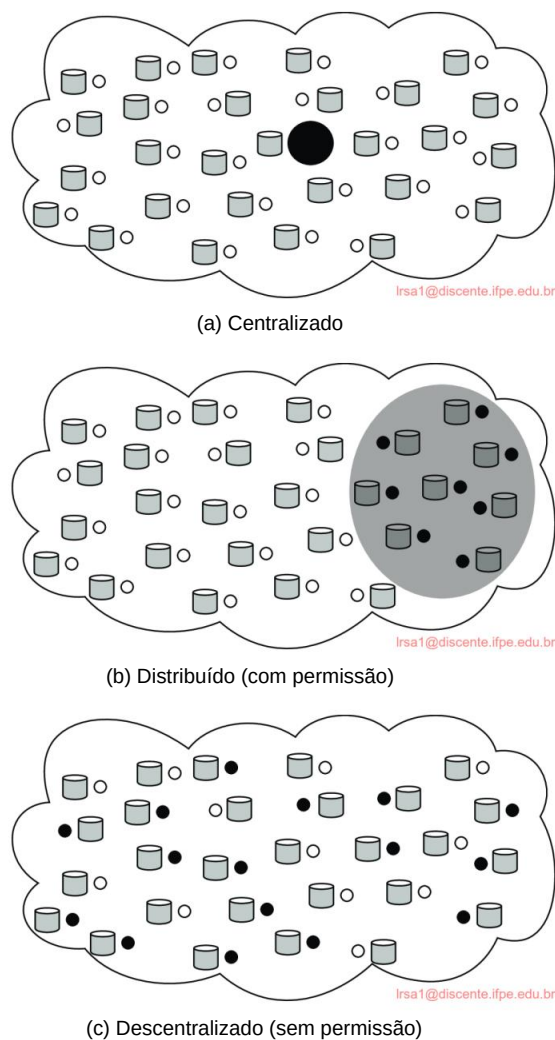


Figura 2.30: As três diferentes organizações de blockchains: (a) centralizada, (b) distribuída, (c) totalmente descentralizada. Os nós preenchidos representam validadores; outros nós são participantes não envolvidos na validação.

frequentemente recompensado por isso). Na prática, nem todos os nós participantes desejam atuar como validadores, mesmo porque o algoritmo de eleição de líder é custoso em termos de recursos. Retornamos às eleições de líder na Seção 5.4.

A arquitetura de um sistema blockchain é, portanto, vista como bastante complexa. Em um sistema permissionado, temos algumas dezenas de nós para validar transações. Nenhum desses nós precisa ser individualmente confiável de antemão, mas pode-se argumentar que eles formam um grupo distribuído centralizado e tolerante a falhas. Confiança

é necessário na medida em que precisamos assumir que não muitos desses nós agem maliciosamente ou conspiram contra quaisquer decisões que eles deveriam tomar.

Por outro lado, blockchains sem permissão podem ser vistos como sendo totalmente descentralizados, mas aqui vemos que medidas especiais são necessárias para garantir alguma forma de justiça entre validadores dispostos. De fato, por meio da dinâmica de blockchains sem permissão, frequentemente vemos que, na prática, apenas um número relativamente pequeno de nós pode realizar tarefas de validação, efetivamente levando também a um sistema mais centralizado. Uma visão geral das várias configurações em blockchains da perspectiva de arquiteturas é dada por Xu et al. [2017].

2.6 Resumo

Sistemas distribuídos podem ser organizados de muitas maneiras. Podemos fazer uma distinção entre arquitetura de software e arquitetura de sistema. A última considera onde os componentes que constituem um sistema distribuído são colocados nas várias máquinas. A primeira está mais preocupada com a organização lógica do software: como os componentes interagem, de que maneiras eles podem ser estruturados, como eles podem ser tornados independentes, e assim por diante?

Uma palavra-chave quando se fala sobre arquiteturas é estilo arquitetônico. Um estilo rege o princípio básico que é seguido na organização da interação entre os componentes de software que compõem um sistema distribuído. Estilos importantes incluem camadas, estilos orientados a serviços e estilos nos quais os eventos de manipulação são proeminentes, exemplificados por são conhecidos como estilos de publicação-assinatura.

Existem muitas organizações de sistemas distribuídos. Uma classe importante é onde as máquinas são divididas em clientes e servidores. Um cliente envia uma solicitação a um servidor, que então produzirá um resultado que é retornado ao cliente. A arquitetura cliente-servidor reflete a maneira tradicional de modularizar software, na qual um módulo chama as funções disponíveis em outro módulo. Ao colocar diferentes componentes em diferentes máquinas, obtemos uma distribuição física natural de funções em uma coleção de máquinas.

Arquiteturas cliente-servidor são frequentemente altamente centralizadas. Em arquiteturas descentralizadas, frequentemente vemos um papel igual desempenhado pelos processos que constituem um sistema distribuído, também conhecidos como sistemas peer-to-peer. Em sistemas peer-to-peer, os processos são organizados em uma rede overlay, que é uma rede lógica na qual cada processo tem uma lista local de outros peers com os quais pode se comunicar. A rede overlay pode ser estruturada, caso em que esquemas determinísticos podem ser implantados para rotear mensagens entre processos. Em redes não estruturadas, a lista de peers é mais ou menos aleatória, o que implica que algoritmos de busca precisam ser implantados para localizar dados ou outros processos.

Em arquiteturas híbridas, elementos de organizações centralizadas e descentralizadas são combinados. Um exemplo típico é o da computação em nuvem, que segue logicamente uma arquitetura cliente-servidor, mas onde o servidor é geralmente

completamente distribuídos em um data center. Na última década, vimos um forte surgimento do que é conhecido como edge computing. As infraestruturas de edge formam várias etapas entre os dispositivos finais e as nuvens e são exigentes do ponto de vista da organização e configuração de sistemas distribuídos. Finalmente, como um exemplo em que a descentralização desempenha um papel proeminente, a arquitetura blockchain cada vez mais popular ilustra outra classe de arquiteturas de sistemas híbridos.

A comunicação entre processos está no coração de todos os sistemas distribuídos. Não faz sentido estudar sistemas distribuídos sem examinar cuidadosamente as maneiras pelas quais processos em diferentes máquinas podem trocar informações. A comunicação em sistemas distribuídos tradicionalmente sempre foi baseada na passagem de mensagens de baixo nível, conforme oferecido pela rede subjacente. Expressar a comunicação por meio da passagem de mensagens é mais difícil do que usar primitivos baseados em memória compartilhada, conforme disponível para plataformas não distribuídas. Os sistemas distribuídos modernos geralmente consistem em milhares ou até milhões de processos espalhados por uma rede com comunicação não confiável, como a Internet. A menos que as facilidades primitivas de comunicação das redes de computadores sejam substituídas por outra coisa, o desenvolvimento de aplicações distribuídas em larga escala é extremamente difícil.

Neste capítulo, começamos discutindo as regras que os processos de comunicação devem seguir, conhecidas como protocolos, e nos concentramos em estruturar esses protocolos na forma de camadas. Em seguida, olhamos para dois modelos amplamente usados para comunicação: Remote Procedure Call (RPC) e Message-Oriented Middleware (MOM). Também discutimos o problema geral de enviar dados para vários receptores, chamado multicasting.

Nosso primeiro modelo para comunicação em sistemas distribuídos é a chamada de procedimento remoto (RPC). Uma RPC visa esconder a maioria das complexidades da passagem de mensagens e é ideal para aplicativos cliente-servidor. No entanto, realizar RPCs transparentemente é mais fácil dizer do que fazer. Observamos vários detalhes importantes que não podem ser ignorados, enquanto mergulhamos no código real para ilustrar até que ponto a transparência da distribuição pode ser realizada de forma que o desempenho ainda seja aceitável.

Em muitas aplicações distribuídas, a comunicação não segue o padrão bastante estrito de interação cliente-servidor. Nesses casos, acontece que pensar em termos de mensagens é mais apropriado. Os recursos de comunicação de baixo nível das redes de computadores não são adequados de muitas maneiras, novamente devido à sua falta de transparência de distribuição. Uma alternativa é usar um modelo de enfileiramento de mensagens de alto nível, no qual a comunicação procede da mesma forma que em sistemas de e-mail. A comunicação orientada a mensagens é um assunto importante o suficiente para garantir uma seção própria. Analisamos vários aspectos, incluindo roteamento em nível de aplicativo.

Finalmente, como nossa compreensão da configuração de instalações de multidifusão melhorou, soluções novas e elegantes para disseminação de dados surgiram. Damos atenção separada a esse assunto na última seção deste capítulo, discutindo meios determinísticos tradicionais de multicasting, bem como abordagens probabilísticas como as usadas em *flooding* e *gossiping*. Estas últimas têm recebido muita atenção nos últimos anos devido à sua elegância, confiabilidade e simplicidade.

4.1 Fundações

Antes de começarmos nossa discussão sobre comunicação em sistemas distribuídos, primeiro recapitulamos algumas questões fundamentais relacionadas à comunicação. Na próxima seção, discutimos brevemente os protocolos de comunicação de rede, pois eles formam a base para qualquer sistema distribuído. Depois disso, adotamos uma abordagem diferente ao classificar os diferentes tipos de comunicação que geralmente ocorrem em sistemas distribuídos.

4.1.1 Protocolos em camadas

Devido à ausência de memória compartilhada, toda a comunicação em sistemas distribuídos é baseada no envio e recebimento de mensagens (baixo nível). Quando o processo P quer se comunicar com o processo Q, ele primeiro cria uma mensagem em seu próprio espaço de endereço. Então ele executa uma chamada de sistema que faz com que o sistema operacional envie a mensagem pela rede para Q. Embora essa ideia básica pareça simples o bastante, para evitar o caos, P e Q precisam concordar sobre o significado dos bits que estão sendo enviados.

O modelo de referência OSI

Para facilitar o tratamento dos vários níveis e problemas envolvidos na comunicação, a International Standards Organization (ISO) desenvolveu um modelo de referência que identifica claramente os vários níveis envolvidos, dá a eles nomes padrão e aponta qual nível deve fazer qual trabalho. Esse modelo é chamado de **Open Systems Interconnection Reference Model** [Day e Zimmerman, 1983], geralmente abreviado como **ISO OSI** ou, às vezes, apenas o **modelo OSI**. Deve-se enfatizar que os protocolos que foram desenvolvidos como parte do modelo OSI nunca foram amplamente usados e estão essencialmente mortos. No entanto, o próprio modelo subjacente provou ser bastante útil para entender redes de computadores. Embora não pretendamos dar uma descrição completa desse modelo e todas as suas implicações aqui, uma breve introdução será útil.

Para mais detalhes, consulte [Tanenbaum et al., 2021].

O modelo OSI é projetado para permitir que sistemas abertos se comuniquem. Um sistema aberto é aquele que está preparado para se comunicar com qualquer outro sistema aberto usando regras padrão que governam o formato, o conteúdo e o significado das mensagens enviadas e recebidas. Essas regras são formalizadas no que são chamados **de protocolos de comunicação**. Para permitir que um grupo de computadores se comunique em uma rede, todos eles devem concordar com os protocolos a serem usados. Diz-se que um protocolo fornece um **serviço de comunicação**. Existem dois tipos de tais serviços. No caso de um **serviço orientado a conexão**, antes de trocar dados, o remetente e o destinatário primeiro estabelecem explicitamente uma conexão e possivelmente negociam parâmetros específicos do protocolo que usarão. Quando terminam, eles liberam (encerram) a conexão. O telefon

serviço de comunicação orientado a conexão típico. Com **serviços sem conexão**, nenhuma configuração prévia é necessária. O remetente apenas transmite a primeira mensagem quando ela estiver pronta. Deixar uma carta em uma caixa de correio é um exemplo de uso de serviço de comunicação sem conexão. Com computadores, tanto a comunicação orientada a conexão quanto a sem conexão são comuns.

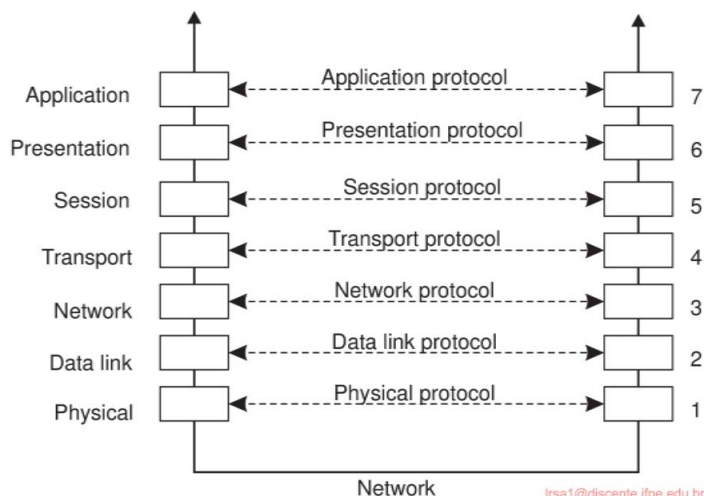


Figura 4.1: Camadas, interfaces e protocolos no modelo OSI.

No modelo OSI, a comunicação é dividida em sete níveis ou camadas, como mostrado na Figura 4.1. Cada camada oferece um ou mais serviços de comunicação específicos para a camada acima dela. Dessa forma, o problema de levar uma mensagem de A para B pode ser dividido em partes gerenciáveis, cada uma das quais pode ser resolvida independentemente das outras. Cada camada fornece uma **interface** para a camada acima dela. A interface consiste em um conjunto de operações que, juntas, definem o serviço que a camada está preparada para oferecer. As sete camadas OSI são:

Camada física Trata da padronização de como dois computadores são conectados e como 0s e 1s são representados.

Camada de enlace de dados Fornece os meios para detectar e possivelmente corrigir erros de transmissão, bem como protocolos para manter o remetente e o destinatário no mesmo ritmo.

Camada de rede Contém os protocolos para rotear uma mensagem através de uma rede de computadores, bem como protocolos para lidar com congestionamento.

Camada de transporte: contém principalmente protocolos para suporte direto a aplicativos, como aqueles que estabelecem comunicação confiável ou oferecem suporte a streaming de dados em tempo real.

Camada de sessão Fornece suporte para sessões entre aplicativos.

Camada de apresentação Prescreve como os dados são representados de uma forma independente dos hosts nos quais os aplicativos de comunicação estão sendo executados.

Camada de aplicação Basicamente, todo o resto: protocolos de e-mail, protocolos de acesso à Web, protocolos de transferência de arquivos e assim por diante.

Quando um processo P quer se comunicar com algum processo remoto Q, ele cria uma mensagem e passa essa mensagem para a camada de aplicação conforme oferecida a ele por meio de uma interface. Essa interface normalmente aparecerá na forma de um procedimento de biblioteca. O software da camada de aplicação então adiciona um cabeçalho à frente da mensagem e passa a mensagem resultante pela interface da camada 6/7 para a camada de apresentação. A camada de apresentação, por sua vez, adiciona seu próprio cabeçalho e passa o resultado para a camada de sessão, e assim por diante. Algumas camadas adicionam não apenas um cabeçalho à frente, mas também um trailer ao final. Quando atinge o fundo, a camada física realmente transmite a mensagem (que agora pode parecer como mostrado na Figura 4.2) colocando-a no meio de transmissão físico.

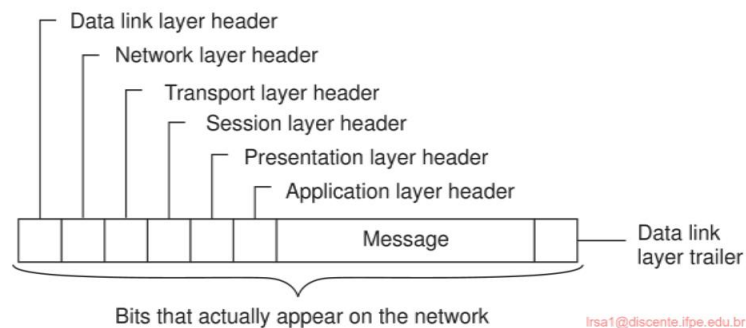


Figura 4.2: Uma mensagem típica conforme aparece na rede.

Quando a mensagem chega à máquina remota que hospeda Q, ela é passada para cima, com cada camada retirando e examinando seu próprio cabeçalho. Finalmente, a mensagem chega ao receptor, o processo Q, que pode responder a ela usando o caminho reverso. As informações no cabeçalho da camada n são usadas para o protocolo da camada n.

No modelo OSI, não há duas camadas, mas sete, como vimos na Figura 4.1. A coleção de protocolos usados em um sistema específico é chamada de **conjunto de protocolos** ou **pilha de protocolos**. É importante distinguir um modelo de referência de seus protocolos reais. Como dito, os protocolos OSI nunca foram populares, em contraste com os protocolos desenvolvidos para a Internet, como TCP e IP.

Nota 4.1 (Mais informações: Protocolos no modelo OSI)

Vamos examinar brevemente cada uma das camadas OSI, começando pela parte inferior. Em vez de dar exemplos de protocolos OSI, quando apropriado, apontaremos alguns protocolos de Internet usados em cada camada.

Protocolos de nível inferior As três camadas mais baixas do conjunto de protocolos OSI implementam as funções básicas que abrangem uma rede de computadores.

A **camada física** se preocupa em transmitir os 0s e 1s. Quantos volts usar para 0 e 1, quantos bits por segundo podem ser enviados e se a transmissão pode ocorrer em ambas as direções simultaneamente são questões-chave na camada física. Além disso, o tamanho e o formato do conector de rede (plugue), bem como o número de pinos e o significado de cada um, são uma preocupação aqui.

O protocolo da camada física lida com a padronização das interfaces elétricas, ópticas, mecânicas e de sinalização para que, quando uma máquina envia um bit 0, ele seja recebido como um bit 0 e não um bit 1. Muitos padrões da camada física foram desenvolvidos (para diferentes mídias), por exemplo, o padrão USB para linhas de comunicação serial.

A camada física apenas envia bits. Enquanto não ocorrerem erros, tudo está bem.

No entanto, redes de comunicação reais estão sujeitas a erros, então algum mecanismo é necessário para detectá-los e corrigi-los. Esse mecanismo é a principal tarefa da **camada de enlace de dados**. O que ela faz é agrupar os bits em unidades, também chamadas de **frames**, e ver se cada frame é recebido corretamente.

A camada de link de dados faz seu trabalho colocando um padrão de bits especial no início e no fim de cada quadro para marcá-los, bem como calculando uma **soma de verificação** somando todos os bytes no quadro de uma certa maneira. A camada de link de dados anexa a soma de verificação ao quadro. Quando o quadro chega, o receptor recalcula a soma de verificação dos dados e compara o resultado com a soma de verificação após o quadro. Se os dois concordarem, o quadro é considerado correto e é aceito. Se eles discordarem, o receptor pede ao remetente para retransmiti-lo. Os quadros recebem números de sequência (no cabeçalho), para que todos possam dizer qual é qual.

Em uma LAN, geralmente não há necessidade de o remetente localizar o destinatário. Ele apenas coloca a mensagem na rede e o destinatário a retira. Uma rede de longa distância, no entanto, consiste em várias máquinas, cada uma com um certo número de linhas para outras máquinas, como um mapa em grande escala mostrando as principais cidades e estradas que as conectam. Para uma mensagem ir do remetente ao destinatário, pode ser necessário fazer vários saltos, em cada um deles escolhendo uma linha de saída para usar.

A questão de como escolher o melhor caminho é chamada de **roteamento** e é essencialmente a tarefa principal da **camada de rede**.

O problema é complicado pelo fato de que a rota mais curta nem sempre é a melhor rota. O que realmente importa é a quantidade de atraso em uma determinada rota, que, por sua vez, está relacionada à quantidade de tráfego e ao número de mensagens enfileiradas para transmissão nas várias linhas. O atraso pode, portanto, mudar ao longo do tempo. Alguns algoritmos de roteamento tentam se adaptar a cargas variáveis, enquanto outros se contentam em tomar decisões com base em médias de longo prazo.

Atualmente, o protocolo de rede mais amplamente usado é o **IP** sem conexão (**Internet Protocol**), que faz parte do conjunto de protocolos da Internet. Um **pacote** IP (o termo técnico para uma mensagem na camada de rede) pode ser enviado sem nenhuma configuração. Cada pacote IP é roteado para seu destino independente de todos os outros. Nenhum caminho interno é selecionado e lembrado.

Protocolos de transporte A **camada de transporte** forma a última parte do que poderia ser chamado de pilha de protocolo de rede básica, no sentido de que implementa todos aqueles serviços que não são fornecidos na interface da camada de rede, mas que são razoavelmente necessários para construir aplicativos de rede. Em outras palavras, a camada de transporte transforma a rede subjacente em algo que um desenvolvedor de aplicativos pode usar.

Pacotes podem ser perdidos no caminho do remetente para o destinatário. Embora alguns aplicativos possam lidar com sua própria recuperação de erros, outros preferem uma conexão confiável. O trabalho da camada de transporte é fornecer esse serviço. A ideia é que a camada de aplicativo deve ser capaz de entregar uma mensagem para a camada de transporte com a expectativa de que ela será entregue sem perdas.

Ao receber uma mensagem da camada de aplicação, a camada de transporte a quebra em pedaços pequenos o suficiente para transmissão, atribui a cada um um número de sequência e então envia todos eles. A discussão no cabeçalho da camada de transporte diz respeito a quais pacotes foram enviados, quais foram recebidos, quantos mais o receptor tem espaço para aceitar, quais devem ser retransmitidos e tópicos semelhantes.

Conexões de transporte confiáveis (que por definição são orientadas à conexão) podem ser construídas sobre serviços de rede orientados à conexão ou sem conexão. No primeiro caso, todos os pacotes chegarão na sequência correta (se chegarem), mas no último caso é possível que um pacote tome uma rota diferente e chegue antes do pacote enviado antes dele. Cabe ao software da camada de transporte colocar tudo de volta para manter a ilusão de que uma conexão de transporte é como um grande tubo — você coloca mensagens nele, e elas saem sem danos e na mesma ordem em que entraram. Fornecer esse comportamento de comunicação de ponta a ponta é um aspecto importante da camada de transporte.

O protocolo de transporte da Internet é chamado **TCP (Transmission Control Protocol)** e é descrito em detalhes por Comer [2013]. A combinação TCP/IP agora é usada como um padrão de fato para comunicação de rede. O conjunto de protocolos da Internet também suporta um protocolo de transporte sem conexão chamado **UDP (Universal Datagram Protocol)**, que é essencialmente apenas IP com algumas pequenas adições. Programas de usuário que não precisam de um protocolo orientado a conexão normalmente usam UDP.

Protocolos de transporte adicionais são propostos regularmente. Por exemplo, para dar suporte à transferência de dados em tempo real, o **Protocolo de Transporte em Tempo Real (RTP)** foi definido. O RTP é um protocolo de estrutura no sentido de que especifica formatos de pacotes para dados em tempo real sem fornecer os mecanismos reais para garantir a entrega de dados. Além disso, ele especifica um protocolo para monitorar e controlar a transferência de dados de pacotes RTP [Schulzrinne et al., 2003]. Da mesma forma, o **Protocolo de Transmissão de Controle de Streaming (SCTP)** foi proposto como uma altern

TCP [Stewart, 2007]. A principal diferença entre SCTP e TCP é que SCTP agrupa dados em mensagens, enquanto TCP apenas move bytes entre processos. Fazer isso pode simplificar o desenvolvimento de aplicativos.

Protocolos de nível superior Acima da camada de transporte, o OSI distingue três camadas adicionais. Na prática, apenas a **camada de aplicação** é usada. Na verdade, no conjunto de protocolos da Internet, tudo acima da camada de transporte é agrupado. Diante dos sistemas de middleware, veremos que nem a abordagem OSI nem a da Internet são realmente apropriadas.

A **camada de sessão** é essencialmente uma versão aprimorada da camada de transporte. Ela fornece controle de diálogo, para manter o controle de qual parte está falando no momento, e fornece recursos de sincronização. Os últimos são úteis para permitir que os usuários insiram pontos de verificação em transferências longas, de modo que, no caso de uma falha, seja necessário voltar apenas ao último ponto de verificação, em vez de voltar todo o caminho até o início. Na prática, poucas aplicações estão interessadas na camada de sessão, e ela raramente é suportada. Ela nem está presente no conjunto de protocolos da Internet. No entanto, no contexto do desenvolvimento de soluções de middleware, o conceito de uma sessão e seus protocolos relacionados se mostraram bastante relevantes, notavelmente ao definir protocolos de comunicação de nível superior.

Ao contrário das camadas inferiores, que se preocupam em levar os bits do remetente ao destinatário de forma confiável e eficiente, a **camada de apresentação** se preocupa com o significado dos bits. A maioria das mensagens não consiste em sequências de bits aleatórias, mas em informações mais estruturadas, como nomes de pessoas, endereços, quantias de dinheiro e assim por diante. Na camada de apresentação, é possível definir registros contendo campos como esses e, então, fazer com que o remetente notifique o destinatário de que uma mensagem contém um registro específico em um determinado formato. Isso torna mais fácil para máquinas com diferentes representações internas se comunicarem entre si.

A camada de aplicação OSI foi originalmente planejada para conter uma coleção de aplicações de rede padrão, como aquelas para correio eletrônico, transferência de arquivos e emulação de terminal. Agora, ela se tornou o contêiner para todas as aplicações e protocolos que, de uma forma ou de outra, não se encaixam em uma das camadas subjacentes. Da perspectiva do modelo de referência OSI, praticamente todos os sistemas distribuídos são apenas aplicativos.

O que está faltando neste modelo é uma distinção clara entre aplicativos, protocolos específicos de aplicativos e protocolos de propósito geral. Por exemplo, o Internet **File Transfer Protocol (FTP)** [Postel e Reynolds, 1985; Horowitz e Lunt, 1997] define um protocolo para transferir arquivos entre uma máquina cliente e servidor. O protocolo não deve ser confundido com o programa ftp, que é um aplicativo de usuário final para transferir arquivos e que também (não por coincidência) implementa o Internet FTP.

Outro exemplo de um protocolo típico específico de aplicação é o **HyperText Transfer Protocol (HTTP)** [Fielding e Reschke, 2014], que é projetado para gerenciar e manipular remotamente a transferência de páginas da Web. O protocolo é implementado por aplicativos como navegadores da Web e servidores da Web. No entanto, o HTTP agora também é usado por sistemas que não estão intrinsecamente vinculados à Web. Por exemplo,

O mecanismo de invocação de objetos do Java pode usar HTTP para solicitar a invocação de objetos remotos protegidos por um firewall.

Há também muitos protocolos de propósito geral que são úteis para muitas aplicações, mas que não podem ser qualificados como protocolos de transporte. Frequentemente, tais protocolos se enquadram na categoria de protocolos de middleware.

Protocolos de middleware

Middleware é um aplicativo que logicamente vive (principalmente) na camada de aplicativo OSI, mas que contém muitos protocolos de propósito geral que garantem suas próprias camadas, independentemente de outros aplicativos mais específicos. Vamos dar uma olhada rápida em alguns exemplos.

O **Sistema de Nomes de Domínio (DNS)** [Liu e Albitz, 2006] é um serviço distribuído usado para procurar um endereço de rede associado a um nome, como o endereço de um chamado **nome de domínio**, como `www.distributed-systems.net`. Em termos do modelo de referência OSI, o DNS é um aplicativo e, portanto, é logicamente colocado na camada de aplicativo. No entanto, deve ser bastante óbvio que o DNS está oferecendo um serviço de uso geral e independente de aplicativo. Pode-se argumentar que ele faz parte do middleware.

Como outro exemplo, há várias maneiras de estabelecer **autenticação**, ou seja, fornecer prova de uma identidade reivindicada. Protocolos de autenticação não estão intimamente ligados a nenhum aplicativo específico, mas, em vez disso, podem ser integrados a um sistema de middleware como um serviço geral. Da mesma forma, **protocolos de autorização** pelos quais usuários e processos autenticados recebem acesso apenas aos recursos para os quais têm autorização, tendem a ter uma natureza geral e independente do aplicativo. Sendo rotulados como aplicativos no modelo de referência OSI, esses são exemplos claros que pertencem ao middleware.

Protocolos de commit distribuídos estabelecem que em um grupo de processos, possivelmente espalhados por várias máquinas, todos os processos realizam uma operação específica ou que a operação não é realizada. Esse fenômeno também é conhecido como **atomicidade** e é amplamente aplicado em transações. Acontece que os protocolos de confirmação podem apresentar uma interface independentemente de aplicativos específicos, fornecendo assim um serviço de transação de propósito geral. Nesse formato, eles normalmente pertencem ao middleware e não à camada de aplicação OSI.

Como último exemplo, considere um **protocolo de bloqueio distribuído**, pelo qual um recurso pode ser protegido contra acesso simultâneo por uma coleção de processos que são distribuídos em várias máquinas. Não é difícil imaginar que tais protocolos podem ser projetados de forma independente de aplicativo e acessíveis por meio de uma interface relativamente simples, novamente independente de aplicativo. Como tal, eles geralmente pertencem ao middleware.

Esses exemplos de protocolo não estão diretamente vinculados à comunicação, mas existem

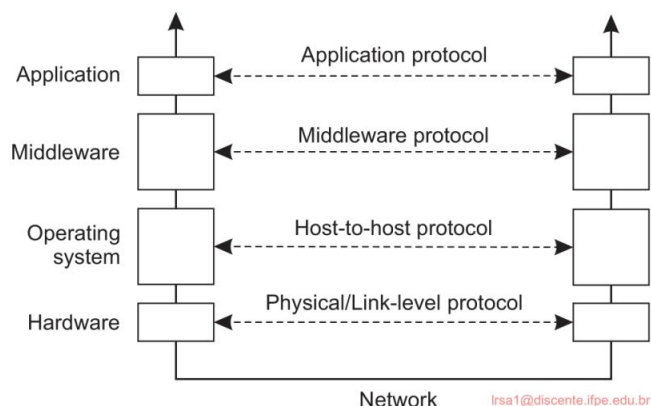


Figura 4.3: Um modelo de referência adaptado para comunicação em rede.

também são muitos protocolos de comunicação de middleware. Por exemplo, com uma **chamada de procedimento remoto**, um processo recebe uma facilidade para chamar localmente um procedimento que é efetivamente implementado em uma máquina remota. Este serviço de comunicação pertence a um dos tipos mais antigos de serviços de middleware e é usado para realizar transparência de acesso. Em uma linha semelhante, existem serviços de comunicação de alto nível para definir e sincronizar fluxos para transferir dados em tempo real, como os necessários para aplicativos multimídia. Como último exemplo, alguns sistemas de middleware oferecem serviços multicast confiáveis que podem ser dimensionados para milhares de receptores espalhados por uma rede de longa distância.

Adotar essa abordagem para camadas leva ao modelo de referência adaptado e simplificado para comunicação, conforme mostrado na Figura 4.3. Comparado ao modelo OSI, a camada de sessão e apresentação foi substituída por uma única camada de middleware que contém protocolos independentes de aplicativo.

Esses protocolos não pertencem às camadas inferiores que acabamos de discutir. Serviços de rede e transporte foram agrupados em serviços de comunicação, como normalmente oferecidos por um sistema operacional, que, por sua vez, gerencia o hardware específico de nível mais baixo usado para estabelecer a comunicação.

4.1.2 Tipos de Comunicação

No restante deste capítulo, nos concentramos em serviços de comunicação de middleware de alto nível. Antes de fazer isso, há outros critérios gerais para distinguir a comunicação (middleware). Para entender as várias alternativas em comunicação que o middleware pode oferecer aos aplicativos, vemos o middleware como um serviço adicional na computação cliente-servidor, conforme mostrado na Figura 4.4. Considere, por exemplo, um sistema de e-mail. Em princípio, o núcleo do sistema de entrega de e-mail pode ser visto como um serviço de comunicação de middleware. Cada host executa um agente de usuário permitindo que os usuários componham, enviem e recebam e-mails. Um agente de usuário de envio passa esse e-mail para o serviço de entrega de e-mail.

sistema, esperando que ele, por sua vez, eventualmente entregue o e-mail ao destinatário pretendido. Da mesma forma, o agente do usuário no lado do destinatário se conecta ao sistema de entrega de e-mail para ver se algum e-mail chegou. Se sim, as mensagens são transferidas para o agente do usuário para que possam ser lidas pelo usuário.

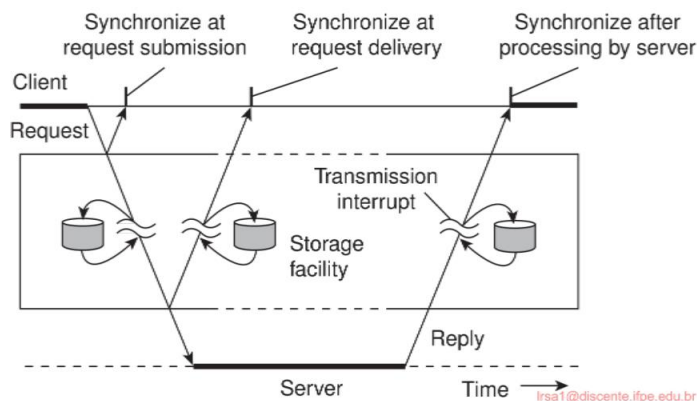


Figura 4.4: Visualizando o middleware como um serviço intermediário (distribuído) na comunicação em nível de aplicativo.

Um sistema de e-mail é um exemplo típico em que a comunicação é persistente. Com a **comunicação persistente**, uma mensagem que foi enviada para transmissão é armazenada pelo middleware de comunicação pelo tempo que for necessário para entregá-la ao receptor. Nesse caso, o middleware armazenará a mensagem em uma ou várias das instalações de armazenamento mostradas na Figura 4.4. Como consequência, não é necessário que o aplicativo de envio continue a execução após enviar a mensagem. Da mesma forma, o aplicativo de recebimento não precisa estar em execução quando a mensagem for enviada.

Em contraste, com a **comunicação transitória**, uma mensagem é armazenada pelo sistema de comunicação somente enquanto o aplicativo de envio e recebimento estiver em execução. Mais precisamente, em termos da Figura 4.4, se o middleware não puder entregar uma mensagem devido a uma interrupção de transmissão, ou porque o destinatário não está ativo no momento, ela será simplesmente descartada. Normalmente, todos os serviços de comunicação em nível de transporte oferecem apenas comunicação transitória. Nesse caso, o sistema de comunicação consiste em roteadores tradicionais de armazenar e encaminhar. Se um roteador não puder entregar uma mensagem para o próximo ou para o host de destino, ele simplesmente descartará a mensagem.

Além de ser persistente ou transitória, a comunicação também pode ser assíncrona ou síncrona. A característica da **comunicação assíncrona** é que um remetente continua imediatamente após ter enviado sua mensagem para transmissão. Isso significa que a mensagem é (temporariamente) armazenada imediatamente pelo middleware após o envio. Com a **comunicação síncrona**, o remetente é bloqueado até que sua solicitação seja conhecida como aceita.

Existem essencialmente três pontos onde a sincronização pode ocorrer. Primeiro,

o remetente pode ser bloqueado até que o middleware notifique que ele assumirá a transmissão da solicitação. Segundo, o remetente pode sincronizar até que sua solicitação tenha sido entregue ao destinatário pretendido. Terceiro, a sincronização pode ocorrer deixando o remetente esperar até que sua solicitação tenha sido totalmente processada, ou seja, até o momento em que o destinatário retornar uma resposta.

Várias combinações de persistência e sincronização ocorrem na prática.

Os mais populares são persistência em combinação com sincronização no envio de solicitação, que é um esquema comum para muitos sistemas de enfileiramento de mensagens, que discutiremos mais adiante neste capítulo. Da mesma forma, comunicação transitória com sincronização após a solicitação ter sido totalmente processada também é amplamente usada. Esse esquema corresponde a chamadas de procedimento remoto, que discutiremos na seção a seguir.

4.2 Chamada de procedimento remoto

Muitos sistemas distribuídos foram baseados em troca explícita de mensagens entre processos. No entanto, as operações `send` e `receive` não ocultam a comunicação de forma alguma, o que é importante para atingir a transparência de acesso em sistemas distribuídos. Esse problema é conhecido há muito tempo, mas pouco foi feito sobre ele até que pesquisadores na década de 1980 [Birrell e Nelson, 1984] introduziram uma maneira totalmente diferente de lidar com a comunicação. Embora a ideia seja agradavelmente simples (uma vez que alguém tenha pensado nela), as implicações são frequentemente sutis. Nesta seção, examinaremos o conceito, sua implementação, seus pontos fortes e fracos.

Em poucas palavras, a proposta era permitir que programas chamassem procedimentos localizados em outras máquinas. Quando um processo em uma máquina A chama um procedimento em uma máquina B, o processo de chamada em A é suspenso, e a execução do procedimento chamado ocorre em B. As informações podem ser transportadas do chamador para o chamado nos parâmetros e podem retornar no resultado do procedimento. Nenhuma mensagem passada é visível para o programador. Este método é conhecido como **chamada de procedimento remoto**, ou frequentemente apenas **RPC**.

Embora a ideia básica pareça simples e elegante, existem problemas sutis. Para começar, como os procedimentos de chamada e chamados são executados em máquinas diferentes, eles são executados em espaços de endereço diferentes, o que causa complicações. Parâmetros e resultados também precisam ser passados, o que pode ser complicado, especialmente se as máquinas não forem idênticas. Finalmente, uma ou ambas as máquinas podem travar, e cada uma das falhas possíveis causa problemas diferentes. Ainda assim, a maioria deles pode ser tratada, e RPC é uma técnica amplamente usada que fundamenta muitos sistemas distribuídos.

4.2.1 Operação básica do RPC

A ideia por trás do RPC é fazer com que uma chamada de procedimento remoto pareça o máximo possível com uma local. Em outras palavras, queremos que o RPC seja transparente — o

o procedimento de chamada não deve estar ciente de que o procedimento chamado está sendo executado em uma máquina diferente ou vice-versa. Suponha que um programa tenha acesso a um banco de dados que permite que ele anexe dados a uma lista armazenada, após o que ele retorna uma referência à lista modificada. A operação é disponibilizada a um programa por uma rotina `append`:

```
novalista = anexar(dados, dbList)
```

Em um sistema tradicional (processador único), `append` é extraído de uma biblioteca pelo linker e inserido no programa objeto. Em princípio, pode ser um procedimento curto, que pode ser implementado por algumas operações de arquivo para acessar o banco de dados.

Embora `append` eventualmente faça apenas algumas operações básicas de arquivo, ele é chamado da maneira usual, empurrando seus parâmetros para a pilha. O programador não conhece os detalhes de implementação de `append`, e é assim, claro, que ele deve ser.

Nota 4.2 (Mais informações: Chamadas de procedimentos convencionais)

Para entender como o RPC funciona e algumas de suas armadilhas, pode ajudar entender primeiro como funciona uma chamada de procedimento convencional (ou seja, de máquina única). Considere a operação `newlist = append(data, dbList)`; Assumimos que o propósito desta chamada é pegar um objeto de lista definido globalmente, aqui chamado de `dbList`, e anexar um elemento de dados simples a ele, representado pela variável `data`. Uma observação importante é que em várias linguagens de programação, como C, `dbList` é implementado como uma referência a um objeto de lista (ou seja, um ponteiro), enquanto `data` pode ser representado diretamente por seu valor (o que assumimos ser o caso aqui). Ao chamar `append`, ambas as representações de `data` e `dbList` são empurradas para a pilha, tornando essas representações acessíveis à implementação de `append`. Para `data`, isso significa que a variável segue uma política de **chamada por valor**, a política para `dbList` é **chamada por referência**. O que acontece antes e durante a chamada é mostrado na Figura 4.5.

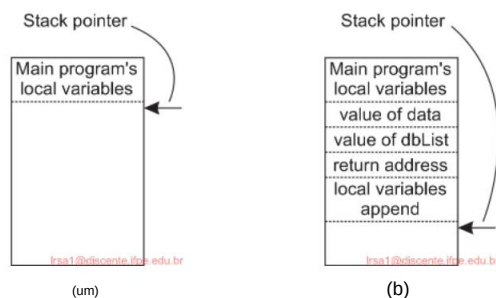


Figura 4.5: (a) Passagem de parâmetros em uma chamada de procedimento local: a pilha antes da chamada para anexar. (b) A pilha enquanto o procedimento chamado está ativo.

Várias coisas valem a pena notar. Por um lado, um parâmetro de valor, como dados, é apenas uma variável local inicializada. O procedimento chamado pode modificá-lo, mas tais mudanças não afetam o valor original no lado da chamada.

Quando um parâmetro como `dbList` é, na verdade, um ponteiro para uma variável em vez do valor da variável, algo mais acontece. O que é empurrado para a pilha é o endereço do objeto da lista, conforme armazenado na memória principal. Quando o valor de data é anexado à lista, uma chamada para `append` modifica o objeto da lista. A diferença entre chamada por valor e chamada por referência é muito importante para RPC.

Um outro mecanismo de passagem de parâmetros também existe, embora não seja usado na maioria das linguagens de programação. É chamado de **chamada por cópia/restauração**. Consiste em ter a variável copiada para a pilha pelo chamador, como em chamada por valor, e então copiada de volta após a chamada, sobrescrevendo o valor original do chamador. Na maioria das condições, isso atinge o mesmo efeito que chamada por referência, mas em algumas situações, como o mesmo parâmetro estar presente várias vezes na lista de parâmetros, a semântica é diferente.

A decisão de qual mecanismo de passagem de parâmetros usar normalmente é feita pelos designers da linguagem e é uma propriedade fixa da linguagem. De vez em quando, depende do tipo de dado que está sendo passado. Em C, por exemplo, inteiros e outros tipos escalares são sempre passados por valor, enquanto arrays são sempre passados por referência. Alguns compiladores Ada usam `copy/restore` para parâmetros inout, mas outros usam `call-by-reference`. A definição da linguagem permite qualquer escolha, o que torna a semântica um pouco confusa. Em Python, todas as variáveis são passadas por referência, mas algumas são realmente copiadas para variáveis locais, imitando assim o comportamento de `copy-by-value`.

O RPC atinge sua transparência analogamente. Quando `append` é, na verdade, um procedimento remoto, uma versão diferente de `append`, chamada de **client stub**, é oferecida ao cliente que faz a chamada. Como o original, ele também é chamado usando uma sequência de chamada normal. No entanto, diferente do original, ele não executa uma operação `append`. Em vez disso, ele empacota os parâmetros em uma mensagem e solicita que essa mensagem seja enviada ao servidor, conforme ilustrado na Figura 4.6. Após a chamada para enviar, o stub do cliente chama `receive`, bloqueando-se até que a resposta seja retornada.

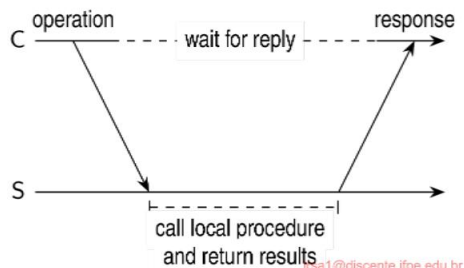


Figura 4.6: O princípio do RPC entre um programa cliente e um programa servidor.

Quando a mensagem chega ao servidor, o sistema operacional do servidor a passa para um **stub de servidor**. Um stub de servidor é o equivalente do lado do servidor de um stub de cliente: é um pedaço de código que transforma solicitações que chegam pela rede em chamadas de procedimento local. Normalmente, o stub de servidor terá chamado recebe e será bloqueado esperando por mensagens de entrada. O stub de servidor descompacta os parâmetros da mensagem e então chama o procedimento do servidor da maneira usual. Da perspectiva do servidor, é como se estivesse sendo chamado diretamente pelo cliente — os parâmetros e o endereço de retorno estão todos na pilha onde pertencem e nada parece incomum. O servidor executa seu trabalho e então retorna o resultado para o chamador (neste caso, o stub de servidor) da maneira usual.

Quando o stub do servidor recupera o controle após a chamada ter sido concluída, ele empacota o resultado em uma mensagem e chama send para retorná-lo ao cliente. Depois disso, o stub do servidor geralmente faz uma chamada para receber novamente, para esperar pela próxima solicitação de entrada.

Quando a mensagem de resultado chega à máquina do cliente, o sistema operacional a passa pela operação de recebimento, que foi chamada anteriormente, para o stub do cliente, e o processo do cliente é posteriormente desbloqueado. O stub do cliente inspeciona a mensagem, descompacta o resultado, copia-o para seu chamador e retorna da maneira usual. Quando o chamador obtém o controle após a chamada para append, tudo o que ele sabe é que ele anexou alguns dados a uma lista. Ele não tem ideia de que o trabalho foi feito remotamente em outra máquina.

Essa ignorância feliz para o cliente é a beleza de todo o esquema. No que lhe diz respeito, serviços remotos são acessados fazendo chamadas de procedimento comuns (ou seja, locais), não chamando send e recebe. Todos os detalhes da passagem de mensagens são escondidos nos dois procedimentos de biblioteca, assim como os detalhes de realmente fazer chamadas de sistema são escondidos em bibliotecas tradicionais.

Para resumir, uma chamada de procedimento remoto ocorre nas seguintes etapas:

1. O procedimento do cliente chama o stub do cliente da maneira normal.
2. O stub do cliente cria uma mensagem e chama o sistema operacional local.
3. O sistema operacional do cliente envia a mensagem para o sistema operacional remoto.
4. O sistema operacional remoto envia a mensagem para o stub do servidor.
5. O stub do servidor descompacta o(s) parâmetro(s) e chama o servidor.
6. O servidor faz o trabalho e retorna o resultado ao stub.
7. O stub do servidor empacota o resultado em uma mensagem e chama seu sistema operacional local.
8. O sistema operacional do servidor envia a mensagem para o sistema operacional do cliente.
9. O sistema operacional do cliente envia a mensagem para o stub do cliente.
10. O stub descompacta o resultado e o retorna ao cliente.

Os primeiros passos são mostrados na Figura 4.7 para um procedimento abstrato de dois parâmetros `doit(a,b)`, onde assumimos que o parâmetro `a` é do tipo `type1`, e `b` do tipo `type2`. O efeito líquido de todos esses passos é converter a chamada local pelo procedimento cliente para o stub cliente, para uma chamada local para o procedimento servidor,

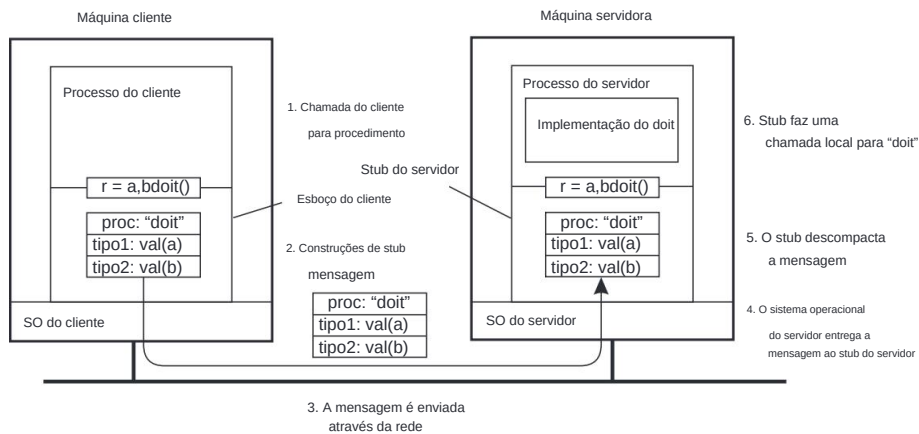


Figura 4.7: Os passos envolvidos na chamada de um procedimento remoto `doit(a,b)`. O caminho de retorno para o resultado não é mostrado.

sem que o cliente ou o servidor estejam cientes das etapas intermediárias ou da existência da rede.

Nota 4.3 (Mais informações: Um exemplo em Python)

Para tornar as coisas concretas, vamos considerar como uma chamada de procedimento remoto poderia ser implementada para a operação `append` discutida anteriormente. Dê uma olhada no código Python mostrado na Figura 4.8 (do qual omitimos fragmentos de código não essenciais).

A classe `DBList` é uma representação simples de um objeto de lista, imitando o que se esperaria ver em uma versão encontrada em um ambiente de banco de dados. O stub do cliente, representado pela classe `Client`, consiste em uma implementação de `append`. Quando chamado com os parâmetros `data` e `dbList`, acontece o seguinte.

A chamada é transformada em uma tupla (`APPEND, data, dbList`) contendo todas as informações que o servidor precisaria para fazer seu trabalho. O stub do cliente então envia o `request off` para o servidor e, subsequentemente, aguarda a resposta. No pacote `channel`, uma operação `recvFrom` sempre retorna um par (`sender,message`), permitindo que o chamador identifique o processo que enviou a mensagem. No nosso caso, quando a resposta chega, o stub do cliente finaliza a chamada para `append` simplesmente passando a mensagem retornada do servidor para o programa que inicialmente chamou o stub.

No lado do servidor, vemos que no stub do servidor, o servidor espera por qualquer mensagem de entrada e inspeciona qual operação ele precisa chamar. Novamente, o pacote `channel` retorna o identificador do remetente original (linha 24). Supondo que o servidor recebeu uma solicitação para chamar `append`, ele então simplesmente faz uma chamada local para sua implementação de `append` com os parâmetros apropriados, como também encontrados na tupla de solicitação. O resultado é então enviado de volta para o cliente identificado.

Observe que um cliente real simplesmente chamaria `c.append(...)` onde `c` é um instância da classe `Client`. De fato, a chamada parece realmente ocorrer localmente.

```

1 canal de importação , picles
2
3 classes DBList:
4 def anexar(self, dados):
5     self.value.extend(dados)
6     retornar a si mesmo
7
8 Cliente de 8ª classe :
9 def append(self, dados, dbList):
10    msglst = (APPEND, data, dbList) # carga útil da mensagem
11    self.channel.sendTo(self.server, msglst) # enviar mensagem para o servidor
12    msgrcv = self.channel.recvFrom(self.server) # aguarde uma mensagem recebida
13
14    # Uma chamada para recvFrom retorna um par [senderID, mensagem]
15    retornar msgrcv[1] # passar mensagem retornada ao chamador
16
17 Servidor de classe 17 :
18 def append(self, dados, dbList):
19     retornar dbList.append(dados)
20
21 def executar(próprio):
22     enquanto Verdadeiro:
23         msgreq = self.channel.recvFromAny() # aguarde qualquer solicitação
24         cliente = msgreq[0] # veja quem é o chamador
25         msgrpc = msgreq[1] # buscar a requisição real
26
27         # Neste ponto, msgreq deve ter o formato (operação, dados, lista)
28         if APPEND == msgrpc[0]: # verifique o que está sendo solicitado
29             resultado = self.append(msgrpc[1], msgrpc[2]) # faz chamada local
30             self.channel.sendTo(cliente, resultado) # retornar resposta

```

Figura 4.8: Um exemplo simples de RPC para operação append.

4.2.2 Passagem de parâmetros

A função do stub do cliente é pegar seus parâmetros e compactá-los em um mensagem e enviá-los para o stub do servidor. Embora isso pareça simples, não é tão simples quanto parece à primeira vista.

Empacotar parâmetros em uma mensagem é chamado de **marshaling de parâmetros**. Voltando à nossa operação append, precisamos garantir que seus dois parâmetros (dados e dbList) são enviados pela rede e interpretados corretamente pelo servidor. A coisa a perceber é que o servidor estará apenas vendo uma série de bytes recebidos que constituem a mensagem original enviada pelo cliente. No entanto, normalmente não há informações adicionais sobre o que esses bytes significam. fornecido com a mensagem. Além disso, estaríamos enfrentando o mesmo problema novamente: como a meta-informação deve ser reconhecida como tal pelo servidor?

Além deste problema de interpretação, também precisamos lidar com o caso em que a colocação de bytes na memória pode diferir entre arquiteturas de máquina. Em particular, precisamos de ter em conta o facto de algumas máquinas, como Os processadores Intel numeram seus bytes da direita para a esquerda, enquanto muitos outros, como os processadores ARM mais antigos, numere-os de outra forma (agora o ARM suporta ambos). O formato Intel é chamado **little endian** e o (mais antigo) ARM formato é chamado **big endian**. A ordenação de bytes também é importante para redes: também aqui podemos testemunhar que as máquinas podem usar uma ordenação diferente quando transmitindo (e assim recebendo) bits e bytes. No entanto, big endian é o que é normalmente usado para transferir bytes através de uma rede.

A solução para este problema é transformar os dados que serão enviados para um formato independente de máquina e rede, além de garantir que ambos as partes que se comunicam esperam que o mesmo tipo de dados de mensagem seja transmitido. Este último pode ser resolvido tipicamente no nível das linguagens de programação. o primeiro é realizado usando rotinas dependentes de máquina que transformam dados de e para formatos independentes de máquina e rede.

O empacotamento e o desempacotamento têm tudo a ver com essa transformação para o neutro formata e constitui uma parte essencial das chamadas de procedimentos remotos.

Nota 4.4 (Mais informações: Um exemplo em Python revisitado)

```

1 canal de importação , pickles
2
3 Cliente de 3 classes :
4 def append(self, dados, dbList):
5     msglst = (APPEND, data, dbList) # carga útil da mensagem
6     msgsnd = pickle.dumps(msglst) # chamada de encapsulamento
7     self.channel.sendTo(self.server, msgsnd) # enviar solicitação ao servidor
8     msgrcv = self.channel.recvFrom(self.server) # aguarde resposta
9     retval = pickle.loads(msgrcv[1]) retornar retval          # desembrolhar valor de retorno
10                                     # passe para o chamador
11
12 Servidor de 12 classes :
13 def executar(próprio):
14     enquanto Verdadeiro:
15         msgreq = self.channel.recvFromAny() # aguarde qualquer solicitação
16         cliente = msgreq[0] # veja quem é o chamador
17         msgrpc = pickle.loads(msgreq[1]) # desembrolhar a chamada
18         if APPEND == msgrpc[0]: # verifique o que está sendo solicitado
19             resultado = self.append(msgrpc[1], msgrpc[2]) # faz chamada local
20             msgres = pickle.dumps(resultado) # encapsula o resultado
21             self.channel.sendTo(client, msgres) # enviar resposta

```

Figura 4.9: Um exemplo simples de RPC para a operação append, mas agora com organização adequada.

Não é difícil ver que a solução para chamada de procedimento remoto, como mostrado na Figura 4.8, não funcionará em geral. Somente se o cliente e o servidor estiverem operando em máquinas que obedecem às mesmas regras de ordenação de bytes e têm as mesmas representações de máquina para estruturas de dados, a troca de mensagens, como mostrado, levará a interpretações corretas. Uma solução robusta é mostrada na Figura 4.9 (onde novamente omitimos o código para brevidade).

Neste exemplo, usamos a biblioteca Python pickle para marshaling e unmarshaling de estruturas de dados. Note que o código dificilmente muda em comparação ao que mostramos na Figura 4.8. As únicas mudanças ocorrem logo antes do envio e após o recebimento de uma mensagem. Note também que tanto o cliente quanto o servidor são programados para trabalhar nas mesmas estruturas de dados, como discutimos acima.

(Observamos que, nos bastidores, sempre que uma instância de classe é passada como parâmetro, o Python geralmente cuida de conservar e depois desconservar a instância. Nesse sentido, nosso uso explícito de pickles não é, estritamente falando, necessário.)

Agora chegamos a um problema difícil: como ponteiros, ou em geral, referências são passados? A resposta é: somente com a maior das dificuldades, se é que são. Um ponteiro é significativo somente dentro do espaço de endereço do processo no qual está sendo usado. Voltando ao nosso exemplo de append, declaramos que o segundo parâmetro, dbList, é implementado por meio de uma referência a uma lista armazenada em um banco de dados. Se essa referência for apenas um ponteiro para uma estrutura de dados local em algum lugar na memória principal do chamador, não podemos simplesmente passá-la para o servidor. O valor do ponteiro transferido provavelmente estará se referindo a algo totalmente diferente.

Uma solução é simplesmente proibir ponteiros e parâmetros de referência em geral. No entanto, eles são tão importantes que essa solução é altamente indesejável. Na verdade, muitas vezes também não é necessária. Primeiro, os parâmetros de referência são frequentemente usados com tipos de dados de tamanho fixo, como matrizes estáticas, ou com tipos de dados dinâmicos para os quais é fácil calcular seu tamanho em tempo de execução, como strings ou matrizes dinâmicas. Nesses casos, podemos simplesmente copiar toda a estrutura de dados à qual o parâmetro está se referindo, substituindo efetivamente o mecanismo de cópia por referência por cópia por valor/restauração. Embora isso nem sempre seja semanticamente idêntico, frequentemente é bom o suficiente. Uma otimização óbvia é que quando o stub do cliente sabe que os dados referenciados serão apenas lidos, não há necessidade de copiá-los de volta quando a chamada for concluída. A cópia por valor é, portanto, boa o suficiente.

Tipos de dados mais intrincados podem frequentemente ser suportados também, e certamente se uma linguagem de programação suportar esses tipos de dados. Por exemplo, uma linguagem como Python ou Java suporta classes definidas pelo usuário, permitindo que um sistema de linguagem forneça marshaling e unmarshaling totalmente automatizados desses tipos de dados. Note, entretanto, que assim que estivermos lidando com estruturas de dados dinâmicas grandes, aninhadas ou de outra forma intrincadas, o (un)marshaling automático pode não estar disponível, ou mesmo desejável.

O problema com ponteiros e referências, como discutido até agora, é que eles fazem sentido apenas localmente: eles se referem a locais de memória que têm

significando apenas para o processo de chamada. Problemas podem ser aliviados usando referências globais: referências que são significativas para o processo de chamada e o processo chamado. Por exemplo, se o cliente e o servidor têm acesso ao mesmo sistema de arquivos, passar um identificador de arquivo em vez de um ponteiro pode resolver. Há uma observação importante: ambos os processos precisam saber exatamente o que fazer quando uma referência global é passada. Em outras palavras, se considerarmos uma referência global tendo um tipo de dado associado, o processo de chamada e o processo chamado devem ter a mesma imagem das operações que podem ser executadas. Além disso, ambos os processos devem ter acordo sobre exatamente o que fazer quando um identificador de arquivo é passado. Novamente, esses são problemas típicos que podem ser resolvidos pelo suporte adequado de linguagem de programação. Retornaremos a esse assunto em breve.

Nota 4.5 (Avançado: Passagem de parâmetros em sistemas baseados em objetos)

Sistemas baseados em objetos frequentemente usam referências globais. Considere a situação em que todos os objetos no sistema podem ser acessados de máquinas remotas. Nesse caso, podemos usar consistentemente **referências de objetos** como parâmetros em invocações de métodos. As referências são passadas por valor e, portanto, copiadas de uma máquina para outra. Quando um processo recebe uma referência de objeto como resultado de uma invocação de método, ele pode simplesmente se vincular ao objeto referenciado quando necessário posteriormente (consulte também a Seção 2.1.2).

Infelizmente, usar apenas objetos distribuídos pode ser altamente ineficiente, especialmente quando os objetos são pequenos, como inteiros ou, pior ainda, booleanos. Cada invocação por um cliente que não está co-localizado no mesmo servidor que o objeto, gera uma solicitação entre diferentes espaços de endereço ou, pior ainda, entre diferentes máquinas. Portanto, referências a objetos remotos e aquelas a objetos locais são frequentemente tratadas de forma diferente.

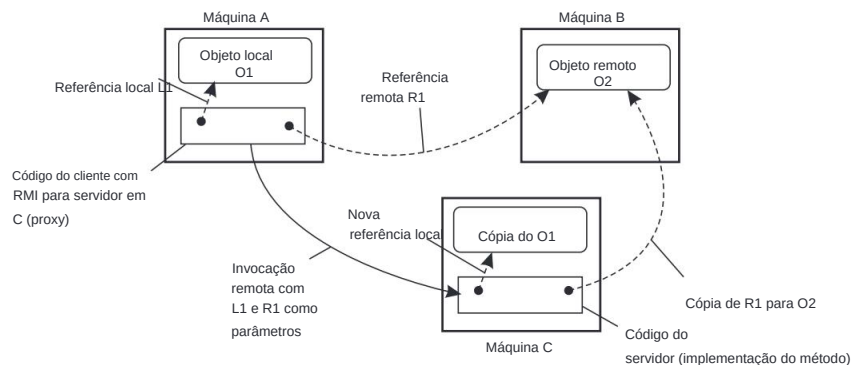


Figura 4.10: Passando um objeto por referência ou por valor.

Ao invocar um método com uma referência de objeto como parâmetro, essa referência é copiada e passada como um parâmetro de valor somente quando se refere a um objeto remoto.

Neste caso, o objeto é literalmente passado por referência. No entanto, quando a referência se refere a um objeto local, que é um objeto no mesmo espaço de endereço que o cliente, o objeto referenciado é copiado como um todo e passado junto com a invocação. Em

em outras palavras, o objeto é passado por valor.

Essas duas situações são ilustradas na Figura 4.10, que mostra um programa cliente em execução em uma máquina A e um programa servidor em uma máquina C. O cliente tem uma referência a um objeto local O1 que ele usa como parâmetro ao chamar o programa servidor na máquina C. Além disso, ele mantém uma referência a um objeto remoto O2 que reside na máquina B, que também é usado como parâmetro. Ao chamar o servidor, uma cópia de O1 é passada para o servidor na máquina C, junto com apenas uma cópia da referência a O2.

Note que se estamos lidando com uma referência a um objeto local ou uma referência a um objeto remoto pode ser altamente transparente, como em Java. Em Java, a distinção é visível apenas porque objetos locais são essencialmente de um tipo de dados diferente de objetos remotos. Caso contrário, ambos os tipos de referências são tratados da mesma forma (veja também [Wollrath et al., 1996]). Por outro lado, ao usar linguagens de programação convencionais como C, uma referência a um objeto local pode ser tão simples quanto um ponteiro, que nunca pode ser usado para se referir a um objeto remoto.

O efeito colateral de invocar um método com uma referência de objeto como parâmetro é que podemos estar copiando um objeto. Obviamente, esconder esse aspecto é inaceitável, de modo que somos consequentemente forçados a fazer uma distinção explícita entre objetos locais e distribuídos. Claramente, essa distinção não apenas viola a transparência da distribuição, mas também dificulta a escrita de aplicativos distribuídos.

Agora também podemos explicar facilmente como referências globais podem ser implementadas ao usar linguagens portáteis e interpretadas, como Python ou Java: use o stub do cliente inteiro como referência. A observação principal é que um stub do cliente é frequentemente apenas outra estrutura de dados que é compilada em bytecode (portátil). Esse código compilado pode realmente ser transferido pela rede e executado no lado do receptor. Em outras palavras, não há mais necessidade de vinculação explícita; simplesmente copiar o stub do cliente para o destinatário é o suficiente para permitir que este último invoque o objeto associado do lado do servidor. Claro, pode ser necessário organizar esse código antes de enviá-lo para a outra máquina, embora, estritamente falando, não haja razão para que essa outra máquina funcione com layouts diferentes.

4.2.3 Suporte a aplicativos baseados em RPC

Pelo que explicamos até agora, está claro que ocultar uma chamada de procedimento remoto requer que o chamador e o chamado concordem com o formato das mensagens que trocam e que sigam os mesmos passos quando se trata de, por exemplo, passar estruturas de dados complexas. Em outras palavras, ambos os lados em um RPC devem seguir o mesmo protocolo ou o RPC não funcionará corretamente. Há pelo menos duas maneiras pelas quais o desenvolvimento de aplicativos baseados em RPC pode ser suportado. A primeira é deixar um desenvolvedor especificar exatamente o que precisa ser chamado remotamente, a partir do qual stubs completos do lado do cliente e do lado do servidor podem ser gerados. Uma segunda abordagem é incorporar a chamada de procedimento remoto como parte de um ambiente de linguagem de programação.

Geração de stub

Considere a função `someFunction` da Figura 4.11(a). Ela tem três parâmetros, um caractere, um número de ponto flutuante e uma matriz de cinco inteiros. Supondo que uma palavra tenha quatro bytes, o protocolo RPC pode prescrever que devemos transmitir um caractere no byte mais à direita de uma palavra (deixando os próximos três bytes vazios), um `float` como uma palavra inteira e uma matriz como um grupo de palavras cujo tamanho é igual ao comprimento da matriz, precedido por uma palavra que fornece o comprimento, conforme mostrado na Figura 4.11(b). Portanto, dadas essas regras, o stub do cliente para `someFunction` sabe que deve usar o formato da Figura 4.11(b), e o stub do servidor sabe que as mensagens recebidas para `someFunction` terão o formato da Figura 4.11(b).

```
void algumaFunção(char x; float y; int z[5])
```

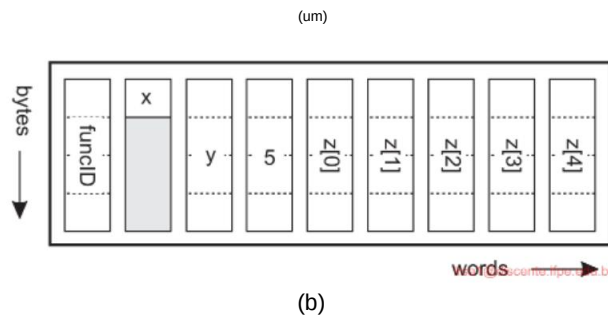


Figura 4.11: (a) Uma função. (b) A mensagem correspondente e a ordem em que bytes e palavras são enviados pela rede.

Definir o formato da mensagem é um aspecto de um protocolo RPC, mas não é suficiente. O que também precisamos é que o cliente e o servidor concordem com a representação de estruturas de dados simples, como inteiros, caracteres, booleanos, etc. Por exemplo, o protocolo poderia prescrever que inteiros são representados em complemento de dois, caracteres em Unicode de 16 bits e `floats` no formato padrão IEEE #754, com tudo armazenado em little endian. Com essas informações adicionais, as mensagens podem ser interpretadas de forma inequívoca.

Com as regras de codificação agora fixadas até o último bit, a única coisa que resta a ser feita é que o chamador e o chamado concordem com a troca real de mensagens. Por exemplo, pode ser decidido usar um serviço de transporte orientado a conexão, como TCP/IP. Uma alternativa é usar um serviço de datagrama não confiável e deixar o cliente e o servidor implementarem um esquema de controle de erros como parte do protocolo RPC. Na prática, existem várias variantes, e cabe ao desenvolvedor indicar o serviço de comunicação subjacente preferido.

Uma vez que o protocolo RPC foi totalmente definido, os stubs do cliente e do servidor precisam ser implementados. Felizmente, stubs para o mesmo protocolo, mas diferentes

procedimentos, normalmente diferem apenas em sua interface com os aplicativos. Uma interface consiste em uma coleção de procedimentos que podem ser chamados por um cliente e que são implementados por um servidor. Uma interface geralmente está disponível na mesma linguagem de programação em que o cliente ou servidor é escrito (embora isso não seja, estritamente falando, necessário). Para simplificar as coisas, as interfaces geralmente são especificadas por meio de uma **Interface Definition Language (IDL)**.

Uma interface especificada em tal IDL é então subsequentemente compilada em um stub de cliente e um stub de servidor, juntamente com as interfaces de tempo de compilação ou tempo de execução apropriadas. Esse processo é esboçado na Figura 4.12.

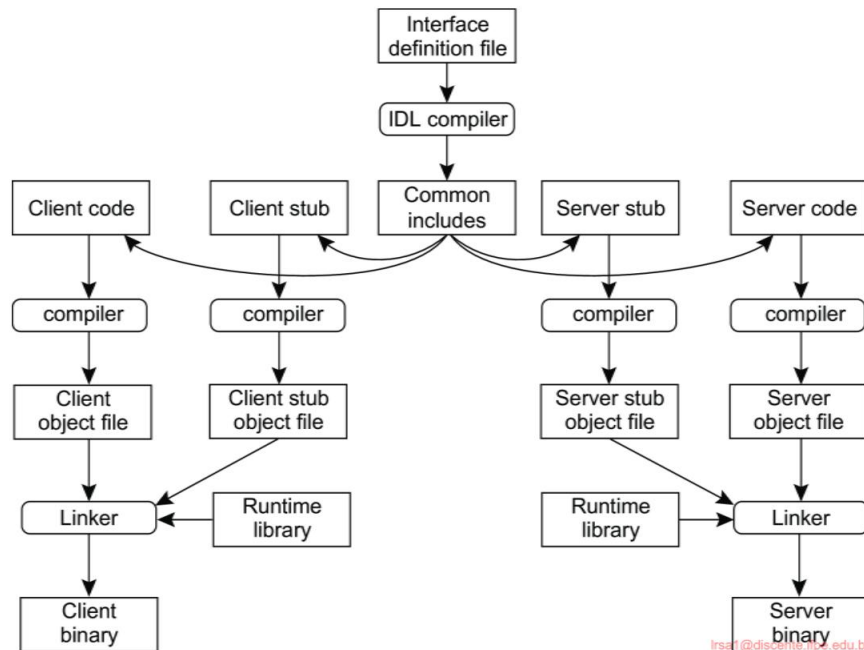


Figura 4.12: O princípio de geração de stubs a partir de um arquivo de definição de interface.

A figura mostra a situação para linguagens compiladas, mas não é difícil imaginar que uma situação muito semelhante se aplique a linguagens interpretadas. Observe também que é possível que o código do cliente e do servidor sejam escritos em linguagens diferentes. Não há nenhuma razão principal pela qual o lado do cliente não possa consistir em componentes escritos em, por exemplo, Python, enquanto o lado do servidor é escrito em C. Claro, nesse caso, a caixa na Figura 4.12 representando inclusões comuns conterá arquivos separados para o cliente e o servidor, respectivamente. Importante neste esquema é a biblioteca de tempo de execução: ela forma a interface para o sistema de tempo de execução real, constituindo o middleware.

A prática mostra que usar uma linguagem de definição de interface simplifica consideravelmente os aplicativos cliente-servidor baseados em RPCs. Como é fácil

gerar stubs de cliente e servidor, todos os sistemas de middleware baseados em RPC oferecem um IDL para dar suporte ao desenvolvimento de aplicativos. Em alguns casos, usar o IDL é ainda mais obrigatório.

Nota 4.6 (Mais informações: RPC baseado em linguagem em Python)

Vejamos com um exemplo como a chamada de procedimento remoto pode ser integrada em um linguagem. Temos usado a linguagem Python para a maioria dos nossos exemplos e continuará a fazê-lo agora também. Na Figura 4.13 mostramos um servidor simples para nosso Estrutura de dados DBList. Neste caso, ele tem duas operações expostas: `exposed_append` para anexar elementos e `exposed_value` para exibir o que está atualmente no lista. Usamos o pacote Python RPyC para incorporar RPCs.

```

1 importação rpyc
2 de rpyc.utils.server importar ForkingServer
3
4 classe DBList(rpyc.Service):
5     valor = [] # Usado para construir uma lista de strings
6
7     def exposed_append(self, dados):
8         self.value.extend(str(data)) # Estenda a lista com os dados
9         return self.value # Retorna a lista atual
10
11 Servidor de 11 classes :
12 # Crie um servidor de bifurcação no momento da inicialização e inicie-o imediatamente.
13 # Para cada solicitação recebida, o servidor gerará outro processo para lidar
14 # essa solicitação. O processo que iniciou o servidor (principal) pode simplesmente matar
15 # quando for a hora de fazer isso.
16 def __init__(próprio):
17     self.server = ForkingServer(DBList, nome do host=SERVIDOR, porta=PORTA)
18
19 def iniciar(próprio):
20     self.servidor.iniciar()
21
22 Cliente da classe 22 :
23 def executar(próprio):
24     conn = rpyc.connect(SERVER, PORT) # Conecte-se ao servidor
25     conexão.root.exposed_append(2) # Chamar uma operação exposta,
26     conn.root.exposed_append(4) # e anexar dois elementos
27     print(conn.root.exposed_value()) # Imprime o resultado

```

Figura 4.13: Incorporando RPCs em uma linguagem

O cliente também é mostrado na Figura 4.13. Quando uma conexão é feita com o servidor, uma nova instância do DBList será criada e o cliente poderá imediatamente anexar valores à lista. As operações expostas são chamadas sem mais delongas.

Observe que quando o cliente interrompe a conexão com o servidor, a lista será perdida. É um **objeto transitório** e medidas especiais precisarão ser tomadas para torná-lo um **objeto persistente**.

Suporte baseado em idioma

A abordagem descrita até agora é amplamente independente de uma linguagem de programação específica. Como alternativa, também podemos incorporar chamadas de procedimentos remotos em uma linguagem em si. O principal benefício é que o desenvolvimento de aplicativos geralmente se torna muito mais simples. Além disso, atingir um alto grau de transparência de acesso geralmente é mais simples, pois muitos problemas relacionados à passagem de parâmetros podem ser contornados completamente.

Um exemplo bem conhecido no qual a chamada de procedimento remoto é totalmente incorporada é Java, onde um RPC é chamado de **invocação de método remoto (RMI)**. Em essência, um cliente sendo executado por sua própria máquina virtual (Java) pode invocar um método de um objeto gerenciado por outra máquina virtual. Ao simplesmente ler o código-fonte de um aplicativo, pode ser difícil ou mesmo impossível ver se uma invocação de método é para um objeto local ou remoto.

4.2.4 Variações no RPC

Como em chamadas de procedimento convencionais, quando um cliente chama um procedimento remoto, o cliente bloqueará até que uma resposta seja retornada. Esse comportamento estrito de solicitação-resposta é desnecessário quando não há resultado para retornar, ou pode prejudicar a eficiência quando vários RPCs precisam ser executados. A seguir, veremos duas variações do esquema RPC que discutimos até agora.

RPC assíncrono

Para dar suporte a situações nas quais simplesmente não há resultado para retornar ao cliente, os sistemas RPC podem fornecer recursos para o que são chamados de **RPCs assíncronos**. Com RPCs assíncronos, o servidor, em princípio, envia imediatamente uma resposta de volta ao cliente no momento em que a solicitação RPC é recebida, após o que ele chama localmente o procedimento solicitado. A resposta atua como um reconhecimento ao cliente de que o servidor irá processar o RPC. O cliente continuará sem mais bloqueios assim que receber o reconhecimento do servidor. A Figura 4.14(b) mostra como o cliente e o servidor interagem no caso de RPCs assíncronos. Para comparação, a Figura 4.14(a) mostra o comportamento normal de solicitação-resposta.

RPCs assíncronos também podem ser úteis quando uma resposta será retornada, mas o cliente não está preparado para esperar por ela e não fazer nada enquanto isso. Um caso típico é quando um cliente precisa contatar vários servidores independentemente. Nesse caso, ele pode enviar as solicitações de chamada uma após a outra, estabelecendo efetivamente que os servidores operam mais ou menos em paralelo. Depois que todas as solicitações de chamada foram enviadas, o cliente pode começar a esperar que os vários resultados sejam retornados. Em casos como esses, faz sentido organizar a comunicação entre o cliente e o servidor por meio de um RPC assíncrono combinado com um **retorno de chamada**, conforme mostrado na Figura 4.15. Nesse esquema, também conhecido como RPC síncrono **diferido**, o cliente primeiro chama o servidor, aguarda a aceitação,

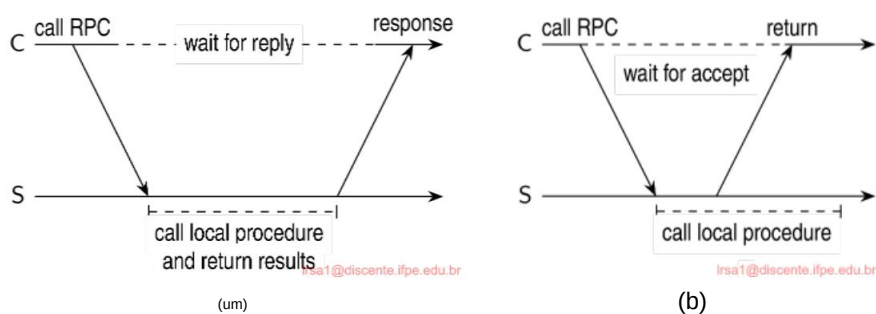


Figura 4.14: (a) A interação entre cliente e servidor em um RPC tradicional. (b) A interação usando RPC assíncrono.

e continua. Quando os resultados se tornam disponíveis, o servidor envia uma mensagem de resposta que leva a um retorno de chamada no lado do cliente. Um retorno de chamada é uma função definida pelo usuário que é invocada quando um evento especial acontece, como uma mensagem recebida. Uma implementação direta é gerar um thread separado e deixá-lo bloquear na ocorrência do evento enquanto o processo principal continua. Quando o evento ocorre, o thread é desbloqueado e chama a função.

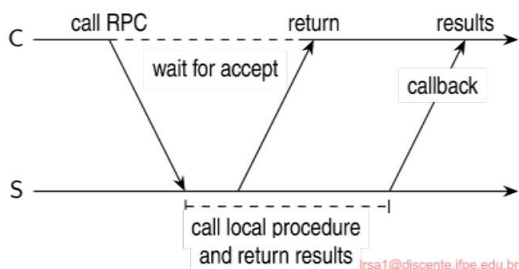


Figura 4.15: Um cliente e um servidor interagindo por meio de um RPC síncrono diferido.

Deve-se observar que existem variantes de RPCs assíncronos nas quais o cliente continua a execução imediatamente após enviar a solicitação ao servidor. Em outras palavras, o cliente não espera por um reconhecimento da aceitação da solicitação pelo servidor. Nós nos referimos a esses RPCs como **RPCs unidirecionais**. O problema com essa abordagem é que quando a confiabilidade não é garantida, o cliente não pode saber com certeza se sua solicitação será processada. Retornaremos a esses assuntos no Capítulo 8. Da mesma forma, no caso de RPC síncrono adiado, o cliente pode consultar o servidor para ver se os resultados já estão disponíveis, em vez de deixar o servidor chamar o cliente de volta.

RPC multidifusão

RPCs assíncronos e síncronos diferidos facilitam outra alternativa às chamadas de procedimentos remotos, ou seja, a execução de vários RPCs ao mesmo tempo. Adotando os RPCs unidirecionais (ou seja, quando um servidor não diz ao cliente que aceitou sua solicitação de chamada, mas imediatamente começa a processá-la, enquanto o cliente continua logo após emitir o RPC), um **RPC multicast** se resume a enviar uma solicitação RPC para um grupo de servidores. Esse princípio é mostrado na Figura 4.16. Neste exemplo, o cliente envia uma solicitação para dois servidores, que subsequentemente processam essa solicitação de forma independente e em paralelo. Quando concluído, o resultado é retornado ao cliente, onde um retorno de chamada ocorre.

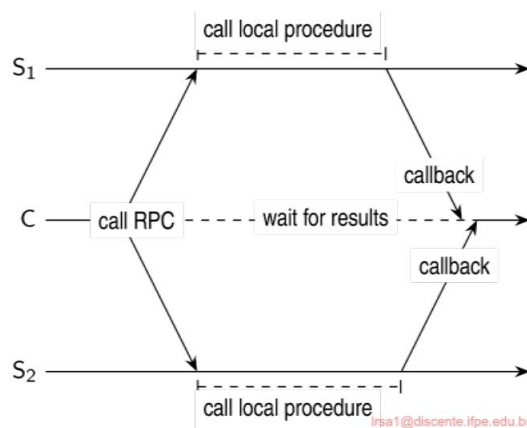


Figura 4.16: O princípio de um RPC multicast.

Há várias questões que precisamos considerar. Primeiro, como antes, o aplicativo cliente pode não estar ciente do fato de que um RPC está realmente sendo encaminhado para mais de um servidor. Por exemplo, para aumentar a tolerância a falhas, podemos decidir ter todas as operações executadas por um servidor de backup que pode assumir quando o servidor principal falhar. Que um servidor foi replicado pode ser completamente escondido de um aplicativo cliente por um stub apropriado. No entanto, mesmo o stub não precisa estar ciente de que o servidor é replicado, por exemplo, porque estamos usando um endereço multicast de nível de transporte.

Segundo, precisamos considerar o que fazer com as respostas. Em particular, o cliente prosseguirá após todas as respostas terem sido recebidas ou esperará apenas por uma? Tudo depende. Quando o servidor tiver sido replicado para tolerância a falhas, podemos decidir esperar apenas pela primeira resposta ou talvez até que a maioria dos servidores retorne o mesmo resultado. Por outro lado, se os servidores tiverem sido replicados para fazer o mesmo trabalho, mas em partes diferentes da entrada, seus resultados podem precisar ser mesclados antes que o cliente possa continuar. Novamente, tais questões podem ser ocultadas no stub do lado do cliente, mas o desenvolvedor do aplicativo terá, no mínimo, que especificar o propósito do RPC multicast.

4.3 Comunicação orientada para mensagens

Chamadas de procedimento remoto e invocações de objeto remoto contribuem para ocultar a comunicação em sistemas distribuídos, ou seja, aumentam a transparência de acesso. Infelizmente, nenhum dos mecanismos é sempre apropriado. Em particular, quando não se pode presumir que o lado receptor está executando no momento em que uma solicitação é emitida, serviços de comunicação alternativos são necessários. Da mesma forma, a natureza síncrona inerente dos RPCs, pela qual um cliente é bloqueado até que sua solicitação seja processada, pode precisar ser substituída por outra coisa.

Esse algo mais é mensagem. Nesta seção, nos concentramos na comunicação orientada a mensagens em sistemas distribuídos, primeiro examinando mais de perto o que exatamente é comportamento síncrono e quais são suas implicações. Em seguida, discutimos sistemas de mensagens que assumem que as partes estão executando no momento da comunicação. Finalmente, examinaremos sistemas de enfileiramento de mensagens que permitem que processos troquem informações, mesmo que a outra parte não esteja executando no momento em que a comunicação é iniciada.

4.3.1 Mensagens transitórias simples com sockets

Muitos sistemas e aplicações distribuídos são construídos diretamente sobre o modelo simples orientado a mensagens oferecido pela camada de transporte. Para entender e apreciar melhor os sistemas orientados a mensagens como parte de soluções de middleware, primeiro discutimos mensagens por meio de soquetes de nível de transporte.

Atenção especial foi dada à padronização da interface da camada de transporte para permitir que os programadores façam uso de todo o seu conjunto de protocolos (de mensagens) por meio de um conjunto simples de operações. Além disso, interfaces padrão facilitam a portabilidade de um aplicativo para uma máquina diferente. Como exemplo, discutimos brevemente a **interface de soquete** conforme introduzida na década de 1970 no Berkeley Unix, e que foi adotada como um padrão POSIX (com apenas pouquíssimas adaptações).

Conceitualmente, um **socket** é um ponto final de comunicação para o qual um aplicativo pode gravar dados que devem ser enviados pela rede subjacente, e do qual os dados de entrada podem ser lidos. Um socket forma uma abstração sobre a porta real que é usada pelo sistema operacional local para um protocolo de transporte específico. No texto a seguir, nos concentramos nas operações de socket para TCP, que são mostradas na Figura 4.17.

Os servidores geralmente executam as quatro primeiras operações, normalmente na ordem dada. Ao chamar a operação de soquete, o chamador cria um novo ponto final de comunicação para um protocolo de transporte específico. Internamente, criar um ponto final de comunicação significa que o sistema operacional local reserva recursos para enviar e receber mensagens para o protocolo específico.

A operação de vinculação associa um endereço local ao soquete recém-criado. Por exemplo, um servidor deve vincular o endereço IP de sua máquina junto com um número de porta (possivelmente bem conhecido) a um soquete. A vinculação informa ao sistema operacional

Descrição da operação	
tomada	Crie um novo ponto final de comunicação
vincular	Anexar um endereço local a um soquete
ouvir	Informe ao sistema operacional qual é o número máximo de pendências os pedidos de conexão devem ser
aceitar	Bloquear o chamador até que uma solicitação de conexão chegue
conectar	Tente ativamente estabelecer uma conexão
enviar	Envie alguns dados pela conexão
receber	Receba alguns dados pela conexão
fechar	Solte a conexão

Figura 4.17: Operações de soquete para TCP/IP.

sistema que o servidor deseja receber mensagens apenas no endereço especiýcado e porta. No caso de comunicação orientada à conexão, o endereço é usado para receber solicitações de conexão de entrada.

A operação de escuta é chamada apenas no caso de conexão orientada comunicação. É uma chamada não bloqueante que permite ao sistema operacional local reservar buffers suficientes para um número máximo especificado de eventos pendentes. solicitações de conexão que o chamador está disposto a aceitar.

Uma chamada para aceitar bloqueia o chamador até que uma solicitação de conexão chegue. Quando chega uma solicitação, o sistema operacional local cria um novo soquete com o mesmas propriedades do original e o retorna ao chamador. Esta abordagem permitirá que o servidor, por exemplo, bifurque um processo que posteriormente lidar com a comunicação real por meio da nova conexão. O servidor pode volte e aguarde outra solicitação de conexão no soquete original.

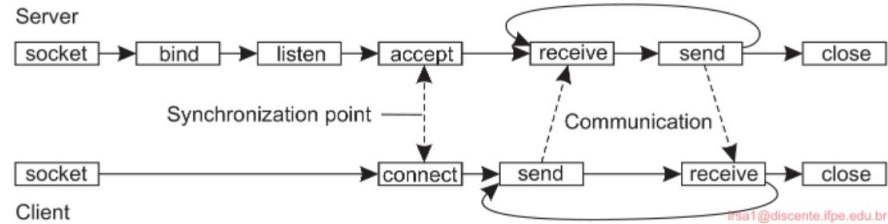


Figura 4.18: Padrão de comunicação orientado à conexão usando sockets.

Vamos agora dar uma olhada no lado do cliente. Aqui, também, um soquete deve primeiro ser criado usando a operação de soquete , mas vinculando explicitamente o soquete a um local endereço não é necessário, pois o sistema operacional pode alocar dinamicamente uma porta quando a conexão é configurada. A operação de conexão requer que o o chamador especifica o endereço de nível de transporte para o qual uma solicitação de conexão deve ser enviada ser enviado. O cliente é bloqueado até que uma conexão seja configurada com sucesso, após o qual ambos os lados podem começar a trocar informações através do envio e receber operações. Finalmente, fechar uma conexão é simétrico ao usar

soquetes e é estabelecido fazendo com que o cliente e o servidor chamem o fechamento operação. Embora existam muitas exceções à regra, o padrão geral seguido por um cliente e servidor para comunicação orientada à conexão usando sockets é como mostrado na Figura 4.18. Detalhes sobre programação de rede usando soquetes e outras interfaces no Unix podem ser encontrados em [Stevens, 1998].

Nota 4.7 (Exemplo: Um sistema cliente-servidor simples baseado em soquete)

Como ilustração do padrão recorrente na Figura 4.18, considere o simples sistema cliente-servidor baseado em soquete mostrado na Figura 4.19 (veja também Nota 2.1). Vemos o servidor começa criando um soquete e, posteriormente, vinculando um endereço a esse soquete. Ele chama a operação de escuta e aguarda uma conexão de entrada solicitação. Quando o servidor aceita uma conexão, a biblioteca de soquetes cria um conexão separada, `conn`, que é usada para receber dados e enviar uma resposta para o cliente conectado. O servidor entra em um loop recebendo e enviando mensagens, até que nenhum dado tenha sido recebido. Então, ele fecha a conexão.

```

1 de importação de soquete *
2
3 Servidor de 3 classes :
4 def executar(próprio):
5     s = soquete(AF_INET, SOCK_STREAM)
6     s.bind((HOST, PORTA))
7     s.ouvir(1)
8     (conn, addr) = s.accept() # retorna novo socket e addr. cliente
9     enquanto Verdadeiro: # para sempre
10        data = conn.recv(1024) # receber dados do cliente
11        se não forem dados: quebrar          # pare se o cliente parou
12        conn.send(dados+b"")                # retorna dados enviados mais um ""
13        conn.close()                        # feche a conexão

```

(um)

1 classe Cliente:

```

2 def executar(próprio):
3     s = soquete(AF_INET, SOCK_STREAM)
4     s.connect((HOST, PORT)) # conectar ao servidor (bloquear até ser aceito)
5     s.send(b"Olá, mundo") # envia os mesmos dados
6     data = s.recv(1024) # recebe a resposta
7     print(dados) # imprima o que você recebeu
8     s.send(b"") # diz ao servidor para fechar
9     s.close() # fecha a conexão

```

(b)

Figura 4.19: Um sistema cliente-servidor simples baseado em soquete.

O cliente novamente segue o padrão da Figura 4.18. Ele cria um soquete e chama `connect` para solicitar uma conexão com o servidor. Uma vez que a conexão tenha estabelecido, ele envia uma única mensagem, aguarda a resposta e, após imprimindo o resultado, fecha a conexão.

Nota 4.8 (Avançado: Implementação de stubs como referências globais revisitada)

Para fornecer uma visão mais aprofundada do funcionamento dos soquetes, vejamos em um exemplo mais elaborado, ou seja, o uso de stubs como referências globais.

```

1 classe DBClient:
2     def __sendrecv(self, message): sock = socket()           # este é um método privado
3         sock.connect((self.host, self.port)) # criar um soquete
4         sock.send(pickle.dumps(message)) # envia alguns dados
5         resultado = pickle.loads(sock.recv(1024)) # recebe a resposta
6         sock.close() # fecha a conexão
7         retornar resultado
8
9
10    def criar(self):
11        self.listID = self.__sendrecv([CRIAR])
12        retornar self.listID
13
14    def obterValor(self):
15        retornar self.__sendrecv([GETVALUE, self.listID])
16
17    def appendData(self, dados):
18        retornar self.__sendrecv([APPEND, dados, self.listID])
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

(um)

Servidor de 1 classe :

```

2     self.conjuntoDeListas = {} # init: nenhuma lista para gerenciar
3
4     def executar(próprio):
5         enquanto Verdadeiro:
6             (conn, addr) = self.sock.accept() # aceitar chamada recebida
7             dados = conn.recv(1024) # buscar dados do cliente
8             solicitação = pickle.loads(dados) # desembulhar a solicitação
9
10            se request[0] == CRIAR: # criar uma lista
11                listID = len(self.setOfLists) + 1 # alocar listID
12                self.setOfLists[ID da lista] = [] # inicializar para vazio
13                conn.send(pickle.dumps(ID da lista)) # ID de retorno
14
15            elif request[0] == APPEND: listID = # solicitação de acréscimo
16                request[2] data = request[1] # buscar listID
17                # buscar dados para anexar
18                self.setOfLists[listID].append(data) # anexa à lista
19                conn.send(pickle.dumps(OK)) # retornar um OK
20
21            elif request[0] == GETVALUE: listID = # solicitação de leitura
22                request[1] resultado = # buscar listID
23                self.setOfLists[listID] # obter os elementos
24                conn.send(pickle.dumps(resultado)) # retorna a lista
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

(b)

Figura 4.20: Implementando um servidor de lista em Python.

Retornamos ao nosso exemplo de implementação de uma lista compartilhada, que agora faça usando um servidor de lista, implementado na forma da classe Python mostrada em Figura 4.20(b). A Figura 4.20(a) mostra a implementação stub de uma lista compartilhada. Novamente, omitimos o código para facilitar a leitura.

A classe DBClient representa um stub do lado do cliente que, uma vez empacotado, pode ser passado entre processos. Ele fornece três operações associadas a uma lista: criar, obter valor e anexar, com semântica óbvia. Um DBClient é assumido como ser associado a uma lista específica, conforme gerenciada pelo servidor. Um identificador para essa lista é retornada quando a lista é criada. Observe como o sendrecv (interno) a operação segue o padrão do lado do cliente explicado na Figura 4.18.

O servidor mantém listas, conforme mostrado na Figura 4.20(b). Seus dados internos estrutura é um setOfLists com cada elemento sendo uma lista criada anteriormente. A o servidor simplesmente espera por solicitações recebidas, desempacota a solicitação e verifica qual operação está sendo solicitada. Os resultados são enviados de volta ao cliente solicitante (que sempre emite a operação sendrecv implementada como parte do DBClient). Novamente, vemos que o servidor segue o padrão mostrado na Figura 4.18: ele cria um soquete, vincula um endereço a ele, informa ao sistema operacional quantos conexões que ele deve ouvir e então esperar para aceitar uma conexão de entrada solicitação. Uma vez estabelecida a conexão, o servidor recebe os dados, envia uma resposta e fecha a conexão novamente.

```

1 classe Cliente:
2 def __init__(self, porta):
3     self.host = 'localhost'          # esta máquina
4     self.port = porta # porta que irá escutar
5     self.sock = socket() # soquete para chamadas recebidas
6     self.sock.bind((self.host, self.port)) # vincular socket a um endereço
7     self.sock.listen(2) # número máximo de conexões
8
9 def sendTo(self, host, porta, dados):
10    meia = soquete()
11    sock.connect((host, porta))      # conectar ao servidor (bloqueando chamada)
12    sock.send(pickle.dumps(data)) # envie alguns dados
13    meia.fechar()
14
15 def recvAny(próprio):
16    (conn, addr) = self.sock.accept()
17    retornar conn.recv(1024)

```

Figura 4.20: (c) Implementando um servidor de lista em Python: o cliente.

Para usar um stub como referência global, representamos cada aplicativo cliente por a classe Client mostrada na Figura 4.20(c). A classe é instanciada da mesma forma processo executando o aplicativo (exemplificado pelo valor de self.host) e irá estar escutando em uma porta específica para mensagens de outros aplicativos, bem como o servidor. Caso contrário, ele apenas envia e recebe mensagens, codificadas por meio do operações sendTo e recvAny, respectivamente.

Agora considere o código mostrado na Figura 4.20(d), que imita dois clientes aplicações. O primeiro cria uma nova lista e anexa dados a ela. Então observe

como `dbClient1` é simplesmente enviado para o outro cliente. Nos bastidores, agora sabemos que é empacotado na operação `sendTo` (linha 12) da classe `Client` mostrada em Figura 4.20(c).

```

1 def cliente1():
2     c1 = Cliente(PORTC1)                # criar cliente
3     dbC1 = DBClient(HOSTS,PORTS) 4      # criar referência
dbC1.create() 5                          # criar nova lista
dbC1.appendData('Cliente 1') 6           # acrescentar alguns dados
c1.sendTo(HOSTC2,PORTC2,dbC1)           # enviar para outro cliente
7
8 def cliente2():
9     c2 = Cliente(PORTC2) 10 dados =    # criar um novo cliente
c2.recvAny() dbC2 = pickle.loads(dados) 12 # bloquear até que os dados sejam enviados
11     dbC2.appendData('Cliente 2')      # receber referência
                                         # anexar dados à mesma lista

```

Figura 4.20: (d) Passagem de stubs como referências.

O segundo cliente simplesmente espera por uma mensagem recebida (linha 12), desempacota o resultado, sabendo que é uma instância `DBClient` e, posteriormente, acrescenta alguns mais dados para a mesma lista que o primeiro cliente anexou dados. Na verdade, um a instância de `DBClient` é vista como passada como uma referência global, aparentemente junto com todas as operações que acompanham a classe associada.

4.3.2 Mensagens transitórias avançadas

A abordagem padrão baseada em soquete para mensagens transitórias é muito básica e, como tal, bastante quebradiço: um erro é facilmente cometido. Além disso, soquetes suportam essencialmente apenas TCP ou UDP, o que significa que qualquer recurso extra para as mensagens precisam ser implementadas separadamente por um programador de aplicativos. Na prática, muitas vezes precisamos de abordagens mais avançadas para mensagens orientadas comunicação para facilitar a programação de rede, para expandir além do funcionalidade oferecida pelos protocolos de rede existentes, para fazer melhor uso recursos locais, e assim por diante.

Usando padrões de mensagens: ZeroMQ

Uma abordagem para tornar a programação de rede mais fácil é baseada em observação de que muitos aplicativos de mensagens, ou seus componentes, podem ser efetivamente organizado de acordo com alguns padrões de comunicação simples. Por posteriormente fornecendo melhorias aos soquetes para cada um desses padrões, pode se tornar mais fácil desenvolver um aplicativo distribuído em rede. Isso

A abordagem foi seguida no ZeroMQ e documentada em [Hintjens, 2013;

[Ägöl, 2013].

Como na abordagem Berkeley, o ZeroMQ também fornece soquetes por meio dos quais toda a comunicação acontece. A transmissão real de mensagens geralmente acontece por meio de conexões TCP e, como o TCP, toda a comunicação é essencialmente orientada à conexão, o que significa que uma conexão será primeiro configurada entre um remetente e um destinatário antes que a transmissão da mensagem possa acontecer.

No entanto, a configuração e a manutenção de conexões são mantidas principalmente sob o capô: um programador de aplicativos não precisa se preocupar com essas questões. Para simplificar ainda mais as coisas, um soquete pode ser vinculado a vários endereços, permitindo efetivamente que um servidor manipule mensagens de fontes muito diferentes por meio de uma única interface. Por exemplo, um servidor pode escutar várias portas usando uma única operação de recebimento de bloqueio. Os soquetes ZeroMQ podem, portanto, suportar comunicação muitos-para-um em vez de apenas comunicação um-para-um, como é o caso dos soquetes Berkeley padrão. Para completar a história: os soquetes ZeroMQ também suportam comunicação um-para-muitos, ou seja, multicasting.

Essencial para ZeroMQ é que a comunicação é **assíncrona**: um remetente normalmente continuará após ter enviado uma mensagem para o subsistema de comunicação subjacente. Um efeito colateral interessante de combinar comunicação assíncrona com comunicação orientada a conexão é que um processo pode solicitar uma configuração de conexão e, subsequentemente, enviar uma mensagem mesmo que o destinatário ainda não esteja instalado e funcionando e pronto para aceitar solicitações de conexão de entrada, muito menos mensagens de entrada. O que acontece, é claro, é que uma solicitação de conexão e mensagens subsequentes são enfileiradas no lado do remetente, enquanto um thread separado como parte da biblioteca do ZeroMQ cuidará para que, eventualmente, a conexão seja configurada e as mensagens sejam transmitidas ao destinatário.

Simplificando as coisas, o ZeroMQ estabelece um nível mais alto de abstração na comunicação baseada em sockets ao parear sockets: um tipo específico de socket usado para enviar mensagens é pareado com um tipo de socket correspondente para receber mensagens. Cada par de tipos de socket corresponde a um **padrão de comunicação**.

Os três padrões de comunicação mais importantes suportados pelo ZeroMQ são solicitação-resposta, publicação-assinatura e pipeline.

O **padrão de solicitação-resposta** é usado na comunicação tradicional cliente-servidor, como as normalmente usadas para chamadas de procedimento remoto. Um aplicativo cliente usa um soquete de solicitação (do tipo REQ) para enviar uma mensagem de solicitação a um servidor e espera que este responda com uma resposta apropriada. Supõe-se que o servidor use um soquete de resposta (do tipo REP). O padrão de solicitação-resposta simplifica as coisas para os desenvolvedores, evitando a necessidade de chamar a operação `listen`, bem como a operação `accept`. Além disso, quando um servidor recebe uma mensagem, uma chamada subsequente para `send` é automaticamente direcionada ao remetente original. Da mesma forma, quando um cliente chama a operação `recv` (para receber uma mensagem) após ter enviado uma mensagem, o ZeroMQ assume que o cliente está esperando por uma resposta do destinatário original. Observe que essa abordagem foi efetivamente codificada na operação `sendrecv` local da Figura 4.20(b), que discutimos na Nota 4.8.

Nota 4.9 (Exemplo: O padrão de solicitação-resposta)

Vejamos um exemplo simples de programação para ilustrar a solicitação-resposta padrão. A Figura 4.21 mostra um servidor que acrescenta um asterisco a uma mensagem recebida. Como antes, ele cria um soquete e o vincula a uma combinação de um protocolo (neste caso caso TCP), e um host e uma porta. Em nosso exemplo, o servidor está disposto a aceitar solicitações de conexão de entrada em duas portas diferentes. Ele então aguarda a entrada mensagens. O padrão de solicitação-resposta vincula efetivamente o recebimento de uma mensagem ao resposta subsequente. Em outras palavras, quando o servidor chama send, ele transmitirá uma mensagem para o mesmo cliente do qual ele havia recebido uma mensagem anteriormente. Claro, essa simplicidade só pode ser alcançada se o programador realmente cumprir o padrão de solicitação-resposta.

```

1 importação zmq
2
3 def servidor():
4     contexto = zmq.Context()
5     soquete = contexto.socket(zmq.REP) 6 soquete.bind("tcp://      # criar soquete de resposta
   *:12345")                               # vincular socket ao endereço
7
8     enquanto Verdadeiro:
9         mensagem = socket.recv() se                                # esperar mensagem recebida
10        não for "STOP" em str(mensagem):                             # se não parar...
11            resposta = str(message.decode())+"*" # anexar "*" à mensagem
12            socket.send(reply.encode()) # envie-o embora (codificado)
13        outro:
14            quebrar                                                # sair do loop e terminar
15
16 def cliente():
17     contexto = zmq.Context()
18     soquete = contexto.socket(zmq.REQ)                               # criar soquete de solicitação
19
20     socket.connect("tcp://localhost:12345") # bloqueia até ser conectado
21     socket.send(b"Olá mundo") 22 mensagem =                         # enviar mensagem
23     socket.recv() 23 socket.send(b"PARAR") 24                         # bloquear até resposta
25     print(mensagem.decode())                                           # diga ao servidor para parar
26                                                                           # imprimir resultado

```

Figura 4.21: Um sistema cliente-servidor ZeroMQ.

O cliente, também mostrado na Figura 4.21, cria um soquete e se conecta ao servidor associado. Quando ele envia uma mensagem, ele pode esperar receber, daquele mesmo servidor, uma resposta. Ao enviar a string "STOP", ele informa ao servidor que está pronto, depois do qual o servidor irá realmente parar.

Curiosamente, a natureza assíncrona do ZeroMQ permite iniciar o cliente antes de iniciar o servidor. Uma implicação é que se, neste exemplo, começássemos o servidor, depois um cliente e depois de um tempo um segundo cliente, que este último será bloqueado até que o servidor seja reiniciado. Além disso, observe que o ZeroMQ não exige que o programador especifique quantos bytes devem ser recebidos. Ao contrário do TCP, o ZeroMQ usa mensagens em vez de fluxos de bytes.

No caso de um **padrão publicar-assinar**, os clientes assinam mensagens específicas que são publicadas por servidores. Nós nos deparamos com esse padrão na Seção 2.1.3 ao discutir coordenação. Na verdade, apenas as mensagens às quais o cliente foi inscrito será transmitido. Se um servidor estiver publicando mensagens para as quais ninguém se inscreveu, essas mensagens serão perdidas. Em sua forma mais simples, isso O padrão estabelece mensagens multicast de um servidor para vários clientes. O o servidor deve usar um soquete do tipo PUB, enquanto cada cliente deve usar SUB soquetes de tipo. Cada soquete do cliente é conectado ao soquete do servidor. Por padrão, um cliente não assina nenhuma mensagem específica. Isso significa que, enquanto como nenhuma assinatura explícita é fornecida, o cliente não receberá uma mensagem publicado pelo servidor.

Nota 4.10 (Exemplo: O padrão publicar-assinar)

Novamente, vamos tornar esse padrão mais concreto por meio de um exemplo simples. A Figura 4.22 mostra um servidor de tempo reconhecidamente ingênuo que publica seu atual, local hora, através de um soquete PUB. A hora local é publicada a cada cinco segundos, para qualquer cliente interessado.

```

1 importação multiprocessing
2 importar zmq, tempo
3
4 def servidor():
5     contexto = zmq.Context()
6     socket = context.socket(zmq.PUB) 7                # criar um soquete de editor
7     socket.bind("tcp://*:12345") 8 while Verdadeiro:    # vincular socket ao endereço
8
9         tempo.sono(5)                                # espere a cada 5 segundos
10        t = "TEMPO " + tempo.asctime()
11        socket.send(t.encode())                        # publicar a hora atual
12
13 def cliente():
14     contexto = zmq.Context()
15     socket = context.socket(zmq.SUB) 16 socket.connect("tcp://    # criar um soquete de assinante
16     localhost:12345") # conectar ao servidor
17     socket.setsockopt(zmq.SUBSCRIBE, b"TIME") # assinar mensagens TIME
18
19     para i em intervalo(5): tempo                # Cinco iterações
20         = socket.recv() # receber uma mensagem relacionada à assinatura
21         print(time.decode()) # imprime o resultado

```

Figura 4.22: Uma configuração baseada em soquete de multidifusão.

Um cliente é igualmente simples, como também mostrado na Figura 4.22. Ele primeiro cria um SUB socket que se conecta ao socket PUB correspondente do servidor. Para receber as mensagens apropriadas, ele precisa assinar mensagens que tenham TEMPO como sua tag. Em nosso exemplo, um cliente simplesmente imprimirá as cinco primeiras mensagens recebidas do servidor. Note que podemos ter quantos clientes quisermos: o servidor a mensagem será multidifundida para todos os assinantes. O mais importante neste exemplo é

que ilustra que as únicas mensagens recebidas através do soquete SUB são as aqueles que o cliente havia assinado.

Finalmente, o **padrão de pipeline** é caracterizado pelo fato de que um processo quer divulgar seus resultados, assumindo que existem outros processos que desejam para obter esses resultados. A essência do padrão de pipeline é que um push o processo realmente não se importa com qual outro processo obtém seus resultados: o primeiro disponível, um fará exatamente o que é preciso. Da mesma forma, qualquer processo que extraia resultados de vários outros processos farão isso desde o primeiro processo de push, tornando-o resultados disponíveis. A intenção do padrão do pipeline é, portanto, vista como mantida muitos processos funcionando tanto quanto possível, empurrando resultados através de um pipeline de processos o mais rápido possível.

Nota 4.11 (Exemplo: O padrão do pipeline)

Como nosso último exemplo, considere o seguinte modelo para manter uma coleção de tarefas do trabalhador ocupadas. A Figura 4.23 mostra o código para uma chamada **tarefas do fazendeiro**: uma processo produzindo tarefas para serem escolhidas por outros. Neste exemplo, simulamos a tarefa permitindo que o produtor escolha um número aleatório modelando a duração, ou carga, do trabalho a ser feito. Esta carga de trabalho é então enviada para o soquete PUSH , sendo efetivamente enfileirado até que outro processo o pegue.

```

1 def produtor():
2     contexto = zmq.Context()
3     socket = context.socket(zmq.PUSH) 4 socket.bind("tcp://          # criar um push socket
127.0.0.1:12345") # vincular socket ao endereço
5
6 enquanto Verdadeiro:
7     carga de trabalho = random.randint(1, 100) # calcula carga de trabalho
8     socket.send(pickle.dumps(workload)) # envia carga de trabalho para o trabalhador
9     time.sleep(carga de trabalho/NWORKERS)          # equilibrar a produção esperando
10
11 def trabalhador(id):
12     contexto = zmq.Context()
13     socket = context.socket(zmq.PULL) 14 socket.connect("tcp://          # criar um soquete pull
localhost:12345") # conectar ao produtor
15
16 enquanto Verdadeiro:
17     trabalho = pickle.loads(socket.recv())          # receber trabalho de uma fonte
18     tempo.sono(trabalho)                            # finja trabalhar

```

Figura 4.23: Um padrão produtor-trabalhador.

Esses outros processos são conhecidos como **tarefas do trabalhador**, das quais também é feito um esboço. dado na Figura 4.23. Uma tarefa de trabalho se conecta a um único soquete PULL . Uma vez que ele escolhe algum trabalho, ele simula que está realmente fazendo algo dormindo por algum tempo proporcional à carga de trabalho recebida.

A semântica desse padrão push-pull é tal que o primeiro trabalhador disponível pegará o trabalho do produtor e, da mesma forma, se houver vários trabalhadores prontos para pegar o trabalho, cada um deles receberá uma tarefa. Como a distribuição do trabalho é realmente feita de forma justa requer alguma atenção específyca, que não discutiremos mais aqui.

A Interface de Passagem de Mensagens (MPI)

Com o advento dos multicomputadores de alto desempenho, os desenvolvedores têm procurado operações orientadas a mensagens que lhes permitam escrever facilmente aplicativos altamente eficientes. Isso significa que as operações devem estar em um nível conveniente de abstração (para facilitar o desenvolvimento de aplicativos) e que sua implementação incorre apenas em sobrecarga mínima. Os soquetes foram considerados insuficientes por dois motivos. Primeiro, eles estavam no nível errado de abstração ao suportar apenas operações simples de envio e recebimento. Segundo, os soquetes foram projetados para se comunicar em redes usando pilhas de protocolos de uso geral, como TCP/IP. Eles não foram considerados adequados para os protocolos proprietários desenvolvidos para redes de interconexão de alta velocidade, como aquelas usadas em clusters de servidores de alto desempenho. Esses protocolos exigiam uma interface que pudesse lidar com recursos mais avançados, como diferentes formas de buffer e sincronização.

O resultado foi que a maioria das redes de interconexão e multicomputadores de alto desempenho foram enviados com bibliotecas de comunicação proprietárias. Essas bibliotecas ofereciam uma riqueza de operações de comunicação de alto nível e geralmente eficientes. Claro, todas as bibliotecas eram mutuamente incompatíveis, de modo que os desenvolvedores de aplicativos agora tinham um problema de portabilidade.

A necessidade de ser independente de hardware e plataforma eventualmente levou à deñição de um padrão para passagem de mensagens, simplesmente chamado de **Message- Passing Interface** ou **MPI**. O MPI é projetado para aplicações paralelas e, como tal, é adaptado para comunicação transitória. Ele faz uso direto da rede subjacente. Além disso, ele assume que falhas sérias, como travamentos de processo ou partições de rede, são fatais e não exigem recuperação automática.

O MPI assume que a comunicação ocorre dentro de um grupo conhecido de processos . Cada grupo recebe um identificador. Cada processo dentro de um grupo também recebe um identificador (local). Um par (groupID, processID) portanto identifica exclusivamente a origem ou o destino de uma mensagem e é usado em vez de um endereço de nível de transporte. Pode haver vários grupos de processos, possivelmente sobrepostos, envolvidos em uma computação e que estão todos sendo executados ao mesmo tempo.

No centro do MPI estão as operações de mensagens para dar suporte à comunicação transitória , das quais as mais intuitivas estão resumidas na Figura 4.25. Para entender sua semântica, é útil manter a Figura 4.4 em mente, com o

middleware formado inteiramente por uma implementação MPI. Em particular, o O middleware mantém seus próprios buffers e a sincronização geralmente está relacionada a quando os dados necessários foram copiados do aplicativo de chamada para o middleware.

Operação	Descrição
MPI_BSEND	Anexar mensagem de saída a um buffer de envio local
MPI_ENVIAR	Envie uma mensagem e aguarde até que seja copiada para o local ou remoto amortecedor
MPI_SSEND	Envie uma mensagem e aguarde até que a transmissão comece
MPI_SENDRECV	Envie uma mensagem e aguarde a resposta
MPI_ÉFIM	Passar referência para mensagem de saída e continuar
MPI_ISEND	Passar referência para mensagem de saída e aguardar até o recebimento começa
MPI_RECV	Receber uma mensagem; bloquear se não houver nenhuma
MPI_Irecv	Verifique se há uma mensagem recebida, mas não bloqueie

Figura 4.24: Algumas das operações de passagem de mensagens mais intuitivas do MPI.

A comunicação assíncrona transitória é suportada pelo MPI_BSEND operação. O remetente envia uma mensagem para transmissão, que geralmente é primeiro copiado para um buffer local no sistema de tempo de execução do MPI. Quando a mensagem foi copiado, o remetente continua. O sistema de tempo de execução MPI local irá remover a mensagem do seu buffer local e cuide da transmissão o mais rápido possível como um receptor chamou uma operação de recebimento.

Existe também uma operação de envio de bloqueio, chamada MPI_SEND, da qual o a semântica depende da implementação. A operação MPI_SEND pode bloquear o chamador até que a mensagem especificada tenha sido copiada para o MPI sistema de tempo de execução do lado do remetente, ou até que o receptor tenha iniciado um recebimento operação. Comunicação síncrona pela qual o remetente bloqueia até que seu a solicitação é aceita para processamento posterior está disponível através do MPI_SSEND operação. Finalmente, a forma mais forte de comunicação síncrona também é suportado: quando um remetente chama MPI_SENDRECV, ele envia uma solicitação ao destinatário e bloqueia até que este último retorne uma resposta. Basicamente, esta operação corresponde para um RPC normal.

Tanto MPI_SEND quanto MPI_SSEND possuem variantes que evitam a cópia de mensagens de buffers de usuário para buffers internos ao sistema de tempo de execução MPI local. Esses variantes correspondem essencialmente a uma forma de comunicação assíncrona. Com MPI_ISEND, um remetente passa um ponteiro para a mensagem, após o qual o MPI o sistema de tempo de execução cuida da comunicação. O remetente continua imediatamente . Para evitar sobrescrever a mensagem antes que a comunicação seja concluída, O MPI oferece operações para verificar a conclusão ou até mesmo bloquear, se necessário. Assim como no MPI_SEND, se a mensagem foi realmente transferida para o receptor ou que foi meramente copiado pelo sistema de tempo de execução MPI local para um buffer interno é deixado sem especificação.

Da mesma forma, com `MPI_ISSEND`, um remetente também passa apenas um ponteiro para o sistema de tempo de execução do MPI. Quando o sistema de tempo de execução indica que processou a mensagem, o remetente tem a garantia de que o destinatário aceitou a mensagem e agora está trabalhando nela.

A operação `MPI_RECV` é chamada para receber uma mensagem; ela bloqueia o chamador até que uma mensagem chegue. Há também uma variante assíncrona, chamada `MPI_Irecv`, pela qual um receptor indica que está preparado para aceitar uma mensagem. O receptor pode verificar se uma mensagem realmente chegou, ou bloquear até que uma chegue.

A semântica das operações de comunicação do MPI nem sempre é direta, e operações diferentes podem às vezes ser trocadas sem afetar a correção de um programa. A razão oficial pela qual tantas formas diferentes de comunicação são suportadas é que isso dá aos implementadores de sistemas MPI possibilidades suficientes para otimizar o desempenho. Os cínicos podem dizer que o comitê não conseguiu se decidir coletivamente, então ele jogou tudo. Agora, o MPI está em sua quarta versão com mais de 650 operações disponíveis.

Sendo projetado para aplicações paralelas de alto desempenho, talvez seja mais fácil entender sua diversidade. Mais sobre MPI pode ser encontrado em [Gropp et al., 2016]. A referência completa do MPI-4 pode ser encontrada em [Message Passing Interface Forum, 2021].

4.3.3 Comunicação persistente orientada a mensagens

Agora chegamos a uma classe importante de serviços de middleware orientados a mensagens, geralmente conhecidos como **sistemas de enfileiramento de mensagens**, ou apenas **middleware orientado a mensagens (MOM)**. Os sistemas de enfileiramento de mensagens fornecem amplo suporte para comunicação assíncrona persistente. A essência desses sistemas é que eles oferecem capacidade de armazenamento de prazo intermediário para mensagens, sem exigir que o remetente ou o destinatário estejam ativos durante a transmissão da mensagem. Uma diferença importante com soquetes e MPI é que os sistemas de enfileiramento de mensagens são tipicamente direcionados para oferecer suporte a transferências de mensagens que podem levar minutos em vez de segundos ou milissegundos.

Modelo de enfileiramento de mensagens

A ideia básica por trás de um sistema de enfileiramento de mensagens é que os aplicativos se comunicam inserindo mensagens em filas específicas. Essas mensagens são encaminhadas por uma série de servidores de comunicação e, eventualmente, são entregues ao destino, mesmo que ele estivesse inativo quando a mensagem foi enviada. Na prática, a maioria dos servidores de comunicação são conectados diretamente uns aos outros. Em outras palavras, uma mensagem é geralmente transferida diretamente para um servidor de destino. Em princípio, cada aplicativo tem sua própria fila privada para a qual outros aplicativos podem enviar mensagens. Uma fila pode ser lida somente por seu aplicativo associado, mas também é possível que vários aplicativos compartilhem uma única fila.

Um aspecto importante dos sistemas de enfileiramento de mensagens é que um remetente geralmente recebe apenas as garantias de que sua mensagem será eventualmente inserida na fila do destinatário. Nenhuma garantia é dada sobre quando, ou mesmo se a mensagem será realmente lida, o que é completamente determinado pelo comportamento do destinatário.

Essas semânticas permitem que a comunicação seja desacoplada no tempo, como também discutido na Seção 2.1.3. Portanto, não há necessidade de o receptor estar executando quando uma mensagem está sendo enviada para sua fila. Da mesma forma, não há necessidade de o remetente estar executando no momento em que sua mensagem é captada pelo receptor. O remetente e o destinatário podem executar de forma completamente independente um do outro. De fato, uma vez que uma mensagem é depositada em uma fila, ela permanecerá lá até ser removida, independentemente de o remetente ou o destinatário estar executando. Isso nos dá quatro combinações em relação ao modo de execução do remetente e do destinatário, conforme mostrado na Figura 4.25.

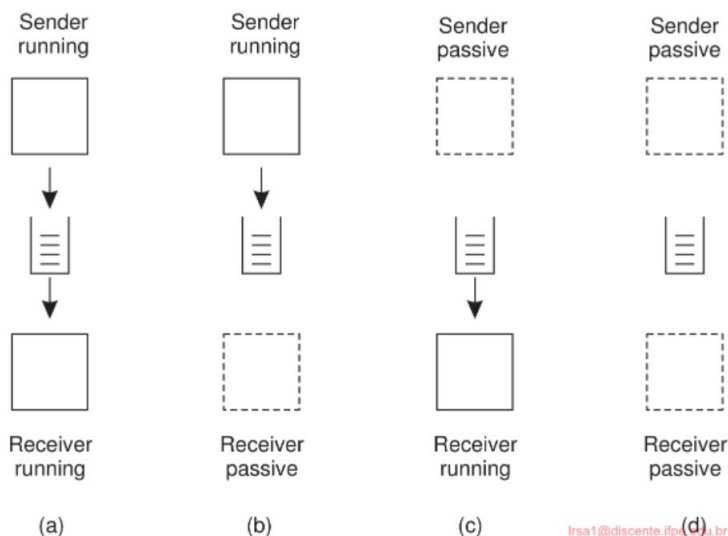


Figura 4.25: Quatro combinações para comunicação fracamente acoplada usando filas.

Na Figura 4.25(a), tanto o remetente quanto o destinatário estão em execução durante toda a transmissão de uma mensagem. Na Figura 4.25(b), apenas o remetente está em execução, enquanto o destinatário está passivo, ou seja, em um estado no qual a entrega da mensagem não é possível. No entanto, o remetente ainda pode enviar mensagens. A combinação de um remetente passivo e um receptor ativo é mostrada na Figura 4.25(c). Neste caso, o receptor pode ler mensagens que foram enviadas a ele, mas não é necessário que seus respectivos remetentes também estejam em execução. Finalmente, na Figura 4.25(d), vemos a situação em que o sistema está armazenando (e possivelmente transmitindo) mensagens mesmo enquanto o remetente e o receptor

são passivos. Pode-se argumentar que somente se esta última configuração for suportada, o sistema de enfileiramento de mensagens realmente fornece mensagens persistentes.

Mensagens podem, em princípio, conter quaisquer dados. O único aspecto importante da perspectiva do middleware é que as mensagens sejam endereçadas corretamente. Na prática, o endereçamento é feito fornecendo um nome exclusivo para todo o sistema da fila de destino. Em alguns casos, o tamanho da mensagem pode ser limitado, embora também seja possível que o sistema subjacente cuide da fragmentação e montagem de mensagens grandes de uma forma que seja completamente transparente para os aplicativos. Um efeito dessa abordagem é que a interface básica oferecida aos aplicativos pode ser simples, como mostrado na Figura 4.26.

Descrição da operação	Acrescentar uma
COLOCAR	mensagem a uma fila especificada Bloquear até que a fila especificada
PEGAR	não esteja vazia e remover a primeira mensagem Verificar uma fila especificada para mensagens e remover a primeira.
ENQUETE	Nunca bloquear
NOTIFICAR	Instale um manipulador para ser chamado quando uma mensagem for colocada na fila especificada

Figura 4.26: Interface básica para uma fila em um sistema de enfileiramento de mensagens.

A operação PUT é chamada por um remetente para passar uma mensagem ao sistema subjacente que deve ser anexada à fila especificada. Como explicamos, esta é uma chamada não bloqueante. A operação GET é uma chamada bloqueante pela qual um processo autorizado pode remover a mensagem pendente mais longa na fila especificada. O processo é bloqueado somente se a fila estiver vazia. Variações nesta chamada permitem a busca por uma mensagem específica na fila, por exemplo, usando uma prioridade ou um padrão correspondente. A variante não bloqueante é dada pela operação POLL .

Se a fila estiver vazia ou se uma mensagem específica não puder ser encontrada, o processo de chamada simplesmente continua.

Finalmente, a maioria dos sistemas de enfileiramento também permite que um processo instale um manipulador como uma função de retorno de chamada, que é automaticamente invocada sempre que uma mensagem é colocada na fila. A organização desse esquema é tratada por meio de uma operação NOTIFY . Os retornos de chamada também podem ser usados para iniciar automaticamente um processo que buscará mensagens da fila se nenhum processo estiver em execução no momento. Essa abordagem geralmente é implementada por meio de um daemon no lado do receptor que monitora continuamente a fila de mensagens recebidas e as manipula adequadamente.

Arquitetura geral de um sistema de enfileiramento de mensagens

Vamos agora dar uma olhada mais de perto em como é um sistema geral de enfileiramento de mensagens . Primeiro, as filas são gerenciadas por **gerenciadores de filas**. Um gerenciador de filas é um processo separado ou é implementado por meio de uma biblioteca vinculada

com um aplicativo. Em segundo lugar, como regra geral, um aplicativo pode colocar mensagens somente em uma fila local. Da mesma forma, obter uma mensagem é possível extraindo-a somente de uma fila local. Como consequência, se um gerenciador de filas QMA manipulando as filas para um aplicativo A for executado como um processo separado, ambos os processos QMA e A geralmente serão colocados na mesma máquina ou, na pior das hipóteses, na mesma LAN. Observe também que se todos os gerenciadores de filas estiverem vinculados a seus respectivos aplicativos, não podemos mais falar de um sistema de mensagens assíncronas persistente.

Se os aplicativos puderem colocar mensagens somente em filas locais, então claramente cada mensagem terá que carregar informações sobre seu destino. É tarefa do gerenciador de filas garantir que uma mensagem chegue ao seu destino. Isso nos leva a várias questões.

Em primeiro lugar, precisamos considerar como a fila de destino é endereçada. Obviamente, para aumentar a transparência de localização, é preferível que as filas tenham nomes lógicos, independentes de localização. Supondo que um gerenciador de filas seja implementado como um processo separado, usar nomes lógicos implica que cada nome deve ser associado a um **endereço de contato**, como um par (host,porta) , e que o mapeamento nome-para-endereço esteja prontamente disponível para um gerenciador de filas, como mostrado na Figura 4.27. Na prática, um endereço de contato carrega mais informações, notavelmente o protocolo a ser usado, como TCP ou UDP. Nós nos deparamos com tais endereços de contato em nossos exemplos de soquetes avançados, por exemplo, na Nota 4.9.

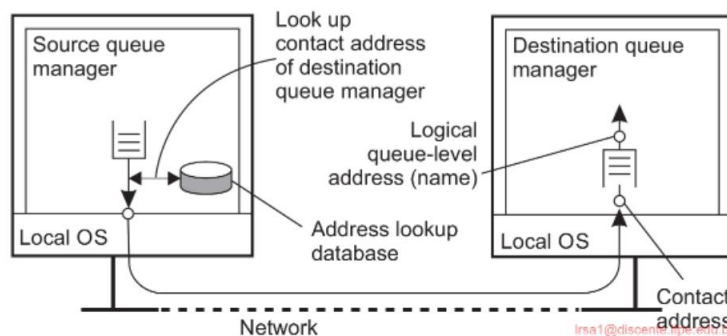


Figura 4.27: A relação entre a nomenclatura em nível de fila e o endereçamento em nível de rede .

Uma segunda questão que precisamos considerar é como o nome para endereço mapeamento é realmente disponibilizado para um gerenciador de filas. Uma abordagem comum é simplesmente implementar o mapeamento como uma tabela de consulta e copiar essa tabela para todos os gerenciadores. Obviamente, isso leva a um problema de manutenção, pois toda vez que uma nova fila é adicionada ou nomeada, muitas, se não todas as tabelas, precisam ser atualizadas. Existem várias maneiras de aliviar esses problemas, que discutiremos no Capítulo 6.

Isso nos leva a uma terceira questão, relacionada aos problemas de manter mapeamentos de nome para endereço de forma eficiente. Assumimos implicitamente que se uma fila de destino no gerenciador QMB for conhecida pelo gerenciador de filas QMA, então o QMA pode contatar diretamente o QMB para transferir mensagens. Na verdade, isso significa que (o endereço de contato de) cada gerenciador de filas deve ser conhecido por todos os outros. Obviamente, ao lidar com grandes sistemas de enfileiramento de mensagens, teremos um problema de escalabilidade. Na prática, geralmente há gerenciadores de filas especiais que operam como **roteadores**: eles encaminham mensagens recebidas para outros gerenciadores de filas. Dessa forma, um sistema de enfileiramento de mensagens pode gradualmente se transformar em uma rede de sobreposição completa em nível de aplicativo.

Se apenas alguns roteadores precisam saber sobre a topologia da rede, então um gerenciador de filas de origem precisa apenas saber para qual roteador adjacente, digamos R, ele deve encaminhar uma mensagem, dada uma fila de destino. O roteador R, por sua vez, pode precisar manter o controle apenas de seus roteadores adjacentes para ver para onde encaminhar a mensagem, e assim por diante. Claro, ainda precisamos ter mapeamentos de nome para endereço para todos os gerenciadores de filas, incluindo os roteadores, mas não é difícil imaginar que tais tabelas podem ser muito menores e mais fáceis de manter.

Corretores de mensagens

Uma importante área de aplicação dos sistemas de enfileiramento de mensagens é integrar aplicativos existentes e novos em um único sistema de informação distribuído e coerente. Se assumirmos que a comunicação com um aplicativo ocorre por meio de mensagens, então a integração requer que os aplicativos possam entender as mensagens que recebem. Na prática, isso requer que o remetente tenha suas mensagens de saída no mesmo formato que o do destinatário, mas também que suas mensagens sigam a mesma semântica que as esperadas pelo destinatário.

O remetente e o destinatário precisam essencialmente falar a mesma língua, ou seja, aderir ao mesmo protocolo de mensagens.

O problema com essa abordagem é que cada vez que um aplicativo A é adicionado ao sistema tendo seu próprio protocolo de mensagens, então para cada outro aplicativo B que deve se comunicar com A, precisaremos fornecer os meios para converter suas respectivas mensagens. Em um sistema com N aplicativos, precisaremos, portanto, de $N \times N$ conversores de protocolo de mensagens.

Uma alternativa é concordar com um protocolo de mensagens comum, como é feito com protocolos de rede tradicionais. Infelizmente, essa abordagem geralmente não funciona para sistemas de enfileiramento de mensagens. O problema é o nível de abstração em que esses sistemas operam. Um protocolo de mensagens comum só faz sentido se a coleção de processos que fazem uso desse protocolo realmente tiver o suficiente em comum. Se a coleção de aplicativos que compõem um sistema de informações distribuído for altamente diversa (o que geralmente é), então inventar uma solução única para todos simplesmente não vai funcionar.

Se nos concentrarmos apenas no formato e no significado das mensagens, a comunalidade pode ser alcançada elevando o nível de abstração, como é feito com mensagens XML.

Neste caso, as mensagens carregam informações sobre sua própria organização, e o que foi padronizado é a maneira como elas podem descrever seu conteúdo. Como consequência, um aplicativo pode fornecer informações sobre a organização de suas mensagens que podem ser processadas automaticamente. Claro, essas informações geralmente não são suficientes: também precisamos ter certeza de que a semântica das mensagens seja bem compreendida.

Diante desses problemas, a abordagem geral é aprender a conviver com as diferenças e tentar fornecer os meios para tornar as conversões o mais simples possível. Em sistemas de enfileiramento de mensagens, as conversões são manipuladas por nós especiais em uma rede de enfileiramento, conhecidos como **corretores de mensagens**. Um corretor de mensagens atua como um gateway de nível de aplicativo em um sistema de enfileiramento de mensagens. Seu principal propósito é converter mensagens recebidas para que elas possam ser entendidas pelo aplicativo de destino. Observe que, para um sistema de enfileiramento de mensagens, um corretor de mensagens é apenas mais um aplicativo, conforme mostrado na Figura 4.28. Em outras palavras, um corretor de mensagens geralmente não é considerado parte integrante do sistema de enfileiramento.

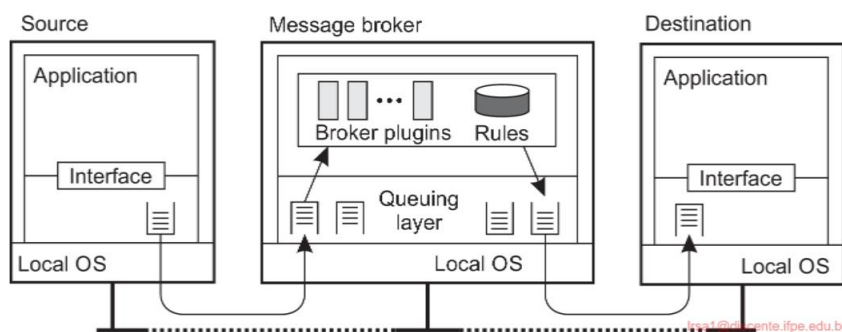


Figura 4.28: A organização geral de um corretor de mensagens em um sistema de enfileiramento de mensagens.

Um corretor de mensagens pode ser tão simples quanto um reformatador para mensagens. Por exemplo, suponha que uma mensagem recebida contenha uma tabela de um banco de dados na qual os registros são separados por um delimitador especial de fim de registro e os campos dentro de um registro têm um comprimento conhecido e fixo. Se o aplicativo de destino espera um delimitador diferente entre os registros e também espera que os campos tenham comprimentos variáveis, um corretor de mensagens pode ser usado para converter mensagens para o formato esperado pelo destino.

Em uma configuração mais avançada, um broker de mensagem pode atuar como um gateway de nível de aplicativo, no qual informações sobre o protocolo de mensagens de vários aplicativos foram codificadas. Em geral, para cada par de aplicativos, teremos um subprograma separado capaz de converter mensagens entre os dois aplicativos. Na Figura 4.28, esse subprograma é desenhado como um **plugin** para enfatizar que tais partes podem ser dinamicamente conectadas ou removidas de um broker.

Por fim, observe que, frequentemente, um broker de mensagens é usado para **integração avançada de aplicativos empresariais (EAI)**, como discutimos na Seção 1.3.2. Nesse caso, em vez de (apenas) converter mensagens, um broker é responsável por combinar aplicativos com base nas mensagens que estão sendo trocadas. Em tal **modelo de publicação-assinatura**, os aplicativos enviam mensagens na forma de publicação. Em particular, eles podem publicar uma mensagem sobre o tópico X, que é então enviada ao corretor. Os aplicativos que declararam seu interesse em mensagens sobre o tópico X, ou seja, que assinaram essas mensagens, receberão essas mensagens do corretor. Formas mais avançadas de mediação também são possíveis.

No coração de um corretor de mensagens está um repositório de regras para transformar uma mensagem de um tipo em outro. O problema é definir as regras e desenvolver os plugins. A maioria dos produtos corretores de mensagens vem com ferramentas de desenvolvimento sofisticadas, mas o ponto principal ainda é que o repositório precisa ser preenchido por especialistas. Aqui vemos um exemplo perfeito onde produtos comerciais são frequentemente enganosamente ditos para fornecer “inteligência”, onde, na verdade, a inteligência deve ser encontrada apenas nas cabeças desses especialistas. Pode ser melhor falar consistentemente sobre sistemas “avançados” (mas mesmo assim, frequentemente vemos que isso significa complexo ou complicado).

Nota 4.12 (Mais informações: Uma nota sobre sistemas de enfileiramento de mensagens)

Considerando o que dissemos sobre sistemas de enfileiramento de mensagens, parece que eles existem há muito tempo na forma de implementações para serviços de e-mail.

Os sistemas de e-mail são geralmente implementados por meio de uma coleção de servidores de e-mail que armazenam e encaminham mensagens em nome dos usuários em hosts conectados diretamente ao servidor. O roteamento geralmente é deixado de fora, pois os sistemas de e-mail podem fazer uso direto dos serviços de transporte subjacentes. Por exemplo, no protocolo de e-mail para a Internet, SMTP [Postel, 1982], uma mensagem é transferida configurando uma conexão TCP direta para o servidor de e-mail de destino.

O que torna os sistemas de e-mail especiais em comparação aos sistemas de enfileiramento de mensagens é que eles são principalmente voltados para fornecer suporte direto para usuários finais. Isso explica, por exemplo, por que vários aplicativos de groupware são baseados diretamente em um sistema de e-mail [Khoshafian e Buckiewicz, 1995]. Além disso, os sistemas de e-mail podem ter requisitos muito específicos, como filtragem automática de mensagens, suporte para bancos de dados de mensagens avançados (por exemplo, para recuperar facilmente mensagens armazenadas anteriormente) e assim por diante.

Os sistemas gerais de enfileiramento de mensagens não visam dar suporte apenas aos usuários finais. Uma questão importante é que eles são configurados para permitir comunicação persistente entre processos, independentemente de um processo estar executando um aplicativo de usuário, manipulando acesso a um banco de dados, realizando cálculos e assim por diante. Essa abordagem leva a um conjunto diferente de requisitos para sistemas de enfileiramento de mensagens do que sistemas de e-mail puros. Por exemplo, sistemas de e-mail geralmente não precisam fornecer entrega de mensagens garantida, prioridades de mensagens, recursos de registro, multicasting eficiente, balanceamento de carga, tolerância a falhas e assim por diante para uso geral.

Os sistemas de enfileiramento de mensagens de uso geral, portanto, têm uma ampla gama

de aplicativos, incluindo e-mail, fluxo de trabalho, groupware e processamento em lote. No entanto, como afirmamos antes, a área de aplicação mais importante é a integração de uma coleção (possivelmente amplamente dispersa) de bancos de dados e aplicativos em um sistema de informações federado [Hohpe e Woolf, 2004]. Por exemplo, uma consulta que expande vários bancos de dados pode precisar ser dividida em subconsultas que são encaminhadas para bancos de dados individuais. Os sistemas de enfileiramento de mensagens auxiliam fornecendo os meios básicos para empacotar cada subconsulta em uma mensagem e roteá-la para o banco de dados apropriado. Outros recursos de comunicação que discutimos neste capítulo são muito menos apropriados.

4.3.4 Exemplo: Protocolo Avançado de Enfileiramento de Mensagens (AMQP)

Uma observação interessante sobre sistemas de enfileiramento de mensagens é que eles foram desenvolvidos em parte para permitir que aplicativos legados interoperem, mas ao mesmo tempo vemos que quando se trata de operações entre diferentes sistemas de enfileiramento de mensagens, muitas vezes batemos em uma parede. Como consequência, uma vez que uma organização escolhe usar um sistema de enfileiramento de mensagens do fabricante X, ela pode ter que se contentar com soluções que somente X fornece. Soluções de enfileiramento de mensagens são, portanto, em grande parte soluções proprietárias. Tanto para abertura.

Em 2006, um grupo de trabalho foi formado para mudar essa situação, o que resultou na especificação do **Advanced Message-Queuing Protocol**, ou simplesmente **AMQP**. Existem diferentes versões do AMQP, com a versão 1.0 sendo a mais recente. Existem também várias implementações do AMQP, notavelmente de versões anteriores à 1.0, que na época em que a versão 1.0 foi estabelecida, ganharam considerável popularidade. Como uma versão pré-1.0 é tão diferente da versão 1.0, mas também tem uma base de usuários estável, podemos ver vários servidores AMQP pré-1.0 existirem ao lado de (seus inevitavelmente incompatíveis) servidores

Nesta seção, descreveremos o AMQP, mas misturaremos mais ou menos deliberadamente as versões pré-1.0 e 1.0, mantendo o essencial e o espírito do AMQP. Detalhes podem ser encontrados nas especificações [AMQP Working Group, 2008; OASIS, 2012]. As implementações do AMQP incluem RabbitMQ [Roy, 2018] e ActiveMQ do Apache.

Noções básicas

O AMQP gira em torno de aplicativos, gerenciadores de filas e filas. Adotando uma abordagem comum para muitas situações de rede, fazemos uma distinção entre o AMQP como um serviço de mensagens, o protocolo de mensagens real e, finalmente, a interface de mensagens conforme oferecida aos aplicativos. Para esse fim, é mais fácil considerar ter apenas um único gerenciador de filas, executando como um único servidor separado, formando a implementação do AMQP como um serviço. Um aplicativo se comunica com esse gerenciador de filas por meio de uma interface local.

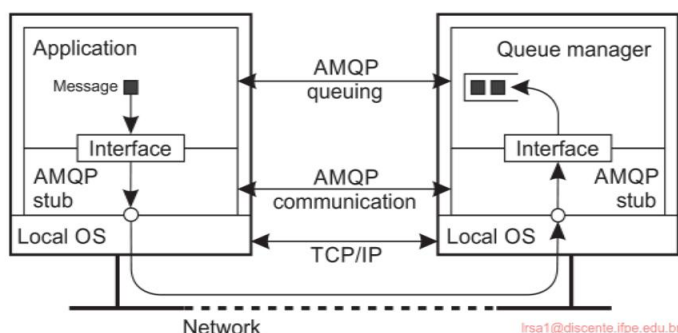


Figura 4.29: Uma visão geral de uma instância AMQP de servidor único.

Entre um aplicativo e o gerenciador de filas, a comunicação prossegue de acordo com o protocolo AMQP.

Esta situação é mostrada na Figura 4.29 e deve parecer familiar. O stub AMQP protege o aplicativo (bem como o gerenciador de filas) dos detalhes relativos à transferência de mensagens e comunicação em geral. Ao mesmo tempo, ele implementa uma interface de enfileiramento de mensagens, permitindo que o aplicativo faça uso do AMQP como um serviço de enfileiramento de mensagens. Embora a distinção entre stub AMQP e gerenciador de filas seja explicitada para gerenciadores de filas, a rigidez da separação é deixada para uma implementação. No entanto, se não for rigorosa, conceitualmente há uma distinção entre lidar com filas e lidar com comunicação relacionada, como deixaremos claro em breve.

Comunicação AMQP

O AMQP permite que um aplicativo configure uma **conexão** com um gerenciador de filas; uma conexão é um contêiner para vários **canais unidirecionais**. Enquanto o tempo de vida de um canal pode ser altamente dinâmico, as conexões são consideradas relativamente estáveis. Essa diferença entre conexão e canal permite implementações eficientes, notavelmente usando uma única conexão TCP de camada de transporte para multiplexar muitos canais diferentes entre um aplicativo e um gerenciador de filas. Na prática, o AMQP assume que o TCP é usado para estabelecer conexões AMQP.

A comunicação bidirecional é estabelecida por meio de **sessões**: um agrupamento lógico de dois canais. Uma conexão pode ter várias sessões, mas observe que um canal não precisa necessariamente fazer parte de uma sessão.

Finalmente, para realmente transferir mensagens, um **link** é necessário. Conceitualmente, um link, ou melhor, seus pontos finais, rastreiam o status das mensagens que estão sendo transferidas. Ele, portanto, fornece controle de fluxo refinado entre um aplicativo e um gerenciador de filas e, de fato, diferentes políticas de controle podem ser colocadas simultaneamente em prática para diferentes mensagens que são transferidas.

através da mesma sessão ou conexão. O controle de fluxo é estabelecido através de créditos: um receptor pode dizer ao remetente quantas mensagens ele tem permissão para enviar através de um link específico.

Quando uma mensagem deve ser transferida, o aplicativo a passa para seu stub AMQP local. Conforme mencionado, cada transferência de mensagem é associada a um link específico. A transferência de mensagem normalmente ocorre em três etapas.

1. No lado do remetente, a mensagem recebe um identificador exclusivo e é registrada localmente para estar em um **estado não resolvido**. O stub subsequentemente transfere a mensagem para o servidor, onde o stub AMQP também a registra como estando em um estado não resolvido. Nesse ponto, o stub do lado do servidor a passa para o gerenciador de filas.
2. O aplicativo receptor (nesse caso, o gerenciador de filas) é assumido para manipular a mensagem e normalmente relata de volta ao seu stub que ela foi finalizada. O stub passa essas informações para o remetente original, em cujo ponto a mensagem no stub AMQP do remetente original entra em um **estado estabelecido**.
3. O stub AMQP do remetente original agora informa ao stub do destinatário original que a transferência da mensagem foi liquidada (o que significa que o remetente original esquecerá da mensagem a partir de agora). O stub do destinatário agora também pode descartar qualquer coisa sobre a mensagem, registrando-a formalmente como liquidada também.

Observe que, como o aplicativo receptor pode indicar à camada de comunicação AMQP subjacente que terminou com uma mensagem, o AMQP permite uma verdadeira confiabilidade de comunicação de ponta a ponta. Em particular, o aplicativo, seja um aplicativo cliente ou um gerenciador de filas real, pode instruir a camada de comunicação AMQP a manter uma mensagem (ou seja, uma mensagem permanece no estado não resolvido).

Mensagens AMQP

O envio de mensagens no AMQP ocorre logicamente na camada acima daquela que manipula a comunicação. É aqui que um aplicativo pode indicar o que precisa ser feito com uma mensagem, mas também pode ver o que aconteceu até agora. O envio de mensagens ocorre formalmente entre dois **nós**, dos quais existem três tipos: um produtor, um consumidor ou uma fila. Normalmente, os nós produtores e consumidores representam aplicativos regulares, enquanto as filas são usadas para armazenar e encaminhar mensagens. De fato, um gerenciador de filas normalmente consiste em vários nós de fila. Para que a transferência de mensagens ocorra, dois nós terão que estabelecer um link entre eles.

O receptor pode indicar ao remetente se sua mensagem foi aceita (o que significa que foi processada com sucesso) ou rejeitada. Observe que isso significa que uma notificação é retornada ao remetente original. O AMQP também

```

1 importação rabbitpy
2
3 def produtor():
4     conexão = rabbitpy.Connection() # Conecte-se ao servidor RabbitMQ
5     canal = conexão.canal()          # Crie um novo canal na conexão
6
7     exchange = rabbitpy.Exchange(channel, 'exchange') # Crie uma exchange
8     troca.declarar()
9
10    queue1 = rabbitpy.Queue(channel, 'example1') # Cria a 1ª fila
11    fila1.declare()
12
13    queue2 = rabbitpy.Queue(channel, 'example2') # Cria a 2ª fila
14    fila2.declare()
15
16    queue1.bind(exchange, 'example-key') # Vincular queue1 a uma única chave
17    queue2.bind(exchange, 'example-key') # Vincular queue2 à mesma chave
18
19    mensagem = rabbitpy.Message(canal, 'Mensagem de teste')
20    message.publish(exchange, 'example-key') # Publica a mensagem usando a chave
21    troca.delete()

```

(um)

```

1 importação rabbitpy
2
3 def consumidor():
4     conexão = rabbitpy.Connection()
5     canal = conexão.canal()
6
7     fila = rabbitpy.Queue(canal, 'exemplo1')
8
9     # Enquanto houver mensagens na fila, busque-as usando Basic.Get
10    enquanto len(fila) > 0:
11        mensagem = queue.get()
12        print('Mensagem Q1: %s' % message.body.decode())
13        mensagem.ack()
14
15        fila = rabbitpy.Queue(canal, 'exemplo2')
16
17    enquanto len(fila) > 0:
18        mensagem = queue.get()
19        print('Mensagem Q2: %s' % message.body.decode())
20        mensagem.ack()

```

(b)

Figura 4.30: Um simples (a) produtor e (b) consumidor para RabbitMQ, adaptado de [Roy, 2018].

suporta fragmentação e montagem de mensagens grandes, para as quais são enviadas notificações adicionais.

Claro, um aspecto importante do AMQP é seu suporte para **mensagens persistentes**. A obtenção de persistência é tratada por meio de vários mecanismos. Primeiro, uma mensagem pode ser marcada como durável, indicando que a fonte espera que qualquer nó intermediário, como uma fila, seja capaz de se recuperar em caso de falha. Um nó intermediário que não pode garantir tal durabilidade terá que rejeitar uma mensagem. Segundo, um nó de origem ou de destino também pode indicar sua durabilidade: se durável, ele manterá seu estado ou também manterá o estado (não estabelecido) de mensagens duráveis? Combinar o último com mensagens duráveis efetivamente estabelece transferência de mensagem confiável e mensagens persistentes.

O AMQP é realmente um protocolo de mensagens no sentido de que ele não suporta por si só, por exemplo, primitivas de publicação-assinatura. Ele espera que tais problemas sejam tratados por gerenciadores de filas proprietários mais avançados, semelhantes aos corretores de mensagens discutidos na Seção 4.3.3.

Finalmente, não há razão pela qual um gerenciador de filas não possa ser conectado a outro gerenciador de filas. Na verdade, é bastante comum organizar gerenciadores de filas em uma rede de sobreposição na qual as mensagens são roteadas de produtores para seus consumidores. O AMQP não especifica como a rede de sobreposição deve ser construída e gerenciada e, de fato, diferentes provedores de sistemas baseados em AMQP oferecem diferentes soluções. De particular importância é especificar como as mensagens devem ser roteadas pela rede. O ponto principal é que os administradores precisarão fazer muitas dessas especificações manualmente. Somente em casos em que as sobreposições têm estruturas regulares, como ciclos ou árvores, fica mais fácil fornecer os detalhes de roteamento necessários.

AMQP na prática

Uma das implementações mais populares de corretagem de mensagens é o servidor RabbitMQ, amplamente descrito por Roy [2018]. O servidor RabbitMQ é baseado na versão 0.9 do AMQP, ao mesmo tempo em que oferece suporte total à versão 1.0. Uma diferença importante entre essas duas versões é o uso de **trocas**. Em vez de manipular filas diretamente, os produtores contatam uma troca, que, por sua vez, coloca mensagens em uma ou várias filas. Em essência, uma troca permite que um produtor use um RPC síncrono ou assíncrono para um gerenciador de filas. Uma troca nunca armazena mensagens; ela simplesmente garante que uma mensagem seja colocada em uma fila apropriada.

Para ilustrar esse conceito, considere a Figura 4.30(a) mostrando um produtor simples. Conforme mencionado anteriormente, o AMQP incorpora canais como parte de conexões mais duráveis, e a primeira coisa que fazemos é criar um meio de comunicação com o servidor (estamos omitindo vários detalhes; neste caso, estabelecemos uma conexão padrão com o servidor). Nas linhas a seguir, uma troca é criada dentro do domínio do servidor, junto com duas filas. Nas Linhas 16

e 17, ambas as filas são vinculadas à mesma troca por uma chave simples chamada *example-key*. Após criar uma mensagem, o produtor diz à troca que ela deve publicar essa mensagem em qualquer fila vinculada pela chave especificada.

Como era de se esperar, a troca colocará a mensagem em ambas as filas, o que é verificado pelo consumidor simples mostrado na Figura 4.30(b). Neste caso, o consumidor simplesmente busca mensagens de qualquer fila e imprime suas conteúdo.

Trocas e filas têm muitas opções, para as quais encaminhamos o leitor interessado para Roy [2018]. É importante notar que usando sistemas como RabbitMQ, mas também outros corretores de mensagens, geralmente é possível configurar redes de sobreposição avançadas para roteamento de mensagens em nível de aplicativo. Ao mesmo tempo, a manutenção dessas redes pode facilmente se tornar um pesadelo.

4.4 Comunicação multicast

Um tópico importante na comunicação em sistemas distribuídos é o suporte ao envio de dados para múltiplos receptores, também conhecido como **comunicação multicast**. Por muitos anos, este tópico pertenceu ao domínio dos protocolos de rede, onde inúmeras propostas para soluções de nível de rede e nível de transporte foram implementadas e avaliadas [Janic, 2005; Obraczka, 1998]. Uma questão importante em todas as soluções era configurar os caminhos de comunicação para disseminação de informações. Na prática, isso envolvia um enorme esforço de gerenciamento, frequentemente exigindo intervenção humana. Além disso, enquanto não houver convergência de propostas, os ISPs têm se mostrado relutantes em dar suporte a multicasting [Diot et al., 2000].

Com o advento da tecnologia peer-to-peer, e notavelmente o gerenciamento de sobreposição estruturado, ficou mais fácil configurar caminhos de comunicação. Como as soluções peer-to-peer são tipicamente implantadas na camada de aplicação, várias técnicas de multicasting de nível de aplicação foram introduzidas. Nesta seção, daremos uma breve olhada nessas técnicas.

A comunicação multicast também pode ser realizada de outras maneiras além de configurar caminhos de comunicação explícitos. Como também exploramos nesta seção, a disseminação de informações baseada em *focos* fornece maneiras simples (mas frequentemente menos eficientes) para multicasting.

4.4.1 Multicast baseado em árvore em nível de aplicativo

A ideia básica no **multicasting em nível de aplicação** é que os nós são organizados em uma rede de sobreposição, que é então usada para disseminar informações para seus membros. Uma observação importante é que os roteadores de rede não estão envolvidos na associação de grupo. Como consequência, as conexões entre nós na rede de sobreposição podem cruzar vários links físicos e, como tal, o roteamento de mensagens dentro da sobreposição pode não ser ideal em comparação ao que poderia ter sido alcançado pelo roteamento em nível de rede.

Uma questão crucial de design é a construção da rede de sobreposição. Em essência, há duas abordagens [El-Sayed et al., 2003; Hosseini et al., 2007; Allani et al., 2009]. Primeiro, os nós podem se organizar diretamente em uma árvore, o que significa que há um caminho único (de sobreposição) entre cada par de nós. Uma abordagem alternativa é que os nós se organizem em uma rede mesh na qual cada nó terá vários vizinhos e, em geral, existem vários caminhos entre cada par de nós. A principal diferença entre os dois é que o último geralmente fornece maior robustez: se uma conexão for interrompida (por exemplo, porque um nó falha), ainda haverá uma oportunidade de disseminar informações sem ter que reorganizar imediatamente toda a rede.

Nota 4.13 (Avançado: Construindo uma árvore multicast no Chord)

Para tornar as coisas concretas, vamos considerar um esquema relativamente simples para construir uma árvore multicast no Chord, que descrevemos na Nota 2.6. Este esquema foi originalmente proposto para o Scribe [Castro et al., 2002b], que é um esquema de multicasting de nível de aplicação construído sobre o Pastry [Rowstron e Druschel, 2001]. Este último também é um sistema peer-to-peer baseado em DHT.

Suponha que um nó queira iniciar uma sessão multicast. Para isso, ele simplesmente gera um identificador multicast, digamos, *mid*, que é apenas uma chave de 160 bits escolhida aleatoriamente. Ele então procura *succ(mid)*, que é o nó responsável por essa chave, e o promove para se tornar a raiz da árvore multicast que será usada para enviar dados para nós interessados. Para se juntar à árvore, um nó *P* simplesmente executa a operação *lookup(mid)*, tendo o efeito de que uma mensagem de *lookup* com a solicitação para se juntar ao grupo multicast *mid* será roteada de *P* para *succ(mid)*. O algoritmo de roteamento em si será explicado em detalhes no Capítulo 6.

Em seu caminho em direção à raiz, a solicitação de junção passará por vários nós. Suponha que ele primeiro alcance o nó *Q*. Se *Q* nunca tivesse visto uma solicitação de junção para *mid* antes, ele se tornará um **encaminhador** para aquele grupo. Nesse ponto, *P* se tornará um filho de *Q*, enquanto este último continuará a encaminhar a solicitação de junção para a raiz. Se o próximo nó na raiz, digamos, *R* também ainda não for um encaminhador, ele se tornará um e registrará *Q* como seu filho e continuará a enviar a solicitação de junção.

Por outro lado, se *Q* (ou *R*) já for um encaminhador para *mid*, ele também registrará o remetente anterior como seu filho (ou seja, *P* ou *Q*, respectivamente), mas não haverá mais necessidade de enviar a solicitação de junção para a raiz, pois *Q* (ou *R*) já será um membro da árvore multicast.

Nós como *P* que solicitaram explicitamente para se juntar à árvore multicast são, por definição, também encaminhadores. O resultado desse esquema é que construímos uma árvore multicast através da rede de sobreposição com dois tipos de nós: encaminhadores puros que agem como auxiliares, e nós que também são encaminhadores, mas solicitaram explicitamente para se juntar à árvore. O multicasting agora é simples: um nó apenas envia uma mensagem multicast para a raiz da árvore executando novamente a operação *lookup(mid)*, após a qual essa mensagem pode ser enviada ao longo da árvore.

Notamos que esta descrição de alto nível de multicasting no Scribe não faz justiça completa ao seu design original. O leitor interessado é encorajado a dar uma olhada nos detalhes, que podem ser encontrados em [Castro et al., 2002b].

Problemas de desempenho em sobreposições

Da descrição de alto nível dada acima, deve ficar claro que, embora construir uma árvore por si só não seja tão difícil uma vez que tenhamos organizado os nós em uma sobreposição, construir uma árvore eficiente pode ser uma história diferente. Observe que em nossa descrição até agora, a seleção de nós que participam da árvore não leva em conta nenhuma métrica de desempenho: ela é puramente baseada no roteamento (lógico) de mensagens através da sobreposição.

Para entender o problema em questão, dê uma olhada na Figura 4.31, que mostra um pequeno conjunto de cinco nós que são organizados em uma rede de sobreposição simples, com o nó A formando a raiz de uma árvore multicast. Os custos para atravessar um link físico também são mostrados. Agora, sempre que A faz multidifusão de uma mensagem para os outros nós, a rota lógica, ou seja, no nível da sobreposição, é simplesmente $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C$. A rota real cruza os links de nível de rede na seguinte ordem: $A \rightarrow Ra \rightarrow Rb \rightarrow B \rightarrow Rb \rightarrow Ra \rightarrow Re \rightarrow E \rightarrow Re \rightarrow Rc \rightarrow Rd \rightarrow D \rightarrow Rd \rightarrow Rc \rightarrow C$. Portanto, é visto que esta mensagem atravessará cada um dos links B , Rb , Ra , Rb , E , Re , Rc , Rd e D , Rd duas vezes. A rede de sobreposição teria sido mais eficiente se não tivéssemos construído os links de sobreposição B , E e D , E , mas sim A , E e C , E . Tal configuração teria poupado a dupla travessia pelos links físicos Ra , Rb e Rc , Rd .

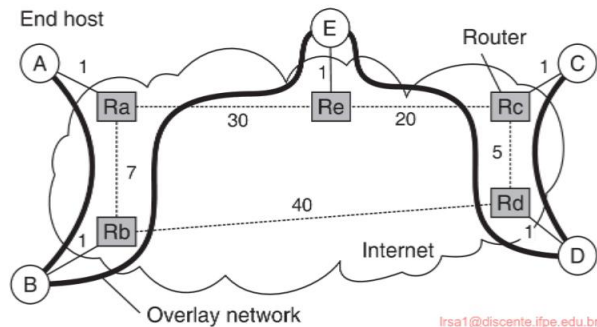


Figura 4.31: A relação entre links em uma sobreposição e rotas em nível de rede.

A qualidade de uma árvore multicast de nível de aplicação é geralmente medida por três métricas diferentes: estresse de link, alongamento e custo de árvore. O **estresse de link** é definido por link e conta com que frequência um pacote cruza o mesmo link [Chu et al., 2002]. Um estresse de link maior que 1 vem do fato de que, embora em um nível lógico um pacote possa ser encaminhado ao longo de duas conexões diferentes, parte dessas conexões pode, na verdade, corresponder ao mesmo link físico, como mostramos na Figura 4.31.

A **penalidade de alongamento** ou atraso relativo (**RDP**) mede a razão do atraso entre dois nós na sobreposição e o atraso que esses dois nós experimentariam na rede subjacente. Por exemplo, mensagens de

B para C seguem a rota B $\rightarrow$ Rb $\rightarrow$ Ra $\rightarrow$ Re $\rightarrow$ E $\rightarrow$ Re $\rightarrow$ Rc $\rightarrow$ Rd $\rightarrow$ D $\rightarrow$ Rd $\rightarrow$ Rc $\rightarrow$ C na rede sobreposta, tendo um custo total de 73 unidades.

No entanto, as mensagens teriam sido roteadas na rede subjacente ao longo do caminho B $\rightarrow$ Rb $\rightarrow$ Rd $\rightarrow$ Rc $\rightarrow$ C, com um custo total de 47 unidades, levando a um alongamento de 1,55. Obviamente, ao construir uma rede de sobreposição, o objetivo é minimizar o alongamento agregado, ou similarmente, o RDP médio medido sobre todos os pares de nós.

Finalmente, o **custo da árvore** é uma métrica global, geralmente relacionada à minimização dos custos de link agregados. Por exemplo, se o custo de um link for considerado o atraso entre seus dois nós finais, então otimizar o custo da árvore se resume a encontrar uma árvore de abrangência mínima na qual o tempo total para disseminar informações para todos os nós é mínimo.

Para simplificar um pouco as coisas, suponha que um grupo multicast tenha um nó associado e bem conhecido que mantém o controle dos nós que se juntaram à árvore. Quando um novo nó emite uma solicitação de junção, ele contata esse **nó de encontro** para obter uma lista (potencialmente parcial) de membros. O objetivo é selecionar o melhor membro que pode operar como o pai do novo nó na árvore. Quem ele deve selecionar? Existem muitas alternativas, e diferentes propostas geralmente seguem soluções muito diferentes.

Considere, por exemplo, um grupo multicast com apenas uma única fonte. Nesse caso, a seleção do melhor nó é óbvia: ele deve ser a fonte (porque nesse caso, podemos ter certeza de que o stretch será igual a 1).

No entanto, ao fazer isso, introduziríamos uma topologia em estrela com a fonte no meio. Embora simples, não é difícil imaginar que a fonte pode facilmente ficar sobrecarregada. Em outras palavras, a seleção de um nó geralmente será restringida de tal forma que somente aqueles nós que têm k ou menos vizinhos podem ser escolhidos, com k sendo um parâmetro de projeto. Essa restrição complica severamente o algoritmo de estabelecimento de árvore, pois uma solução adequada pode exigir que parte da árvore existente seja reconfigurada. Tan et al. [2003] fornecem uma visão geral e avaliação abrangentes de várias soluções para esse problema.

Nota 4.14 (Avançado: Switch-trees)

Como ilustração, vamos dar uma olhada mais de perto em uma família específica, conhecida como **árvores de comutação** [Helder e Jamin, 2002]. A ideia básica é simples. Suponha que já temos uma árvore multicast com uma única fonte como raiz. Nessa árvore, um nó P pode alternar os pais descartando o link para seu pai atual em favor de um link para outro nó. As únicas restrições impostas à comutação de links são que o novo pai nunca pode ser um membro da subárvore com raiz em P (pois isso particionaria a árvore e criaria um loop) e que o novo pai não terá muitos filhos imediatos. Esse último requisito é necessário para limitar a carga de mensagens de encaminhamento por qualquer nó único.

Existem diferentes critérios para decidir mudar de pais. Um simples é

otimizar a rota para a fonte, minimizando efetivamente o atraso quando uma mensagem deve ser multicast. Para esse fim, cada nó recebe regularmente informações sobre outros nós (explicaremos uma maneira específica de fazer isso abaixo). Nesse ponto, o nó pode avaliar se outro nó seria um pai melhor em termos de atraso ao longo da rota para a fonte e, se for o caso, inicia uma troca.

Outro critério poderia ser se o atraso para o outro pai potencial é menor do que para o pai atual. Se cada nó tomar isso como um critério, então os atrasos agregados da árvore resultante devem ser idealmente mínimos. Em outras palavras, este é um exemplo de otimização do custo da árvore, como explicamos acima. No entanto, mais informações seriam necessárias para construir tal árvore, mas, como se vê, este esquema simples é uma heurística razoável que leva a uma boa aproximação de uma árvore de abrangência mínima.

Como exemplo, considere o caso em que um nó P recebe informações sobre os vizinhos de seu pai. Observe que os vizinhos consistem no avô de P, junto com os outros irmãos do pai de P. O nó P pode então avaliar os atrasos para cada um desses nós e, subsequentemente, escolher aquele com o menor atraso, digamos Q, como seu novo pai. Para esse fim, ele envia uma solicitação de troca para Q. Para evitar que loops sejam formados devido a solicitações de troca simultâneas, um nó que tenha uma solicitação de troca pendente simplesmente se recusará a processar quaisquer solicitações recebidas. Na verdade, isso leva a uma situação em que apenas trocas completamente independentes podem ser realizadas simultaneamente. Além disso, P fornecerá a Q informações suficientes para permitir que este último conclua que ambos os nós têm o mesmo pai, ou que Q é o avô.

Um problema importante que ainda não abordamos é a falha do nó. No caso de árvores de comutação, uma solução simples é proposta: sempre que um nó percebe que seu pai falhou, ele se conecta à raiz. Nesse ponto, o protocolo de otimização pode prosseguir normalmente e eventualmente colocará o nó em um bom ponto na árvore multicast. Experimentos descritos em [Helder e Jamin, 2002] mostram que a árvore resultante está de fato próxima de uma árvore de abrangência mínima.

4.4.2 Multicast baseado em flooding

Até agora, assumimos que quando uma mensagem deve ser multicast, ela deve ser recebida por todos os nós na rede de sobreposição. Estritamente falando, isso corresponde à **transmissão**. Em geral, a multicast se refere ao envio de uma mensagem para um subconjunto de todos os nós, ou seja, um grupo específico de nós. Uma questão de design fundamental quando se trata de multicast é minimizar o uso de nós intermediários para os quais a mensagem não se destina. Para deixar isso claro, se a sobreposição for organizada como uma árvore multinível, mas apenas os nós folha forem os que devem receber uma mensagem multicast, então claramente pode haver alguns nós que precisam armazenar e, posteriormente, encaminhar uma mensagem que não se destina a eles.

Uma maneira simples de evitar tal ineficiência é construir uma rede de sobreposição por grupo multicast. Como consequência, a multidifusão de uma mensagem m para um

grupo G é o mesmo que transmitir m para G . A desvantagem dessa solução é que um nó pertencente a vários grupos precisará, em princípio, manter uma lista separada de seus vizinhos para cada grupo do qual é membro.

Se assumirmos que uma sobreposição corresponde a um grupo multicast e, portanto, que precisamos transmitir uma mensagem, uma maneira ingênua de fazer isso é aplicar **flooding**. Nesse caso, cada nó simplesmente encaminha uma mensagem m para cada um de seus vizinhos, exceto para aquele do qual recebeu m . Além disso, se um nó mantém o controle das mensagens que recebeu e encaminhou, ele pode simplesmente ignorar duplicatas.

Para entender o desempenho do flooding, modelamos uma rede de sobreposição como um grafo não direcionado conectado $G = (V, E)$ com nós V e links E .

Seja v_0 o nó que inicia o flooding. Em princípio, cada nó diferente de v_0 enviará a mensagem para seus vizinhos uma vez, exceto para o vizinho do qual recebeu a mensagem. Veremos, portanto, um total de $\sum_{v \in V} \deg(v)$ mensagens, com $\deg(v)$ o número de vizinhos do nó v . Para qualquer grafo não direcionado, $\sum_{v \in V} \deg(v) = 2 \cdot |E|$, com $|E|$ denotando o número total de arestas. Em outras palavras, veremos $2|E| + 1$ mensagens sendo enviadas, tornando o flooding bastante ineficiente. Somente se G for uma árvore, a inundação será ótima, pois nesse caso, $|E| = |V| - 1$. No pior caso, quando G estiver totalmente conectado, temos $|E| = \binom{|V|}{2}$ levando a uma ordem de $|V|^2$ mensagens.

Suponha agora que não temos informações sobre a estrutura da rede de sobreposição e que o melhor que podemos assumir é que ela pode ser representada como um **gráfico aleatório**, que (para manter a simplicidade) é um gráfico que tem uma probabilidade p de que dois vértices sejam unidos por uma aresta, também conhecido como um **gráfico Erdős-Rényi** [Erdős e Rényi, 1959]. Observe que estamos realmente considerando nossa rede de sobreposição como uma rede ponto a ponto não estruturada e que não temos nenhuma informação sobre como ela está sendo construída. Com uma probabilidade p de que dois nós sejam unidos e um total de $\binom{|V|}{2}$ arestas, não é difícil ver que podemos esperar que nossa sobreposição tenha $|E| = p \cdot \binom{|V|}{2}$ arestas. Para dar uma ideia do que estamos lidando, a Figura 4.32 mostra a relação entre o número de nós e arestas para diferentes valores de p . Como pode ser visto, o número de arestas pode facilmente se tornar grande, mesmo para pequenos valores de p .

Para reduzir o número de mensagens, também podemos usar o **flooding probabilístico** conforme introduzido por Banaei-Kashani e Shahab [2003] e formalmente analisado por Oikonomou e Stavrakakis [2007]. A ideia é simples: quando um nó está flooding uma mensagem m e precisa encaminhar m para um vizinho específico, ele o fará com uma probabilidade p . O efeito pode ser dramático: o número total de mensagens enviadas cairá linearmente em p . No entanto, também há um risco: quanto menor o p , maior a chance de que nem todos os nós na rede sejam alcançados. Esse risco é causado pelo simples fato de que todos os vizinhos de um nó específico Q podem ter decidido não encaminhar m para Q . Se Q tiver n vizinhos, isso pode acontecer aproximadamente com uma probabilidade de $(1 - p)^n$. Claramente, o

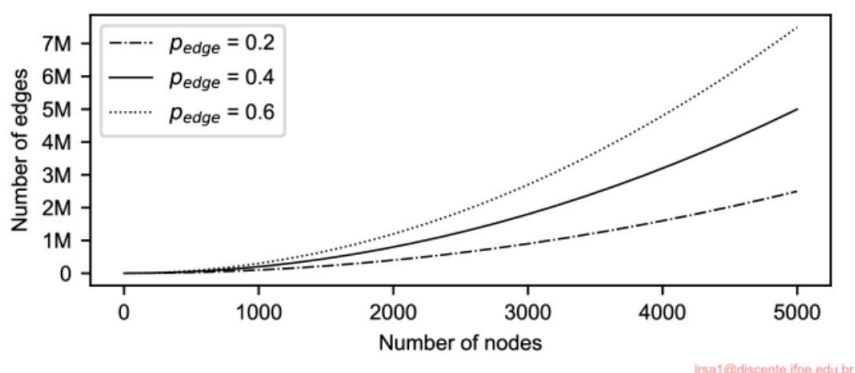


Figura 4.32: O tamanho de uma sobreposição aleatória em função do número de nós.

O número de vizinhos desempenha um papel importante na decisão de encaminhar ou não uma mensagem e, de fato, podemos substituir a probabilidade estática de encaminhamento por uma que leve em consideração o grau do vizinho. Isso foi desenvolvido e analisado por Sereno e Gaeta [2011]. Para dar uma ideia da eficiência da transmissão probabilística: em uma rede aleatória de 10.000 nós e $p_{edge} = 0,1$, precisamos apenas definir $p_{good} = 0,01$ para estabelecer uma redução de mais de 50 vezes no número de mensagens enviadas em comparação ao flooding completo.

Ao lidar com uma sobreposição estruturada, ou seja, uma que tem uma topologia mais ou menos determinística, projetar esquemas de inundação eficientes é mais simples. Como exemplo, considere um hipercubo n -dimensional, mostrado na Figura 4.33 para o caso $n = 4$, como também discutido na Seção 2.4.1.

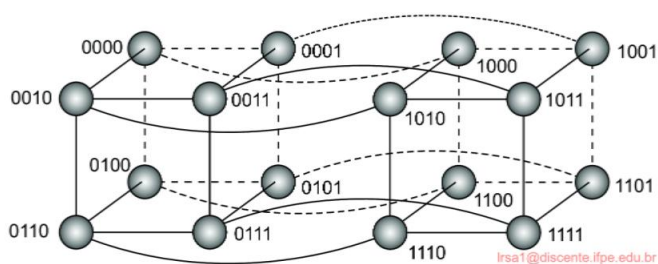


Figura 4.33: Um hipercubo simples, ponto a ponto, quadridimensional.

Um esquema de transmissão simples e eficiente foi projetado por Schlosser et al. [2002] e depende do acompanhamento de vizinhos por dimensão. Isso é melhor explicado considerando que cada nó em um hipercubo n -dimensional é representado por uma sequência de bits de comprimento n . Cada aresta na sobreposição é rotulada com sua dimensão. Para o caso $n = 4$, o nó 0000 terá como vizinhos o conjunto {0001, 0010, 0100, 1000}. A aresta entre 0000 e 0001 é rotulada

“4” corresponde à mudança do 4º bit ao comparar 0000 com 0001 e vice-versa. Da mesma forma, a aresta $\langle 0000, 0100 \rangle$ é rotulada como “2”, e assim por diante. Um nó inicialmente transmite uma mensagem m para todos os seus vizinhos e marca m com o rótulo da aresta sobre a qual ele envia a mensagem. Em nosso exemplo, se o nó 1001 transmite uma mensagem, ele enviará o seguinte:

- $(m,1)$ a 0001 •
- $(m,2)$ a 1101 •
- $(m,3)$ a 1011 •
- $(m,4)$ a 1000

Quando um nó recebe uma mensagem de transmissão, ele a encaminhará somente ao longo de arestas que tenham uma dimensão maior. Em outras palavras, em nosso exemplo, o nó 1101 encaminhará m somente para os nós 1111 (unidos a 1101 por uma aresta rotulada “3”) e 1100 (unidos por uma aresta com rótulo “4”). Usando esse esquema, pode ser mostrado que cada transmissão requer precisamente $N \cdot \log N$ mensagens, onde $N = 2^n$, que é o número de nós em um hipercubo n -dimensional. Esse esquema de transmissão é, portanto, ótimo em termos do número de mensagens enviadas.

Nota 4.15 (Avançado: Inundação baseada em anel)

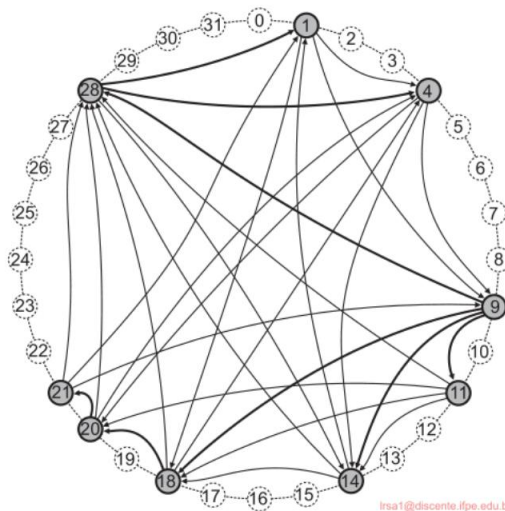


Figura 4.34: Um anel de acordes no qual o nó 9 transmite uma mensagem.

Um hipercubo é um exemplo direto de como podemos usar efetivamente o conhecimento da estrutura de uma rede de sobreposição para estabelecer um abastecimento eficiente. No caso do Chord, podemos seguir uma abordagem proposta por Ghodsi [2010]. Lembre-se de que no Chord cada nó é identificado por um número p , e cada recurso (tipicamente

um arquivo), é atribuída uma chave k do mesmo espaço usado para identificadores de nó. O sucessor $\text{succ}(k)$ de uma chave k é o nó com o menor identificador p $\hat{y}$ k . Considere o pequeno anel Chord mostrado na Figura 4.34 e suponha que o nó 9 deseja enviar uma mensagem para todos os outros nós.

Em nosso exemplo, o nó 9 divide o espaço de identificadores em quatro segmentos (um para cada um de seus vizinhos). O nó 28 é solicitado a garantir que a mensagem chegue a todos os nós com identificadores $28 \hat{y} k < 9$ (lembre-se de que estamos aplicando aritmética de módulo); o nó 18 cuida dos nós com identificadores $18 \hat{y} k < 28$; o nó 14 para $14 \hat{y} k < 18$; e 11 para identificadores $11 \hat{y} k < 14$.

O nó 28 dividirá posteriormente a parte do espaço do identificador que ele é solicitado a manipular em dois subsegmentos: um para seu nó vizinho 1 e outro para 4. Da mesma forma, o nó 18, responsável pelo segmento $[18, 28)$ irá “dividir” esse segmento em apenas uma parte: ele tem apenas um vizinho para delegar o fluxo e encaminha a mensagem para o nó 20, dizendo a ele que deve manipular o segmento $[20, 28)$.

Na última etapa, apenas o nó 20 tem trabalho a fazer. Ele encaminha a mensagem para o nó 21, dizendo-lhe para encaminhá-la para nós conhecidos por ele no segmento $[21, 28)$. Como não há mais tais nós, a transmissão é concluída.

Como no caso do nosso exemplo do hipercubo, vemos que a inundação é feita com $N \hat{y} 1$ mensagens, sendo N o número de nós no sistema.

4.4.3 Disseminação de dados baseada em fofocas

Uma técnica importante para disseminar informações é confiar no **comportamento epidêmico**, também conhecido como **fofoca**. Observando como as doenças se espalham entre as pessoas, os pesquisadores há muito tempo investigam se técnicas simples poderiam ser desenvolvidas para disseminar informações em sistemas distribuídos de larga escala. O principal objetivo desses **protocolos epidêmicos** é propagar rapidamente informações entre uma grande coleção de nós usando apenas informações locais. Em outras palavras, não há um componente central pelo qual a disseminação de informações seja coordenada.

Para explicar os princípios gerais desses algoritmos, assumimos que todas as atualizações para um item de dados específico são iniciadas em um único nó. Dessa forma, simplesmente evitamos conflitos de gravação-gravação. A apresentação a seguir é baseada no artigo clássico de Demers et al. [1987] sobre algoritmos epidêmicos. Uma visão geral da disseminação de informações epidêmicas pode ser encontrada em [Eugster et al., 2004].

Modelos de divulgação de informação

Como o nome sugere, algoritmos epidêmicos são baseados na teoria de epi-demias, que estuda a disseminação de doenças infecciosas. No caso de sistemas distribuídos em larga escala, em vez de espalhar doenças, eles espalham informações. Pesquisas sobre epidemias para sistemas distribuídos também visam a um objetivo totalmente diferente: enquanto organizações de saúde farão o melhor para evitar que doenças infecciosas se espalhem por grandes grupos de pessoas,

Os projetistas de algoritmos epidêmicos para sistemas distribuídos tentarão "infectar" todos os nós com novas informações o mais rápido possível.

Usando a terminologia de epidemias, um nó que faz parte de um sistema distribuído é chamado de **infectado** se ele contém dados que ele está disposto a espalhar para outros nós. Um nó que ainda não viu esses dados é chamado de **suscetível**. Finalmente, um nó atualizado que não está disposto ou não é capaz de espalhar seus dados é dito ter sido **removido**. Observe que assumimos que podemos distinguir dados antigos de novos, por exemplo, porque eles foram marcados com data e hora ou versionados. Sob essa luz, os nós também são ditos espalhar atualizações.

Um modelo de propagação popular é o da **anti-entropia**. Neste modelo, um nó P escolhe outro nó Q aleatoriamente e, subsequentemente, troca atualizações com Q. Existem três abordagens para trocar atualizações:

1. P só puxa novas atualizações de Q
2. P só empurra suas próprias atualizações para Q
3. P e Q enviam atualizações um para o outro (ou seja, uma abordagem push-pull)

Quando se trata de espalhar atualizações rapidamente, apenas enviar atualizações por push acaba sendo uma escolha ruim. Intuitivamente, isso pode ser entendido da seguinte forma. Primeiro, observe que em uma abordagem baseada em push puro, as atualizações podem ser propagadas apenas por nós infectados. No entanto, se muitos nós forem infectados, a probabilidade de cada um selecionar um nó suscetível é relativamente pequena. Consequentemente, as chances são de que um nó específico permaneça suscetível por um longo período simplesmente porque um nó infectado não o seleciona.

Em contraste, a abordagem baseada em pull funciona muito melhor quando muitos nós são infectados. Nesse caso, a disseminação de atualizações é essencialmente acionada por nós suscetíveis. As chances são grandes de que tal nó entre em contato com um infectado para, posteriormente, puxar as atualizações e também ser infectado.

Se apenas um único nó for infectado, as atualizações se espalharão rapidamente por todos os nós usando qualquer forma de antientropia, embora push-pull continue sendo a melhor estratégia [Jelasity et al., 2007]. Defina uma **rodada** como abrangendo um período no qual cada nó terá tomado a iniciativa uma vez para trocar atualizações com um outro nó escolhido aleatoriamente. Pode-se então mostrar que o número de rodadas para propagar uma única atualização para todos os nós é da ordem $O(\log(N))$, onde N é o número de nós no sistema. Isso indica de fato que a propagação de atualizações é rápida, mas acima de tudo escalável.

Nota 4.16 (Avançado: Uma análise da anti-entropia)

Uma análise simples e direta dará uma ideia de quão bem a antientropia funciona. Considere um sistema com N nós. Um desses nós inicia a propagação de uma mensagem m para todos os outros nós. Seja p_i a probabilidade de que um nó P ainda não tenha recebido m após a rodada i . Distinguímos os três casos a seguir:

O

- Com uma abordagem puramente pull, $\pi_{i+1} = (\pi_i)^2$: não apenas P ainda não havia sido atualizado na rodada anterior, como também P contata um nó que ainda não havia recebido m.
- Com uma abordagem baseada em push puro, $\pi_{i+1} = \pi_i \cdot (1 - \frac{1}{N-1})^{N(1-\pi_i)}$: novamente, P não deveria ter sido atualizado na rodada anterior, mas também nenhum dos nós atualizados deveria contatar P. A probabilidade de que nenhum nó N_{i+1} ; podemos esperar que haja $N(1 - \pi_i)$ nós atualizados contate P é $1 - \pi_i$ na rodada i.
- Em uma abordagem push-pull, podemos simplesmente combinar os dois: P não deve entrar em contato com um nó atualizado e não deve ser contatado por um.

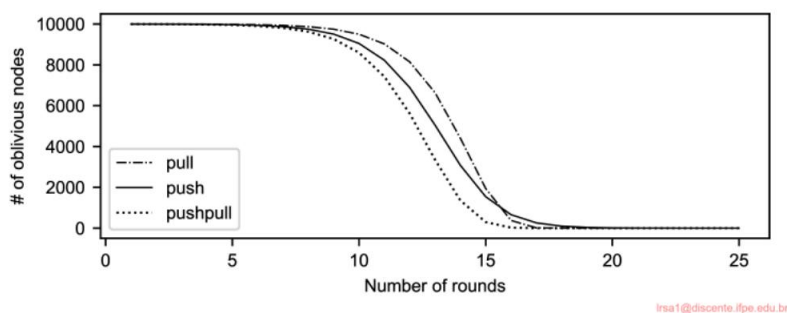


Figura 4.35: (a) O número de nós que não foram atualizados em função do número de rodadas de disseminação.

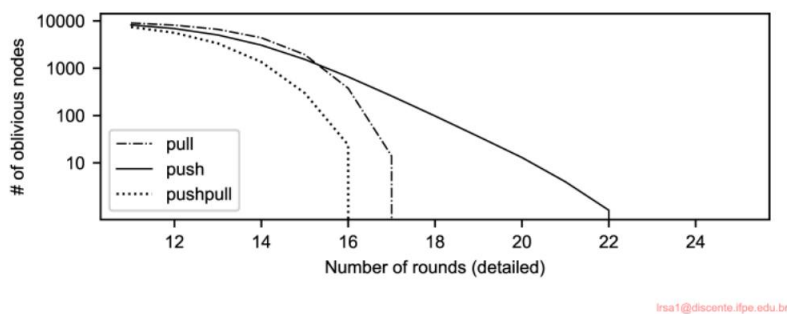


Figura 4.35: (b) Ampliando as diferenças entre as três abordagens anti-entropia quando quase todos os nós foram atualizados.

A Figura 4.35(a) mostra quão rapidamente a probabilidade de ainda não ter sido atualizado cai como uma função do número de rodadas em uma rede de 10.000 nós. Após um início lento, a informação é rapidamente disseminada. Na Figura 4.35(b), ampliamos o que acontece no último número de rodadas enquanto também usamos uma escala logarítmica para o eixo y. De fato, assumindo que os nós estão ativos e funcionando o tempo todo,

Acontece que uma estratégia push-pull anti-entropia pode ser um protocolo de disseminação extremamente eficaz.

Uma variante específica de protocolos epidêmicos é chamada de **espalhamento de rumores**. Ela funciona da seguinte maneira. Se o nó P acabou de ser atualizado para um item de dados x, ele contata um outro nó arbitrário Q e tenta enviar a atualização para Q. No entanto, é possível que Q já tenha sido atualizado por outro nó. Nesse caso, P pode perder o interesse em espalhar a atualização ainda mais, digamos com probabilidade p_{stop} . Em outras palavras, ele então é removido.

A propagação de boatos é a fofoca, como a vivenciamos principalmente na vida real. Quando Bob tem uma notícia quente para espalhar, ele pode ligar para sua amiga Alice, contando tudo a ela. Alice, assim como Bob, ficará realmente animada para espalhar o boato para seus amigos também. No entanto, ela ficará desapontada ao ligar para um amigo, digamos Chuck, apenas para ouvir que a notícia já chegou até ele. As chances são de que ela pare de ligar para outros amigos, pois de que adianta se eles já sabem?

A disseminação de rumores acaba sendo uma excelente maneira de espalhar notícias rapidamente. No entanto, não pode garantir que todos os nós serão realmente atualizados [Demers et al., 1987]. De fato, quando há muitos nós que participam das epidemias, a fração s de nós que permanecerão ignorantes de uma atualização, ou seja, permanecerão suscetíveis, satisfaz a

$$s = e^{-\frac{1}{p_{stop} + 1}}$$

Para ter uma ideia do que isso significa, dê uma olhada na Figura 4.36, que mostra s como uma função de p_{stop} . Mesmo para valores altos de p_{stop} , vemos que a fração de nós que permanece ignorante é relativamente baixa e sempre menor que aproximadamente 0,2. Para $p_{stop} = 0,20$, pode ser mostrado que $s = 0,0025$. No entanto, nos casos em que p_{stop} é relativamente alto, medidas adicionais precisarão ser tomadas para garantir que todos os nós sejam atualizados.

Nota 4.17 (Avançado: Análise da propagação de rumores)

Para analisar formalmente a situação de disseminação de rumores, vamos denotar a fração de nós que ainda não foram atualizados, ou seja, a fração de nós suscetíveis. Da mesma forma, deixe i denotar a fração de nós infectados: aqueles que foram atualizados e ainda estão contatando outros nós para espalhar informações. Finalmente, r é a fração de nós que foram atualizados, mas desistiram, ou seja, eles são passivos e não desempenham mais um papel na disseminação de informações. Obviamente, $s + i + r = 1$. Usando a teoria de epidemias, não é difícil ver o seguinte:

$$\begin{aligned} (1) \frac{ds}{dt} &= -\gamma s \cdot i & (2) \frac{di}{dt} &= \gamma s \cdot i - \gamma p_{stop} \cdot i \\ \frac{di}{ds} &= \frac{\gamma s \cdot i}{-\gamma s \cdot i} = -1 & \frac{di}{ds} &= \frac{\gamma s \cdot i}{-\gamma s \cdot i} = -1 \\ \frac{di}{ds} &= \frac{\gamma s \cdot i}{-\gamma s \cdot i} = -1 & \frac{di}{ds} &= \frac{\gamma s \cdot i}{-\gamma s \cdot i} = -1 \end{aligned}$$

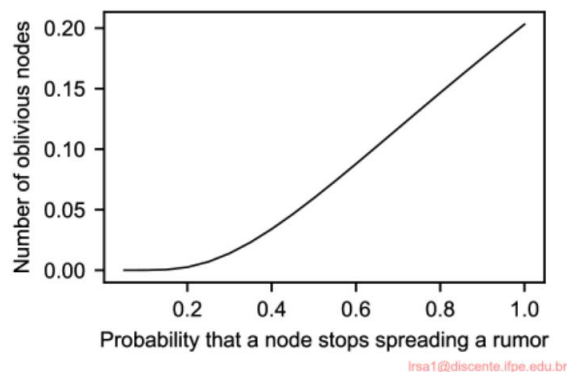


Figura 4.36: A relação entre a fração s de nós ignorantes de atualização e a probabilidade p_{stop} de que um nó pare de focar quando entrar em contato com um nó que já foi atualizado.

onde usamos a notação $i(s)$ para expressar i como uma função de s . Quando $s = 1$, nenhum nó foi infectado ainda, o que significa que $i(1) = 0$. Isso nos permite derivar que $C = 1 + p_{stop}$ e, portanto,

$$i(s) = (1 + p_{parada}) \cdot (1 - s) + p_{parada} \cdot \ln(s)$$

Estamos procurando a situação em que não haja mais rumores se espalhando, ou seja, quando $i(s) = 0$. Ter uma expressão fechada para $i(s)$ leva a

$$s = e^{-\frac{1}{p_{parada}+1}(1-s)}$$

Uma das principais vantagens dos algoritmos epidêmicos é sua escalabilidade, já que o número de sincronizações entre processos é relativamente pequeno em comparação a outros métodos de propagação. Para sistemas de área ampla, Lin e Marzullo [1999] mostraram que faz sentido levar em conta a topologia de rede real para obter melhores resultados. Nesse caso, os nós que estão conectados a apenas alguns outros nós são contatados com uma probabilidade relativamente alta. A suposição subjacente é que esses nós formam uma ponte para outras partes remotas da rede; portanto, eles devem ser contatados o mais rápido possível. Essa abordagem é chamada de **fofoca direcional** e vem em diferentes variantes.

Este problema toca em uma suposição importante que a maioria das soluções epidêmicas faz, a saber, que um nó pode selecionar aleatoriamente qualquer outro nó para focar. Isso implica que, em princípio, o conjunto completo de nós deve ser conhecido por cada membro. Em um sistema grande, essa suposição nunca pode ser válida, e medidas especiais precisarão ser tomadas para imitar tais propriedades. Retornaremos a essa questão na Seção 5.5.2 quando discutirmos um **serviço de amostragem por pares**.

Removendo dados

Algoritmos epidêmicos são fantásticos para espalhar atualizações. No entanto, eles têm um efeito colateral um tanto estranho: espalhar a exclusão de um item de dados é difícil. A essência do problema está no fato de que a exclusão de um item de dados destrói todas as informações sobre esse item. Consequentemente, quando um item de dados é simplesmente removido de um nó, esse nó eventualmente receberá cópias antigas do item de dados e as interpretará como atualizações sobre algo que ele não tinha antes.

O truque é registrar a exclusão de um item de dados como apenas mais uma atualização e manter um registro dessa exclusão. Dessa forma, cópias antigas não serão interpretadas como algo novo, mas meramente tratadas como versões que foram atualizadas por uma operação de exclusão. O registro de uma exclusão é feito espalhando certidões **de óbito**.

Claro, o problema com as certidões de óbito é que elas devem ser eventualmente limpas, ou então cada nó irá gradualmente construir um enorme nó local. banco de dados de informações históricas sobre itens de dados excluídos que, de outra forma, não seriam usado. Demers et al. [1987] propõem usar o que são chamados de certidões **de óbito inativas**. Cada certidão de óbito é carimbada com data e hora quando é criada. Se for possível assumir que as atualizações se propagam para todos os nós dentro de um tempo finito conhecido, então as certidões de óbito podem ser removidas após esse tempo máximo de propagação ter decorrido.

No entanto, para fornecer garantias rígidas de que as exclusões são de fato espalhadas para todos os nós, apenas alguns poucos nós mantêm certidões de óbito inativas que nunca são descartadas. Suponha que o nó P tenha tal certidão para um item de dados x. Se por acaso uma atualização obsoleta para x chegar a P, P reagirá simplesmente espalhando a certidão de óbito para x novamente.

4.5 Resumo

Ter recursos poderosos e flexíveis para comunicação entre processos é essencial para qualquer sistema distribuído. Em aplicações de rede tradicionais, a comunicação é frequentemente baseada em primitivas de passagem de mensagens de baixo nível oferecidas pela camada de transporte. Uma questão importante em sistemas de middleware é oferecer um nível mais alto de abstração que tornará mais fácil expressar a comunicação entre processos do que o suporte oferecido pela interface para a camada de transporte.

Uma das abstrações mais amplamente utilizadas é a Chamada de Procedimento Remoto (RPC). A essência de uma RPC é que um serviço é implementado por meio de um procedimento, cujo corpo é executado em um servidor. O cliente recebe apenas a assinatura do procedimento, ou seja, o nome do procedimento junto com seus parâmetros. Quando o cliente chama o procedimento, a implementação do lado do cliente, chamada de stub, cuida de encapsular os valores dos parâmetros em uma mensagem e enviá-la ao servidor. Este último chama o procedimento real e retorna os resultados, novamente em uma mensagem. O stub do cliente

extrai os valores do resultado da mensagem de retorno e os passa de volta para o aplicativo cliente que faz a chamada.

RPCs oferecem facilidades de comunicação síncrona, pelas quais um cliente é bloqueado até que o servidor tenha enviado uma resposta. Embora existam variações de qualquer mecanismo pelas quais esse modelo síncrono estrito é relaxado, acontece que modelos de propósito geral, orientados a mensagens de alto nível, são frequentemente mais convenientes.

Em modelos orientados a mensagens, as questões são se a comunicação é persistente e se a comunicação é síncrona. A essência da comunicação persistente é que uma mensagem que é enviada para transmissão é armazenada pelo sistema de comunicação pelo tempo que leva para entregá-la. Em outras palavras, nem o remetente nem o destinatário precisam estar ativos e funcionando para que a transmissão da mensagem ocorra. Na comunicação transitória, nenhuma facilidade de armazenamento é oferecida, de modo que o destinatário deve estar preparado para aceitar a mensagem quando ela for enviada.

Na comunicação assíncrona, o remetente tem permissão para continuar imediatamente após a mensagem ter sido enviada para transmissão, possivelmente antes mesmo de ter sido enviada. Na comunicação síncrona, o remetente é bloqueado pelo menos até que uma mensagem tenha sido recebida. Alternativamente, o remetente pode ser bloqueado até que a entrega da mensagem tenha ocorrido ou até mesmo até que o destinatário tenha respondido, como com RPCs.

Modelos de middleware orientados a mensagens geralmente oferecem comunicação assíncrona persistente e são usados onde RPCs não são apropriados. Eles são frequentemente usados para auxiliar na integração de coleções (amplamente dispersas) de bancos de dados em sistemas de informação de larga escala.

Finalmente, uma classe importante de protocolos de comunicação em sistemas distribuídos é o multicasting. A ideia básica é disseminar informações de um remetente para vários receptores. Discutimos duas abordagens diferentes. Primeiro, o multicasting pode ser alcançado configurando uma árvore do remetente para os destinatários. Considerando que agora é bem compreendido como os nós podem se auto-organizar em um sistema peer-to-peer, soluções também surgiram para configurar árvores dinamicamente de forma descentralizada. Segundo, o *ýoding* de mensagens pela rede é extremamente robusto, mas requer atenção especial se quisermos evitar desperdício severo de recursos, pois os nós podem ver mensagens várias vezes. O *ýoding* probabilístico, pelo qual um nó encaminha uma mensagem com uma certa probabilidade, muitas vezes prova combinar simplicidade e eficiência, sendo ao mesmo tempo altamente eficaz.

Outra classe importante de soluções de disseminação implementa epidemias protocolos. Esses protocolos provaram ser simples e extremamente robustos.